ADV JAJA

BY

SHEKAR

SATYA TECHNOLOGIES

KONARK XEROX

9949585888

2011-07

1. J2SE / JSE → Desktop, client-Ser (2-tier) idk s/w - JSE
2. J2ME / Java ME → Mobiles, Digital s, PDA's (constrained resources)
3. J2EE / Java EE → web applications, stributed * Enterprise
   ↓ Adv. Java
   (Real-time app ions) (client-server) (2-tier, 3-tier, N-tier)
   EJB

→ JDBC, Servlet, JSP, EJB

Java to database → J2SE
web apps → J2EE
Distributed → JEE

Introduction :-

"SUN" microsystems ha divided java technology into 3 modules.

1. J2SE / Java SE (Standard Editi
2. J2ME / Java ME (Micro Edition
3. J2EE / Java EE (Enterprise E n).

1. Java SE :-

JSE is suitable for nd-alone ie., desktop *. 2-tier (client-server) applications. JSE is an installable s/w.

* Java SE module is for develo both console and windows
  ↓ CUI Standalone
  ↓ GUI
  applications in Java
* JDBC - JSE

2. Java ME :-

JME is suitable f the development of app.'s for resource constrained devices. (Personal Digital Assistants),

Ex:- For mobile applications, PI Digital set-top boxes e.t

Java EE is suitable for the develpment of web-applications, Distributed & Enterprise applications.

* Java EE is not an installable S/w.
* JavaSE is the base for Java EE
  Servlet, JSP
* Java EE is a S/w in the form of jar (Java Archeive) files
* Java EE is totally for the develpment of client-server applications only ie., 2-tier, 3-tier & n-tier
* web applications are servlet, JSP
* Distributed & Enterprise applications are: EJB
* In Java EE, we have the folbwing 3 technologies
  1. Servlet
  2. JSP
  3. EJB
* Servlet, JSP technologies are for develpment of web applications
* EJB technology is used for develpment of distributed & Enterprise application.

Client-Server models:-

client-server model applications by depending on the 3 logics of the project.

1. presentation logic/layer
2. Business logic
3. Data Access logic/persistance logic

Model:-

Model means an application (or) software.

→ To develop client-server applications we do not requir physically multiple systems are machines.

→ In a single physical machine, we can able to develop entire applications also.

→ The client-server model will be decided either by using the no. of s/w's communicating or no. of application communicating, but not no. of systems communicating.

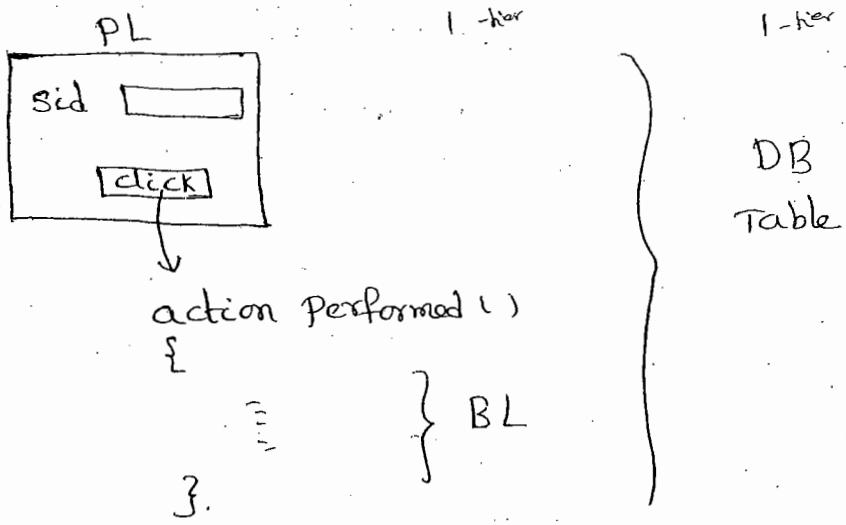
→ Single-tier applications are nothing but the applications that contains all the 3 logics inside it.

ex :- MS-Access.

→ In client-server model applications, depends on the logics, the applications are consider as a two-tier (or) 3-tier (or) n-tier.

→ In advance java, we use JDBC, Servlet, JSP technologies

→ JDBC technology is for the development of 2-tier.



→ Servlet & JSP technologies are 3-tier & n-tier applications.

HTML | Servlet | DB

PL BL DAL

Web :-

A web application is a server side application. it runs at server or executes at server and provides service to the clients across internet.

→ web applications produces web pages onto the client browser.

For example, every web site is a web application.

→ Generally, websites are providing services like mail and chating.

For example, the websites like gmail & yahoo are providing both mailing & chating services to the clients across the n/w

→ whenever, we want to provide service to multiple clients across internet then we need to choose web app's.

→ If at all we want to provide service to an single client at a time. then we need to choose a desktop applications.

→ In todays world, most of the web application development is done by using java technologies like servlets, JSP's

→ The difference b/w a web application & a website is a web application is an application that has not yet placed into an internet server. But once, it is placed in an internet server then it becomes as a website.

Q what is the difference between S/w project & S/w product?

Ans :- 1. If an application is developed, according to the requirements of a client organization then we call that application as a S/w project.

2. If an application is developed, not according to

The requirements of an organization otherwise client organization then we call that application as a s/w product

* For example, Finacle this is the client for Infosys.

Finacle is a s/w project developed by Infosys for a client called ICICI.

* oracle is a s/w product and it can be used by any time.

→ web applications are divided into 2 types:

1. Presentation oriented
2. Service oriented.

→ To develop Presentation (→ HTML) oriented web applications we do not require any technologies. Just HTML pages are enough.

For example, online tutorials (→ www.roseindia.net) or presentation oriented webapplications.

→ Service-oriented (→ Servlets, JSP) applications development uses web technologies.

These type of applications are concentrating on providing services to the client.

for example, gmail, yahoo, rediff.

→ To develop service-oriented web applications, so far we got the following technologies.

(i) CGI (Common Gateway Interface)

(ii) Servlet

(iii) Server pages

→ Java developers uses Servlet & JSP technologies & .Net people uses asp.net for the development of service-oriented web applications.

Reflection is a mechanism used for finding reflective information about a class or an interface of java.

→ Reflective information is nothing but getting metadata of a class or an interface.

→ Meta data of a class means, finding the details about the class like it's base class, interfaces implemented by the class, constructors of the class. Methods of the class & variables of the class (fields).

→ Reflection is used for work finding metadata of a pre-defined class or metadata of a user-defined class also.

→ we can find metadata of the current class which is running or we can find metadata of another class, by loading it.

→ Reflection, is required for the programmers who are interested in the development of s/w products

→ Reflection is used, mostly wider developing java class browsers, java debuggers & java Compilers.

→ Reflection is internal used in the development of IDE s/w's.

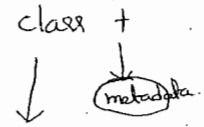
Loading a class:-

whenever, we want to find details (metadata) of a class or an interface then first of all that class or interface must be loaded into JVM

* In order to load a class or an interface into JVM at run-time (dynamic), we need to use a method forName().

forName()

forName() is a static method even in class. So, we need to call forName() method, with respect to class name



→ whenever a class is loaded into JVM then internally JVM seperates metadata into an object and then that object will be return back to the programmer.

→ If a programmer wants to know the metadata then the programmer receive the object & otherwise no need to receive the object.

→ while loading a class, first JVM verifies whether the class is already loaded or not. If not loaded then it loads the class now. If it is already loaded then the class is not loaded once again.

```
class c = class.forName("B");
" " " " " " " ";
```

} multiple times given it can't be executed.

→ A JVM loads a class for only once.

class → Keyword.

class → class name (java.lang)

↓ already defined.

```
class A
{
    P.S.V.m()
}
class C = class.forName("B")
    ↓       ↓        ↓
  object          static method.
}
}
```

41.

ERROR : Could not find String

C:\> javap java.lang.String ↵ (→ fully qualified class (including package))

= } metadata

Ex:- public class st extends String (→ final ↓ doesn't extend.)

→ we can also find metadata of a class or an interface using a tool given by JDK or Javap (Java profiler).

→ For example, if we want to know metadata of String class then the following command is required.

C:\> javap java.lang.String

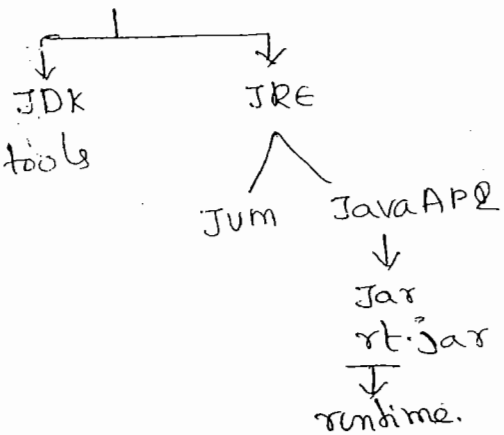
→ Reflection mechanism = Javap tool. But the difference is, Javap is only used at command prompt, but not written a java application.

→ The basic step to Enter into the reflection programming is loading a class. This can be done by using the following

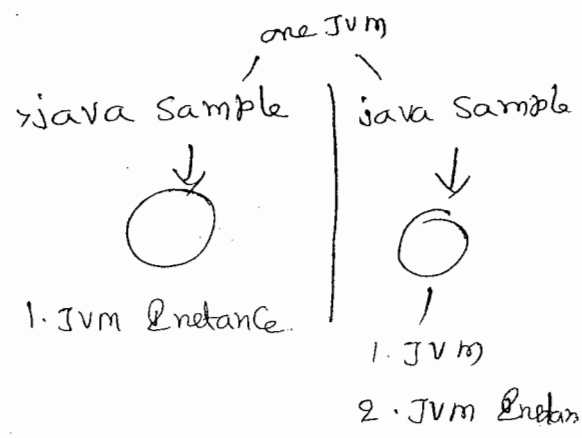
Syntax

Class c = Class.forName("fully qualified classname");

→ Java S/w



→ java Sample



For ex:

wrapper class

```
Class c = Class.forName("java.lang.Integer");
```

Q. what is a factory method?

Ans:- A method which produces an object is called as an factory method.

* Factory methods are of 2 types:
  1. Static factory methods
  2. Instance factory methods

* The following statements are examples for static factory methods in java.

1. Class c = Class.forName("java.lang.String");
2. Calender c = Calender.getInstance();
   ↓ abstract class & interface not create objects
3. Thread t = Thread.currentThread();

Q. In how many ways dynamic loading of a Java class into JVM is possible?

Ans:- 2 ways

1. By using forName() method given by Class c
2. By using loadClass() method given by ClassLoader

```
Class c = Class.forName("FQCN");
```
FQCN → fully qualified class name
```
Class c = ClassLoader.loadClass("FQCN");
```

Note:- whenever a class is loaded into JVM then not only that class, but also it's related classes & interfaces are also automatically loaded into JVM.

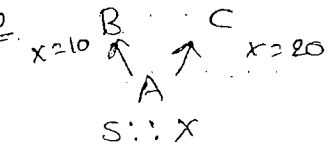
Reflexive superclass / base class

1. Whenever a class is loaded into JVM then automatically its superclass is loaded into JVM.
2. If we want to get superclass metadata then we need to call a method getSuperclass()
3. In java, one class can extend a maximum of one base class at a time. The reason is java doesn't support multiple inheritance at class level.

Q. Why Java doesn't support multiple inheritance at class level?

Ans:- If multiple inheritance is supported then there is a chance of getting ambiguity error. To solve this error, we need scope resolution (::) operator. But this operator is not supported in java. So multiple inheritance is not supported in java at class level.

CPP



Note:- At interfaces level, java supports multiple inheritance. But in an interface all variables are static & final. So, no need of scope resolution operator

→ The following code is to print superclassName for the loaded class.

```
class c = class.forName("java.lang.Integer");
Class sc = c.getSuperclass();
String str = sc.getName();
S.O.P(str);
```

→ The following example is for printing superclass name for the given class as a command line argument.

```
// Reflect1.java
class Reflect1
{
    static public void main(String[] args)
                    throws Exception
    {
        class c = class.forName(args[0]);
        class sc = c.getSuperclass();
        String s1 = c.getName();
        String s2 = sc.getName();
        S.O.P(s1 + "extends" + s2);
    }
}; //optional
```

It is given by JDK
String.... args.
(or)
we can also write args because it array var

JVM will handle → throws Exception

class.forName(java.util.String)
cause exception because we can use long

O/P:-

→ javac Reflect1.java
→ java Reflect1 java.lang.Integer ↵

java.lang.Integer extends java.lang.Number.

→ java Reflect1 java.lang.Exception ↵

java.lang.Exception extends java.lang.Throwable.

→ In the above example, we have used throws exception stmt. for the main method. The reason is forName() method throws ClassNotFoundException if the given class is not found.

→ In java end of the class can be represented with semicolon (;) But it is optional.

1. whenever a class is loaded into JVM then automatically all implemented interfaces are also loaded.
2. If we want to find metadata i.e., the interface names implemented by the class then we need to call a method called getInterfaces();
3. The method getInterfaces() returns an array of of classtype. By using a loop we can get name of each interface.

→ The following example is for finding the interface names implemented by a class, which is given as command line argument

```
// Reflect2.java
class Reflect2
{
  p.s.v.m (String [] args) throws Exception
  {
    Class c = Class.forName(args[0]);
    Class it[] = c.getInterfaces();
    for (int i=0; i<it.length; i++)
    {
      String s1 = it[i].getName();
      System.out.println(s1);
    }
  }
}
```

O/P:-

```
c:\> javac Reflect2.java
c:\> java Reflect2 java.lang.String
java.io.Serializable
java.lang.Comparable
java.lang.charSequence
c:\> java Reflect2 java.lang.Thread
java.io.Runnable ← interface
```

Q :- Difference b/w length and length() :

Ans :- → length property is used for finding size of an array

→ length() method is used for finding for size of a string.

For example :- String str[] = { "Java", "oracle", "cpp" };

str.length

str[1].length()

Q In Java, why we are importing parents child packages seperately ?

Ans :- Because, at packages level java doesn't support Inheritance so, we are importing parent & child packages seperately in java.

class extends class

interface extends interface.

class implements interface

Package _____ Package (Not supported)

Note :- Reflection always getting data from heap (instance variables stored (or) fields store

Finding fields data :-

1. A field is nothing but an instance variable. It means a variable created (or) declared inside the class.
2. In Java, we can also declare variables inside a method (or) block. we call those variables as "local variables"
3. Reflection can be used to find the variables details that are created inside a class. we call these variables as "fields."
4. Reflection doesn't provide details of local variables. because, local variables are stored in "stack memory"

```
{
    int x;      → field (class scope)
    void m1()
    {
        int y;  → local variable (method scope)
        if(---)
        {
            int z; → local variable (block scope)
        }
    }
}
```

→ while finding details of fields, we need to store the each field metadata into an object of type Field class

→ Field is a class given in java.lang.reflect package.

→ java.lang.reflect is a subpackage of java.lang (Default Package)

→ Reflection API contains a single package called ___

java.lang.reflect

In Java, API → classes & interfaces → Packages

→ java → API → classes & interfaces → Packages

C++ → API → functions & classes → header files

C → API → functions → header files.

java 1.5 - 3562 classes

1.6 - 3562 + ___ classes

Note:- when we install java s/w, we will get the java API into our system in the form "rt.jar" (runtime)

→ In Reflection, we have to import "java.lang.reflect" package explicitly into a java program. the reason is, if a parent package is imported, then child package is not automatically imported

The following example is for finding the details of the fields of a loader class.

```
//Reflect3.java
import java.lang.reflect.*;
class Reflect3
{
    P.S.V.m(String[] args) throws exception
    {
        Throws Exception
        {
            class c = class.forName(args[0]);
            Field f[] = c.getFields();
            for(int i=0; i<f.length; i++)
            {
                String fname = f[i].getName();
                String dtname = f[i].getTYPE().getName();
                S.O.P(dtname + " " + fname);
            }
        }
    }
}
```

O/P:-

```
c:\> javac Reflect3.java
c:\> java Reflect3 java.lang.Thread
    int MIN_PRIORITY
    int NORM_PRIORITY
    int MAX_PRIORITY
```

Q:- what is a Singleton variable in Java?

Ans:- If a variable is public, static & final then that variable is called "Singleton variable"

Q what is Singleton class?

Ans:- For a class, if it is possible to create only one object then we call that class as a "Singleton class"

Q ...

Ans:- Yes

Actually, main() is called by JVM but here we have to call the main() with out JVM

Example:-

```
class Sample1
{
   static
   {
      String str[] = {"abc", "xyz"};
      main(str);
      System.exit(0);
   }
   p.s.v.m(String[] args)
   {
      S.O.P("I am main()....");
   }
}
```

```
c:\> javac Sample1.java
c:\> java Sample1
I am main()..... } called by Programmer
```

Q what is the difference between getFields() method & getDeclaredFields() method?

Ans:- getFields() method returns the metadata of derived class fields including its base class fields also But, getDeclaredFields() method returns only metadata of the fields related to the current class only, but not its base class.

1. Once a class is loaded, if we want to find the details of the constructors of a loaded class then, we need to call getConstructors()

2. For each constructor metadata, an object of type Constructor class is created.

3. Constructor is a class given in "java.lang.reflect" package.

```
Class c = class.forName(" ");
Constructor cons[] = c.getConstructors();
```

Note:- Constructors doesn't have return type but method has a return type. By default constructors can be created at Compile-time.

Note:- Constructors can't be inherited, but called from base class. Methods, variables can be inherited from base class. Constructors can be executed from top to bottom and destructors can be executed from bottom to top and in Java, we use finalize() instead of Destructors.

Example:- The following example is for printing details of constructors of a loaded class.

```
//Reflect4.java
import java.lang.reflect.*;
class Reflect4
{
    public static void main(String[] args) throws Exceptio
    {
        Class c = Class.forName(args[0]);
        Constructor cons[] = c.getConstructors();
        for(int i=0; i<cons.length; i++)
        {
            String conname = cons[i].getName();
            System.out.print(conname + "(");
```

Class type of an array ← ↓ → To get the Parameters of the Constructor.

```
        for(int j=0; j<p.length; j++)
        {
            String ptype = p[j].getName();
            S.O.print(ptype + " ");
        } //inner
        System.out.println(")");
    } //outer
}
}
```

Note:- String is final class that's why we can't be inherited.

O/P:-

```
C:\> javac Reflect4.java
C:\> java Reflect java.lang.String
java.lang.String([C, int, int)
```

(↑ characterarray)

Note:- java.lang.math & java.lang.System, both classes contains private constructors in the class. so, we cannot create an object of Math & System classes directly.

* All methods are in Math & System classes are static, because we call by using classname

(X) Math m = new Math()

```
Math.Pow(a,b)
System.out.println();
```

↓ obj of PrintStream class created in System class.

Note:- Math and System classes contains all methods and variables as static. so, we can call the methods directly by using classname.

Note:- Static methods doesn't allow this and super keywords of Java.

* Static methods allows only static variables of the class but instance variables are not allowed.
* If variables declared inside the static it is local & it can be accessed

```
class
static int a=50;
static void print()
{
  int m=40;
}
```

```
P.S.V.m(String args)
{ int a,b,c;
}
```

Q Why sizeof() is removed from java?

Ans:- In Java, memory for all primitive types are constant and doesn't change from one O.S to another O.S. But, In C & C++, according to the O.S, memory will be changed. Because of this reason, sizeof() method is given (or) available in C & C++ and it is removed from java.

	16 bits DOS	32 bits UNIX	64 bits windows	
C int a;	2 bytes	4 bytes	8 bytes	→ P.D.
Java.	4	4	4.	→ P.I.

Note:-
Java can executed faster in UNIX O.S than it in any OS. The execution speed depends on the data transfer.

Finding methods:-

1. Once a class is loaded into JVM, if we want to find the methods details of the class then we need to call getMethods().
2. getMethods() returns an array of Method type.
3. Method is a class given in java.lang.reflect.*

inherited but constructors cannot be inherited.

5. We can also call getDeclaredMethods(), for finding the details of the methods of a loaded class.

*6. The difference between getMethods() & getDeclaredMethods() is, getMethods() means, it returns current class methods and also base class methods. whose as, getDeclaredMethods() means, it returns only the current class methods, but not including the base class methods.

7. finding methods is similar to finding constructors. But, the difference is, for a constructor we do not have any return type. whose as, for a method we need return type also.

The following example is for printing the methods of a given class.

```
// Reflect5.java
import java.lang.reflect.*;
class Reflect5
 {
    Public static void main (String[] args) throws Exception
    {
       class c = Class.forName(args[0]);
       Method m[] = c.getDeclaredMethods();
       for(int i=0; i<m.length; i++)
        {
           String mname = m[i].getName();
           String rtype = m[i].getReturnType.getName();
           System.out.print(rtype + " " + mname + "(");
```

```
Class P[] = mL[j].getParameterTypes();
for (int j=0; j<P.length; j++)
{
    String ptype = P[i].getName();
    System.out.print(ptyte + " ");
} //inner
System.out.println(" )");
} //outer
}
```

O/P:-

```
c:\> javac Reflect5.java
c:\> java Reflect5 java.lang.Exception
no methods in Exception class.
```

10/6/11

Finding Modifiers:-

1. In Java, we do not have any access specifiers instead we have all access modifiers.
2. For a class (or) an interface of java, we cannot provide "private" & "protected" modifiers.
3. A class (or) an interface in java can be either "public" (or) "default"
4. In a Java application (or) program, if there multiple classes then only one class can be made as "public". The public class name and the program name, both must be same.

Note:-

Sample.java

```
class A
{
}
class Demo
{
    main();
```

javac Sample.java

java Demo

java.lang.reflect.* ← Modifier → class

static methods

find modifiers of the class then we need to call the method getModifiers().

→ getModifiers() method returns an integer value. This integer value indicates whether a class is public (or) final (or) static (or) abstract.

```
class c = class.forName(args[0]);
int k = c.getModifiers();
```

→ In java.lang.reflect package, we have a class given called "Modifier" and this class provides "static methods" to find whether the int value belongs to which modifier.

→ For example, if we want to check whether a class is public (or) not then, the following condition is used.

```
if(Modifier.isPublic(K))
{
    S.O.P("Public class");
}
```

K → 1 Public
1 + 16 (Public Final)
6 (Static)
1 + 5 (Public abstract)

→ The following example is for finding modifiers of a class, by taking input as command line argument.

```
// Reflect6.java
import java.lang.reflect.*; (In real-time code we can't use the .*, we can provide particular class)
class Reflect6
{
    public static void main(String[] args) throws Exception
    {
        class c = class.forName(args[0]);
        int k = c.getModifiers();
        if(Modifier.isPublic(k))
        {
            S.O.P("Public class");
```

```
if (Modifier.isStatic(k))
{
    S.O.P ("static class");
}
if (Modifier.isFinal(k))
{
    S.O.P ("final class");
}
if (Modifier.isAbstract(k))
{
    S.O.P ("Abstract class");
}
}
}
```

Note:- It's not compulsory to save the class name as a program name.

O/P:-

```
C:\> javac Reflect6.java
C:\> java Reflect6 java.lang.String
public class
final class
C:\> java Reflect6 java.util.Calender
public class
Abstract class
```

→ Note:- In Java, Abstract and final both modifiers are not allowed for a class at a time. The reason is, abstract and final are opposite keywords.

* If a class is an abstract, then, that class must be extended (inherited). If a class is final, then that class cannot be extended. So, both keywords cannot be applied at a time.

Q what is the difference between an inner class and a Nested class?

static inner class is called "Nested"

Ex:-

```
class A                          class A
{                                {
  class B → innerclass             static class B → Nested class.
  {                                {
  }                                }
}                                }
```

Q Can we find sizeof an object of class in java (or) not.

Ans:- Yes, it is possible. we need to use "Runtime" class of java.lang.* package.

For example:- jdk 1.4 mechanism.

↑ method returns an object called factory method.

```
Runtime r = Runtime.getRuntime();
long x = r.totalMemory - r.freeMemory();
Sample s1 = new Sample();
long y = r.totalMemory - r.freeMemory();
long z = y - x;
S.O.P(z + "bytes");
```

x → It is for storing used the bytes memory before creating an object

y → It is for storing used memory after creating an object

In jdk 1.5 mechanism, java program execution starts from Premain(). It has two main()

1. premain()
2. main()

The size of an object can be find in jdk 1.5 by using Instrumentation interface. it is in java.lang.Instrument.*

```
long x = ins.getObjectSize();
```

Q. In how many ways we can create an object for a class in java?

Ans:- There are 5 ways to create an object for a class in java.

1. new Keyword
2. newInstance()
3. factory method
4. clone()
5. deserialization()

Note:- Among all the above 5 ways, new keyword is very expensive to create an object for a java class. So, in Real-time applications, the new keyword is used for less no. of cases.

18/6/19 → when a java programmer wants to create an object for a class at client side applications. Then mostly new keyword is used.

→ new keyword is very expensive, in terms of performance.

→ new keyword allocates memory in byte by byte manner.

→ newInstance() method is used by servers, while creating objects for the classes.

→ The advantage of newInstance() is, class name can be provided at run time dynamically to create an object of the class.

* The difference b/w new keyword & newInstance() is, in a new keyword class name must be known before compiling, to create an object for the class. But in a newInstance(), class name can be given at run_time for creating an object of the class.

we can not create an object for the class from outside of the class. In this case, we use factory method for creating an object of the class.

* Mostly we use factory method, to convert a class into a singleton class.
* If we want to create more number of objects for the same class, then instead of using new keyword for multiple times; we prefer clone() method.
* cloning is the process creating a duplicate object for an existing object.
* The advantage of clone() is the entire memory will be copied at once. So, it increases the performance of an application.

Ex:-
```
Sample S1 = new Sample();
Sample S2 = S1.clone();
```

Deserialization:-

It is a mechanism used in distributed programming of java.

* when an object is transfer from one system to another system in a network then the bytes of an object are transferred across n/w. At the other end these bytes are received and stored into an object. This process is called "Deserialization"



→ The following example is to create an object for a class using factory method.

```
// FactoryTest.java
class Sathya
{
    private sathya()
    {
    }
    Public static Sathya getobject()
    {
        return new sathya();
    }
    Public void display()
    {
        System.out.println("display");
    }
};
class FactoryTest
{
    Public static void main(String[]args);
    {
        // Sathya s2 = new Sathya(); error
        Sathya s1 = new Sathya()
        Sathya s1 = sathya.getobject();
        s1.display();
    }
}
```

O/P:-

```
C:\> javac FactoryTest.java
C:\> java FactoryTest ↵
display
```

In the above example, getobject() is

a static factory method.

* If getobject() method is called for 2 times then 2 times object is created for the class Sathya. So, class Sathya is not a singleton class.
* In the above example, we can convert the class Sathya into singleton. To do this create an object at outside of the getobject() method.

For example :-

```
class Sathya
{
  static Sathya s=new Sathya();
  return s;
}
```

(continue)

* Static method will allow only static variables of the class. Because of this reason, we made 's' as a static object.

→ The following code (or) example is to create an object by using clone()

```
//cloneTest.java
class Sample implements Cloneable
{
    public Sample()
    {
        System.out.println("object created");
    }
    public Sample clone() throws CloneNotSupportedException
    {
```

```
        Sample s = (Sample)super.clone();
        return s;
    }
};
class cloneTest
{
    public static void main(String []args) throws Exception
    {
        Sample s1 = new Sample();
        Sample s2 = s1.clone();
        Sample s3 = s1.clone();
    }
}
```

---- O/P:-

```
c:\> javac cloneTest.java
c:\> java cloneTest
```

object created.

Note:- In the above example totally 3 objects are created. But Constructor is executed for only once. the reason is, we use clone() method for creating two objects.

**** Q can we create an object for a class, by with out calling constructor of the class?

Ans:- Yes, Possible. It can be done by using clone() method.

→ This program is for factory method.

```
class Sathya
{
    static Sathya s = new Sathya();
    private Sathya()
```

(Continue)

```
        {

        }
    Public static Sathya getobject()
    {
        return s;
    }
    Public void display()
    {
        System.Out.Println("display");
    }
};

class FactoryTest
{
    Public static void main(String[] args)
    {
        Sathya s1 = Sathya.getobject();
        s1.display();
        Sathya s2 = Sathya.getobject();
        s2.display();
    }
}
```

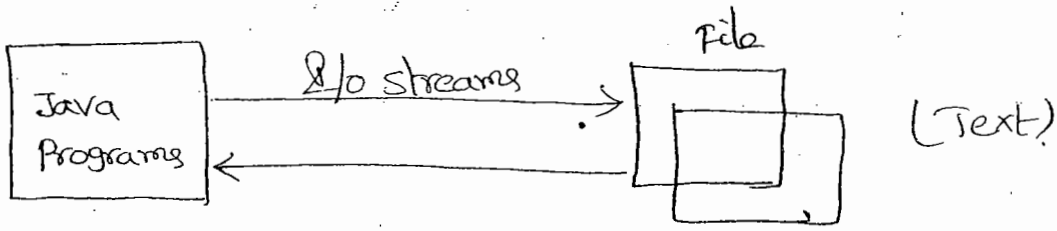
13/01

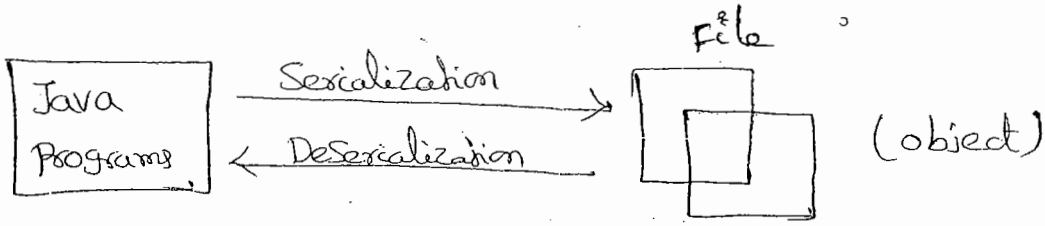
Java DataBase Connectivity (JDBC):-

File Management System (FMS):-

Storing data in a permanent area is called "Persistency." This permanent data is called as Persistant data.

* As a Java Programmer, if we want to store the data permanently then we can make use of a file as a storage area.
* Java programmer uses I/O streams for reading & writing the data using a file. We call the data format as a "Text."
* A Java Programmer can also store an object into a file. Later, when it is required we can read the object back from a file. This process of reading and writing objects is called "Serialization."

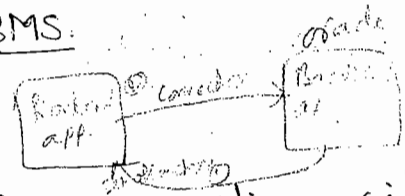




working with files is having the following drawbacks:

1. files are simple, we [illegible]
programmer has to care about file opening & file closing.
2. files doesn't support Query language. So, Reading & writing the data using file is complex.
3. It is not possible to maintain, the Relationship b/w the data stored in files.

→ In order to overcome the above drawbacks, we are shifted from FMS into DBMS.



Why JDBC?

1. When a front-end application connecting with back-end, at initial days, with front-end application used set of functions given by database Vendor to connect with the database.
2. Even today, A C & C++ applications uses "orcl.h" header file given by "oracle" to connect with the oracle database.
3. The drawback of using Vendor given functions (API) is, the front end application becomes Database Dependent.
4. In order to overcome the above drawback, with professionals from TOP MNC companies, a group was formed called "X/OPEN" Community.
5. The X/OPEN group was formed, to make a front-end application as database Independent.
6. X/OPEN group suggested a set of functions called as X/OPEN API. But, this API was

rejected by the DataBase vendors. Because, the implementation requires ALP (Assembly Language Programming). So, X/OPEN group was failed.

7. After X/OPEN failed, Microsoft itself has formed another group with its own Professions was ODBC group (Open DataBase Connectivity)
8. ODBC group created a set of fenctions called as ODBC API and the DataBase vendors accepted ODBC API. Because, the Implementation of ODBC requires 'C' language with pointers coding. It means ODBC API was success.
9. If a front-end application uses ODBC then it can connect with any database.
10. Java application uses ODBC. then we have following two drawbacks.
    (i) Java has no pointers but, ODBC is in pointers. So, non-pointers to pointers conversion is provi required It becomes slow connectivity from Java to DataBase.
    (ii) Java is platform Independent but, ODBC is Platform Dependent. So, if Java uses ODBC then Java also becomes platform Dependent. This is against Java Principle ie., write once Run Anywhere.

* In order to overcome the above dracobacks, SUN micro Systems introduced its own technology as "JDBC"

as front-end applications in Real-time?

Ans:- If C & C++ are used as front-end then the front-end application becomes Database Dependent. Because, of this reason both C & CPP are not suitable as front-end for real-time.

**** Q what is the difference b/w ODBC & JDBC?

Ans:- 1. ODBC is platform dependent but JDBC is platform Independent.

2. ODBC implementation is in 'C' pointers, But JDBC implementation is in Java language.

→ what is JDBC?

1. JDBC is not a seperate programming language.
2. JDBC is not an operating system.
3. JDBC is an in-built technology of Java SE.
4. we can work with JDBC technology by installing JDK S/w.

14/6/11 Definition:-

JDBC = API + ...

JDBC is an API a specification, used for transferring sql commands between Java and Database.

→ JDBC acts as a vehicle to transfer the data "to and fro" between a Java application & a Database.

→ "SUN" Microsystems given JDBC technology for the following two kinds of people.

1. Application writers — JDBC API
2. Driver writers — ...

→ Application writers are responsible for developing applications (Programs). For this kind of people, "SUN" microsystems has given JDBC API

→ Driver writers are responsible to develop JDBC drivers. For this kind of people, 'SUN' microSystems has given JDBC specification.

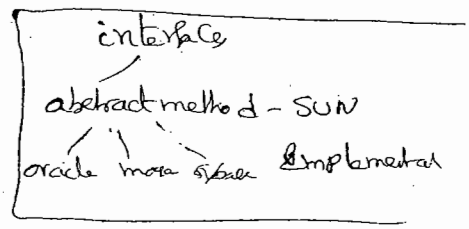
→ JDBC API contains two Packages:

1. java.sql ← classes → its implementation given by 'SUN'
   interface → its implemented by vendor.
2. javax.sql

→ The above two packages contains classes & interfaces for developing JDBC applications.

→ JDBC specification contains Rules & Guidelines & these rules are followed by the Driver Vendors. (oracle, IBM, mysql...)

→ JDBC Driver Vendor provides implementations for the interfaces given in the JDBC API.



→ Driver vendors should follow the specification given by SUN, while creating JDBC Driver.

(vendor) IBM — DB2
Sybase.
↳ It follow rules & guidelines by "SUN"

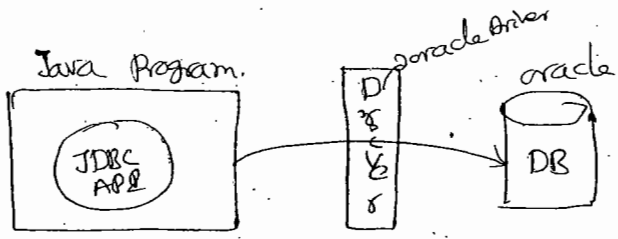
Q: why 'SUN' microSystems has not given implementations for interfaces in JDBC?

Ans:- If 'SUN' microsystem provided implementation also then, the Java program always connects with a single Database. It makes a Java program as Database Dependent. Because, of this reason 'SUN' has not provided implementations.

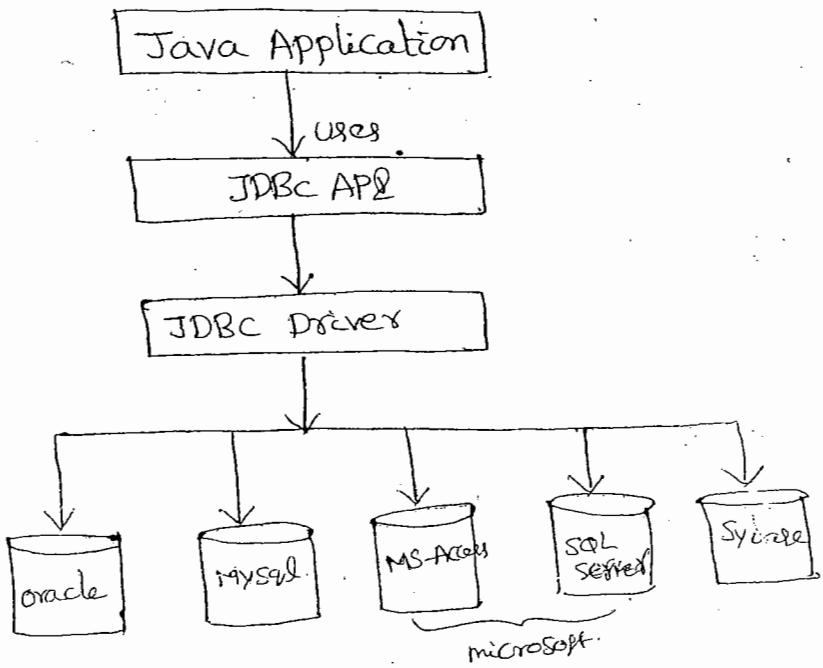
* If vendors are providing implementations then Java

Q:- What is a JDBC

Ans:- A JDBC Driver is a S/w Program, created by Vendors and it acts as a mediator for connecting Java and Database.



* While creating JDBC Driver Programs, vendors can use either 100% Java Coding (or) partially Java and partially some other native language.
* We have both partial JDBC drivers and pure JDBC Drivers
* A JDBC Driver is mandatory, while connecting from Java to a Database.



Note:- whenever a Java Programmer wants to connect with a database then, Java Programmer need the following two components. 1. JDBC API

2. JDBC Driver

Types of JDBC Drivers:-

There are 4 types of Drivers.

1. JDBC-ODBC Bridge Driver. } Partial Java Drivers
2. Native-API Partly Java Driver }
3. Net Protocol Pure Java Driver } Pure Java Drivers.
4. Native Protocol Pure Java Driver }

Common points:-

1. Type-I and Type-II are Partial Java Drivers. Type-III & Type-IV are pure Java Drivers.
2. Type-I is the slowest Driver, because Java has no pointers & ODBC has C+ Pointers and Type-IV is the Fastest Driver among the 4 types of drivers.
3. Type-I, II & IV follows 2-tier architecture model. But, Type-III follows 3-tier model. (Java | middle server | DB)

Classes and Interfaces java.sql package:-

Interfaces:

1. Driver
2. Connection
3. Statement
4. PreparedStatement
5. CallableStatement
6. ResultSet
7. ResultSetMetaData
8. DatabaseMetaData
9. ParameterMetaData.

Classes:-

1. DriverManager
2. DriverPropertyInfo

java.util.* & java.sql.*

4. Time
5. TimeStamp
6. Types

package contains classes, interfaces, Exceptions & Sub-Packages

Exceptions :-

1. SQLException
2. SQLWarning
3. DataTruncation
4. BatchUpdateException

15/6/11

→ TYPE-1 :-

SUN → interfaces → implementation → only for testing → ex., Java is connected with db (or) not.
not for real-time

Vendors → implementation for real-time

TYPE-1 : JDBC-ODBC BRidge Driver :-

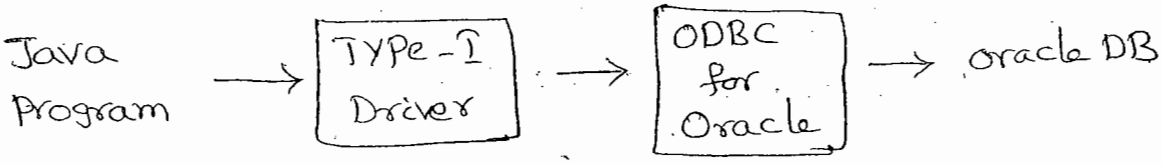
1. This type of driver is given by "SUN" micro Systems.
2. "SUN" given this driver for only "Testing" whether a Java application is connecting with a database (or) not.
3. This Type-1 driver is not suitable for production environment (Real-time).
4. The functionality of this driver is, it converts JDBC calls (functions) into ODBC calls (functions), while connecting with a DataBase.

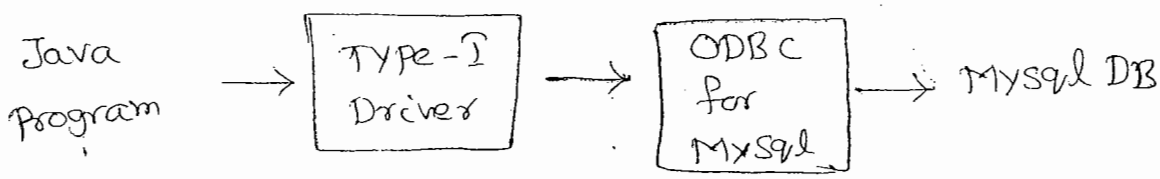
5. JDBC calls are implemented in Java Programming and ODBC calls are implemented in C language with Pointers. So, the conversion takes some amount of time. So, this TYPE-1 driver is slow.

6. TYPE-1 driver internally uses ODBC driver for Connecting with a database.

Java Program → TYPE-I Driver → ODBC Driver → Database.

* In the middle TYPE-I Driver duties, Converting non-pointers code into pointers code and in reverse pointers code into non-pointers.
* ODBC driver is a database dependent driver. It means we have seperate ODBC driver for each database.
* The relationship between ODBC driver and a database is 1:1 ie., one ODBC driver has one db.
* TYPE-I driver is common for all databases. It means the relationship b/w TYPE-I driver and ODBC driver is 1:n





Advantages :- (Pros)

1. This TYPE-I driver is database independent.
2. This TYPE-I driver automatically Comes along with JDK s/w.

Thes drivers.

Disadvantages:-(cons)

1. This TYPE-I driver is the slowest driver among all JDBC drivers.
2. This TYPE-I driver internally uses ODBC, so this driver is platform-dependent.
3. This TYPE-I driver is not suitable for Real-time applications.

Note:-

* The TYPE-I driver is suitable only for local System applications.
* 'SUN' microsystems has given the following driver class.

Sun.jdbc.odbc.JdbcOdbcDriver → Fully Qualified class name (FQCN)

Packagename classname

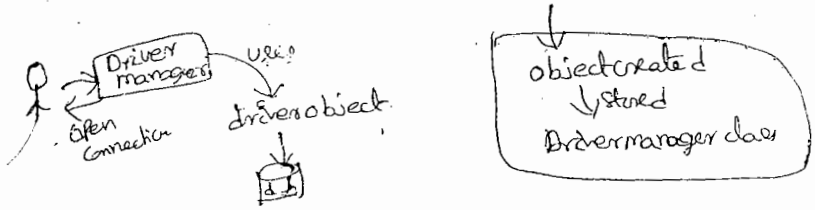
Steps to write a Jdbc program:-

Step-1:- Import Jdbc packages (API)

Syn: import java.sql.*;
import javax.sql.*;

step-2:- Load and Register a Jdbc driver

Syn:- 1. Class.forName("Sun.jdbc.odbc.JdbcOdbcDriver"); (FQCN)



2. DriverManager.registerDriver(object of driver class)

For ex:-

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

(or)

```
Sun.jdbc.odbc.JdbcOdbcDriver Od = new sun.jdbc.odbc.JdbcOdb
                                      Driver();

DriverManager.regieterDriver(Od);
```

Step-3:-

Obtain a connection with the database.

Syn:- `Connection Con = DriverManager.getConnection(url, username, Password);`

Step4:-

→ Create a statement object.

→ This Statement object has the methods to transfer SQL Commands from Java to database.

Syn:- `Statement stmt = con.createStatement();`

Step-5:-

Execute the SQL commands on Database

Syn:- `output = stmt.xxxx("sql command");`

Step-6:-

Print the output.

Syn:- `System.out.println(output);`

Step-7:-

close the connection with Database.

Syn:- `Con.close();`

1. executeUpdate ("Sql Command");
2. executeQuery ("Sql Command");
3. execute ("Sql command");

1. executeUpdate():-

* This method is used for executing non-selecting options on db.
* Non-Select options means both DDL & DML options on db.
* This method returns an integer values that integer value can be either -1 (or) a positive number (>=0)
* This method returns -1, if the command executed on db is a DDL command. If the command is DML then this method returns a positive integer.

Syntax:-

```
int x = stmt.executeUpdate("Sql command");
```

Ex:-

1. `int x = stmt.executeUpdate("create table sathya( sid number(5), sname varchar2(10))");`

→ Here, if the table is successfully created in database then the variable 'x' contain -1 ie., x = -1 (DDL cmd).

2. `int x = stmt.executeUpdate("insert into sathya values(111,'abcd')");`

Here if the row is successfully inserted into the table then the 'x' contains 1 ie., x = 1 (DML cmd)

2. executeQuery():-

* This method is only used for executing Select option on database.
* In Jdbc, the selected data or records from the table will be stored in a ResultSetObject. It means the return type of this method is ResultSet.

Syntax:

```
ResultSet rs = stmt.executeQuery("sql command");
```

Ex:-

```
ResultSet rs = stmt.executeQuery("select *from emp");
```

3. execute():-

* This method is used for both executing select & non-select options on database.
* This method returns boolean. If select option is executed then this method returns "True". & If non-select option command is executed then this method return "false"

Syntax:-

```
boolean b = stmt.execute("sql command");
```

Ex:-

1. boolean b = stmt.execute("alter table ----");
   Here b contains false.
2. boolean b = stmt.execute("delete from ----");
   Here b contains false.
3. boolean b = stmt.execute("select *from customer");
   Here b contains True.

→ The following Jdbc program is for creating a table in oracle database, using type1 driver.

```
//step-1
import java.sql.*;
class JdbcExample1
{
    p.s.v.m(String[] args) throws Exception
    {
        //step-2
        class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
        //step-3
        Con = DriverManager.getConnection(
```

```
//step-4
    Statement stmt = con.createStatement();
//step-5
    int x = stmt.executeUpdate("create table test123
            (sno number(3), sname varchar2(10), marks number(3))");
                                                    //it returns -1
//step-6.
    if(x==-1)
        S.O.P("table created");
//step-7
        stmt.close();
        con.close();
        S.O.P("Connection closed");
    }
}
```

→ Jdbc handle with checked Exceptions.

O/P :-

```
D:\> javac JdbcExample1.java
D:\> java JdbcExample1 ↵
     Connection created.
     Table created
     Connection closed.
SQL > desc test123;
     Name
     SNO
     Sname
     Marks.
D:\> java JdbcExample1 ↵
     Name already used by an object, Exception
     occured
```

→ If we want to delete the table go to SQL

SQL > drop table test123 ↵

Table is dropped.

7/6/4 DSN (Data Source Name) :-

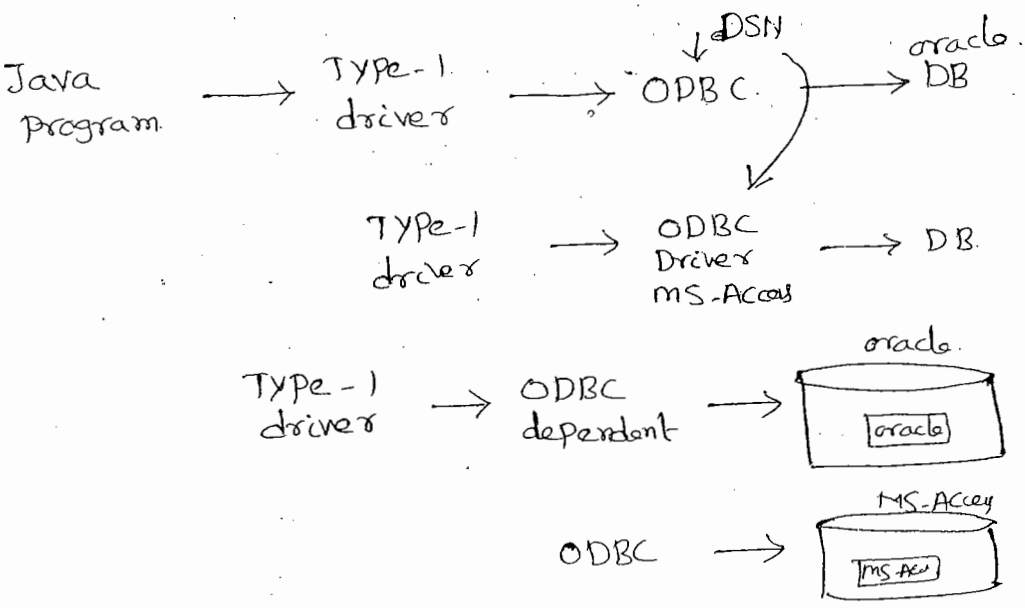
Q what is DSN ?

Ans: A DSN is a datastructure, it contains information of a db's and it is used by ODBC driver, for opening connection with the database.

→ with out DSN, an ODBC driver cannot open connection with a database.

→ DSN's are of 3 types:

1. System DSN
2. User DSN
3. File DSN.



* A System DSN is a global DSN, it is sharable for all users of the system.
* An User DSN is a local DSN & it is only for one user of the system.
* when we create a System DSN (or) User DSN

this type of DSN then the DSN information will be stored in file & we can transfer this file from one system to another system in a n/w.

* In file DSN, the DSN will be stored in a file
  * its extension is .DSN.

Steps to create DSN:-

Start → settings →
Control Panel
↓
Administrative Tools
↓
Data Sources (ODBC)
↓
Add Button
↓
Select MicroSoft ODBC for oracle
↓
Finish
↓
Select Data Source Name: [orads n]
↓
any db we can take.
User name: [SCOTT]
↓
OK.

Note:-

1. while creating DSN, if any error like oracle client & n/w components not found is occured then we need to verify whether Path variable contains oracle path or not, in the environment variables.
2. Eventhough, if oracle path exists and if get the

error then we need to copy or move the oracle Path to the 1st place in that value.

→ The following Jdbc program is for inserting dynamic values into a table using TYPE-1 driver.

```
//step1
import java.sql.*;
import java.util.*;
class JdbcExample2
{
    p.s.v.m (String[] args) throws Exception
    {
      //step-2
       class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
       //step-3
       Connection con=DriverManager.getConnection(
                          "sun.jdbc:odbc:oradsn","scott","tiger");
        S.O.P ("connection created");
       //step-4
       Statement stmt = con.createStatement();
       Scanner s=new Scanner(System.in);
        S.O.P("enter student no");
         int n=s.nextInt();
        S.O.P ("enter student name");
         String str=s.next();
        S.O.P ("Enter marks");
         int m=s.nextInt();
       //step-5
        int x=stmt.executeUpdate("insert into test123
                          values("+n+","+str+","+m+")";
       //step-6
        S.O.P(x+"row inserted");
```

```
        stmt.close();
        con.close();
    }
}
```

O/P:-

```
javac jdbcExample2.java ↵
java jdbcExample2 ↵
Connection created
Enter student no:
303
Enter student name
ABC
Enter student marks
200
1 row is inserted.
```

In SQL PLUS

```
select *from test123;
```

Sno	Sname	Smarks
303	ABC	200

Selecting Data from Table:-

* we need to store the data selected from a table into ResultSet Object.
* whenever data is stored in a ResultSet Object then the ResultSet cursor points at before 1st record by default.

* To move the cursor to the next records one by one then we need to call next() method.

* next() method returns either true or false. It returns true, if next record is available and otherwise it returns false.

* If you want to read the data from each row after ResultSet then we need to call getX() methods.

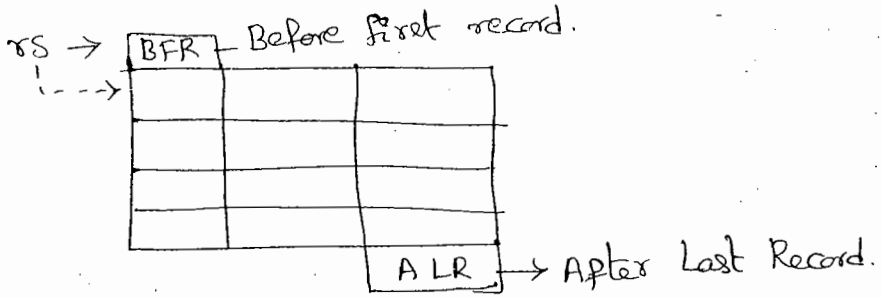
For ex:-

getInt() for int value
getString() for string value e.t.c

* If we do not know the value type then we use Common method as getString().

Ex:-

```
ResultSet rs = stmt.executeQuery("select * from test123");
```



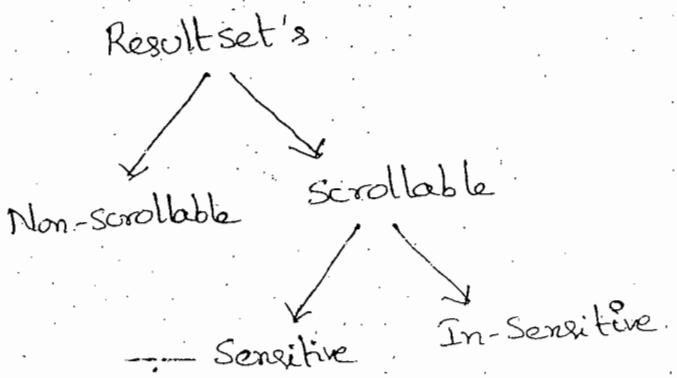

```
while(rs.next())
{
    S.O.P(rs.getInt(1) + "  " + rs.getString(2) + "  "
                + rs.getInt(3));
}
```

18/6/11

* In Jdbc, ResultSets are of two types:

1. Non-Scrollable ResultSet (default)
2. Scrollable ResultSet

* By default, every Resultset is a Non-scrollable. But, it is possible to convert from Non-scrollable into Scrollable type.



→ The following Jdbc Program is for selecting the data from table using Type-I driver.

```
// JdbcExample3.java
import java.sql.*;
class JdbcExample3
{
    P.S.V.m(String[] args) throws Exception
    {
        sun.jdbc.odbc.JdbcOdbcDriver od = new
                        sun.jdbc.odbc.JdbcOdbcDriver();
        DriverManager.registerDriver(od);
        Connection con = DriverManager.getConnection("
                        jdbc:odbc:oradsn", "scott", "tiger")
        Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("select *from test123");
while(rs.next())
{
    System.out.println(rs.getInt(1) +" "+rs.getString(2)
                       +" " +rs.getInt(3));
    System.out.println("------");
}
rs.close();
stmt.close();
con.close();
} //main
} //class
```

O/P:-

```
Javac JdbcExample3.java
Java JdbcExample3
101 abc 345
202 Sathya 780
303 tech 560
```

Explanation of execute() method:-

* execute() method is used for both select & non-select operations on database.
* execute() method is not actually executing the command. Instead, it verifies the command.
* If the command is "Select" then returns True and If the command is "non-select" then returns False.
* execute() method functionality is called "Lazy Execution"

* If we use execute() method ...

another method also for executing the sql command on db.

* For, non-select operations, the method is

getUpdateCount()

* For, select operations, the method is getResultSet().

For example:-

Ex:-1 :-

```
boolean b = stmt.execute("select *from emp");
if(b==true)
{
   ResultSet rs = stmt.getResultSet();
   while(rs.next())
   {
      - - -
      - - -
   }
}
```

Ex:-2 :-

```
boolean b = stmt.execute("delete from emp where eno=101");
                          (or)
                          create
                          (or)
                          insert.
if(b==false)
{
   int x = stmt.getUpdateCount();
   if(x==-1)
      S.O.P("DDL cmd executed");
   else
      S.O.P("DML cmd executed");
}
```

DDL :- -1
DML :- Positive number.

→ The following Jdbc Program is for executing the given command on database.

```
//JdbcExample4.java
import java.sql.*;
import java.util.*;
class JdbcExample4
{
    P.S.V.m (String[] args) throws Exception
    {
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
        Connection con = DriverManager.Connection("jdbc:odbc
                                    oradsn", "scott", "tiger");
        Statement stmt = con.createStatement();
        Scanner s = new Scanner(System.in);
        S.O.P("Enter a command of your choice");
        String str = s.nextLine();
                    ↳ It is for reading the command in next line.
        boolean b = stmt.execute(str);
        if(b==true)
        {
            ResultSet rs = stmt.getResultSet();
            while(rs.next())
            {
                S.O.P(rs.getInt(1)+" "+rs.getString(2)
                                    " "+rs.getInt(3));

                S.O.P("-------");
            }//while
            rs.close();
        }//if
```

```
else
{
    boolean
    int x = stmt.getUpdateCount();
    if(x==-1)
        S.O.P("DDL cmd executed");
    else
        S.O.P("DML cmd executed");
  }//else
 stmt.close();
 con.close();
}//main
}//class.
```

O/P:-

javaC JdbcExample4.java

javac JdbcExample4 ↵

Enter a command of your choice

insert into test123 values (404, 'tech', 670)

DML cmd executed.

→ for verifying go to SQLplus

Select *from test123.

no	name	marks
404	tech	670

D:\> java JdbcExample4

enter a command of your choice.

Select *from test123

101 abc 345

202 sathya 780

303 tech. 560

404 tech 670.

→ Enter a Command of your choice.

create table Java (SID Number(3), Sname varchar2(10));

DDL cmd executed

Go to SQL-PLUS.

desc java ;

SID

SName

Type-2 :- Native-API Partly java driver :-

This driver translates Jdbc calls into direct database calls for connecting with the db.

* This driver is Database Dependent. It means their ratio b/w Driver and db is 1:1
* Type-2 driver doesn't use any other driver in the middle. It directly connects with the db.

Java program → Jdbc calls → Type-2 Driver → Native calls → Database

Java program → Type-2 Driver for Oracle → Oracle

Java program → Type-2 Driver for MYSQL → MySQL

Advantage :-

1. This driver is faster than Type-1 driver.

Disadvantages :-

1. This Type-2 driver is Database Dependent & also platform dependent

3. This driver is slow compare with TYPE-3 & TYPE-4 drivers.
4. This TYPE-2 driver is not a portable driver.

20/6/11

Type-2 for oracle:-

Oracle database vendors has provided two Job Jdbc drivers for connecting from java to oracle db.

1. oracle oci driver
2. oracle thin driver

* oci driver belongs to type-2 category & thin driver belongs to type-4 driver.
* oracle oci driver converts jdbc calls into oci calls (oracle call Interface), for establishing the communication b/w java & db.

Java program —Idbc calls→ oracle oci driver —oci calls→ oracle DB.

→ The following information is required to connect from java to oracle db using type-2 driver.

Driver classname : oracle.jdbc.OracleDriver

url : jdbc:oracle:oci:@<SID>
↓
Service ID.

Username : scott

Password : tiger

→ The follo

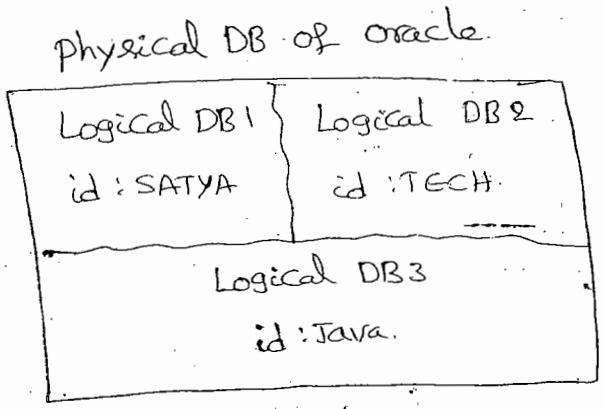
→ whenever oracle database installed then along with physical db, one logical db also created.

→ when we are working with oracle, it means that we are working with logical db but not physical db directly

→ for each logical db, there is a unique ID to identify it.

→ This ID is called Service ID (SID).

→ By default when database is installed one logical db is created & later any no. of logical database can be created.



Different ways of finding SID:

1. At SQL plus type the following command to know SID.

```
SQL > select *from global_name;

GLOBAL_NAME
-----------
SATYA.OS.ORACLE.com.
```

2. start → settings → control-panel
↓
Administrative tools
↓
Services
↓
Select oracle service from the lists find sid (OracleServiceSATYA)
SID.

\admin folder & find servicename in the file.

→ Type-2 driver is a platform dependent because, it uses different databases.

Java program → Type-2 drive for Sybase → CLI calls → Sybase

→ The following example is for deleting an employee from emp table using type-2 driver.

```
JdbcExample5.java
import java.sql.*;
class JdbcExample5
{
    PSVm(String[] args) throws Exception
    {
        Class.forName("oracle.jdbc.oracleDriver");
        Connection con = DriverManager.getConnection
                ("jdbc:oracle:oci:@SATYA",
                 "scott","tiger");
        Statement stmt = con.createStatement();
        int x = stmt.executeUpdate("delete from
                emp where empno=7876");
        S.O.P(x + "row deleted");
        stmt.close();
        con.close();
    }
}
```

O/P:- C:\> javac JdbcExample5.java ↵

java JdbcExample5 ↵

Exception in thread main java.lang.classNotFoundException

→ If we run the above java Program then we will get classNotFoundException. Because we are using oracleDriver class, it is not given java API. So, JVM (Sun) unable to recognize the class. Because of this reason an exception is thrown.

→ In order to inform JVM about a third party class (Not For Sun). we need to set classpath.

→ we need to set ojdbc14.jar given by Oracle into classpath. So that JVM goes to that jar file & takes its bytecode (class code) & loads it into JVM.

→ C:\> set classpath = D:\oracle\ora92\jdbc\lib\ojdbc.jar; %classpath%

↓

do not modify previous value of the classpath.

D:\> java JdbcExample5

1 row deleted

2/1/6/11

Q When path & classpath settings are required?

Ans:- while, working with any .exe files (or) a batch file (or) command file, if an OS is not recognizing then we will get "not recognized" error. To solve this error, we use path setting.

Ex:- javac test.java

∴ javac is not recognized

while compiling a java program, if we get "package does not exist" error (or) while executing a Java program, if we get no class definition found error (or) if we get classNotFoundException then we need to set classpath.

is related to JVM.

Q when ClassNotFoundException is raised by the JVM?

Ans:- Two reasons.

1. Loading a class given by JavaAPI, but the class is not included in that package then we will get ClassNotFoundException.

   For example,

   Class.forName("java.applet.String");

2. In a Java program, if we use a class which is not a part of JavaAPI then ClassNotFoundException is raised.

   Ex:- import oracle.jdbc.*;

```
class test
{
    p.s.v.m(string[] args)
    {
    }
}
```

javac test.java ↵

1: Package oracle.jdbc does not exist.

c:\> set classpath = d:\- - - - - - -

Note:-

c:\> Path = d:\Programfile\java\jdk1.5.1\bin;%Path%.

↵

It is used for not overwritten by previous Path.

→ c:\> set classpath = %classpath%;d:\.

c:\> set Path = %Path%;d:\.

→ C:\> java Test

Exception in thread "main": java.lang.NoClassDefFoundError

This problem is find when we are setting the classpath in the current direction

→ C:\> set classpath = %classpath%; .
↳ current location setting class path

In this we set classpath in the current directory.

Note:- For jdk tools, only, we set the path. For other exceptions, we set the classpath.

Prepared_Statement Interface:-

1. Prepared_statement is an interface given in java.sql package and it is extended from Statement.

2. In JDBC, there are 3 types of statements:

1. Statement
2. PreparedStatement
3. CallableStatement

Statement
↑
PreparedStatement
↑
CallableStatement

Q Differences b/w statement & PreparedStatement:

Ans:-

Statement	PreparedStatement
1. Statement interface doesn't support binary datatypes of the database. so, we	1. PreparedStatement supports binary types of the database. So, we can insert a

2. In Statement interface, if we want to set dynamic values into SQL query, then very Complex command is required.

3. Statement interface compiles the SQL Command, for each time before executing internally. Even though, same command is executing repeatedly, but stmt interface compiles the command internally each time. It decreases (or) reduces the performance at other side.

2. In PreparedStatement, a simple query is required for setting dynamic values into the query. because, in PreparedStatement, we can use "?" Symbol in the SQL Command.

3. In PreparedStatement, it Compiles the sql command for only once. Later, it can be executed for any no of times, by without recompiling once again. It increases the performance. It means prepared Statement is faster than Statement interface.

PreparedStatement:-

Syntax:-

```
PreparedStatement pstmt = con.prepareStatement("SQL Command");
```

for ex:-

```
PreparedStatement pstmt = con.prepareStatement("insert into test123
                                values(?,?,?));
```

'?' - placeResolution Operator (or) Replacing operator

It is also called as "pre-compiling"

Ex:-

```
PreparedStatement pstmt = con.prepareStatement("insert into
                          test123 values(?,??));
for(i=1;i<=10;i++)
{
  pstmt.setInt(1,501);
  pstmt.setString(2,str);
```

```
pstmt.setInt(3, 5000);
int i1 = pstmt.executeUpdate();
```

2/6/11

→ while creating PreparedStatement object we are passing the SQL query with a special symbol called '?'. we call this '?' symbol as place resolution operator (or) replacement operator.

* At run time, if we want to set values into '?' place, we need to setXxx() methods given by PreparedStatement

→ The following Jdbc program is for updating salary of an employee by taking input values at Run-time, Using type-2 driver.

```
// JdbcExample7.java
import java.sql.*;
import java.util.*;
class JdbcExample7
{
    psvm(String[] args) throws Exception
    {
        Class.forName("oracle.jdbc.OracleDriver");
        Connection con = DriverManager.getConnection("jdbc:
                oracle:oci:@satya", "scott", "tiger");
        PreparedStatement pstmt = con.prepareStatement(
                "update emp set sal=? where
                empno=?");
        Scanner s = new Scanner(System.in);
        S.O.P("Enter salary");
        double d = s.nextDouble();
        S.O.P("Enter empno");
        int no = s.nextInt();
```

```
        pstmt.setDouble(1,d);
        pstmt.setInt(2,eno);
    }
    int x = pstmt.executeUpdate();
    S.O.P(" x + "row updated");
        pstmt.close();
        con.close();
    }
}
```

O/P:-

```
D:\> javac JdbcExample7.java ↵
D:\> java JdbcExample7 ↵
    classNotFoundException.
D:\> set classpath = d:\oracle\ora92\jdbc\lib\ojdbc14.jar;
                                        -/.classpath%.

D:\> java JdbcExample7
Enter emp salary
9999
Enter eno
7788
1 row updated
verify SQLplus
select sal from emp where empno = 7788;

    SAL
    ----
    9999
```

→ Inserting a picture:-

1. Inserting a picture means storing binary data of a picture into a column of a table.
2. If we want to store binary data into a column then, that column datatype must be BLOB (Binary large object)

3. Storing an image file, converting a picture into a table is only supported by PreparedStatement, but not possible with Statement.

4. To insert binary data of a picture, into a database column then we need to call a method setBinaryStream() given by PreparedStatement interface.

5. For inserting a picture into a table, we need the following two class objects.

   1. File class
   2. FileInputStream class

6. The above two classes are given in java.io package.

Example:-

```
PreparedStatement pstmt = con.prepareStatement("insert into Companies
                                                values(?, ?)");
pstmt.setString(1, "abc Ltd");
File f = new File("C:/duke.gif"); // To find size of picture
FileInputStream fis = new FileInputStream(f);
int l = (int) f.length(); // 'l' must be in int  That's why we do explict casting
pstmt.setBinaryStream(2, fis, l);
                          ↳ int
int x = pstmt.executeUpdate();
S.O.P(x + "row inserted");
```

→ Image size should be less than ≤ 4kB.

→ The following Jdbc program is for inserting a Company name and its logo into a table called Companies using PreparedStatement.

The following command is required for store creating a table in oracle database

SQL > Create table companies (compname varchar2(10),
logo BLOB);

Table created.

SQL >

```
//PictureInsert.java
import java.io.*;
import java.sql.*;
class PictureInsert
{
   psvm(String[] args) throws Exception
   {
      Class.forName("oracle.jdbc.driver.OracleDriver");

      Connection con = DriverManager.getConnection(
            "jdbc:oracle:oci:@SATYA", "scott", "tiger");

      PreparedStatement pst = con.prepareStatement(
            "insert into companies values(?,?)");

      pst.setString(1, "abc ltd.");
      //creating an object for file
      File f = new File("c:/duke.gif");
      // finding size of the file.
      long l = f.length();
       int s = (int) l; //explicit type casts.
      FileInputStream fis = new FileInputStream(f);
                        //creating FileIS object for reading the file
      //setting binary data into 2nd question mark.
       pst.setBinaryStream(2, fis, s);

       int i = pst.executeUpdate();
       S.o.p(i + " row inserted");
       fis.close();
```

```
        pst.close();
        con.close();
    }
}
```

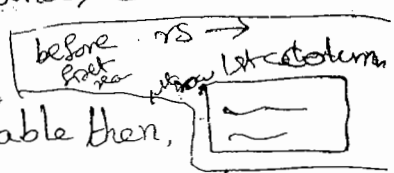
O/P:-

```
javac pictureInsert.java
java pictureInsert
1 row inserted.
SQL> Select * from companies;
  column or attribute type cannot be displayed by SQL Plus
SQL> Select compname from companies;
COMPNAME
abc ltd.
```

23/6/11

→ Selecting a Picture:-

1. In order to select a picture from a database table, we need to use PreparedStatement.
2. Selecting a picture means reading binary data from a table & Constructing an image by using that binary data.
3. For Constructing an image, we need FileOutputStream. Constructing an image is nothing but writing binary data into a file.
4. Whenever, binary data is selected from a table then, ResultSet stores the binary data into a Column of ResultSet. Afterthat, if we want to get the binary data from ResultSet then, we need to use getBinaryStream() method.



5. While, creating (or) Constructing a new image file with binary data Then we need to check whether the binary data

→ The following Jdbc program is for selecting a picture from the table using TYPE-2 driver.

```
// PictureSelect.java
import java.io.*;
import java.sql.*;
class PictureSelect
{
    PSVm (String[] args) throws Exception
    {
        class.forName("oracle.jdbc.driver.OracleDriver");
        Connection con = DriverManager.getConnection("jdbc:oracle:
                                    oci:@SATYA", "scott", "tiger");
        PreparedStatement pst = con.prepareStatement("select
                                    logo from companies where compname=?");
        pst.setString(1, "abc ltd");
        ResultSet rs = pst.executeQuery();
        rs.next();
        InputStream is = rs.getBinaryStream(1);
        FileOutputStream fos = new FileOutputStream(
                                    "e:/test/sample.gif");
        int k;
        while ((k = is.read()) = -1)
        {
            fos.write(k);
        }
        System.out.println("Image received");
```

```
        fos.close();
        pst.close();
        con.close();
    }
}
```

o/p :-

javac PictureSelect.java ↵

java PictureSelect

Image received.

for checking image :-

∈ :

+ → Test Folder

+ → Sample.gif

↓

open.

duke.gif, Sample.gif are same.

→

```
p c Test implements Satya
{
}
Satya s = new Test();
```

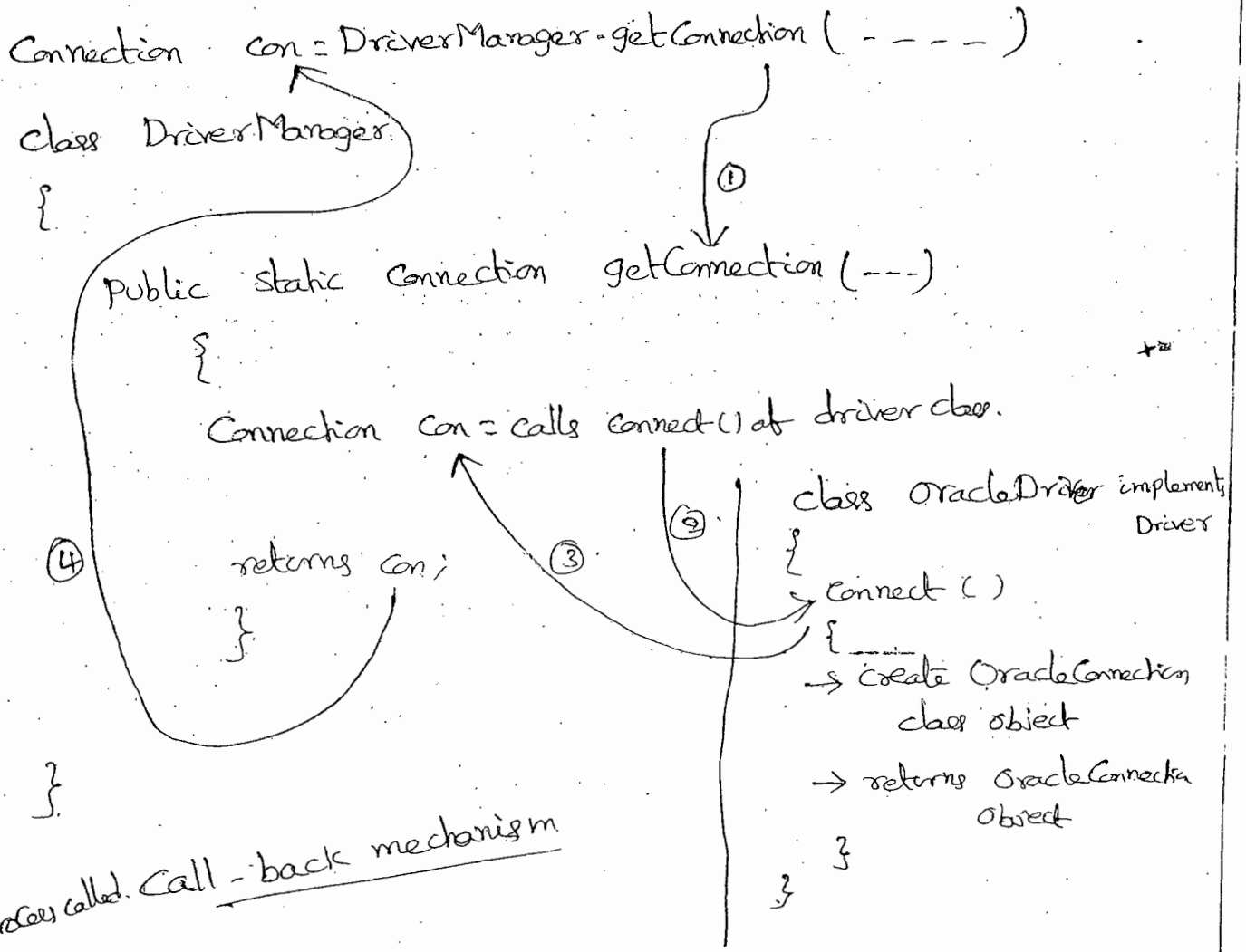
→ classes are created by vendors. Interfaces are created by "Sun".

Q In Connection con = DriverManager.getConnection(); statement, 'con' is a reference (or) an object.

Ans :- * 'con' is not an object. It is a reference.

* Internally, whenever getConnection() method is called, then it internally calls connect() method of Driver. The connect() method returns an object of connection class (OracleConnection class object).

returned object into a reference of Connection interface. Finally, that reference is given to java Program.



This process called Call-back mechanism

Q what is Jdbc Call back mechanism?

Ans:- It means calling a method, creating an object internally for a class and returning an interface reference, back to the Java Program.

Ex:- when we call getConnection() of DriverManager. then, internally OracleConnection object is created (for Oracle) and to the java program Connection reference is given back. It is a "Call-back mechanism."

24/6/14

TYPE:4 Native-protocol pure Java Driver :-

* This driver is a pure Java driver. It means "this driver implementation is completely done using Java Port.
* This TYPE-4 driver doesn't convert Jdbc calls into any other type of calls.
* This type-4 driver uses the following 3 informations, for connecting a Java application with database.
    1. protocol of the database Server
    2. IP address of the database System
    3. Port number of the database Server.
* This TYPE-4 driver doesn't use any mediator driver for connecting from java to a database.

Java Program —JDBC API→ TYPE-4 driver —Native protocol→ Database.

* For example, while connecting from Java to oracle db, OracleDriver uses ttc (two-task-Common) protocol for Connecting with oracle Database.
  Here ttc is a Native Protocol of Oracle database Server.
* For each database, we have a standard (or) Seperate protocol and port number. So, this TYPE-4 drivers are database dependent drivers.
* For example, Oracle database Server runs on a Standard Port no. called 1521. * MySQL Database Server runs on Port no. called 3306

1. Type-4 driver is the fastest driver among all types of Jdbc drivers.
2. Type-4 driver is a pure-Java driver. So, it is platform-independent.
3. Type-4 driver is suitable for internet applications. It means this driver is used in production environment (Real-time).

Disadvantage :-

1. This Type-4 driver is a database dependent driver.

Type-4 driver for Oracle :-

* Oracle database vendor has provided two Jdbc drivers for connecting with oracle.
    1. Oracle oci driver
    2. Oracle thin driver.
* oci driver is a type-2 driver and converts Jdbc calls into native calls.
* thin driver is a type-4 driver and it directly connects with the database for any where in the n/w by using Protocol, Ip address & Port no.
* Between oci driver & thin driver, there is only a change in the url. Remaining Driver class, username & password are one and same.

* Type-2 OCI driver URL is:

```
jdbc:oracle:oci:@satya
                <sid>
```

* Thin driver url is:

```
                jdbc:oracle:thin:@<IP address>:<Port no>:<sid>

oracle:         jdbc:oracle:thin:@ipaddress:1521:satya

MySQL:          jdbc:oracle mysql://localhost:3306/test
```

→ The following Jdbc program is for selecting data from a table using Type-4 oracle driver.

```
//JdbcExample8.java
import java.sql.*;
class JdbcExample8
{
    PSVM(String[] args) throws Exception.
    {
        Driver d = new oracle.jdbc.OracleDriver();
        DriverManager.regieterDriver(d);
        Connection con = DriverManager.getConnection("jdbc:oracle:
                thin:@localhost:1521:satya","scott",
                "tiger");
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("Select * from
                                    dept");

        while(rs.next())
        {
            S.O.P(rs.getString(1)+" "+rs.getString(2)+
                " "+rs.getString(3));

            S.O.P("=----");
```

```
rs.close();
stmt.close();
con.close();
}
}
```

O/P :-

```
javac JdbcExample8.java
java JdbcExample8
10 ACCOUNTING NEWYORK
12 SALES DELHI
```

Callable statement interface :-

1. Callable_Statement is an interface given in java.sql package & it is extended from PreparedStatement interface.
2. In CallableStatement, there is one additional feature when compared with PreparedStatement.
3. The additional feature of CallableStatement is we can call both procedures & functions of a database
4. CallableStatement is for both executing sql commands and also for calling procedures & functions.

25/6/11

Different ways of creating Callable Statement :-

* CallableStatement is used for both executing SQL Commands on database and also for calling procedures and fenctions of the database.

pg
24/6/20

Type-4

Native protocol pure Java driver

→ This driver is a pure Java driver it means this driver implementation is completely done using Java code

→ This type-4 driver doesn't convert JDBC calls into any other type of calls

→ This type-4 driver uses the following three informations for connecting a Java application with a database

(i) protocol of the database server

(ii) Ip address of the database system

(iii) port number of the database server

(iv) This type-4 driver doesn't use any mediator driver for connecting from Java to a database.

Java program → JDBC API → Type-4 driver → Native Protocol → Database

⇒ For example,

* While connecting from Java to Oracle database, Oracle Driver uses ttc protocol (two task common) for connecting with oracle database. Here ttc is a native protocol of Oracle Database server.

* For each database, we have a standard or separate protocol and port number. So this type-4 drivers are database dependent drivers

⇒ For example,

* Oracle database server runs on standard port number called 1521 and my SQL server runs on 3306 port number etc.

Advantages

(i) Type-4 driver is the fastest driver among all types of JDBC drivers.

(ii) Type-4 driver is a pure Java driver. So it is platform independent

(iii) Type-4 driver is suitable for internet applications. It means these driver used in production environment (real time)

Disadvantage

- This driver is a database dependent driver

Type-4 Driver for Oracle

- Oracle database vendors has provided two JDBC drivers for connecting with Oracle
  - (i) Oracle oci Driver
  - (ii) oracle thin Driver

oci driver is a type-2 driver and converts JDBC calls into native calls

- thin driver is a type-4 driver and it directly connects with the database from any where in the nlw by using protocol, IP address and port number
- Between oci driver and thin driver, there is only a change in the url. Remaing driver class, username and password are one and same
- oci driver url is : jdbc:oracle:oci:@ <sid>
- thin driver url is : jdbc:oracle:thin:@ <ip address>:<port num>:<sid>

1) The following JDBC program is for selecting data from a table using type-4 Oracle driver ?

```
// JDBC Examples.java
import java.sql.*;
class JdbcExamples
{
    public static void main (String args[]) throws Exception
    {
        Driver d = new oracle.jdbc.OracleDriver();
        DriverManager.registerDriver(d);
        Connection con = DriverManager.getConnection("jdbc:oracle:
        thin:@localhost:1521:satya", "scott", "tiger");
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("select * from dept");
        while (rs.next())
        {
            sop(rs.getString(1)+" "+rs.getString(2)+" "+rs.getString(3));
        }
        rs.close();
        stmt.close();
        con.close();
    }
}
```

CallableStatement Interface

⇒ CallableStatement is a interface given in java.sql.*; and it is extended from PrepareStatement interface

- In CallableStatement there is one additional feature when compared with PrepareStatement
- The additional feature of CallableStatement is, we can call both procedure and functions of a database
- CallableStatement is for both executing sql commands and also for calling procedures and functions

Different ways of creating callable Statement

→ CallableStatement is used for both executing sql commands on database and also for calling procedure and functions of the database.

- In order to get CallableStatement reference (object), we need to prepareCall() given by connection interface.

we have two syntaxes for creating CallableStatement object one for sql commands and other one is for calling procedures and functions

Syntax 1 :-

```
CallableStatement cstmt = con.prepareCall("sql command");
```

Syntax 2 :-

```
CallableStatement cstmt = con.prepareCall("{call procname
                          (params) / funname (params)}");
```

- In programing languages, the difference between a procedure and function is, procedure does not return any value but a function returns a single value.

* According to database programing, the difference b/w a procedure and a function is a procedure can return zero or more values. But a function exactly returns only one value.

Syntax to create a procedure

```
Create or replace procedure procedurename (param in any)
is
    varible declaration;
Begin
    stmts;
end;
```

parameters for a or a procedure are having three modes

① In
② out ③ Inout

⇒ In parameter can act for accepting the input, out parameters are for producing the output and Inout parameter are both accepting input aswell as producing output.

⇒ By default a parameter is an in mode

⇒ By depending no. of out parameters we can say that a procedure is returning these many out values

⇒ Inside the body of a procedure, we no need to write any return statement. Procedury implicitly returns the values.

Ex: The following procedure in database accepts input as employee number and returns experience of that employee as output

DUAL :- Dynamic Used Allocation List

SQL> (create or replace procedure experience pro (eno in number, e out number)

```
d1 date;
d2 date;
begin
select hiredate into d1 from emp where empno = eno;
select sysdate into d2 from dual;
e := (d2-d1) / 365;
end;
```

[we have to type above code in one file]

Execution

SQL> @ sample1

14 /

procedure created.

Note :- While creating procedures and functions in database, don't provide size for the parameters

7/6/19

Important points for calling a procedure or a function

→ While calling a procedure or a function from a Java application then we need to call setXX() s and we need to pass values for in parameters

→ If any out parameters are there then we need to register the output parameters by calling registerOutParameter() method

→ To call a procedure of function of the database, we need to use execute()

→ After executing the procedure or a function, we need to call getXX() for reading out parameters

The following JDBC program is for calling the above experience procedure of the database.

```
// CallableTest1.java
import java.sql.*;
import java.util.*;
class CallableTest1
{
  public static void main(String[] args) throws Exception
  {
    DriverManager.registerDriver(new oracle.jdbc.OracleDriver());
    Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:sathya","scott","tiger");
    CallableStatement cstmt=con.prepareCall("{call experince(?,?)}");
    Scanner s=new Scanner(System.in);
    System.out.println("enter emp number");
    int n=s.nextInt();
    cstmt.registerOutParameter(2,Types.INTEGER);
    cstmt.execute();
    int k=cstmt.getInt(2);
    sop("output : experince of "+n+" is "+k+" years");
    cstmt.close();
    con.close();
  }
}
```

```
C:\> javac CallableTest1.java
C:\> java CallableTest1
   7788
Output : experince of 7788 is 24 years
```

⇒ The following procedure accepts input as an employee num and returns bonus as output and it is calculated according to the salary of the given employee

[For this procedure code refer page no- 52 Line num 51]

⇒ For JDBC application to call the bonus procedure refer page 52 ~~example~~-10.

→ while compiling this example we will go some more steps. Because, we used and old ~~JDBC~~ Java called DataInputStream for reading input value at runtime.

→ While executing this procedure, we need to set input as a employee num, we get output as bonus. So we need two question marks while calling this procedure

Working with InOut parameter :-

→ If a procedure contains InOut parameter then in a JDBC application, we need the following two steps

① set Input value for that parameter

② Register the same parameter as outparameter by calling registerOutParameter()

The following procedure accepts salary as input and returns salary as Output ?

```
create or replace procedure testpro(sal in out number)
is
begin
    sal := sal+1000;
end;
```

[Samples is a file]

Cmd's

```
SQL> @ samples
```

The following JDBC program is for calling above procedure

```
CallableStatement cstmt = con.prepareCall("{call testpro(2,3)}");
cstmt.setInt(1,8000);
cstmt.registerOutParameter(1, Types.INTEGER);
cstmt.execute();
int x = cstmt.getInt(1);
sop(x);
cstmt.close();
con.close();
```

output

9000

Syntax for creating a function :

```
create or replace function functionname( parameters if any)
return returntype is
variable declaration;
begin
    stmts;
end;
```

The following accepts input as dept number and returns sum of salary of all employee working in that dept

[For this function code refer page num 53 and line 90]

* In a function, all parameters are of type Int
* We can't write In keyword for parameters
* In a function definition, return stmt is mandatory
* While calling a function from a JDBC program, we use an additional ? and it is for storing output value returned by a function

Ex: CallableStatement cstmt = con.prepareCall("{ ? = call fun(?)}");

Date 8/8/11

Transcation Management in JDBC

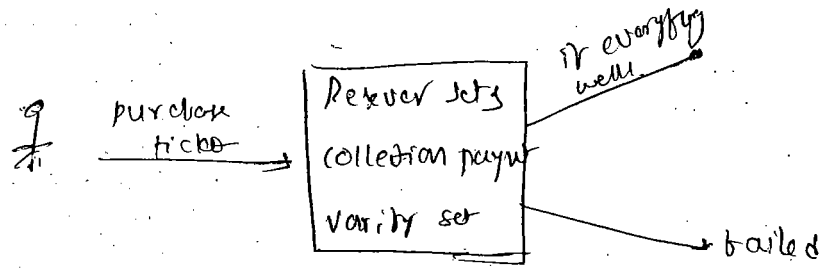
Defi:- A Transcation is a group of operations made as single unit and produces a single outcome. This outcome is either success or failed.

→ If all operations in a transcation are success or correct then that transcation reaching to success point.

→ If any one operation is ~~if~~ failed then all ~~the~~ operations in the group are cancled and that transcation reaching to failed point.

→ For example in on-line ticket reservation if all operations are successfull then the transcation becomes success. If any one operation failed then all operations are cancled and the transcation become failed.

→ Each transcation follows a principle all or nothing



Transcation.

Types of Transcations

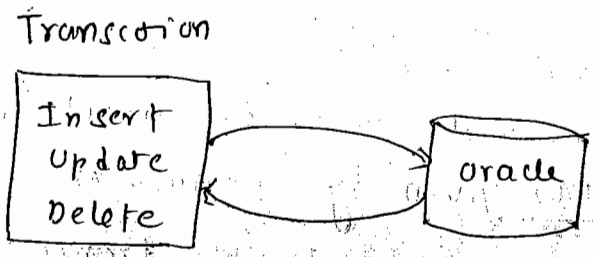
① Local Transcation

② Global or Distributed transcations

- a) Flat transcation
- b) Nested transcation

① Local transcation:- If all operations are executed against one database then we called Local transcation.

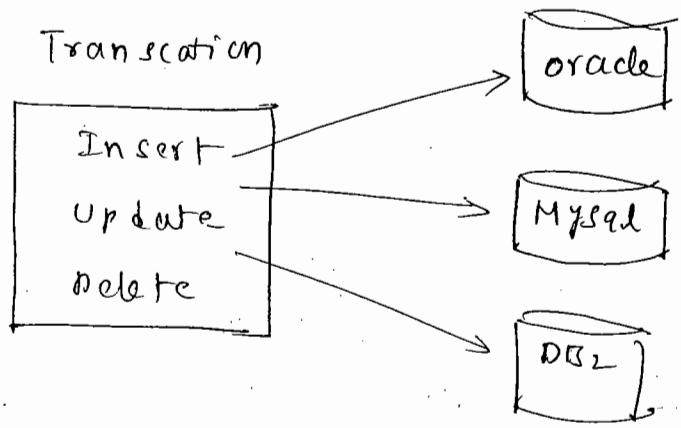
→ JDBC technology onely supports local transcation



Distributed transcation

→ If all operations in a transcation or not executing against a single database that is executing on more then one database the it is called Distributed transcation.

→ EJB technology and spring frame work, both supports distributed transcations.



→ By default, All operations that are executed on a database from a JDBC program are perminently executed, by default

→ The reason for executing operations on database perminently is, because in Java program (JDBC programs), by default auto commit mode is enbled. So the commands are executed perminently on the database.

→ If we want to apply transcation management in a JDBC program then the first step is we need to disable the autocommit mode

→ If we want to disble auto commit mode then we need to call a method setAutoCommit(false). This method is given by Connection interface

→ If any error is occure while executing a transcation on a database then we need to cancel an operations in

by calling rollback() given by Connection interface.

→ If no problem is accured then we need to save all operations in the transcation as perminent on to the database. This can be done by calling commit() given by Connection interface

→ In a JDBC program, we need to put the transcation operation with en try and catch blocks before we start the transcation operations first of all AutoCommit mode must be disable.

Ex:-
```
con.setAutoCommit(false);
try
{
   =
   =
con.commit();
}
catch(Exception e)
{
   con.rollback();
}
```

* According to the above code, if any exception is accured then the control entries into catch block and the transcation is cancled.
* If no exception is accured then the commit stmts in the try block is executed. It means the transcation is success that is committed.

→ The following JDBC program to perform transcation management in JDBC this application has three operations and if any one operation failed transcation fail by if all operations are success then transcation become success.

```
// JdbcTxExample.java

import java.sql.*;
class JdbcTxExample
{
  public static void main(String[] args) throws Exception;
  {
```

```
class.forName ("oracle.jdbc.OracleDriver");
Connection con = DriverManager.getConnection ("jdbc:oracle:
                    thin:@localhost:1521:satya", "scott", "tiger");
Statement stmt = con.createStatement();
con.setAutoCommit(false);
try
{
   int k1 = stmt.executeUpdate ("insert into dept values (
                                  99, 'IT', '400');
   int k2 = stmt.executeUpdate ("delete from sathya where
                                                  sid = 111");
   int k3 = stmt.executeUpdate ("Update emp set sal = 8760 where
    con.commit();                                  empno = 7788");
    sop ("Transcation success");
  }
  catch (Exception e)
   {
     try
       {
          con.rollback();
          sop ("Transcation is cancled");
       }
     catch (Exception e1)
       {
          sop (e1);
       }
     stmt.close();
     con.close();
   }
}
```

16/11

→ The The bollowing example fely transcation manamger methody of JDBC using swings.

Source code refer page no. 55 Ex-19

Imp points

→ In this transcation example, we used two static methods in the class
  (i) main()
  (ii) check()

→ Both are static method and both method are exist in the same class. So we can call one method in another & method directly.

→ In this example, connection is opend in the main() and it closed in the check(). So we created connection variable at outside the main()

→ In Java, static methods allows onely static varibles of the class. So we made our connection varible as static

→ In order to get conbormation dialog box, we used a class called JOptionpane. we imported Javax.swing package.

→ To get conbormation dialog box, we used a static method of the class called showConbormDialog()

→ Our application either commits delete operation or cancley delete operation by depending on the option selected by the user

→ By default a brame is invisible. So, To make it ay visible we called setVisible() and set width & height bor brame we called SetSize()

→ To exit the program execution whenever a frame is closed, we called a method setDefaultCloseOperation()

MetaData in JDBC

⇒ Data about data is called as meta data

→ In JDBC 1.0, we have two metadata interfaces given in java.sql package

i) ResultSetMetaData rsmt

ii) DatabaseMetaData

i) ResultSetMetaData :-

* When we select the data from a table or database, the rows selected from the table are stored in a ResultSet object. But to print that data, the programmer must known about that table structure
* If the table is unknown table then it becomes complex to print the data from a ResultSet object.
* So inorder to overcome above problem, JDBC has provided ResultSetMetaData interface under its reference (object) will store metadata of the data stored in ResultSet. By using this MetaData object a programmer can find or print the data stored in Resultset
* In order to get ResultSetMetaData, we need to call a method getMetaData() given by ResultSet object
* For example.

```
ResultSet rs = stmt.executeQuery("select * from xyz");
ResultSetMetaData rsmd = rs.getMetaData();
```

* According to the above code, the object rs contains data selected from the xyz table and the object rsmd contains data about the data stored in the rs object.

* To get the meta data stored in rsmd object we use the following methods

→ ① getColumnCount() :-

It returns no. of columns in each row of the data stored Resultset object. This method returns an integer value

```
int x = rsmd.getColumnCount();
```

② getColumnName(int columnno)

This method is for getting column name of the given column number. This method returns a String.

Ex

```
String s1 = rsmd.getColumnName(1);
String s2 = rsmd.getColumnName(x);
```

* Here s_1 contains first columnName and s_2 contains last column name.

③ getColumnTypeName(int columnnum)

This method is for getting data type of name of the given column. It is also returns string.

Ex

```
String str1 = rsmd.getColumnTypeName(1);
String str2 = rsmd.getColumnTypeName(x);
```

* Here str_1 contains first column data type name and str_2 contains last column data-type name.

④ getColumnDisplaySize(int column)

It returns the datatype size for the given column. This method integer value.

int k = rsmd.getColumnDisplaySize(3);

⇒ For example of ResultSetMetaData refer pag no. 53 exm - 12

DataBase MetaData interface

This is used to get data about database

⇒ This interface contains meta information like databasename, version, drivername, version etc.

→ In order to get database meta data object, we need to call getMetaData() given by connection interface

Syntax:-

DatabaseMetaData dbmd = con.getMetaData();

→ Inorder to get MetaData information stored in dbmd object, we need to call the following methods

① getDatabaseProductName():-

It returns database name

② getDatabaseProductVersion():- It returns version of database

③ getDriverName():- It returns jdbc driver name

④ getDriverVersion():- It returns version of the driver

⑤ getMaxColumnsInTable():- It return maximum no. of columns allowed while creating a table in the database.

6) getMaxTableNameLength() :- It returns the maximum no. of characters allowed while writing a table name etc.

Prog.
⇒ For this Database Metadata refer pg 53 ex-13.

New features added in JDBC 2.0

(1) Scrollable Resultset is included. By using this Resultset the cursor can be moved in any direction

(2) Batch processing is added. It means the no. of round trips between Java and database are reduced or decreased

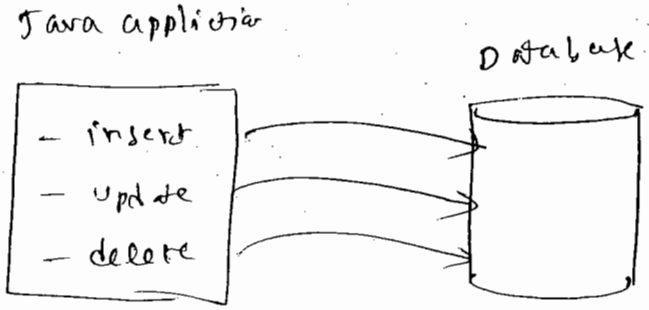
(3) Updatable Resultset. It means if do any modification in the Resultset object then automatically the changes are effected on the database table also

Date 11/7/11

Batch processing

→ While executing multiple operations on database, one by one operation will be transfer on to the database at a time. It will increase no. of round trips between a Java application and a database

→ If no. of round trips are increased between a Java application and a database then automatically the performance of the application will be decreased



→ Inorder to overcome the drawback of JDBC 1.0, in JDBC 2.0 the new feature got called as batch processing

→ In Batch processing, It is possible to execute more than one operation on to the database at a time by making the operation as single batch

→ Advantage of batch processing, is it reduces the no. of round trips between a java application and database so that performance of the application will be increased

→ For batch processing, the following two methods given by Statement interface.

(i) addBatch()

(ii) executeBatch()

→ addBatch() is for construction of a Batch

→ executeBatch() is for execution of a batch on to database

→ addBatch() is called for multiple times, inorder to construct multiple commands as a batch.

→ The entire batch is at a time executed on database, by calling executeBatch()

→ The returntype of ~~execute()~~ executeBatch() is an Integer array. This array contains the integer values returned by the database, for each operation in the batch

Ex:-

```
Statement stmt = con.createStatement();
stmt.addBatch("insert ....");
stmt.addBatch("Update ....");
stmt.addBatch("delete ....");
int k[] = stmt.executeBatch();
```

→ while constructing a batch only DML operations are allowed that is insert, update and delete.

→ select is not a batch processing operation.

→ In batch processing we will never get a ResultSet so. we need to pass type and more parameters for createStatement()

→ While executing Batch on a database, if any exception one operation is failed then the next operations in the are cancled and before operations are executed and an exception is thrown to the Java programmer called as BatchUpdateException

→ If we want to cancle total batch, if any exception is accured then we need to apply transcation management along with batch processing

→ For Batch processing without transcation management, refer S4 ex:- 16

→ The following example is our transcation management along with batch processing

import java.sql.*;

Updatable ResultSet :- In this feature of JDBC 2.0, we may changes on ResultSet and the changes are automatically effected on database side.

→ In this Updatable ResultSet there is no difference between sensitive and insensitive ResultSet

→ This JDBC 2.0 feature is only supported by type-2 JDBC driver. Other drivers are not supported.

Date
11/7/11

Connection pooling

→ Whenever a Java program wants to store the data (or persesiting the data) in a database, first of all Java programe need a database connection. So we have the following two approaches

(i). Create a new connection and execute operation on database and then close it

(ii) use already opened connection and execute operation database

⇒ In the first way, opening a new connection means takes some time. So performance of the Java programe will be decreased

→ When useing already opened connection then no need to wait for until connection is opend. Because connection is already opened.

⇒ In the second approach, there is a problem when multithreading is applied with synchronization. Here only one thread can use that connection and remaing threads hay to wait. So it also effects performance on Java programe. So the solution per above two problems is a connection pool.

→ In connection pooling, a Java programe will ask for a a connection and pool will give a connection to the Java-programe. After the work is completed Java program will give that connection again back to the a pool.

→ A pool containg some set of database connections, so at a time multiple threads can also use connections.

→ Connection pool is avoiding the problem in first approach and problem in second approach so the connection pool is best solution while connecting with database from a Java application.

problems with DriverManager

→ In JDBC programing, whenever we use driverManager class it immediately opens a new connection with a database. After the work is completed then we need to close it.

→ If suppose we are not closeing that connection then that connection will become as a burden to the database server.

→ DriverManager always opens a new connection but it does not reuse the connections

→ If we want to open a connection with a database using driverManager then we have to load the driver, we need to pass the url, username & password so it increases burden on the client programmer

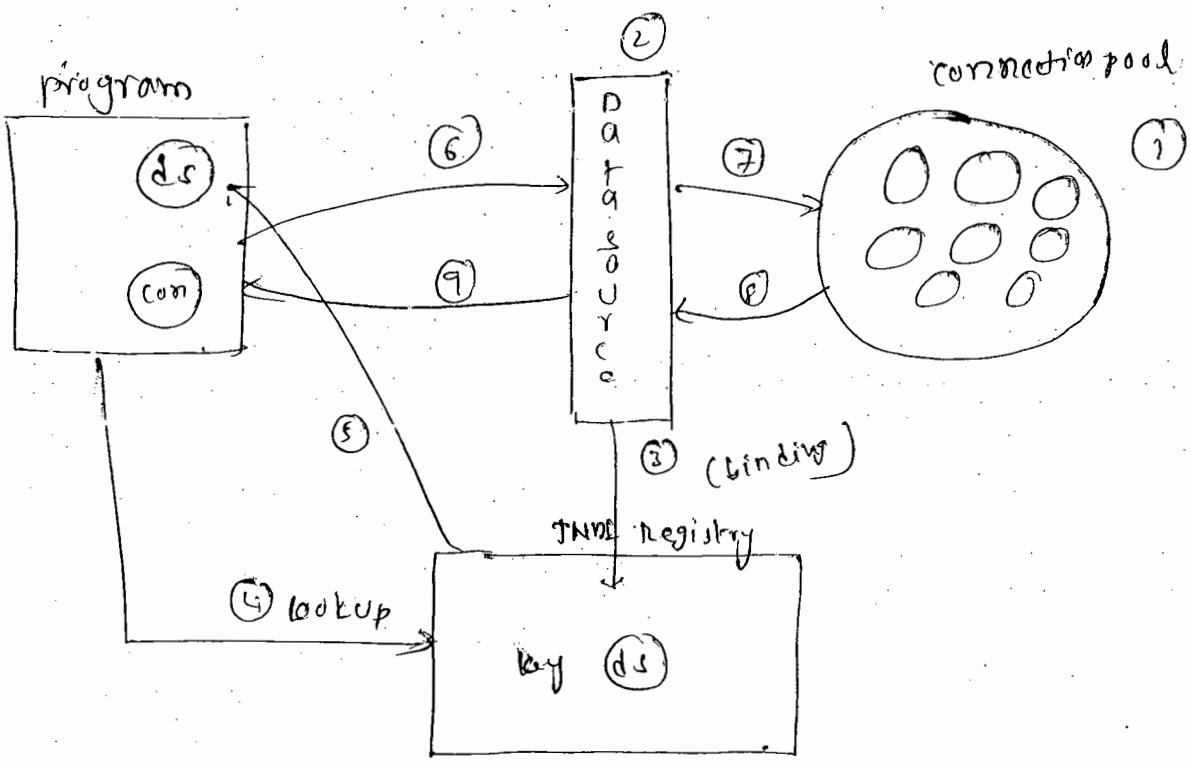
sun Micro systems given a solution for a driverManager called DataSource Interface

→ At present if a Java programmer wants to obtain a database connection, JDBC API has given two ways

(i) use DriverManager class given in java.sql package

(ii) use DataSource Interface given in javax.sql package

→ It



connection pooling Diagram.

→ In real time applications, server administretor and Java programmer both are dibberent people so in real time there is a less burdon on Java programmers

→ All real time applications of Java uses only connection pooling because in real time at a time multiple clients will send request to a web site or project

With respect to the above diagram

i) The administretor creates a connection pool with a database

ii) Administretor creates DataSource object. This DataSource object is mediator between Java applications and connection pool

iii) The DataSource object is binded (stored) into JNDI registry with some key. we call that key as a JNDI name

iv) A Java application uses JNDI API and perbormence lookup operation on the registry

v) Java application will get DataSource object back brom the registry

vi) Java application now communicates with DataSource object

vii & viii) DataSource object takes a connection brom pool and creates a logical connection (proxy connection internally).

ix) Finally DataSource object provides the logical connection backe to the Java application

Note:-

1. Whenever a Java application closes a logical connection then DataSource object will send the orginal connection back to the pool

2. One connection pool contains one database & releted connections only. It is not possible to store multiple databases connection into a single connection pool.

3- An Administrator can create any no. of connection pools depends on the requirement

4- If multiple database connections are required then multiple connection pools are required in the server

Oracle ConnectionPool DataSource class

- OCPDS is the class given in oracle.jdbc.pool package
- oracleCPDS class is the combination of both connection pool and DataSource implementation. It means whenever an object of this class is created then internally a connection pool and DataSource both are created.
- This class is given by oracle for getting connection pool behaviour for a standard alone application
- By using this class we can get a connection from pool and we can create a logical connection required
- In connection pooling, for one physical connection at a time only one logical connection can be created. It means, after closing the first logical connection, it is possible to open a second logical connection.

Ex // example for creating connection pool with oracle in stand-alone application

```
import java.sql.*;
import javax.sql.*;
import oracle.jdbc.pool.*;
class CPExample
{
    public static void main(String []args) throws Exception
    {
        OracleConnectionPoolDataSource ocp = new OracleConnection
                                             PoolDataSource();
```

```
ocp.setURL("jdbc:oracle:thin:@localhost:1521:SATYA");
ocp.setUser("scott");
ocp.setPassword("tiger");
PooledConnection pc=ocp.getPooledConnection();
Connection lcon1 = pc.getConnection();
Statement st1 = lcon1.createStatement();
st1.execute("create table sathya123(sno number(3), sname
                                    varchar2(20))");
sop("table created");
st1.close();
lcon1.close();
sop("first logical connection closed");
Connection lcon2 = pc.getConnection();
Statement st2 = lcon2.createStatement();
int k = st2.executeUpdate("insert into sathya123 values
                                    (111,'sathya')");
sop(k+" row inserted");
st2.close();
lcon2.close();
sop("second logical connection closed");
}
}
```

→ To run above example we need set OTDBC14.jar in the class path

Connecting with MS-Access

→ To connect with MS-Access from a Java application there is only one driver supported called type-1

→ MS-Access is a physical database we can create a logical database in that

→ while creating DSN, we should select the logical database name

MS-Access (physical DB)
↳ Test.mdb (logical DB)
↳ Student, product } Tables

→ while connecting with MS-Access no need of exchanging username password because authentication is not required

1. Open MS-Access and click on file → new → select blank database → Enter file name (Test), save it our own drive → create
2. Before creating the DSN, close the database test
3. Start → settings → control panel → Administrator tools → Datasource → Add Button → select Microsoft Access driver → Finish → Enter Data source name (Accessdsn) → click on select button and select the drive and database → OK → OK → OK

```
// AccessTest.java
import java.sql.*;
class AccessTest
{
    public static void main(String[] args) throws Exception
    {
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
        DriverManager conn DriverManager.getConnection("jdbc:odbc:
                                                        accdsn");
        sop("connection established");
```

```
Statement stmt = con.createStatement();
int ic = stmt.executeUpdate("create table Test (sno number,
                                sname text, marks number)");
if (ic==0)
  sop("table created");
  int x= stmt.executeUpdate("insert into Test values (101,'aaa',500)");
  sop(x+" row inserted");
  stmt.close();
  con.close();
 }
}
```

Selecting Data from a CSV file

(comma separated value)

We can use CSV file as a database and we can select the data from it using JDBC API.

→ we need to use type-1 driver for selecting the data from a CSV file

→ while creating DSN, we need to select microsoft text driver

→ A CSV file containing data or rows and each value in the row is separated with a comma (,)

SNO,	sname,	marks
101,	aaa,	300
202,	bbb,	400
303,	ccc,	500.

```
// CSVTest.java
import java.sql.*;
class CsvTest
{
  public static void main(String[]args) throws Exception
  {
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    Connection con = DriverManager.getConnection("jdbc:odbc:txtdsn");
    Statement stmt = con.createStatement();
```

```
ResultSet rs= stmt.executeQuery ("select * from test.csv4");
while (rs.next())
{
    sop(rs.getString(1)+" "+rs.getString(2)+" "+rs.getString(3));
}
rs.close();
stmt.close();
con.close();
}
}
```

Connecting with MS-Excel

1525 - oracle

MS-Excel (physical database)
↳ Book1.xls (logical database)
↳ sheet1, sheet2, sheet3 } tables

start → settings → control panel → Administrative tools → Datasources → Add button → Microsoft xl driver → finish → enter DSN (xldsn) → click on select work book → select the drive and work book name → de select Read only check box → ok → ok → ok.

Rs rs= stmt.executeQuery ("select * from [sheet1]");

This is difference b/w Excel and Access

Connecting with Mysql

Mysql database vendor has provided a driver belongs to JDBC of type-4 category called Mysql connector/J (Java)

To get a connection with Mysql database, the following connection property are required.

driver: com.mysql.jdbc.Driver
(package name, class name)

url: jdbc:mysql://localhost:3306/test

```
// programme to connect with mysql
// mysqlcreate.java
import java.sql.*;
class mysqlcreate
{
  public static void main (String [] args) throws Exception
  {
    Class.forName ("com.mysql.jdbc.Driver");
    Connection con = DriverManager.getConnection ("jdbc:mysql:
                     //localhost:3306/test", "root", "");
    Statement st = con.createStatement();
    st.execute ("create table sathya(sno INT, sname
                 CHAR(20)");
    sop ("table created");
    st.close();
    con.close();
  }
}
```

→ If we execute the above java programme then u will get class not found exception, because mysql driver is unknown to the JVM.

→ we need to set driver related jar file in the class path but this jar file doesn't come along with mysql database

→ so we need to download a zip file called Mysql-connector-java bile.

→ After extracting, in the root folder that we will get the jar file related to the driver

→ we need to set the jar to the classpath

```
cls set classpath = c:\mysql-connector-java-3.0.8-stable\mysql-
                    connector-java-3.0.8-stable-bin.jar; %classpath%
```

→ To variby wheather table is created in MySql or not

→ clicle on start button → programs → MySql Frent → MySql Frent → connect (button) → expand (+) → test → and variby the table

Date 7/11

Working with Date values

⇒ while working with databases from a Java application, the date formate accepted by Java and the date formate accepted by the database both are different

⇒ while inserting or ~~date~~ secleting ~~delete~~ date value using a JDBC application then we need some conversions between Java and database formates.

→ In JDBC, Date class is given ~~in~~ java.sql package, it acts as a meditor between Java Date formate and Database date formate

→ In Java, Data class is given in two packages

(i) Java.util

(ii) Java.sql

→ From a Java programe if we want to insert date value into a database table. then in the middle they following conversions are required

(i) convert the Date value from string type into Java.Date class object

(ii) To do this conversion, we need a meditor class called SimpleDate formate class given in Java.Text package

(iii) convert Java.util.Date object into Java.Sql.Date object the Java.Sql.Date object will take care about converting the Date value Database related formate

iv) NOW insert Java.Sql.Date object into Database table

Keyboard → dob → String → SimpleDateFormate → Java.util.Date → Java.sql.Date → Database

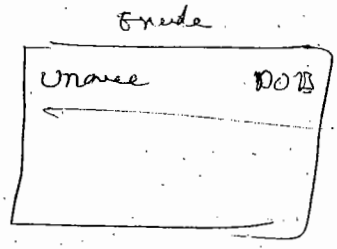
From String to Java.util.Date object

```
Scanner s = new Scanner (System.in);
sop ("Enter date of birth (dd/mm/yyyy)");
String str1 = s.next();
SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
Java.util.Date d1 = sdf.parse(str1);
```

From Java.util.Date into Java.sql.Date object:

```
long ms = d1.getTime();
Java.sql.Date d22 new Java.sql.Date (ms);
```

Setting data into database table :-




```
PreparedStatement pstmt = con.PrepareStatement("insert into example values (?,?)");
pstmt.setString (1, "abcd");
pstmt.setDate (2, d2);
```

① The following JDBC programe is for inserting User name and DoB example table in Database ?

sql> create table example (uname varchar2 (10), dob date);

```
import java.sql.*;
import java.util.*;
import java.text.*;
class JDBCTest
{
    PS VM (String [] args) throws Exception
    {
```

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection con = DriverManager.getConnection("jdbc:odbc:ordas1",
                                              "scott", "tiger");
PreparedStatement pstmt = con.prepareStatement("insert into exam values(?,?)");
Scanner s = new Scanner(System.in);
    sop("enter username");
String str1 = s.next();
    sop("enter date of Birth(dd-mm-yyyy)");
String str2 = s.next();
SimpleDateFormate sdf = new SimpleDateFormate("dd-MM-yyyy");
java.util.Date d1 = sdf.parse(str2);
    long ms = d1.getTime();
java.sql.Date d2 = new java.sql.Date(ms);
    pstmt.setString(1, str1);
    pstmt.setDate(2, d2);
    int k = pstmt.executeUpdate();
    sop(k + " row inserted");
    pstmt.close();
    con.close();
}
}
```

output

```
c:\ java jdbcDataTest
    enter username
    'abcd
Date of Birth (dd-mm-yyyy)
10-3-2009
1 row inserted
```

SQL> select * from example;

UNAME	DOB
abcd	10-MAR-09

Reading Date value from table

i) whenever we read data from a table the we will get the data into resultset object

ii) To read Date values from Result object, we use getDate() while reading, we have to re convert java.sql.Date object back to java.util.Date object

```
Resultset rs = stmt.executeQuery("select * from example");
rs.next();
java.sql.Date d2 = rs.getDate(2);
java.util.Date d1 = (java.util.Date) d2;
sop(d1.toString());
```

rs → BFR → abcd 10-MAR-09

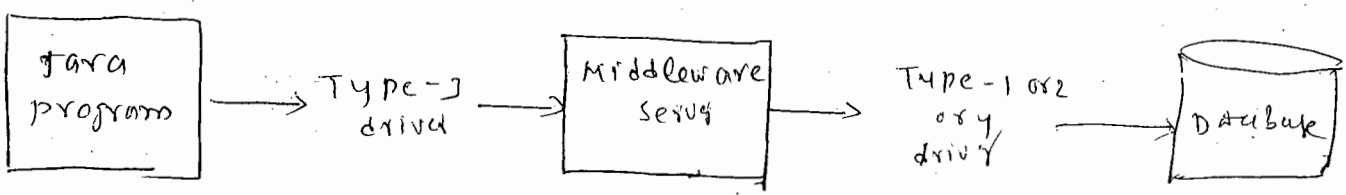
Type-3 Net protocol pure Java driver

→ This type-3 driver uses network protocol bw connecting a Java-programe with a middleware server

→ Type-3 driver follows three-tier architecture or three-tier modem it means Java application connected with middle ware server and middle server connect with Database

→ The type-3 driver implemented 100% Java programing language so it is platform independent

→ Type-3 driver connects Java programe with a middle ware server but not the Database directly. So it is independent of databases



ex:

Java programe → IDS JDBC driver → IDS server → Type-1 → DB

→ In case of Type-3 driver, the original database connectivity taken care by middleware server

⇒ Type-3 drivers will come along with middle ware servers it means Type-3 drivers are server dependent.

⇒ Type-3 driver does not take care about to which database the Java programe is connecting instead it always connects Java program with middle ware server

Advantages :-

1) Type-3 driver is both database independent and platform independent
2) Type-3 driver is suitable for production environment (real time)

Drawbacks :-

1) Type-3 driver is server dependent
2) A middleware server need to be instaled to work with type-3 driver
3) Type-3 driver is slow then type-4 driver
4) Type-3 driver is uses a network protocol for connecting a Java programe with an application that is running at middle server. After that the application running at server will take care remaining Database connectivity

Ex: In case of, IDS server, DB2 JDBC Driver connects Java programe & servlet application running in that Server. After that servlet application takes care about connecting with the database.

Sir: IDSsoftware.com

IDS Server installation

* IDS server is open source Middleware server, provides type-3 JDBC driver called IDS driver
* whenever we installed the IDS server then it will be automatically started and it runs on port number 12
* we can get IDS server from www.idssoftware.com/download.html
* To varify wheather IDS server has started or not, we can open windows services from administretive tools and we can find out in list of servicy IDS server started.

⇒ The following java program represent Type-3 driver connection with oracle database using IDS server as middleware

Programe :-

```
import java.sql.*;
class Type3Test2
{
  public static void main (String [] args) throws Exception
  {
    Class.forName ("ids.sql.IDSDriver");
    Connection con = DriverManager.getConnection ("jdbc:ids://localhost:12
                    /conn?dsn='oradsn'&user='scott'&pwd='tiger'");
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery ("select * from dept");
    while (rs.next())
    {
      System.out.println (rs.getInt(1) + " " + rs.getString(2) + " " + rs.getString(3));
    }
    rs.close();
    con.close();
  }
}
```

→ In the above Java program, we have used dsn name as oradsn, it must be

→ the server does not accept user dsn for connectivity

Date
8/7/11

Introduction to Web appilications

- In standard alone (desktop) two-tier application development Mostly client-server applications are resided in a single system or probably different mechnings in a local area nqw
- ⇒ In standard alone two-tier application development, like java socket programing (networking) or JDBC programing, the client applications are think clients or Fat clients.
- With standard alone client-server applications, we have the following two major problems
  - (i) client application can't be accessed from any where in the network. it means on each systems the client application need to be installed seperately
  - (ii) on one system, if a client application is modified then it will not be directed on other systems where the client applications are installed, we need to go to each system seperatly and we need to make the changes indiadly. It is a time consuming process
- ⇒ To overcome the above problems of standard alone client server applications, we moved on to client server web applications

web application :-

When ever we want to provide service of application to a single client at a time then we can choose a standard alone application and with doesn't require any internet access When we want to provide service to multiple clients

Web application. Web application always runs on a central computer called as server.

Defn:- A web application is a serverside application, which runs at server and provides service to multiple clients across internet simultenously

Types of web applications

(i) prasention oriented web applications
(ii) service oriented web applications

→ prasention oriented

* these type of web applications are also called as static web applications.
* These type of web applications will constrate on providing static content on to the browser
* These type of web applications contains html files and image files
* The first type of web applications introduce are prasentation oriented web applications
* prasentation oriented web applications do not accept any input from the client, rather web pages are displayed on to the client browser
* For example:- An online tutorial is a prasentation oriented web applications. Because it contains static pages, which are already created or generated and one by one page will be shown on to the browser whenever request is given

(iii) Service oriented

* This type of web applications are also called as dynamic web applications
* This type of web applications will consontrate on prouiding dynamic services to the clients
* This type of applications are developed by using server technology like servlets, JSPs, ESPs, PHP etc.
* In two days programing, all most all web applications are belogst to service oriented web applications
* For example Gmail, yahoo are Service oriented web applications and these web sites will prouided mailing and chatting services to the client across network.

Requirements for prasentation oriented application

- Browser software
- Web Server software
- Client side web technology
-

Requirements for service oriented application

1. Browser software
2. Web server software
3. Client side application web technology
4. Server side web technologies
5. Database software

Introduction to web applications

=> In standalone (Desktop) two-tiers application development mostly client & server applications are resided in a single system are probably different machines in a local area n/w's.

=> In standalone two-tiers application's development's, Java ~~app~~ programing (or) Jdbc programing, the client appln's are "thick client" or "Fat client".

=> with standalone server appln's, we have the following two major problems.

(i) client applications cannot be access from anywhere in the network. It means on each system if client applications need to be install seperately.

=> on one system, if the client appli is modified then it will not be effected on other systems, where the client appln's are installed

we need to make the changes. Individually it is a time consuming process.

=> To overcome the above problems in standalone client-server appl's so we move on to client-server web-applm's.

*** Q) What is web-application?

A:- Whenever we want to provide service to single client at a time then we can choose a standalone applications and it doesn't require any internet access.

=> Whenever we want to provide services to multiple clients across network simultaneous then we need to choose web-application.

=> web-appl always runs on a central computer called a server.

Definition

A webapplication is a server side application, which runs at server side and provides service to multiple clients across at network simultaneously.

=> Types of webapplications:-

(i) Presentation webapplication.

(ii) Service oriented webapplication.

(i) presentation oriented web applns:-

This type of web-appln's are also called as static webappln's.

This type of web appln's will concentrate on providing static content on to the Browsers.

=> These type of web applications contains html files & image files.

=> The first type of web applications introduced are presentationoriented web applications.

=> presentation oriented webapplications do not accept any input from the client, rather web pages are displayed on the client browsers.

-> For Example, An online tutorial is a presentation oriented webapplication because it contains static pages, which are already generated and one by one

(2) Get method appends form data in the address bar URL whenever a request is submited. if any secret data Entered in the form it is also displayed in the addressbar. Get method is a Non-Secure method.

(3) Get method doesn't support file uploading

(4) Get is an Idempotent method. it means get method goes to server and it the requested data

(2) When we submit a form post method do not effect form data in address-bar so post method is secure method than get method.

(3) post method supports file uploading.

back to the server. it can not modify the Data available at the server.

(4) post method is Non-idempotent method, post method can modify the Data available at server.

Note: post method can take the data from server and also it can modify the Data available at server.

→ In an online exam, if we want to change from previous answer to new answer for a question then HTTP-post method is required because, when answer is set for first-time & when we moved on to the next question that answer will be stored in server.

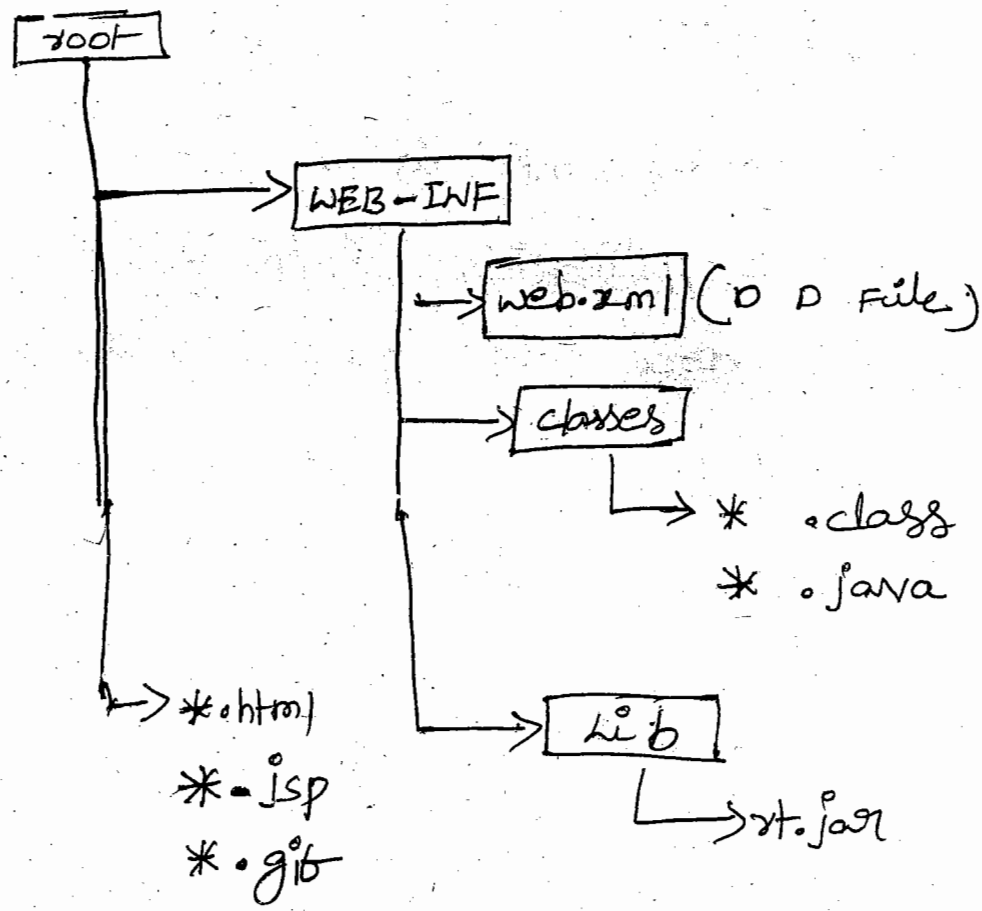
→ when we come-back to the previous que ~~and~~ when we change the answer & the previous answer in the server must be replaced with new answer for that Question. In

→ In an interview questions website for a we
if the answer is not correct then new answe
can be typed by the user, so that at server
the previous answer is replaced with new
answer for that question. In this case Http
post method is required. because post is a
non-idempotent method.

Directory structure of a web-application

→ while developing web-applications in Java, we must follow a standard directory structure given by SUN. (W3C).

→ The advantage of following a standard directory structure is, our application can be executed on any server directly, it means our webapplication becomes server independent by following the standard Directory we will get a moto or feature (WODA) (write once Deploy anyw



→ In the above Directory structure root Directory and its subdirectory WEB-INF or mandatory for a web applications.

For each web-application one web.xml file is manadatory.

⇒ web.xml file is called as deployment descriptor file.

Deployment :-

Deployment is a process of installing a web-application into a server. Deployment can be done in following three ways

(i) Hard Deployment (ii) console deployment

(iii) Tool-based deployment.

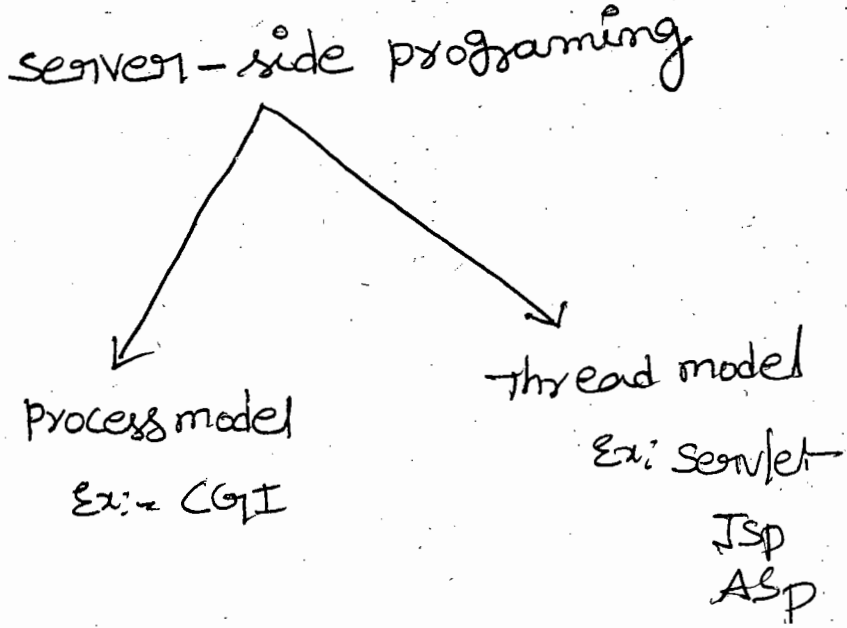
⇒ In Real-time application tool based tools are used (ANT, MAVEEN)

↓

Another need tool.

why servelet Technology ?

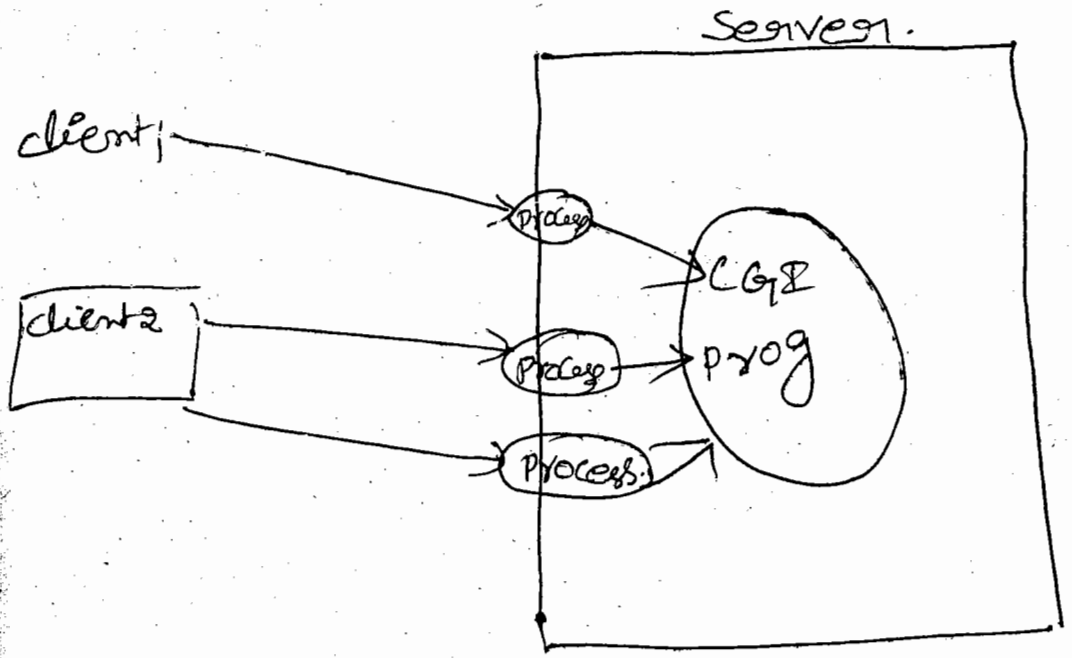
→ For Developing server-side applications we have two models.



-> in the history of www (world wide web) first technology introduced was CGI by NCSA

-> CGI technology follows process model, it means for each request a process is created at O.S. level.

-> If the number of clients are increasing then automatically number of processes at O.S level are also got increased.



Drawbacks of CGI

(i) CGI creates a new process for each request at server, so if the number of clients are increased or number of processes at server are also got increased. so

(i) CGI uses PERL scripting (practical extraction & reporting language) so CGI doesn't provide any security because security is not possible through scripting languages.

In order to overcome the drawbacks of CGI technology SUN micro-systems introduced Servlet Technology.

22-7-14

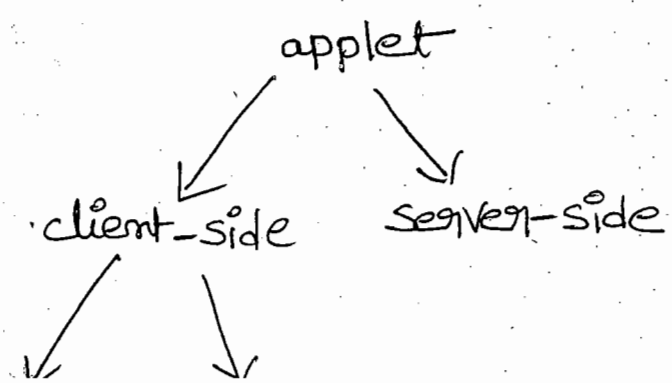
* <u>what is Servlet?</u>

In java, applets are devided into two types

(i) client-side applets.
(ii) server-side "

=> client-side applets are again devided into two types

(i) un-trusted applet.
(ii) trusted "



client-side applets in Java are called GUI applets or applets & server side applets are called servlet.

Applets are by default untrusted, it means an applet doesn't allow native code.

It is possible to convert an untrusted applet into a trusted applet by making changes in the applet policy file.

** what is diff b/w an applet & servlet

Applet	Servlet
(i) An applet runs on a web-browser	(ii) A servlet runs on a web-server.
(ii) An applet increases (or) extends the browser functionality	(ii) A servlet increases (or) extends a server functionality.
(iii) An applet provides service to one client at a time.	(iii) A servlet provides service to multiple clients

3) An applet contain width & height, it means it visible on browser, it means it contain face.

4) A servlet doesn't contain width & height. It is invisible in the server, so a servlet is face

Def 1:

A servlet is a small, platform independent java class it runs on a server & provides service to the clients across network.

Def 2:

A servlet is nothing but, it is a server side applet.

Def 3:

A servlet is a class which follows a model called "Single instance & multiple Threads".

Def 4:

A servlet is a Dynamic web resource of a web-application, which provides servi

Servlet Technology:-

⇒ Servlet is a Technology provided by Sun, after C.G.I Technology, Servlet Technology given by S.U.N, for Developing web Components.

⇒ Servlet Technology is a part of J2EE (or) JEE. J2EE has devided into following two types of Technologies.

(i) component Technology,
(ii) Servlet "

Component Technologies

(i) Servlet
(ii) jsp (Java Server pages)
(iii) EJB (Enterprise Java bean).

⇒ servlet & jsp Technologies are used for web-Component Developnet & EJB Technology is used for Enterprise component development.

(2) Service Technology.

(i) JNDI (Java naming & Directory Interface)
(ii) JMS (Java messag

(iii) Java Mail

(iv) Java authentication authorization service (JAAS)

(v) Java Connector Architecture (JCA).

Servlet API

Technology = API + Specifications

Servlet Technology contains both servlet API & servlet specifications.

Servlet API contains set of classes & interfaces.

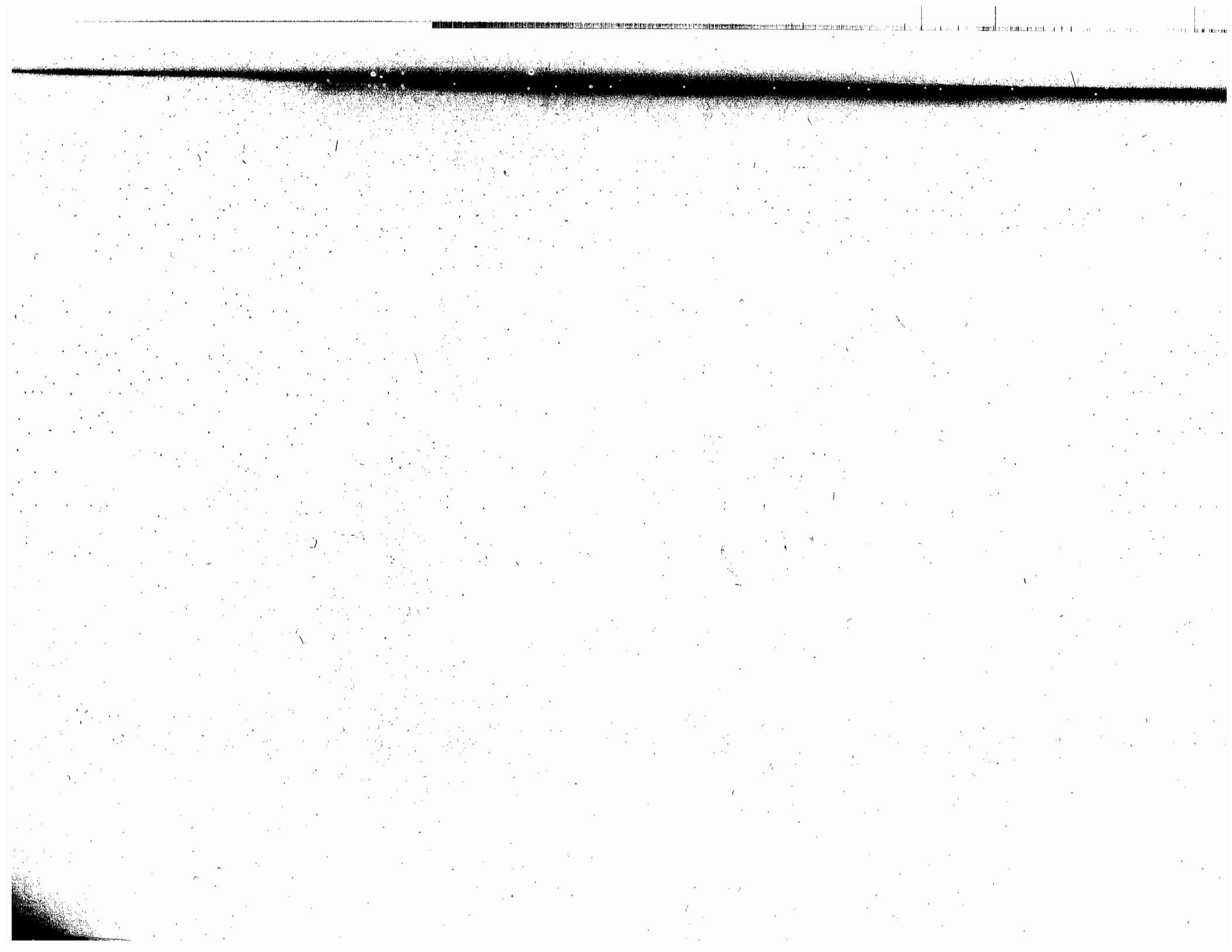
servlet specification contain set of rules & guidelines.

servlet API consists the following two packages

```
javax.servlet;
javax.servlet.http;
```

→ The servlet specification will be followed by the server vendor, by providing implementation of interfaces of servlet API.

→ Similar to servlet technology JDBC Technology also driver vendor will take care about implementation of JDBC interfaces.



Different ways of Creating a Servlet:-

→ Every servlet class is either directly (or) indirectly implemented from Servlet interface given by ServletAPI.

→ For programmer convienence, servlet API as provided some abstract base classes, for creating a servlet.

→ While developing web applications, a programmer can create a servlet class, by using any one of the following three approaches

(i) By Directly implementing our class from Servlet interface.

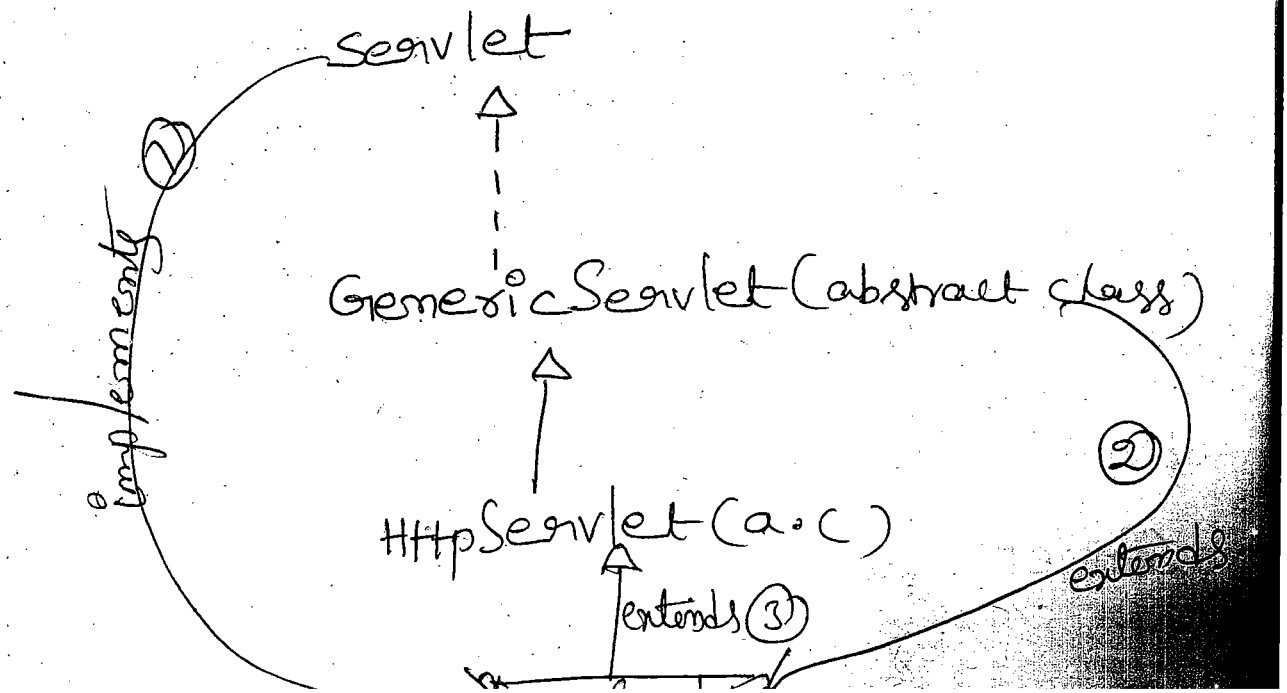
```
public class MyServ implements Servlet
{
}
```

(2) By extending our Servlet class from a baseclass called GenericServlet.

```
public class MyServ extends GenericSer
                                vlet
{
}.
```

(3) By extending our Servlet class from a baseclass called HttpServlet

```
public class MyServ extends HttpServlet
{
}.
```



Note:- (i) According to interviews point of you there is only one way of Creating a Servlet.

i.e., By either Directly (or) indirectly implementing Servlet Interface.

⇒ (ii) While Creating any Servlet classes we assume that there are 3 ways of Creating Servlet.

(i) By implementing Servlet Interface

(ii) By extending GenericServlet

(iii) By extending HttpServlet.

⇒ Servlet Interface and GenricServlet abstract class, both are given in javax.servlet package.

⇒ HttpServlet is an abstract class given

(ii) javax.servlet.http package.

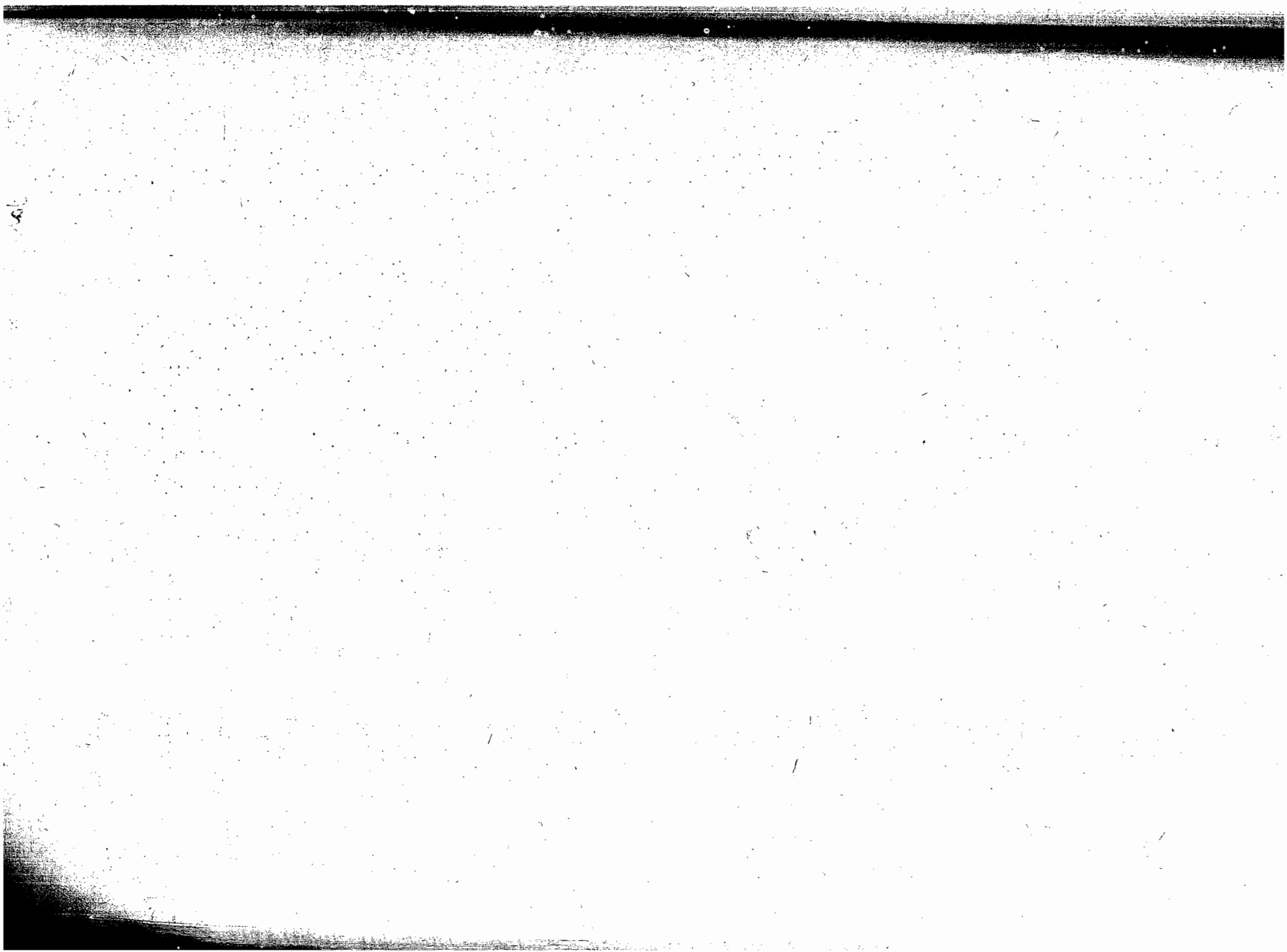
-> In Servlet API, GenericServlet is an abstract class, because it has one abstract method, but in HttpServlet class there are no abstract methods, but the class is given an abstract class

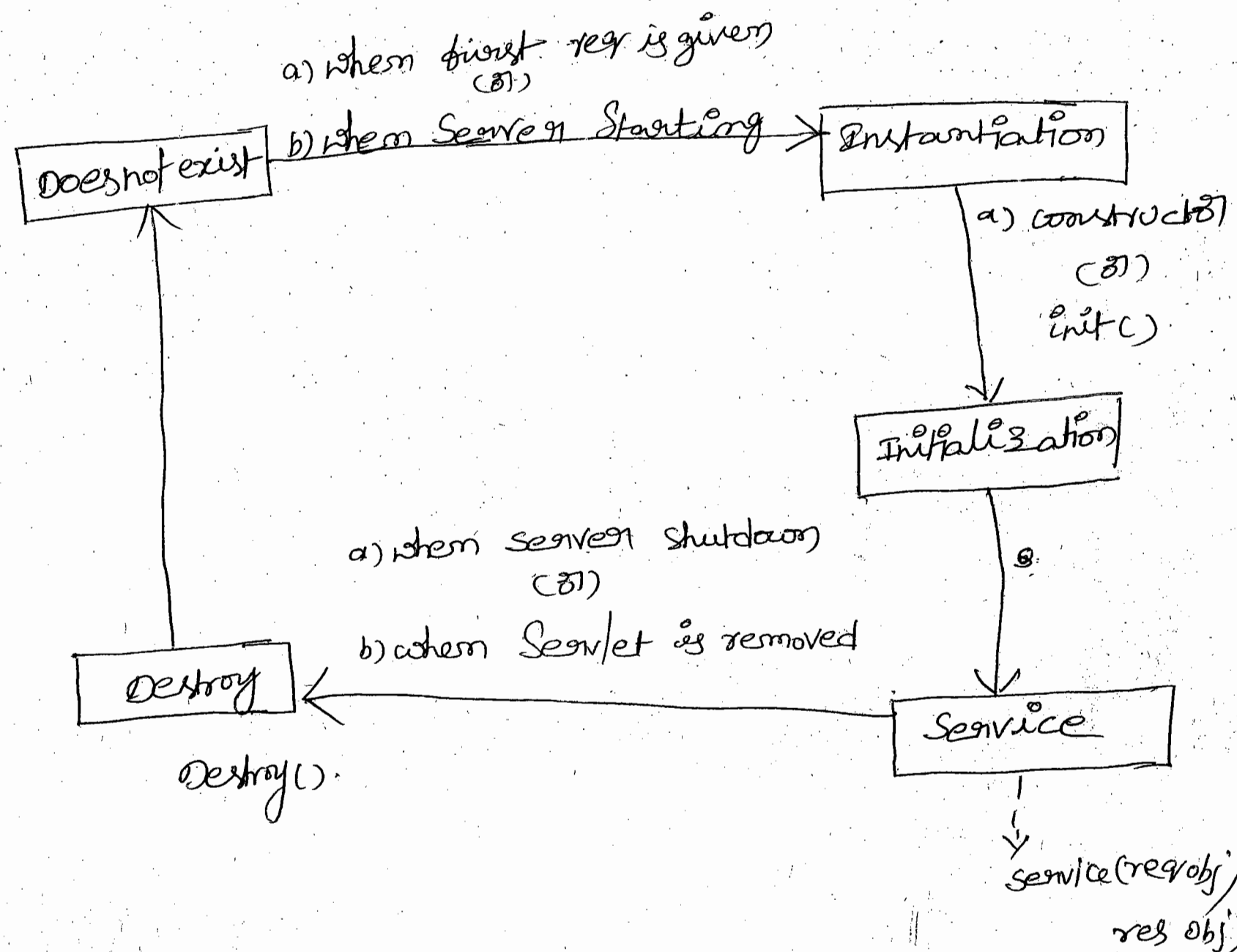
life-cycle of a Servlet:-

(i) Doesn't exist
(ii) instantiation
(iii) initialization
(iv) Service
(v) Destroy.

=> By default webContainer Creats servlet object, whenever a first request is given to the servlet.

=> first client has to wait until




a) when first req is given
(or)
b) when Server Starting
Does not exist
Instantiation
a) constructor
(or)
init()
Initialization
a) when server shutdown
(or)
b) when Servlet is removed
Destroy
destroy().
Service
service(req obj, res obj)

→ After Servlet object created by the container, container initializes the servlet object by calling constructor of the class after that its life-cycle method called init().

→ After a servlet object is initialized, A servlet object will receive the client request & Performs processing & finally provides response back to the client.

→ we call these stage as service.

→ During service stage, container calls another life-cycle method of a servlet called service() method.
(service(req,res))

→ After processing all the requests which are given by the client, whenever the server is shutdown (or) A servlet is removed from the server then container will destroy the servlet object

Container invokes

another life-cycle Method of the Servlet called destroy().

→ whenever A Servlet object is destroyed it will be moved into again doesn't exist stage.

→ Servlet life-cycle is a circular process it means, the life-cycle starts at doesn't exist & finally ends at doesn't exist.

Q) In what cases A Servlet object is Destroyed?

A:- (i) when we shutdown the Server.
(ii) " " undeploy a servlet from a Server
(iii) when Server crash occured.
(iv) when we stop the execution of a Servlet
(v) If Servlet is waiting for a long Period of time in the Server.

Life-cycle Methods:-

For a Servlet there are 3-life cycle methods and these life-cycle methods are provided by Servlet interface.

Servlet interface has totally provided 5 methods. & among 5 methods 3 methods are called life-cycle Methods.

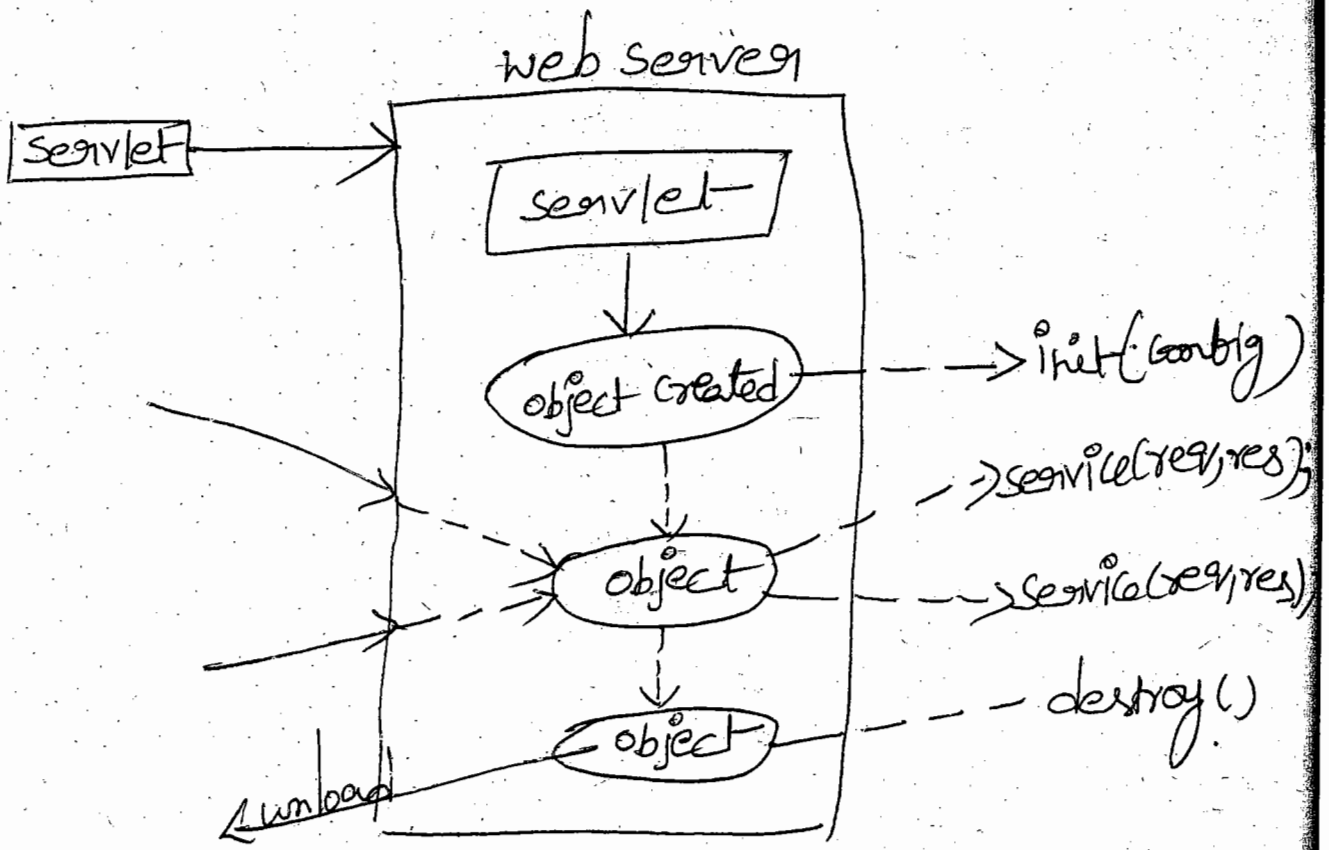
1) public void init(ServletConfig config)
throws ServletException

2) public void service(ServletRequest request,
ServletResponse response)
throws ServletException,
IOException

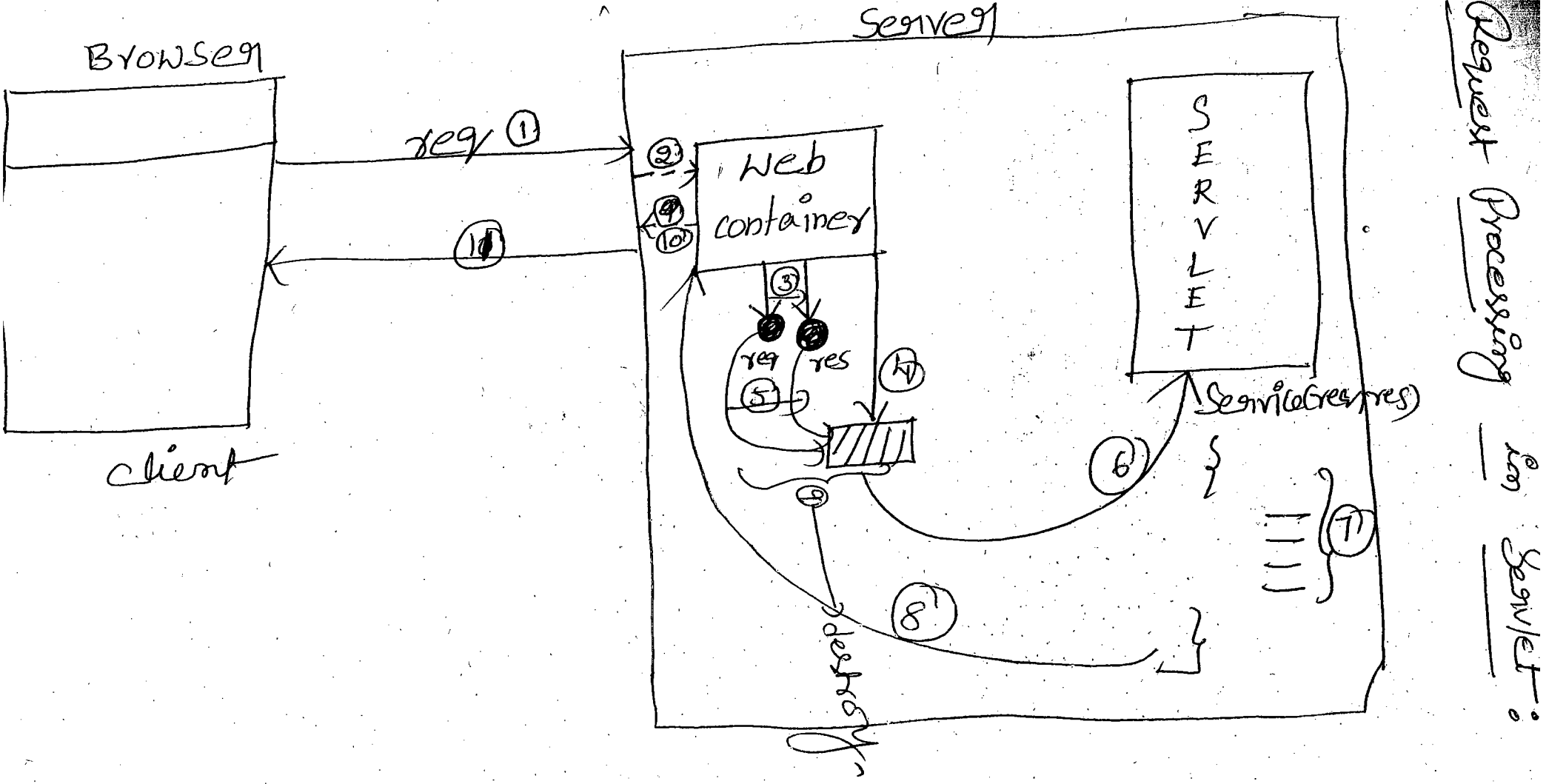
3) public void destroy();

→ init() & destroy() methods are executed for only once., but Service() method will be executed for each request.

The controlling life-cycle method of a servlet is service method.



Request Processing in Servlet:



(i) when a client request is given to the servlet, server received the request

(ii) server verified that the request is for dynamic resource, so server transfered the request to web-container.

(iii) container immediately creates a request object, response object & a Thread.

(iv) Container stores req & res objects into Thread object

(v) The Thread will call service method of servlet by passing req & res objects as parameters.

(vi) The business logic implemented in service method is executed.

(vii) The Container will take the generated response from the servlet.

(viii) The container will provide resp data to server.

(ix) Containers immediatly Destroyes the req, res obj, Thread obj

the server will construct the response data in the form of a webpages then server will send the webpage data to the client.

Date

8/8/11:

```
class A
{
    int x=40;
    class B
    {
        int x=50;
        void m1()
        {
            S.o.p(x); //How to print 40.
        }
    }
}
```

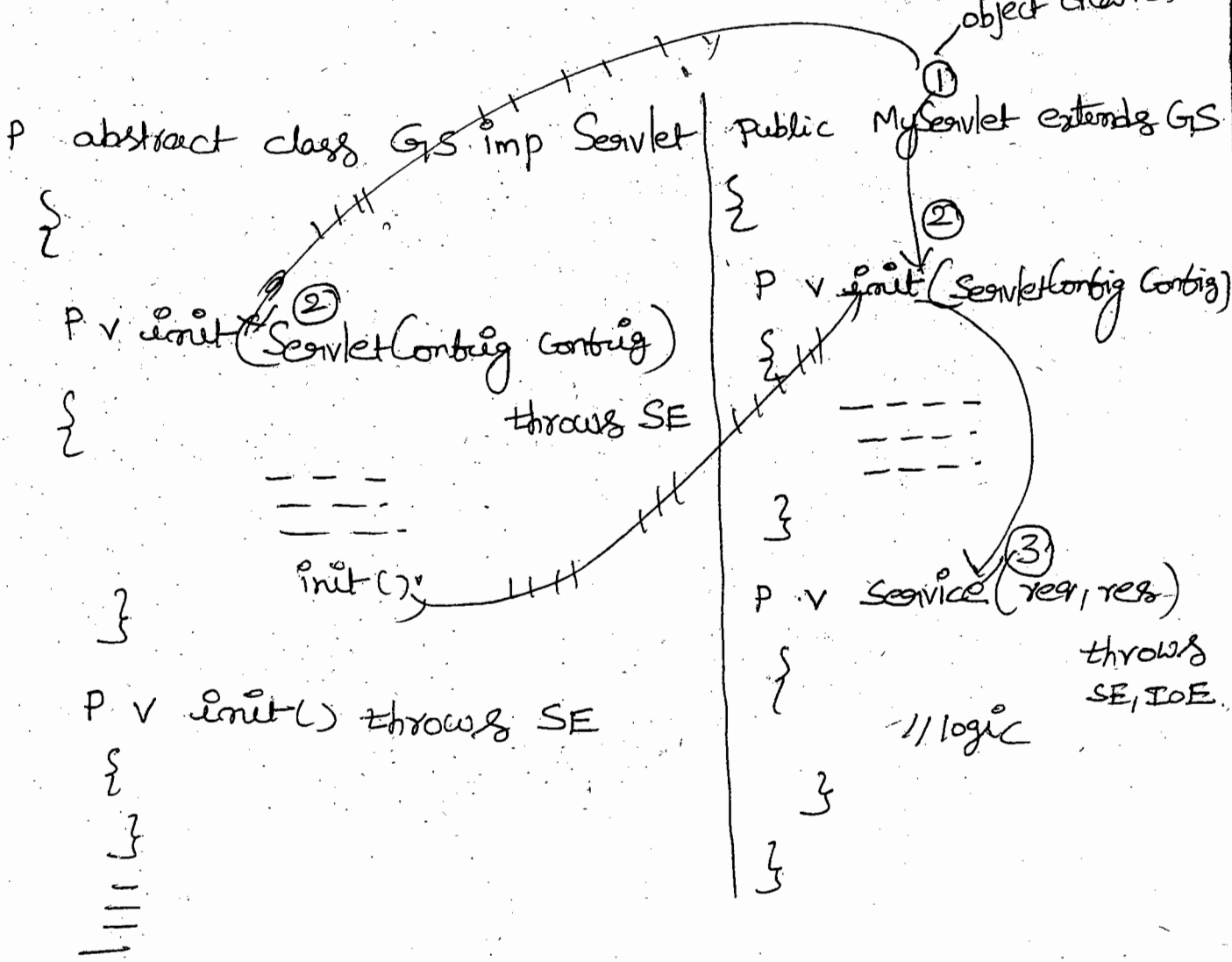
Difference between init(ServletConfig config) & init()

→ In GenericServlet class we have two init() methods, where 1 init() is a life-cycle method which is given by Servlet Interface & another init() is a non-life cycle init() added by Generic Servlet and these init() doesn't contain any parameter.

→ In GenericServlet ~~are~~ class life-cycle init() method internally calls Non-lifecycle init().

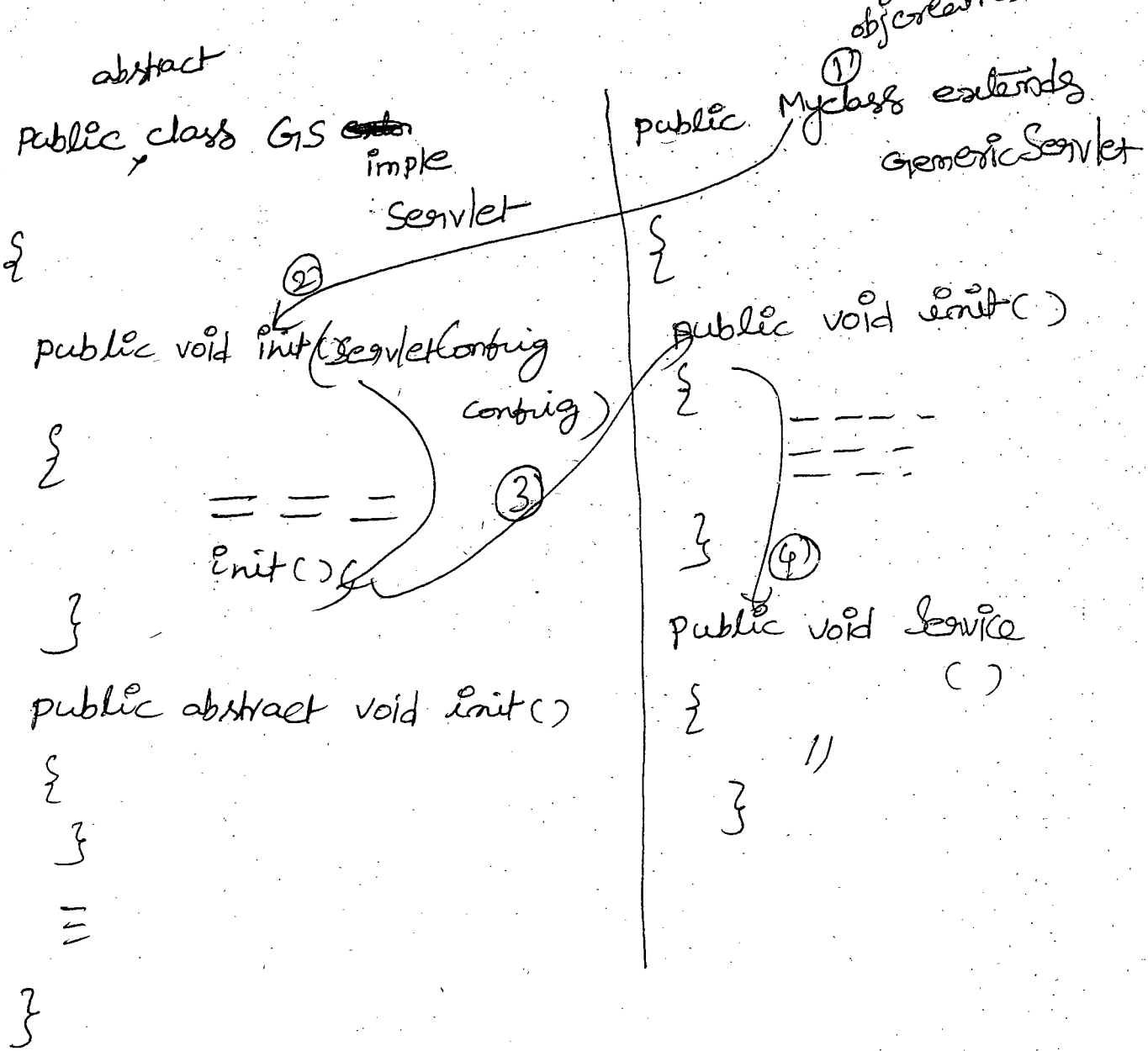
→ while creating our Servlet class, if we Directly override life-cycle init() then container calls the init() available in our class & we are loosing the logic implemented in the life-cycle init() of Base Class.

→ For example



⇒ Inorder to overcome the above problem while creating our Servlet class instead of overriding life-cycle init() in our class we need to override non-life cycle init() given by GenericServlet class into our class.

The advantage of overriding non-life cycle init() is we are not going to loose the base class implementation of the life-cycle init(). Base class life-cycle init() executed after that our class init() is also executed.



→ while overriding base class method in derived class low level access specifier replaced with same level or high-level access specifier

→ High-level access specifier can-not replaced with low-level access specifier.

→ For example.

```
P C MyServlet extends GS
{
    void init() throws SE //not allowed
    {
    }
    P V service(req, res) throws SE, IOE
    {
    }
}
```

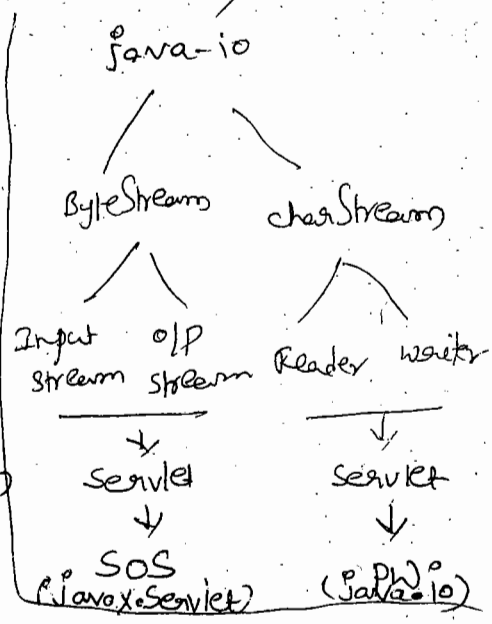
→ In the above servlet the init() an overriding concept. so when we compile our servlet we will get an error.

→ The reason is, in the base class init() is public & we are overriding with default access specifier, it means we are replacing high-level access specifier with low-level access specifier it is wrong.

* ServletOutputStream & PrintWriter classes:

(SOS) (PW)

→ In Servlet Programming, we have the two classes to generate the response i.e., (i) PrintWriter
(ii) ServletOutputStream.



→ PrintWriter is a class from java.io stream & it belongs to character Stream category.

→ ServletOutputStream is from ServletApI and its belongs to byteStream category.

→ In Servlet class to generate response content we must need either print-writer class object or SOS class object.

→ PW & SOS objects are provided by the

⇒ PrintWriter pw = response.getWriter();

ServletOutputStream sos = response.getOutputStream()

PrintWriter	ServletOutputStream
(i) It is a character Stream	(i) It is a Byte Stream.
(ii) PrintWriter is slow in Network	(ii) It is fast in network.
(iii) It is only suitable for Text format of response.	(iii) It is suitable for both Text & Binary(image) format of response.

Note:

If we want to send an image like gif, jpeg etc as a response back to browser from a servlet then we can not use PrintWriter, it is only possible through ServletOutputStream.

Date
9/8/11

Extending HttpServlet

-> HttpServlet is an abstract class given in javax.servlet.http package. and it is extended from GenericServlet.

-> HttpServlet is also an abstract class and it has provided the implementation for the life-cycle service() method, which is an abstract method in the GenericServlet.

-> In HttpServlet class for providing service to the clients requests, there are 9 methods.

-> In HttpServlet class, an additional service() method, i.e., non-life cycle service method is added and 7 doXxx() methods are added and all these methods are for providing service to the client request.

→ In Http Servlet class 2 init() methods are there but these 2 init() methods are not given by HttpServlet these are came from GenericServlet.

→ In HttpServlet class 2 service() methods are there one is life-cycle and other one is Non-life-cycle service method.

→ In HttpServlet 7 Empty doX() methods are given and these 7 methods are used to process the request transfered by 7 Http Protocol methods.

→ For example, HttpGet~~Se~~ request, The processing method is doGet(). and for postrequest the method is doPost() etc.

→ In HttpServlet class, its own methods added with protected access specifier.

```
public abstract class HttpServlet extends
                                GenericServlet
{
    public void service(SR req, SRe res)
                                throws SE, IOE
    {
        =
    }
    protected void service (HttpServletRequest
                            req, HttpServletResp
                                onse res)
                            throws SE, IO=
    {
        ==
    }
    protected void doXxx (HSReq req, HSRs res)
                                throws SE, IOE
    { }
    =
}
```

→ Q) Why 7 doXxx() methods are given has Empty methods in Servlet why not abstract?

A:— If HttpServlet Developed provided seven doXxx() methods has abstract then when a programmer is extending the class from HttpServlet, programer has to override all 7 methods are mandatory, it increases the burden on a Servlet Programer inorder to reduce the burden on Servlet programer, This 7 methods are given has Empty but not abstract.

→ while Creating our Servlet class by extending HttpServlet, we mostly prefered to override either doGet() method or doPost() method. because Http protocol mostly uses either Get() or post() method to transfer the request.

```
public class MyServlet extends HttpServlet
{
    public void doGet(HSR req, HSRe res)
                                throws SE, IOE
    {
        ≡
    }
    public void doPost(HSR req, HSRe res)
                                throws SE, IOE
    {
        ≡
    }
}
```

Note :-

In HttpServlet class the doXxx() methods are Protected and while overriding in our class either we can use protected access specifier or public access specifier.

→ In the above servlet, it gets Request is given then doGet() method is call, then post request is given doPost() method is call.

→ doGet() & doPost() methods are Non-life cycle methods so a web-container doesn't Call them directly.

→ The client request reaches to either doGet() or doPost() in the following way

(i) when a client request is given then container calls life-cycle service() method.

(ii) life-cycle service method converts ServletRequest into HttpServletRequest and ServletResponse into HttpServletResponse and then calls Non-life-cycle service() method.

(iii) Non-life-cycle service() method verifies client request is given get() method or [illegible] and then calls the doGet()

public abstract class HttpServlet extends GS

{

public void service (SR req, SRe res)

throws SE, IOE

{

req ----> hreq

res ----> hres

service (hreq, hres);

}

protected void service (HSR req, HSR res)

throws SE, IOE

{

// read the http method from the given request

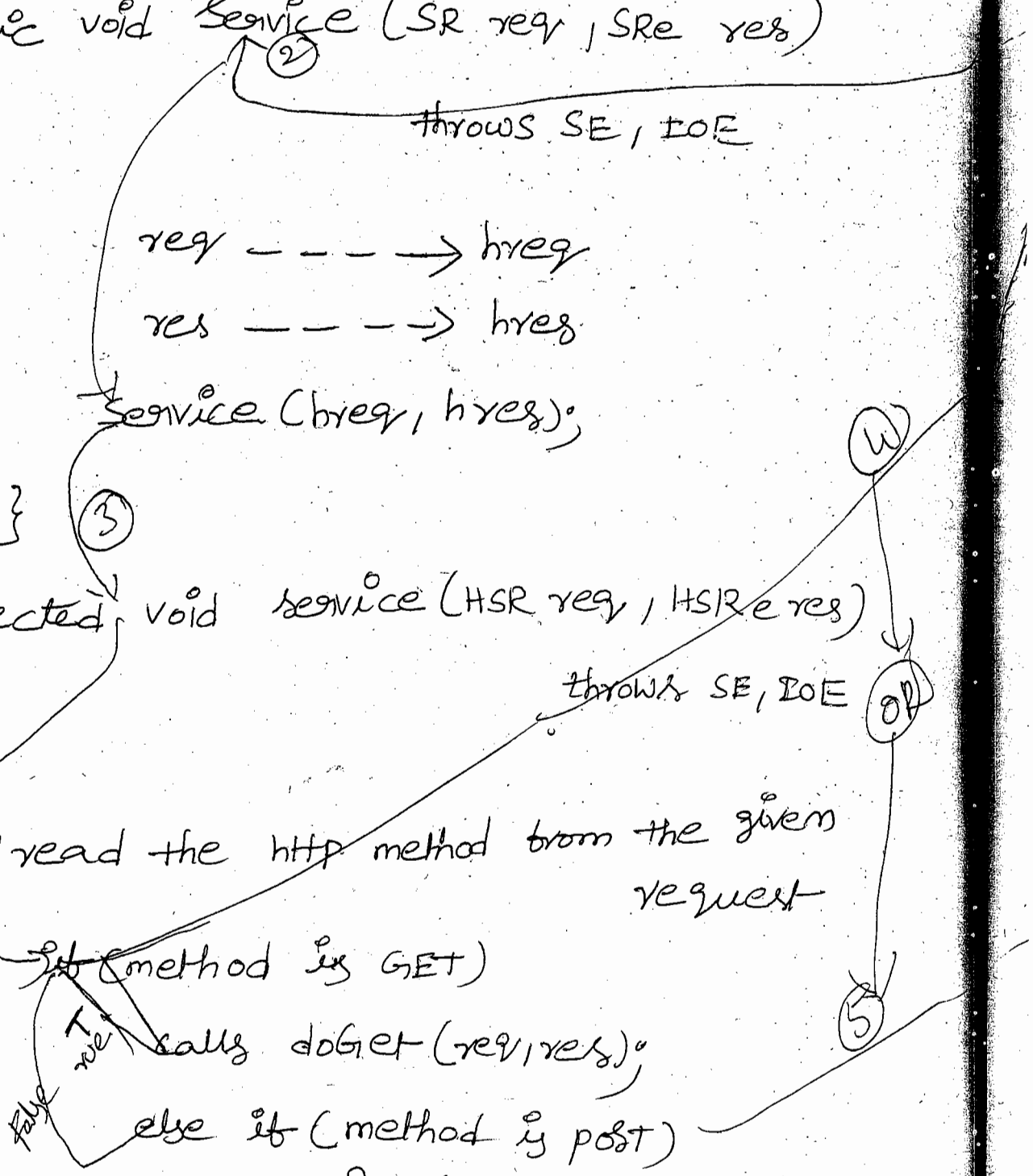
if (method is GET)

calls doGet (req, res);

else if (method is post)

calls doPost (req, res)

}



```
class MyServlet extends HttpServlet
{
    public void doGet(HSR req, HSRe res)
                                    throws SE, IOE
    {
        ----
    }
    public void doPost(HSR req, HSRe res)
                                    throws SE,
                                    IOE
    {
        =
    }
```

**

Q:- Why HttpServlet class given has abstract, eventhough there are no abstract methods in it?

A:- If HttpServlet class is not given has abstract then A servlet programmer can create any number of objects for it directly. It violates servlet principle or model.

so inorder to disallow servlet prog

made as an abstract class.

Because the class is an abstract class it is possible for us only extend it not possible for it to create object directly.

Date 11/8/11

⇒ If a servlet programer is unknown about HTTP method i.e., either get or post. then while creating the servlet the programer has to add doGet() and doPost() methods in the class with logic.

⇒ Instead of defining the logic both doGet() & dopost() methods we can reduce the size of servlet class in the following 2 ways.

(i) Call an user-defined method from both doGet() & doPost() method.

```
public class MyServlet extends HS
{
    p v doGet(req, res) throws SE, IOE
    {
        process(req, res);
```

```
p v  doPost(req, res) throws SE, IOE
{
        process(req, res);
}

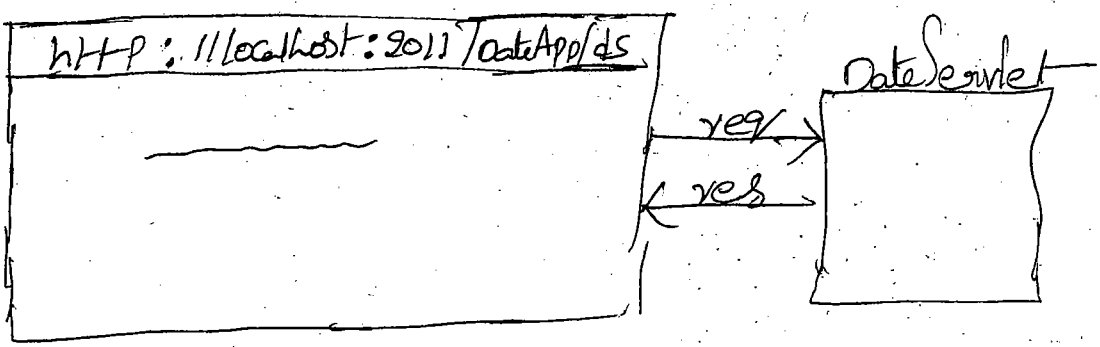
p v  process(req, res) throws SE, IOE
{
        //
}
}
```

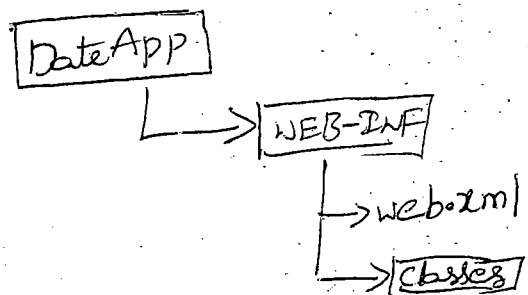
(ii) Call either ~~doGet~~ doPost() from doGet()
~~or~~ or vice versa.

```
public class MyServlet extends HS
{
    p v  doGet(req, res) throws SE, IOE
    {
            doPost(req, res);
    }

    p v  doPost(req, res) throws SE, IOE
    {
            ≡
    }
```

The following web-application is to display Date & Time from servlet on to the browser and to refresh the response for each second?





DateServlet.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;
import java.io.*;

public class Myclass extends HttpServlet
{
    public void doGet(HttpServletRequest
                      req, HttpServlet-
                      Response res)
                      throws SE, IOE
    {
```

```
Date d= new Date();
res.setHeader("Refresh", "1");
pw.println("<h2>");
pw.println(d.toString());
pw.println("</h2>");
pw.close();
}
}
```

② web.xml

⇒ when request is given directly from a browser to a servlet always request is transfered by using HttpGet() method. so we have overridden our Servlet class doGet() method.

⇒ to refresh the response of a servlet automatically on to the browser we have set a header in the servlet & the response or response object.

→ In the above servlet we have added one "Refresh" value 1, which means Response will be automatically refreshed for each second.

* MIME SETTING:-

Before displaying a servlet generated response on to the browser, the web-server will construct a web-page and transfers that web-page on to the browser.

→ If we dont set any MIME (multipurpose Internet mail Extensions) then by default MIME type is considered text/html

→ If we explicitly set MIME type for the response data then the server will construct a web-page of that type. and server will send that page on to the browser.

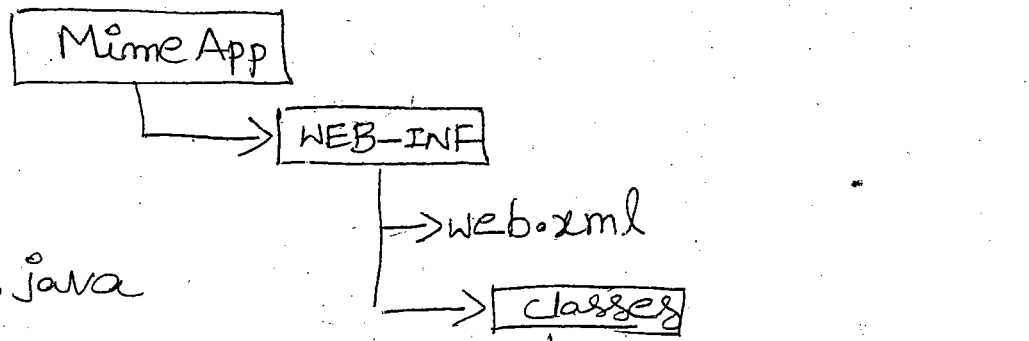
Some of the MIME Types given by world wide web Consortium(W3C) are:

1. text/html
2. Text/plain
3. text/xml
4. image/gif
5. image/jpeg
6. application/msword
7. " /vnd.msexcel
8. " /pdf
9. etc...

→ If we want to set MIME type explicitly we need to call setContentType() method which is given by ServletResponse interface

response.setContentType("text/plain");

* The following web-application is for displaying response using diff MIME types



//HtmlServlet.java

```
import javax.servlet.*;
  //      //  .  //.http.*;
  //      java.io.*;

P c  HtmlServlet extends HttpServlet
{
   public void service(HttpServletRequest req,
                       HttpServletResponse res)
                    throws SE, IOE
   {
     res.setContentType("text/html");
```

```
pw.println("<table border=2>");
pw.println("<tr><th>Sno</th><th>Sname</th></tr>");
pw.println("<tr><td>101</td><td>venu</td></tr>");
pw.println("<tr><td>102</td><td>siva</td></tr>");
pw.close();
}
}
```

// ExcelServlet.java

```
res.setContentType("application/vnd.ms-excel");
```

// WordServlet.java

```
res.setContentType("application/msword");
```

// XmlServlet.java

```
res.setContentType("text/xml");
pw.println("<table>");
```

web.xml

<U-P> /hurl </U-P> for html/Servlet
<U-P> /eurl </U-P> " Excel "
<U-P> /wurl </U-P> " word "
<U-P> /xurl </U-P> " XML "

Date 12/8/11

Console Deployment in Tom-Cat

Step 1:

Create a war file (webarchive).

Step 2:

In java, we have the following three types of compressed components.

a) jar --- java archive
b) war -- web archive
c) ear -- enterprise archive.

To create the archive components, we need to use a tool given by jdk called jar tool.

In web-applications, we need to create a war-file and it is a compressed format of the Entire web-applications.

It is similar to win-zip.

To create a war file, we need to place the [illegible] into the root directory of the web-app

c:\LoginApp> jar cvf logtest.war *.*

c → create, v - verbose f - file.

ear = jar + jar
car = jar + war

Step 3:

Start Tomcat server.

Step 4:

open the browser window & type the following url.

http://localhost:2011

Step 5:

click on Tomcat Manager → Enter username → Tab → password

Step 6:

Goto war file to deploy section → click on Browse → select war file → click Deploy.

Step 7:

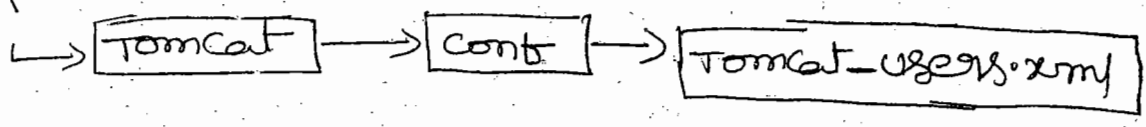
open the Browser & Type the following url in address bar.

http://localhost:2011/logtest/login.html.

Note:

After deploying the war file if we get running state false then there is a mistake in web.xml, so we need to correct web.xml

-> If Manager username & password is unknown then we can find it through tomcat-users.xml



String S = "satya" + "100" + "Tech"

How many objects Created

only one object Created because whenever Arithmetic operator(+) is occured String is converted into StringBuffer, so StringBuffer

Methods to read Url information

-> When Sending a Request to the Servlet from client browser, we type some url in the Address bar and it contains a diff part

-) In Http Servlet Request The following methods are available to read diff parts of the url.

1) request.getScheme() --> returns protocol name like 'http, https-

2) " .getServerName() --> returns server machine Ip Address used by the request

3) " .getServerPort() --> Returns port number of the server

4) " .getQueryString() --> Returns query string added to the request.

5) " .getRequestUri() --> returns uri of the request (contextpath servletPath (uri)

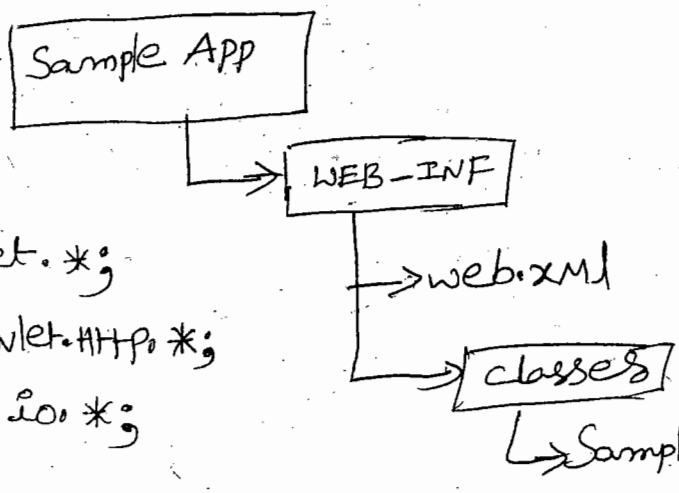
6) " .getRequestUrl() --> Returns complete url.

7) " .getRemoteADDR() --> Returns ipaddress of the machine who

Note:

- getRequestUrl() → method returns StringBuffer and all the Remaining methods return String

* The following web-application is to print the different parts of the url of a request.




```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class SampleServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req,
                      HttpServletResponse resp)
                      throws SE, IOE
    {
        process(request, response);
    }
    public void doPost(HSR req, HSRP resp)
                      throws SE, IOE
    {
        process(req, resp);
    }
```

```
P.V process( HSR req, HSRp resp) throws SE, IOE
{
    ServletOutputStream sos = response.getOutputStream();
    sos.println("Method:" + request.getMethod());
    sos.println("<br>");
    sos.println("QueryString:" + req.getQueryString());
    sos.println("<br>");
    sos.println("RequestUri:" + req.getRequestURI());
    sos.println("<br>");
    sos.println("RequestUrl" + req.getRequestURL().toString());
    sos.println("<br>");
    sos.println("RemoteAddress:" + req.getRemoteAddr());
    sos.println("<br>");
    sos.println("Server name:" + request.getServerName());
    sos.println("<br>");
    sos.println("Server port:" + request.getServerPort());
    sos.println("<br>");
```

```
sos.println("servlet path:"+req.getServletPath());
sos.close();
}
}
```

http://localhost:2011/SampleApp/SameSrv?uname="satya"&age=10.

O/P method: GET

queryString: user=Satya&age=10

RequestUri: /SampleApp/samplesrv

Request Url: http://localhost:2011/SameApp/sameSrv

RemoteAddr: 127.0.0.1 (Standalone or Desktop systems (i.e., those systems not connected in the internet))

Server name: localhost

Server port: 2011

Servlet path: /samplesrv

2:- How to reject a request given to a servlet from a particular client system?

A:- If we want to reject client request given from a particular system to a servlet, Then we need to find IpAddress of the client machine, then we need to reject the

→ To find IpAddress of a system we call getRemoteADDr() method.

for ex:

```
String s = req.getRemoteAddr();
if(s.equals("110.15.12.7"))
pw.println("ur request is rejected");
```

Q: How to reject request given by HTTP, (or) FTP protocol?

A:- we need to call get-scheme method to read the protocol name, from the given request and then we need to verify the protocol to reject the request.

for ex:

```
String s = request.getScheme();
if(s.equals("http") || s.equals("ftp"))
pw.println("ur request rejected");
```

18/8/11 General purpose IDE without supporting Java ~~EE~~ Capabilities

Myeclipse = Eclipse + plugin

↓

ADDed JavaEE Capability.

⇒ in eclipse all works are done by manually.

⇒ The diff b/w Eclipse & Edit-plus is in Eclipse we have Auto-compilation, Automatic setter, getter methods.

⇒ ☐ NO Expiration time for Eclipse.

My-Eclipse Trail Period - 1 Month.

web-application Development in My-eclipse6.0

(i) My-eclipse IDE (Integrated Development Environment) is a Java-related J2EE Enabled IDE and it provides Rapid-application development for J2EE applications.

(ii) Eclipse is a Genral purpose IDE and it doesn't Contain inbuilt, J2EE

(iii) My-Eclipse is Created by adding plug-ins to the eclipse IDE.

My-Eclipse = Eclipse + Plugin.

→ With Myeclipse IDE it is possible for us to develop, deploy & Test a web-application within the IDE window.

→ My Eclipse is the most popular IDE used by Java-developer in the real-time Environment than other-IDE's.

step 1:- start → programs → myeclipse-6.0 → myeclipse-6.0

Enter workspace-name → D:\> venu\servlets → Carry use

debined wo

ok.

step 2:

File → New → project → webProject → Next → Enter

Proj name (TestApp) → Finish

Step 3:

At left side we will get package explorer window and in this window classes folder is invisible under WEB-INF. So shift from PackageExplorer window to Navigator window.

window → show view → other → ~~[illegible]~~

(Expand)
General → Navigator. → ok.

Step 4:

- Right click on projectName → New → Servlet

→ Enter name (ServletClassName) → Next →

Enter * Servlet-name (logins) & Mapping-URL (/logSrv) → Finish.

Step 5:

insert the required servlet-logic, either in doget() or in doPost() methods.

Step 6:

```
doGet()
{
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String s1 = request.getParameter("uname");
    String s2 = request.getParameter("pwd");
```

out.println("Login success")

else

out.println("Login not success")

Step 6:

Right click on projectname → New → HTML → Enter file name (login.html) → Finish.

```
<form action="logSrv(url-pato)">
username: <input type=text name="uname">
<br>
password: <input type=text/password name="pwd">
<br>
<input type=submit value="click">
</form>
```

Step 7:

click on window menu → preferences → Myeclipse (left-side) Expand it → Application Server → Tomcat → Tomcat 6.x → select Enable → click on browse button → select Tomcat root Directory (Tomcat 6.0) → Apply → ok.

Step 8:

Rightclick on projectName -> Myeclipse -> Add & Remove Project Deployments -> Select project Name(TestApp) -> Add button -> Select servers -> Tomcat 6.x -> finish. -> ok.

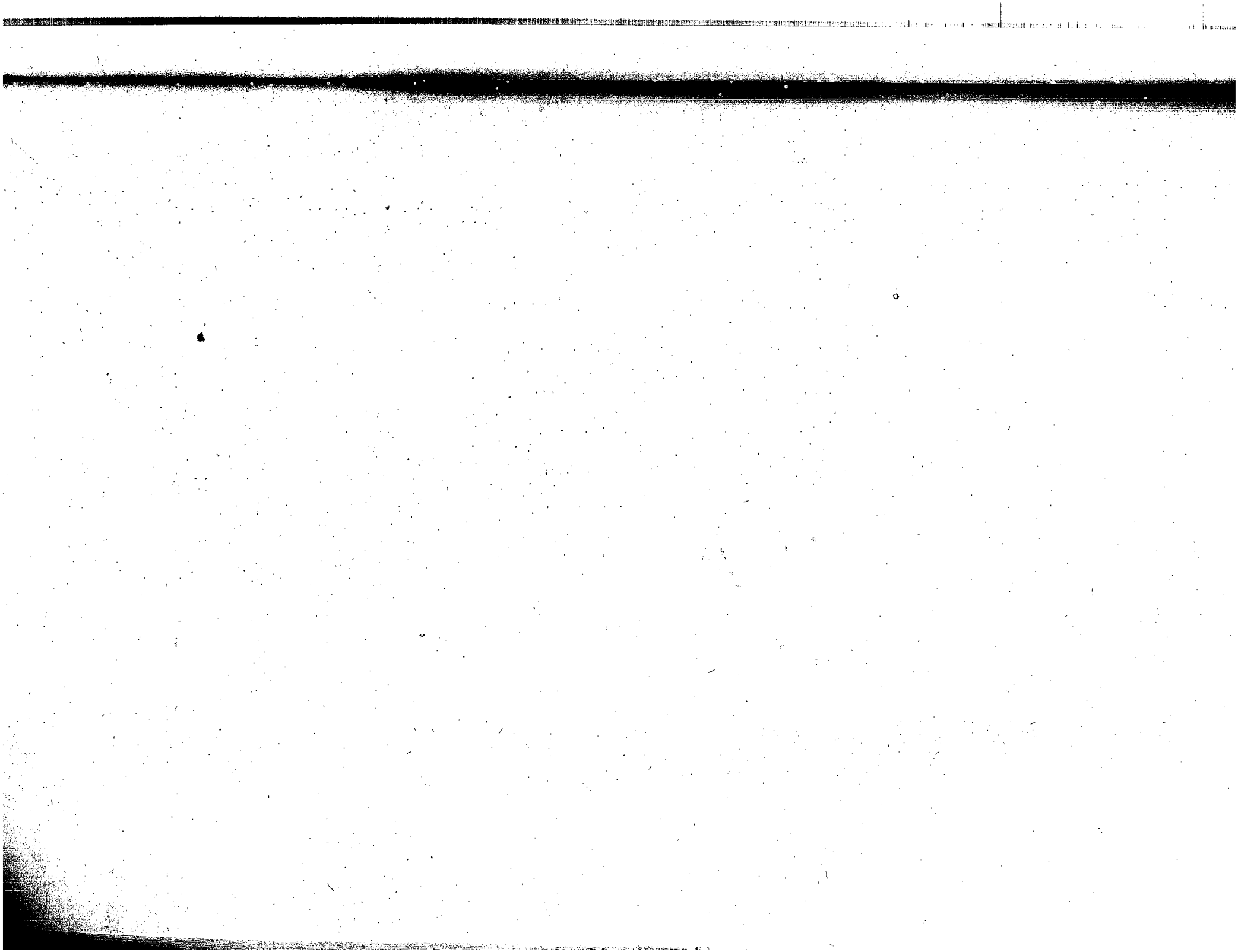
Step 9:

click ON DOWN arrow of myeclipse servers icon ( ) -> Tomcat 6.x -> start.

Step 10:

click on window menue -> show view -> webBrowser -> Enter the following url at address bar (http://localhost:2011/TestApp/login.html -> Enter.

Note:- if we do any modifications in the servlet code then we no need to redeploy & restart the server. The changes are automatically effected.



(2) → Server identifies resource. so Server transferred the request to container.

(3) → container immediately creates a request object, response object and a thread.

(4) → container stores request and response objects into thread object

(5) → The thread will call service() method of servlet by passing req and res objects as parameters

(6) → The business logic implemented in service() method is executed.

(7) → The container will take the generated response from the servlet.

(8) → The container will provide the response data to server.

(9) → container immediately destroys / removes the req objects, res object and thread object / Thread

(10) → Server will construct the response data in the form of a web page and then server will send the web page back to client browser.

Note:

→ For every request, container creates a pair of request and response objects and a thread object.

→ For example, if three requests are given to a servlet then three pairs of request and response objects are created and three threads are created by the container.

→ All request and response objects and threads will share a single as level process. So, there will be a less burden on the server.

Extending our servlet from GenericServlet:-

→ GenericServlet is a predefined class given by the servlet API and this class is implemented from the following two interfaces.

i) Servlet interface

ii) ServletConfig interface.

→ GenericServlet class is an abstract class, because this class is containing one abstract method called service(req, res) method.

→ Servlet interface has provided five abstract [illegible] implemented

four methods and one method is not implemented
i.e., service() method.

→ ServletConfig interface has given four abstract methods and GenericServlet class implemented all of its four methods.

→ so, there is only one abstract method in the class called service.

→ if we Extend our class from GenericServlet class then we must implement service() method in our servlet class.

→ If we do not override service() method in our class then our class becomes an abstract class.

→ If our servlet class is an abstract then container doesn't create any object for our servlet class. so we must override service() method in our class.

public abstract class GenericServlet implements Servlet, ServletConfig

{

	given	implemented	abstract
interface Servlet	5	4	1

```
public class MyServlet extends GenericServlet
{
    public void service(ServletRequest request, ServletResponse response)
                    throws ServletException, IOException
    {
        ---------
        ---------
    }
}
```

→ while extending our servlet class from GenericServlet, the life cycle methods init() and destroy() methods are optional to override but service() method is mandatory to override in our class.

Q: Why service() method is made as abstract method in the GenericServlet class?

Ans: service() method is an important life cycle method, which is executed for each request

→ if a programmer wants to provide business logic in a servlet then the

programmer need one method in the class.
so, the servlet API developers made service()
method as abstract and informed that
the servlet programmer must override this
service() method to implement the business logic
required.

→ if the API developers only given service()
method implementation also then the servlet
programmer doesn't have any methods to
implement his business logic. so, they made
service() method as abstract.

Printing response on to a browser(Client):-

→ while writing a servlet class, after processing is
completed the servlet must print its response
on to a client browser.

→ if we write System.out.println() statement
in a servlet then it will be printed on
server console, but not on to the client browser.

→ If we want to print any response on to the browser,
we should store the response data into response

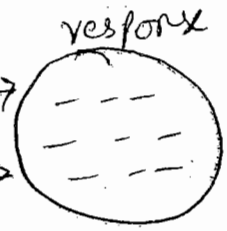
→ In order to write the data into response object, we need a stream object called printWriter.

→ The PrintWriter object will be given to a servlet programmer by response object only. so we need to get printwriter object through response object in our servlet by using the following statement.

PrintWriter pw = response.getWriter();

→ if we write any output using PrintWriter object then it will be stored into response object.

pw.println("---");
pw.println("---");

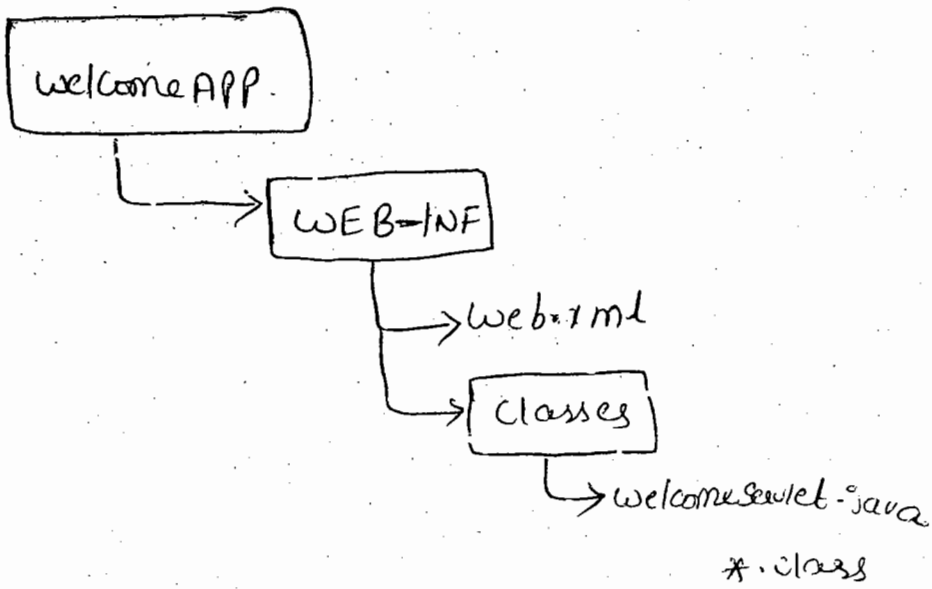


→ The web container will receive response object from our servlet and provides the response data to the server.

→ The server constructs / creates a web page with the data given by the container and then the server will send that web page back to a client browser.

Example1:

The following web application is for printing a welcome message along with system date and time on to the client browser.




```
//welcomeServlet.java
import javax.servlet.*;
import java.io.*;
import java.util.*;
public class welcomeServlet extends GenericServlet
{
    public void service(ServletRequest request, ServletResponse
                        response) throws ServletException, IOException
    {
        Date d=new Date();
        PrintWriter pw=response.getWriter();
        pw.println("Date & time:"+d.toString());
```

```
        pw.println("welcome to servlets");
        pw.close();
    }
}
```

Deployment Descriptor file (web.xml):-

→ For each web application, there must be one web.xml file is required.

→ whenever we deploy our web application into a server then the web container first reads web.xml file immediately.

→ Each servlet created by the programmer must be configured into web.xml file.

→ If we do not configure a servlet in web.xml file then container doesn't create an object for that servlet class

→ while configuring a servlet in web.xml, we should provide three names for it.

1) Original class name (fully qualified class name)
2) Logical name & alias name & dummy name

→ The logical name will be used by the container as object name. It means object name will be provided by the programmer and object of the class will be created by the container.

→ we call logical name also as an instance names of servlet created by the container.

→ The web container identifies whether a client request is given to the servlet class or not, by using url pattern provided for the servlet in web.xml

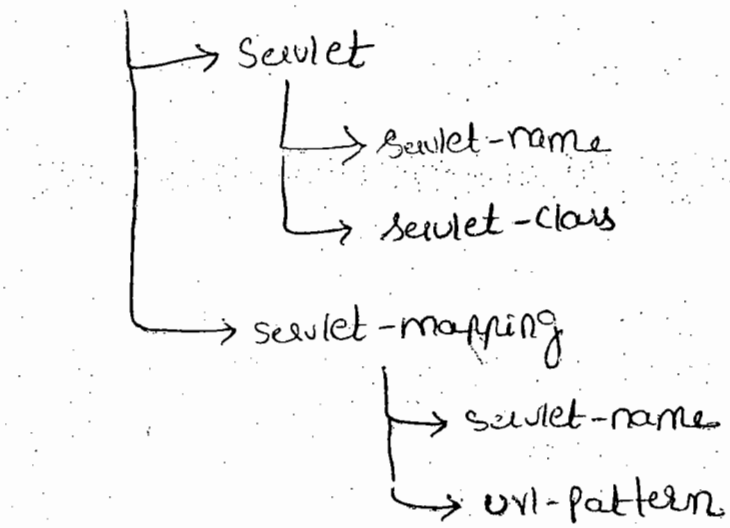
→ In order to configure a servlet in web.xml file, we need to use the following two elements (tags)

1) <servlet>

2) <servlet-mapping>

→ The root Element of web.xml is <web-app> and we should write the above two elements inside the root Element.

web.xml



Note:

The comments in html, jsp, xml all are same

web.xml

```
<!-- web.xml -->  ---> comment

<web-app>
    <servlet>
      <servlet-name> welcomes </servlet-name>
      <servlet-class> WelcomeServlet </servlet-class>
    </servlet>
    <servlet-mapping>
      <servlet-name> welcomes </servlet-name>
      <url-pattern> /srv1 </url-pattern>
    </servlet-mapping>
</web-app>
```

compiling a servlet :-

→ In case of JEE/J2EE technologies, the software is nothing but one or more jar files

→ For servlet & jsp & EJB & any frame work, software will be in the form of jar files.

→ For Technologies like servlet, jsp and EJB, the software will be provided by sever vendors. It means we will get jar files along with a sever software.

→ In Case of Tomcat sever, servlet software is servlet-api.jar

→ Before we compile our servlet, we need to set the servlet-api.jar file in the class path.

```
D:\welcomeAPP\WEB-INF\classes> set classpath=
        C:\Tomcat 6.0\lib\servlet-api.jar;%
        classpath%
```

↵

```
D:\welcomeAPP\WEB-INF\classes> javac WelcomeServlet.java
```

Deployment into TOMCat :-

→ Copy the web application root directory (welcomeAPP) into C:\TOmcat 6.0\webapps folder (Hard deployment)

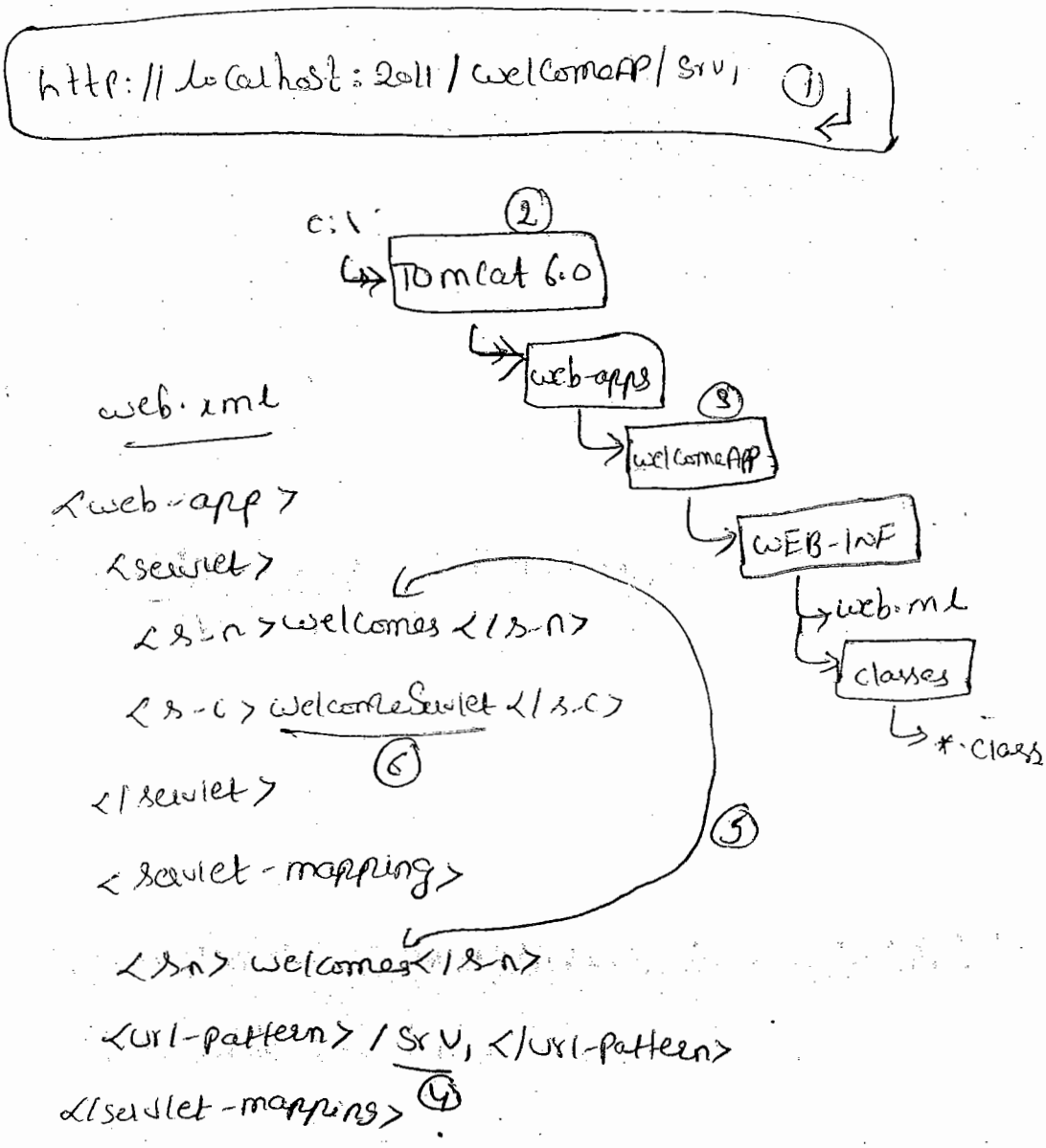
Starting the server:-

Double click on Tomcat 6.exe available at c:\Tomcat6.0\bin folder

sending a request to servlet:-

→ Open the browser and type the following address.

http://localhost:2011/welcomeAPP/srv,



Q: Can we put multiple url patterns to a single servlet class or not?

A: yes, possible. In web.xml file, we have to repeat <servlet-mapping> tag for multiple times

→ For example, if we want to put multiple url patterns to the above servlet then in web.xml we should add the following <servlet-mapping> elements.

```
<servlet-mapping>
  <servlet-name> welcomes </servlet-name>
  <url-pattern> /srv2 </url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name> welcomes </servlet-name>
  <url-pattern> /srv3 </url-pattern>
</servlet-mapping>
```

Adding parameters to a request from browser

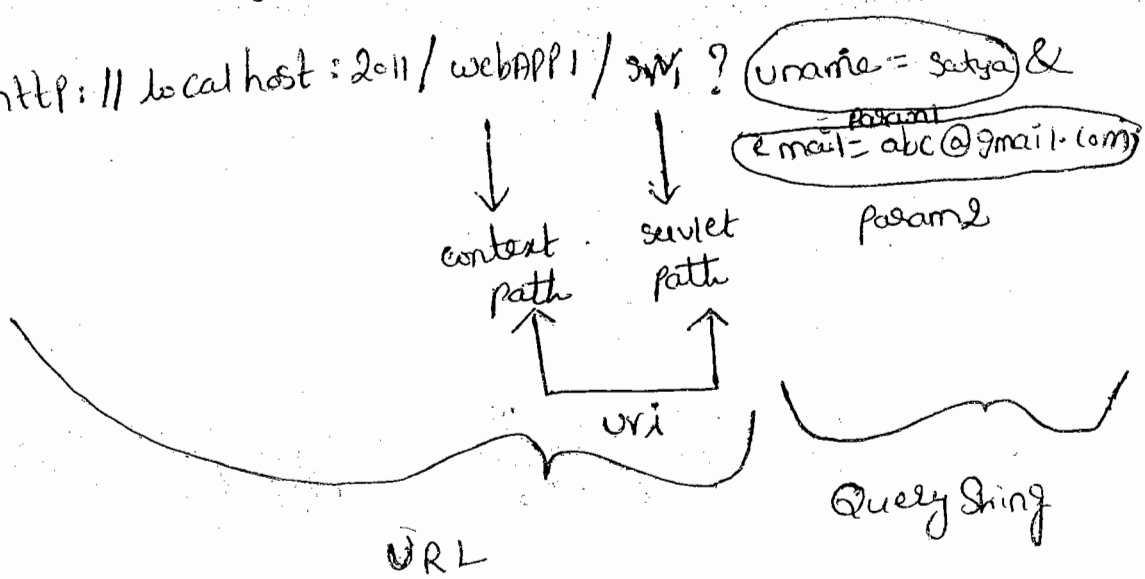
→ while sending a request to a servlet from browser, along with request we can also pass additional parameters to the url of

the servlet

→ If we want to add any parameters to the url in the address bar, we need to seperate the url and the parameters by putting "?" symbol

→ If we want to add any parameters to the url in address bar then we call it is a query string

→ we can add multiple parameters also to the url in address bar. In this case, we need to seperate one parameter and another parameter by putting '&' symbol



→ when we pass any parameters to a servlet from the browser, then container will store all the given parameters in request object

→ If a servlet wants to read the input values given by the client then in

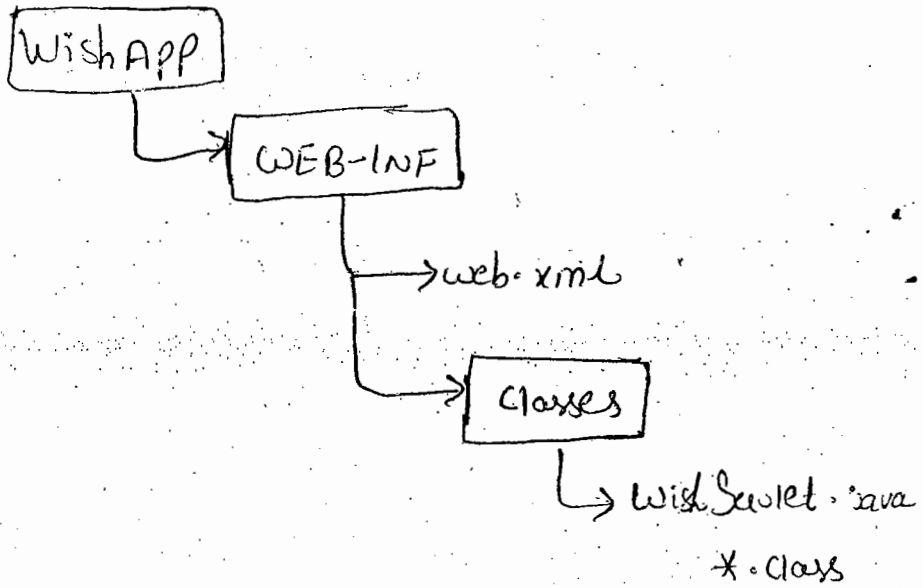
service() method of ,
input values by calling getParameter() method given by ServletRequest interface.

→ In service() method of a servlet, we need the following code for receiving the values of uname and email parameters.

```
public void service(ServletRequest request, ServletResponse response)
                    throws SE, IOE
{
    String s1 = request.getParameter("uname");
    String s2 = request.getParameter("email");
    ---
    ---
    ---
}
```

Ex:-2

The following web application accepts username parameter from the browser along with request to servlet and prints some welcome message for the given user name.




```
// WishServlet.java
import javax.servlet.*;
import java.io.*;
import java.util.*;
public class WishServlet extends GenericServlet
{
    public void service(ServletRequest req, ServletResponse res)
                        throws ServletException, IOException
    {
        String user = req.getParameter("username");
        Calendar c = Calendar.getInstance();
        int h = c.get(Calendar.HOUR_OF_DAY);
        PrintWriter pw = res.getWriter();
        if(h >= 6 && h <= 12)
        {
            pw.println("<font color='green' size=8>");
```

```
        pw.println("</font>");
    }
    else if(h>=13 && h<=18)
    {
        pw.println("<font color='blue' size=8>");
        pw.println("Good Afternoon:"+user);
        pw.println("</font>");
    }
    else
    {
        pw.println("<font color='red' size=8>");
        pw.println("Good night:"+user);
        pw.println("</font>");
    }
    pw.close();
}// service..
}// class..
```

web.xml

```
<!-- web.xml -->
<web-app>
    <servlet>
        <servlet-name>wishes</servlet-name>
        <servlet-class>WishServlet</servlet-class>
```

01

```
<servlet-mapping>
    <servlet-name> wishes </servlet-name>
    <url-pattern> /srv2 </url-pattern>
</servlet-mapping>
</web-app>
```

→ In the above servlet class, we imported java.util package, because of Calender class.

→ Calender is an abstract class, so we cannot create an object with new keyword.

→ To get a reference of Calender class, we called a factory() method called getInstance()

→ A factory() method means it is a method of class returns an object.

→ In java we have two types of factory methods

1) static factory() method
2) Instance factory() method.

→ compile the above servlet and deploy the web application into tomcat

→ Start the tomcat server

→ open the browser and type the following

http://localhost:2011/wishlist/2...

O/P:

Good night : Sweet

White ~~?~~

29/10/11 Tomcat Server

→ while installing Tomcat server by default the server will be installed at a location

C:\ program files \ Apache Software Fundation \ Tomcat 6.0

→ while installing the server, we can change the location (for Example, C:\ Tomcat 6.0)

→ when we install the tomcat server then it will be by default installed with port no 8080. if the system has Database, Oracle also then Oracle http server also runs on 8080 port number. so there is a clash between the two port numbers.

→ If we send a request with port no: 8080 then the request will be given to the Oracle http server, but not to the Tomcat server. so we need to change the Tomcat server port number.

→ If we want to change the Tomcat server port number then we need to open server.xml file available at

C:\ Tomcat 6.0 \ conf folder

→ In Server.xml, select & go to <Connector> element (line no: 69) and change the port number.

<Connector port="2011" Protocol="HTTP/1.1"

→ If already server is started then we need to shut-down and restart the server again.

Starting the Tomcat server:

Three ways:

way 1:

Start → programs → ApacheTomcat6.0 → configure Tomcat → (click)

General → Start / stop.

way 2:

Start → programs → ApacheTomcat6.0 → Monitor Tomcat →

In the system tray of task bar, right click on Tomcat Server icon → start service

way 3:

Double click on Tomcat6.Exe available at

C:\Tomcat6.0\bin folder

Note:

→ The third approach is the standard approach to start the Tomcat server.

→ In Tomcat server, catalina is called a servlet container and jasper is called a jsp engine.

Note:

Apache web server is different from Tomcat web server →

→ The implementation classes provided by Tomcat server for the servlet API are available at catalina.jar

ServletRequest and ServletResponse:-

→ In the service() method of a servlet, there are two parameters and those are references of ServletRequest and ServletResponse interfaces.

→ Both ServletRequest and ServletResponse interfaces are interfaces given in 'javax.servlet package.

→ when a client request is given then the container creates implementation classes objects of ServletRequest and ServletResponse interfaces. container will pass those objects method and we receive into

the interface references.

→ If we directly write the class names in the service() method, instance of interface names then our servlet becomes server dependent. so we use interface names, to make our servlet as server independent.

→ If we want to print the implementation class names of ServletRequest and ServletResponse on to the browser then we need the following statements in the service() method.

```
Pw.println("ServletRequest(i) implementation class is:"+
                request.getClass().getName());
Pw.println("ServletResponse(i) implementation class is:"+
                response.getClass().getName());
```

<load-on-startup> element:-

→ By default a web container creates an object for servlet, whenever first request is given to it.

→ For first client the response will be given slow. because the client has to wait until servlet object is created and initialized.

→ For second request onwards, the response will be faster than first request. In order to eliminate this difference, we need to inform the container that, create servlet object before first request is given. For this we need to add <load-on-start up> element into the web.xml file (DD file).

→ If we add <load-on-startup> Element then the container creates servlet object, whenever our web application is deployed & whenever the server is started.

→ <load-on-startup> Element should be included inside <Servlet> Element.

→ For Example,

```
<servlet>
  <servlet-name> welcome </servlet-name>
  <servlet-class> WelcomeServlet </servlet-class>
  <load-on-startup> 1 </load-on-startup>
</servlet>
```

→ <load-on-startup> behavior will be like the following:

a) The value must be a positive integer

b) Range of <load-on-startup> is 0 to 65,535 (it is server dependent)

c) low value gets high priority

d) high value gets low priority

e) if same priority given for multiple servlets then the order will be decided by the container

** f) if we give a negative value then the container ignores load on startup and creates a servlet object when first request is given. but container doesn't throw any exception.

Note:

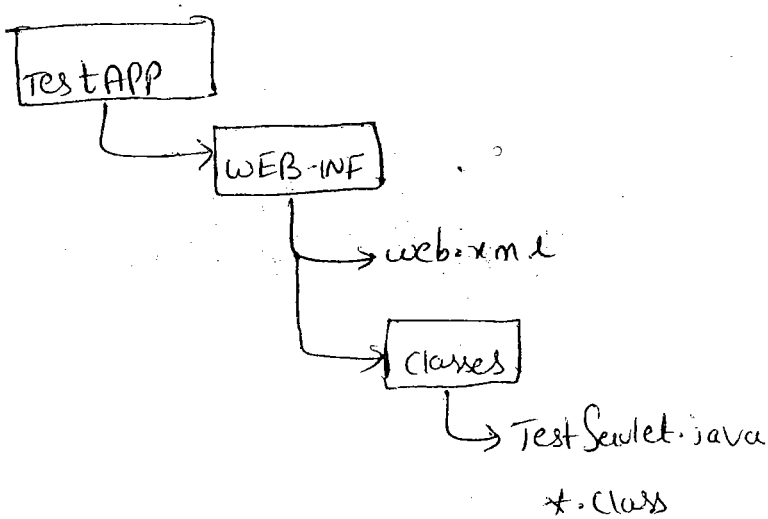
→ up to Tomcat 5.5, if we provide load on startup value as 0 then container provided least priority and that servlet object is created after creation of other servlet object with <load-on-startup>. but from some Tomcat 6.0 for <load-on-startup> value 0, the container provides highest priority to create object.

servlets	<load-on-startup>	Tomcat 5.5 order	Tomcat 6.0 order
servlet 1	10	2	3
servlet 2	5	1	2
servlet 3	-1 (ignores)	lt(when request is given)	when request is given
servlet	0	3	1

30/07/11

Ex:-3

→ The following web application is to test load on startup of a servlet.



```
//TestServlet.java
import javax.servlet.*;
import java.io.*;
public class TestServlet extends GenericServlet
{
    public void init(ServletConfig config) throws ServletException
    {
        System.out.println("init method called...");
    }//init()

    public void service(ServletRequest request, ServletResponse response)
                throws ServletException, IOException
    {
        System.out.println("service method called....");
        PrintWriter pw=response.getWriter();
        pw.println("<h2>"+this+"</h2>");
        pw.close();
    }//service()
    public void destroy()
    {
        System.out.println("destroy method called...");
    }//destroy()
}
```

web.xml:

```
<!--web.xml-->
<web-app>
    <servlet>
        <servlet-name>Test</servlet-name>
        <servlet-class>TestServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>Test</servlet-name>
        <url-pattern>/testSrv</url-pattern>
    </servlet-mapping>
</web-app>
```

→ In web.xml file, <servlet-mapping> tag is fixed. It means we cannot add any additional element into this tag.

→ compile the above servlet, and deploy the web application into the server.

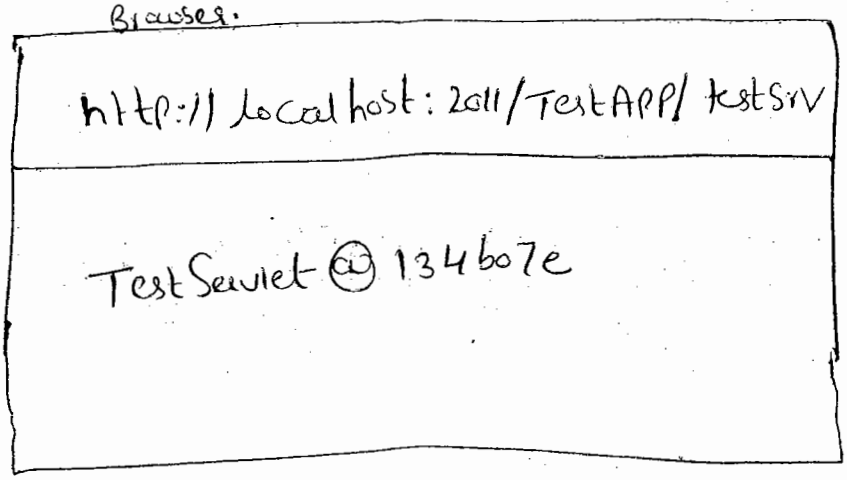
→ start the server and then observe the server console, in that we will get an S.o.p message init method called, before any request is given to the servlet. because

we have added <load-on-startup> tag in web.xml.

→ After sending a request, if we observe the console then we can find service() method called statement/message on the console.

→ If we send multiple times request from the browser then the service() method called statement will be printed for multiple times on the console.

→ Go to webapps folder of the Tomcat server and delete the web application (TestAPP) then on server console we can find a statement destroy() method called.



Q:

How can we prove that only one object is created for a servlet.

Ans:

we can prove by printing the memory block address of the servlet object on to the browser/console.

→ To print the memory block address, in service() method of servlet we need to add the following statement.

```
pw.println(this);
```

Q: why each servlet class must be public?

Ans: For a servlet, an object is created by the web container. If the container wants to create an object then the servlet class must be visible to the container.

→ To make our servlet class as visible to the container, our servlet class must be public

Q: How container is creating an object of a servlet?

Ans:
container first loads the servlet class and then calls newInstance() method for creating an object of a servlet.

```
TestServlet test = (TestServlet) class.forName("TestServlet").
                                    newInstance();
```

Q:
can I write an user defined method in a servlet class or not?

Ans: It is possible to write user defined methods in a servlet class. but container doesn't call the user defined method. because user defined method is not a life cycle method.

→ user defined method can be called from life cycle methods by the programmer

```
public class TestServlet extends GenericServlet
{
  public void service(ServletRequest req, ServletResponse res)
  {
  }
  m1();
}
```

public void m1()
{

}

Q: can we write main method in a servlet class or not?

Ans: Yes, because main method is not a life cycle method, so container doesn't call the method.

→ According to servlet Specification, main method is not recommended to place in a servlet class.

Q: Can we place a parameterized constructor in a servlet class or not?

Ans: In each servlet class, default constructor / no argument constructor is mandatory. if we do not add any constructor in the class then, while compiling the java class & servlet class, the compiler automatically adds default constructor

→ To select multiple options in a list box, we need to hold control key.

→ Each option need a value and whenever we submit the form then the option values will be submitted to server (Servlet).

```
// BookServlet.java

import javax.servlet.*;
import java.io.*;
public class BookServlet Extends GenericServlet
{
    public void service(ServletRequest req, ServletResponse res)
                    throws ServletException, IOException
    {
        double price=0;
        String user = req.getParameter("uname");
        String books[] = req.getParameterValues("book");
        for(int i=0; i<books.length; i++)
        {
            if(books[i].Equals("java"))
                price = price+250;
            if(books[i].equals("net"))
```

```
if(books[i].equals("oracle"))
    price = price+150;
    if(books[i].equals("cpp"))
        price = price+100;
}
PrintWriter pw = res.getWriter();
pw.println("<font color='blue'>");
pw.println("username:"+user);
pw.println("<br>");
pw.println("selected books:");
for(int i=0; i<books.length; i++)
    pw.println(books[i]+" ");
    pw.println("<br>");
    pw.println("</font>");
    pw.close();
}
}
```

15/8/11:-

working with multiple submit buttons in a Form:-

(i) while designing Html form to accepts input value from the client, we can place multiple submit buttons for multiple operations required. and we can send a request to a servlet. whenever the submit button is clicked.

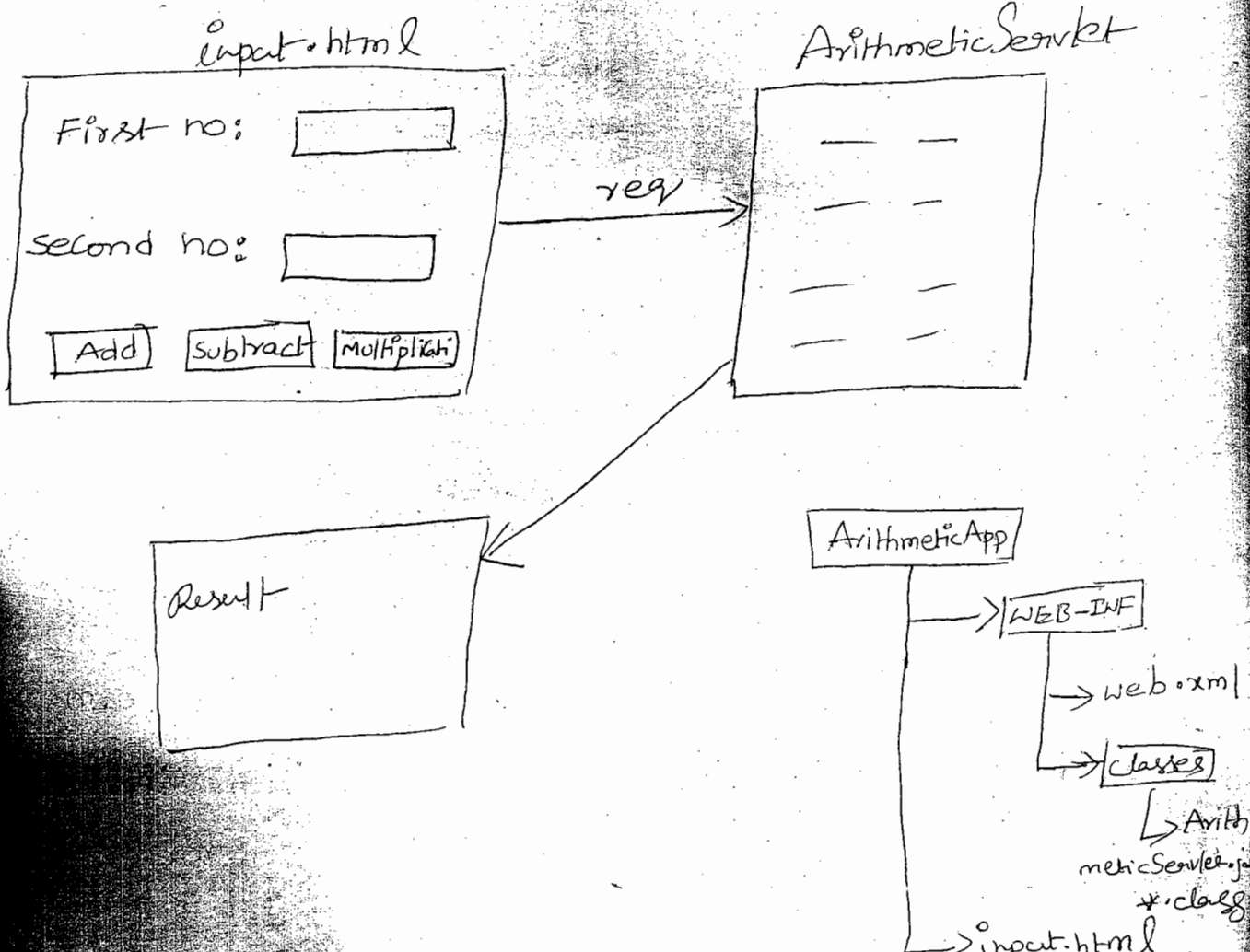
(ii) Eventhough we place multiple submit buttons in a form, but it is possible to click on single submit button at a time.

(iii) The reason is, to a servlet, If one request is send, we need to wait for response. we call this Communication as a Synchronous Communication.

In ASynchronous Communication we can send next request to a server, without getting response to the previous request. this is possible. by using AJAX (ASynchronous Java-Script & XML) Technology.

... to send a request to the servlet, ... must use same name for all submit buttons creations.

... At servlet we should read the value of the ~~servlet~~ submit button using getParameter() and through conditions we need to develop the logic in a servlet.



<!-- input.html -->

<Body>

<form action="Srv1">

First no: <input type=text name="fno">

Second no: <input type=text name="sno">

<input type=submit value="Add"

name="s1">

    (non-breakable space)

<input type=Submit value="sub"

name="s1">

  nbsp;

<input type=Submit value="mul"

name="s1">

</form>

</Body>

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
class ArithmeticServlet extends HttpServlet
{
public void doGet(HttpServletRequest req,
                  HttpServletResponse res)
                  throws SE, IOE
{
String str1 = req.getParameter("fno");
String str2 = req.getParameter("sno");
int a = Integer.parseInt(str1.trim());
int b = Integer.parseInt(str2.trim());
String str = request.getParameter("s1")
response.setContentType("text/html");
PrintWriter pw = res.getWriter();
if("Add".equals(str))
{
pw.println("Addition: "+(a+b));
}
```

```
    else
    {
      pw.println("multiplication":(a*b));
    }
    pw.close();
    }
  }
```

web.xml

```
<web-app>
  <servlet>
    <servlet-name>dummy</servlet-name>
    <servlet-class>ArithmeticServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>dummy</servlet-name>
    <url-pattern>/srv</url-pattern>
  </servlet-mapping>
</web-app>
```

1) "Add".equals(str3); ⟶ Best one

2) str3.equals("Add");

if str3 = NULL, There is a chance of getting NULL pointer exception. in 2

→ There is chance of getting NULLpointer exception

=> Using Multipe Hyperlinks in a page

If we want to put multiple hyperlinks to Communicate with a Servlet then we need to pass a parameter to a Servlet along with the request, the name of the parameter should be same, for all hyperlinks. when we click on hyperlink automatically, the form data is not submited. because hyperlink is not a form element.

=> We can send static values to the Servlet by adding query string.

 Add

 Sub

 Mul

ServletConfig Interface

--> ServletConfig is an interface given in javax.servlet package and the container will create the implementation class object internally.

-> The webContainer creates ServletConfig object, at just before calling the life-cycle init() method of a Servlet.

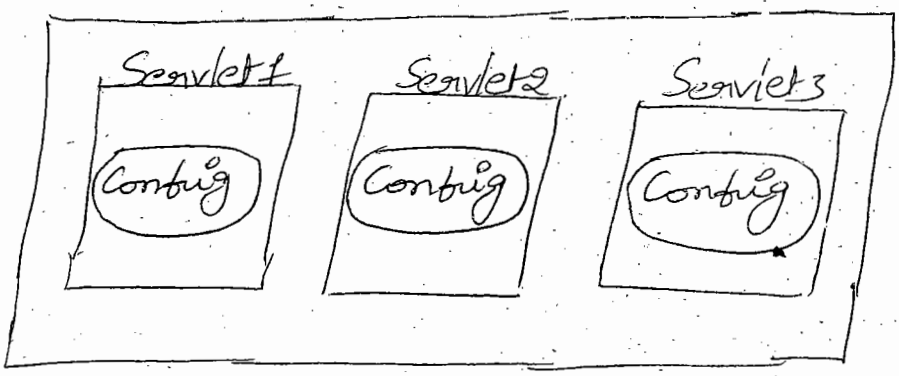
-> The web-container creates ServletConfig object, in between execution of the constructor and the life-cycle init() method.

=> Container first create Servlet~~Config~~ object and after that it creates servlet+ config object.

-> The WebContainer creates a ServletConfig object for each Servlet seperately, it means ServletConfig is local to each Servlet.

-> If a web application has 3 servlets then Container

Container
↓ Creates
Servlet object
↓ Calls
defaultConstructor
↓ Create
ServletConfig object
↓ Calls
init (ServletConfig config)



→ The data stored in servlet config object of the one servlet can not be accessed by another servlet because ServletConfig object local object.

⇒ The webContainer will store the following three types of information into ServletConfig object.

(i) Logical name of servlet
(ii) init parameters of servlet
(iii) ServletContext reference.

→ A webContainer will collect logical, name of servlet, init parameters of servlet from web.xml file.

→ when we deploy our application into a server or when the servlet is started then the container creates ~~Config~~ ServletConfig object immediatly, whenever Config object is created then container adds its reference

1) logical name of the servlet
2) init parameters of servlet
3) ServletContext interface.

* what is the diff b/w Constructor & init() method of a servlet?

A:- In Constructor of a servlet we can not use ServletConfig object, because ServletConfig object is created by the container after calling the Constructor.

→ In the init() method of servlet we can access the data stored in a ServletConfig object.

⇒ init parameters are passed to a servlet from web.xml file, by adding <init-param> tag under <servlet-tag>.

→ one parameter will be in the form of name/value pair & we can pass any number of init parameters to the servlet from the web.xml file.

Generally we use init parameters in web.xml to pass technical values to a servlet.

→ For example, we can pass jdbc connection properties from web.xml to a servlet.

The advantage of passing init parameters from web.xml file to a servlet is, it is possible to modify the values easily or change the values easily by without doing the modifications at servlet, because if we modify the servlet code, then we need to recompile, reload & restart the server,

→ For example if we pass driver properties from XML file, then it will be easy to change from one driver to another driver or even from one database to another database.

→ If we make any changes in xml file then no need of recompiling, redeploying, restarting of web application.

web.xml

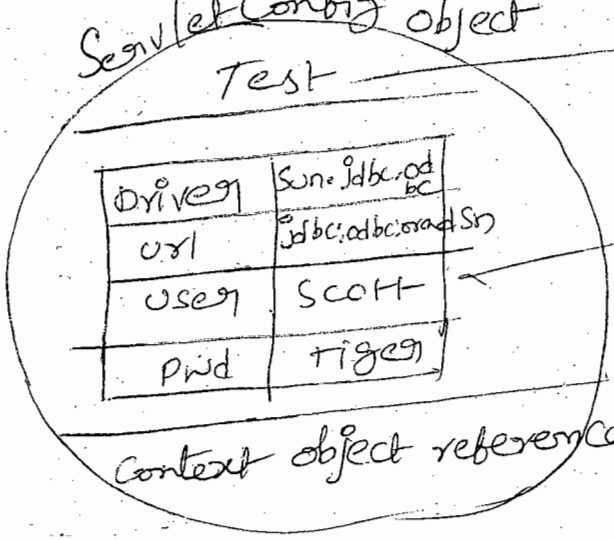
```
<web-app>
  <servlet>
    <s-n> Test </s-n>
    <s-c> TestServlet </s-c>
  <init-param>

<param-name>driver </param-name>
<param-value> son.jdbc.odbc.jdbcOdbcDriver

</init-param>                    </param-value>

  <i-p>
    <p-n> url </p-n>
    <p-v> jdbc:odbc:oradsn </p-v>
  </i-p>
  <i-p>
      <p-n> user </p-n>
      <p-v> scott </p-v>
  </i-p>
  <i-p>
      <p-n> pwd </p-n>
      <p-v> tiger </p-v>
</servlets>
                    <s-n> Test </s-n>
```

→ For the above servlet, the web container stores the following information into ServletConfig object.



⇒ Different ways of obtaining ServletConfig object into Service method?

→ A WebContainer will pass ServletConfig object to init() method of a servlet.

⇒ But we use the data of Config object in service method mostly.

→ In a servlet we implement most of the business logic into service method.

→ So we need ServletConfig object also sometimes into the Service method. it is primary Responsibility to get servlet-Config object into service method. and

→ As a Programer, we can obtain ServletConfig object into ~~servlet~~ service() method by following any one the following 3 approaches.

1. Approach

→ In this approach, we override life-cycle init() method in our Servlet.

→ Container will pass ServletConfig object by calling init() method of our Servlet class.

→ our Servlet class receives ServletConfig object into local variable. by using parameters of init() method, we get the Config object

→ To use ServletConfig object in service() method we need to store local ServletConfig variable into global ServletConfig variable. so that we can use that global object into service() method.

for example:

```
P C  TestServlet extends GS
{
    PrivateServletConfig gConfig;
P V  init(ServletConfig config) throws SE
{
```

```
}
public void service(req,res) throws SE,IOE
{
    // use getConfig obj here.
}
}
```

Date: 17/8/11

2) Approach-II

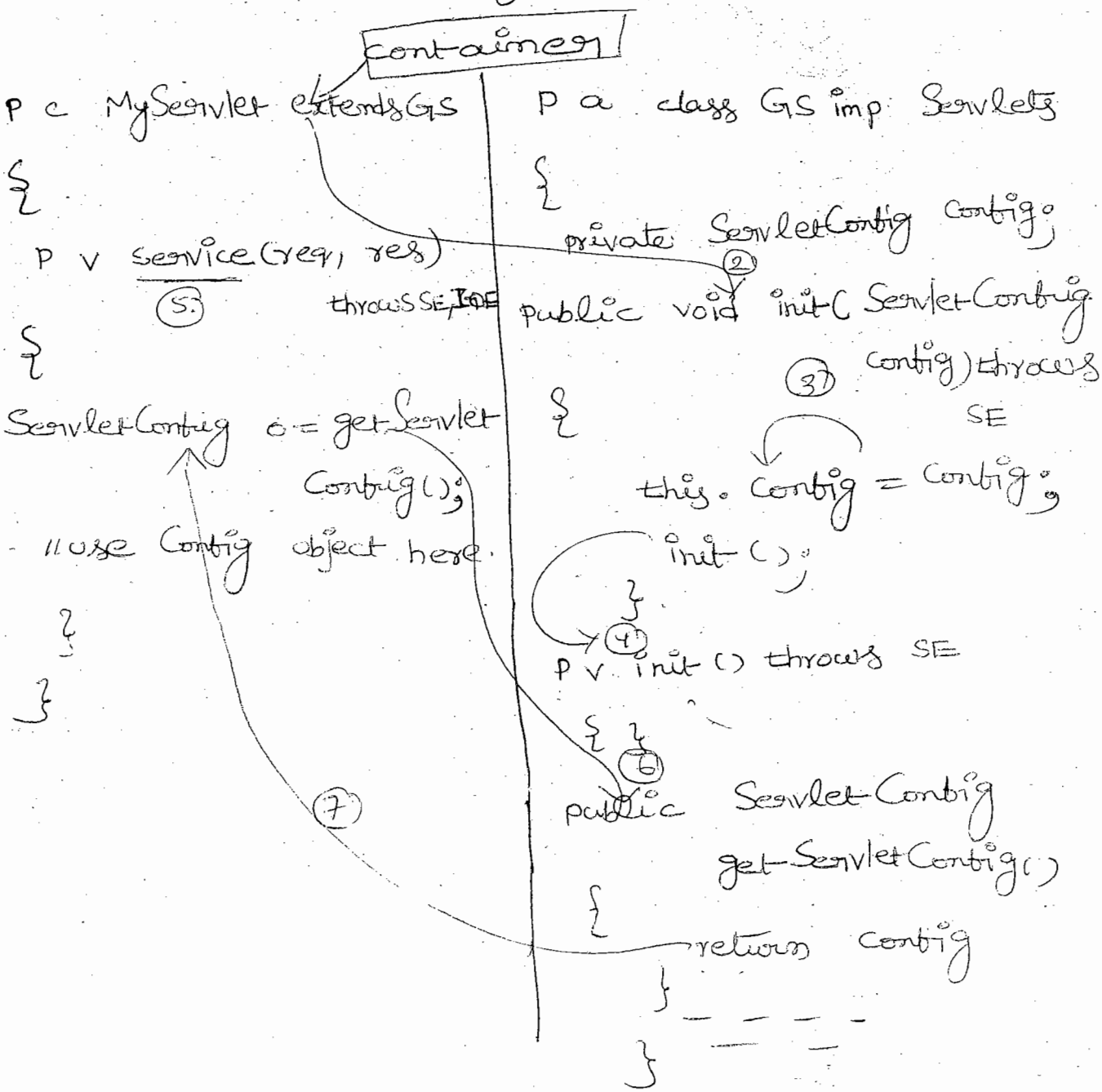
-> In this approach we do not override init() of lifecycle init() in our class.

-> If we dont override init() in our class then container calls base class init() by passing ServletConfig object as a parameter.

-> The base class init() will store the local servlet object into a global servlet config object and it is of private type

-> we can't use the private variable of the base class in the derived class directly. So we can't use config object directly in our service() method.

called getServletConfig() and we call that method to get ServletConfig object into our service().



⇒ In java, we can call a method directly by without using classname (or) object name, provided if both methods are available in same class (or) if method is available in the base class.

=> In the above approach, we are calling getServletConfig() in our Servlet class directly, because it is provided by the base class.

=> GenericServlet class implemented from Servlet interface. so in GenericServlet, there are five methods which are given by servlet interface are available.

=> GenericServlet class implemented four methods given by Servlet interface, but one method is not implemented i.e., service(). so GenericServlet is an abstract class

=> Servlet interface has provided the following five methods

(i) init(ServletConfig)

(ii) service(request, response)

(iii) destroy()

(iv) getServletConfig()

(v) getServletInfo().

3) Approach III

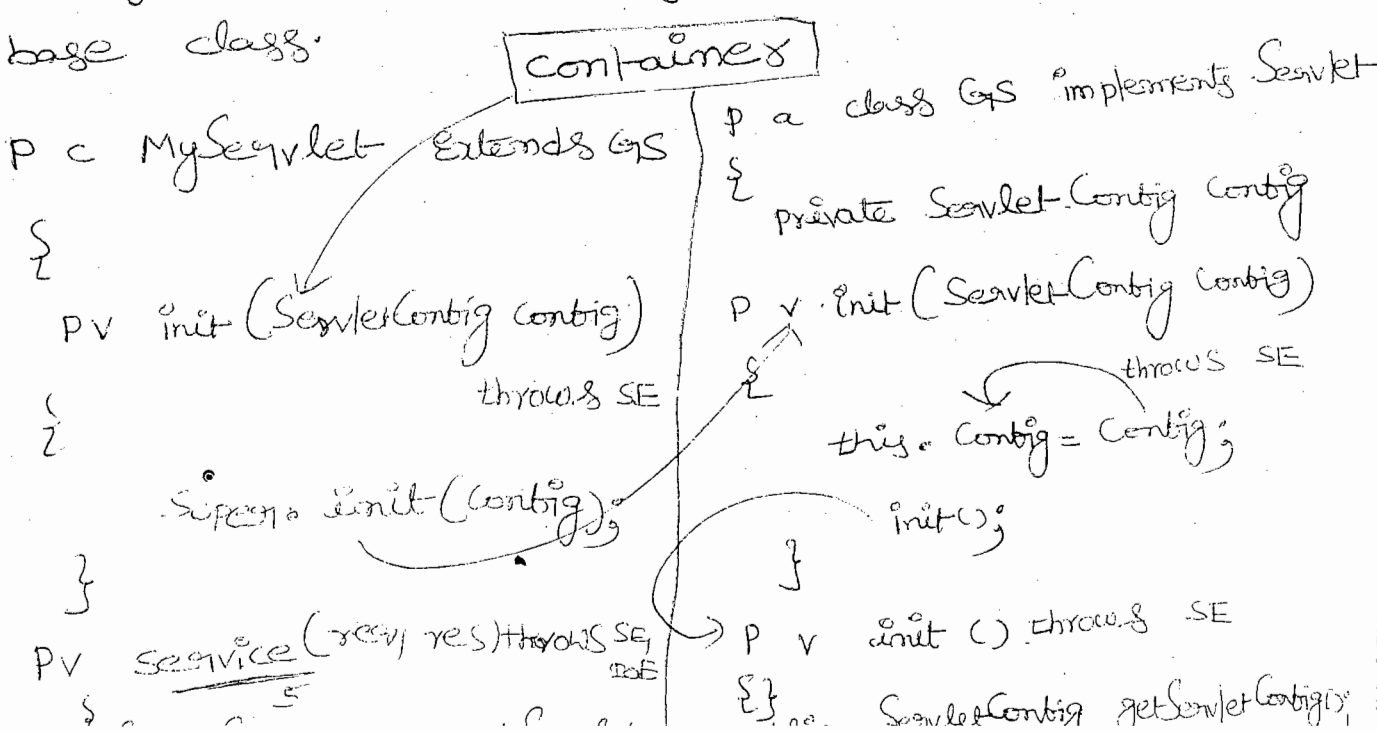
→ In this approach, we override lifecycle init() in our class.

→ If we override lifecycle init() in our class then container calls our class init() methods only.

→ Because of Thread safety reasons, we are not interested to store local variable into global variable.

→ In this case we need to call base class init() directly from our class init() by using super.init(Config).

→ In to the service() we get the servlet Config object, by calling getServletConfig() of the base class.



Methods to read data from ServletConfig object

1) getServletName()

-> This method is used for reading logical name of a servlet from ServletConfig object.

-> This method returns String.

```
String str = Config.getServletName();
```

2) getInitParameter()

-> This method is used to read the value of Init parameter () stored in the Config object.

=> while reading the init parameter, we will get value in String type and it required them we need wrapped into the primitive types.

```
String v1 = Config.getInitParameter("P1");
```

3) getInitParameterNames()

-> This Method is used to read all init parameter names into a "legacy collection"

It is used, when there are large no of init parameters exists.

reference and we need a loop to read the collection

```
Enumeration e = Config.getInitParameterNames();
while(e.hasMoreElements())
{
    object o = e.nextElements();
    String k = (String)o;
    String v = Config.getInitParameter("k");
    ---
    ---
    ---
}
```

4) getServletContext():

→ This method is used to get context reference stored in the config object.

```
ServletContext ctx = Config.getServletContext();
```

ate
/08/2011

ServletContext interface :-

-> ServletContext is an interface given in javax.servlet.*; and Container creates its implementation class object internally, which is provided by the server.

-> when we deploy our web applns into a server, which is already started then immediately the webcontainer reads web.xml file and creates the ServletContext object immediately.

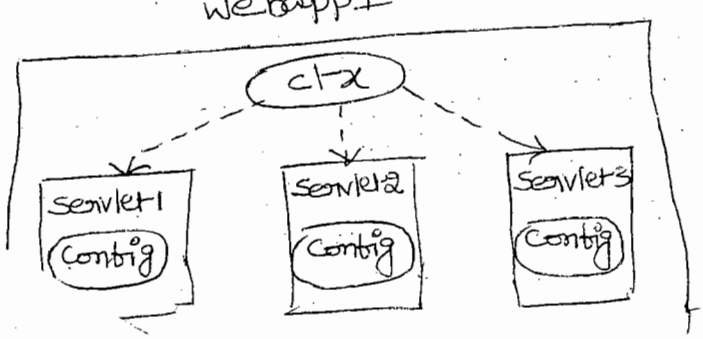
-> If we deploy our web application into a server, which is not yet started then whenever the server is started, then immediately the web container reads web.xml file from the web-application and creates immediately ServletContext objects.

-> The webcontainer creates one servletContext object per each web appln.

-> For a web appln, the first object created by the container is ServletContext object only.

-> we call ServletContext object has a global object bcz a webapplication.

-> In a webappln, each servlet has (or) contains its own ServletConfig object, but all servlets will share a single ServletContext object.



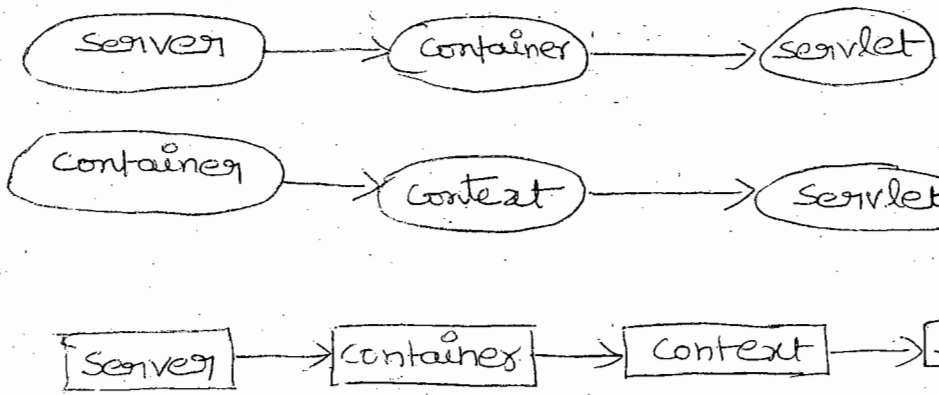
objects

1> Through ServletContext object, we can share some data across multiple Servlets of a webapplication.

2) Through ServletContext object, we can get the services which are provided by the container into a servlet.

⇒ servlet Context object acts as mediator between a webContainer and a servlet.

⇒ A webContainer will acts as mediator between a Server and a servlet.



⇒ The following is the sequence of the objects created by a container for a web-application.

1) ServletContext objects.
2) Servlet object
3) ServletConfig object
4) Request and Response object
5) Thread object.

→ Even though a web-appln doesn't contain any servlet and jsp's, but still ServletContext object will be Created by the Container.

To Create ServletContext object, a web-container

but not to servlets (or) jsp's.

IAQ

Into ServletContext object, the data can be stored in the form of key/value pairs for sharability across multiple files.

⇒ while storing the data in the form of key/value pairs into a ServletContext object, we have the following two approaches.

1) Declarative approach :-

→ In this approach, the context parameters are added into web.xml file.

→ Context parameters are global to all servlets, so we include context parameters at outside of ServletConfigurations.

→ To pass context parameters, we need an element called <Context_Param>.

web.xml

```
<web-app>
    <context-param>
        <param-name> PI </param-name>
        <param-value> 100 </param-value>
    </context-param>
    <servlet>
        <servl
```

file them at the time of Creating ServletContext object by the Container, the web-container will store the Context parameters into the Context object.

→ From web.xml, it is possible to add two types of parameters,

(i) Context Parameters.
(ii) init Parameters.

Qs what is diff b/w Context Parameter and init parameters?

A:- Context Parameters are passed at outside of the servlet tag and this are stored in Context-Object. but init parameters are passed at inside of Servlet tag and this are stored in ServletConfig objects.

⇒ Context Parameters are global to all Servlets and init parameters are local to each Servlets

Qs How many types of parameters exists in Servlet-App?

A:- Three types (i) Request Parameters
(ii) Context "
(iii) init "

Progamatic approach.

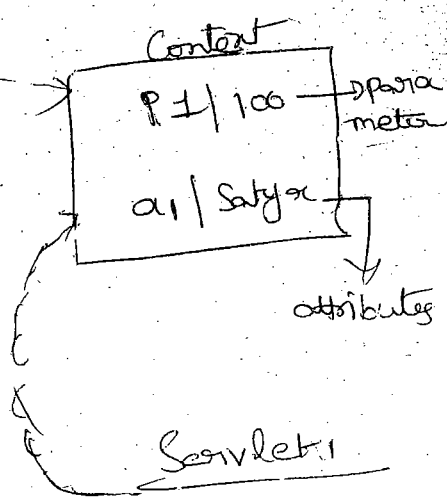
→ In this approach the values are added(or) stored from a Servlet or a JSP.

-> If we store the data from a servlet or jsp then we call it as "an attribute".

-> we can add attribute into a context object, to share the information from the one servlet (or) a jsp to other servlets (or) a jsp/s available in the webapplication

web.xml

```
<context-param>
    <param-name> P1 </P-n>
    <p-value> 100 </P-V>
</context-param>
```



ctx.SetAttribute ("a1", "Satya");

=> parameters can not modify the servlets. parameters are Thread Safe (only one thread act at a time).

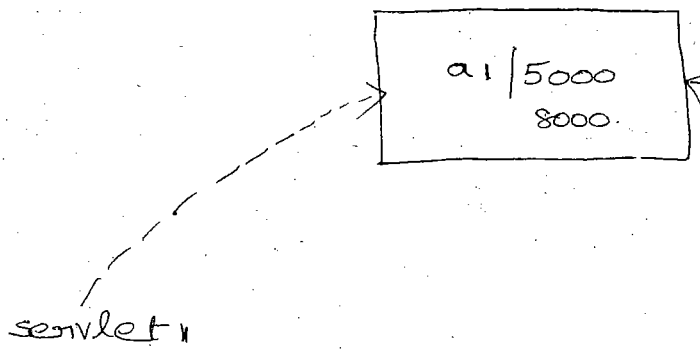
=> attributes can modify the servlets. Attribute are not Thread safe.

19/08/14

Q) what is the difference b/w a parameters and a Attribute?

Parameters	Attribute
-> parameters are passed from web.xml file declaratively	-> Attributes are passed from a servlet programatically.
-> In parameter, both key & value are of String type.	-> In an Attribute, key is a String and value is an object.
-> A parameters cannot be modify from a servlet. It means parameters are thread safe	-> An attribute can be modified from a servlet. So attributes are not thread safe

Ex:-



ctx.setAttribute("a1", newInteger(5000));
key value

ctx.setAttribute("a1" newInteger(8000);

-> If we want to store, read & remove then we have the following four methods.

1) setAttribute(K,V);
2) getAttribute(K);
3) removeAttribute(K);

⇒ while storing value must be an object.

⇒ while reading the value of an attribute by using its key then the value will be return in the form of an object.

⇒ For Example Object o = ctx.setAttribute("a1", o);

⇒ While reading multiple attribute at once, we use getAttributeNames() and it returns Enumeration

⇒ For example

```
Enumeration e = ctx.getAttributeNames();
```

⇒ In servlet API, there are 3 objects which are supporting attributes

(i) request object.
(ii) session object.
(iii) Context object.

⇒ Difference b/w ServletConfig and ServletContext

ServletConfig	ServletContext
1) Config object is a local object to a servlet.	1) Context object is a global object for all servlets.
2) Config object doesn't exists without a servletobject.	2) context object exists without a servlet object.
3) Config object doesn't store attributes.	3) context objects stores attributes also.
...	... context object we have

Note: If a web-app is an empty webapplication, then also ServletContext object will be created by the Container.

Obtaining ServletContext object into Service() method:—

1) ServletContext ctx = Config.getServletContext();

2) ServletContext ctx = getServletContext();

⇒ A webContainer doesn't pass ServletContext object directly to the servlet. It indirectly passes context object through Config object.

⇒ when init(life cycle) is called by the Container Config object will be passed as a parameter and it internally contains context object reference.

⇒ If we override init() in our servlet class and if you store local variable into a global config then we can get context object into service() by using the "first" syntax.

⇒ If we don't override init() (or) If we call back class init() from our class init() then we use second syntax to get ServletContext object.

Date 20/08/11 → In our servlet class, if we override init() and if we store the local servlet_Config variable into the global servletConfig variable then we follows syntax to get ServletConfig into service() method.

→ for example

```
p c MyServlets Extends GIS
{
private ServletConfig config;

p v init (ServletConfig config) throws SE
 {
    this.config = config;
 }

p v service( request, response) throws SE,
                                      IOE
{
  ServletContext ctx = config.getServletContext();
 }
}
```

=> If we are ~~not~~ overriding lifecycle init() are if we store the local servlet config variable into the global servletConfig variable then we follow syntax to get ServletConfig into service() method.

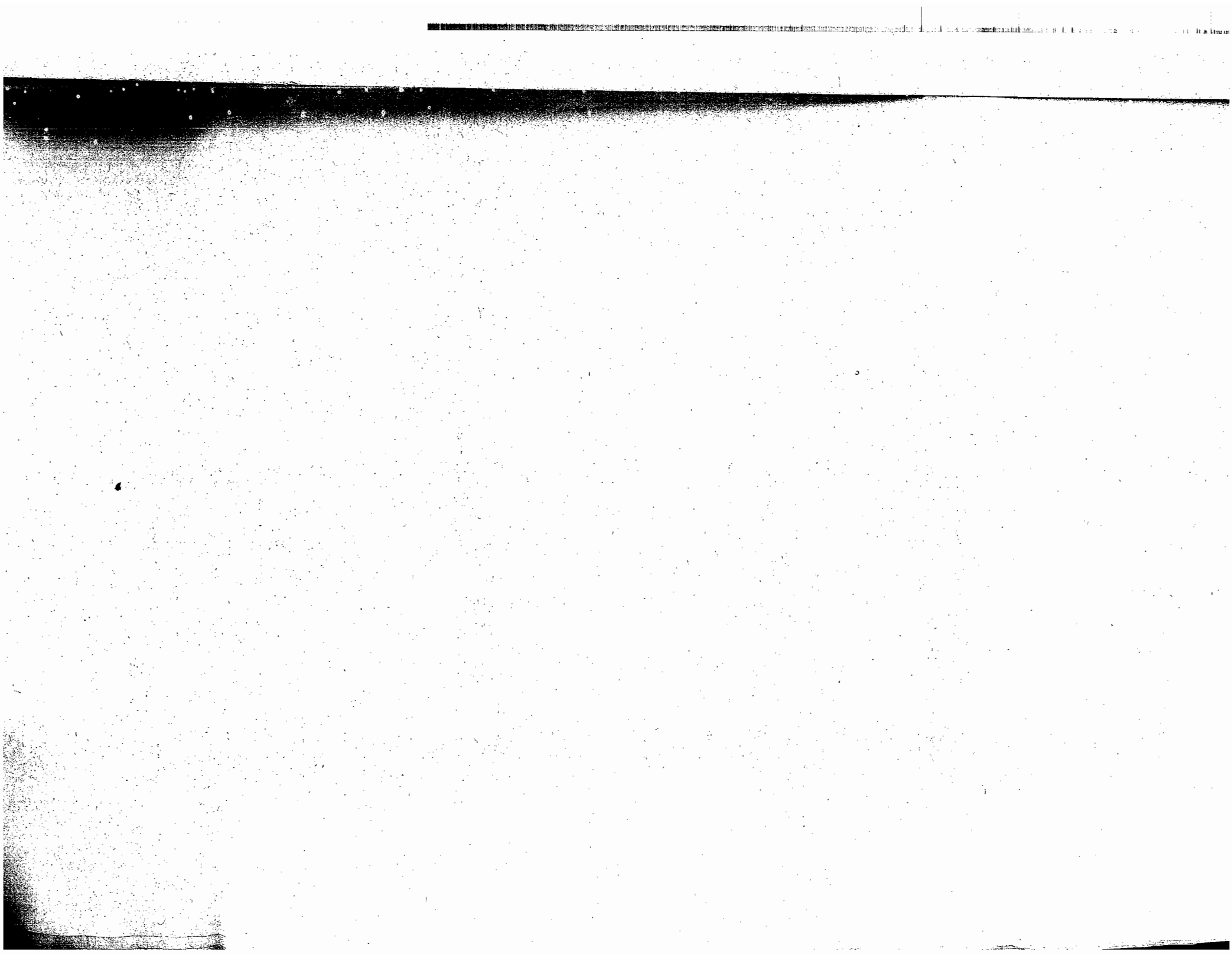
Ex:-

```
p c MyServlets Extends GIS
{
```

are calling base class init() from our class init() then we can use and syntax for getting

public class MyServlet Extends GnS

{



```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;
import org.apache.commons.beanutils.*;
public class newUserServlet extends HttpServlet
{
    List l;
    public void init(ServletConfig sc) throws
                                        SE
    {
        l=new ArrayList();
        super.init(sc);
    }
public void doGet(HttpServletRequest req,
                  HttpServletResponse res)
                  throws SE, IOE
{
    try
    {
    // read input and store in a map object
    Map m=req.getParameterMap();
    // Create pojo class object
    UserBean ob= new UserBean();
// copy input from map object into pojo obj
```

```
// add pojo object into list
        l.add(ub);
ServletContext ctx = getServletContext();
// store list object into context
        ctx.setAttribute("users", l);
PrintWriter pw = res.getWriter();
pw.println("new user added");
pw.close();
}
catch(Exception e)
{
  e.printStackTrace();
}
}
}
```

=> In the above servlet we have created ArrayList object for once by using init() method because we need an arraylist for once inorder to store all pojo class objects.

=> In the above servlet we have imported a new package called org.apache.commons.beanutils.*. Because

object into a pojo class object, directly by calling a static method of BeanUtils class called populate()

→ In order to copy the data from Map object into POJO class object, the keys of Map object and variablenames of pojo class, both must be same.

→ the pojo class object is added to ~~Arraylist~~ Arraylist object and finally Arraylist object is stored in ServletContext in the form of an attribute.

//UserBean.java (POJO class)

```
public class UserBean
{
    private String uname;
    private String email;
    private String[] cl;
    public void setUname(String uname)
    {
        this.uname=uname;
    }
    public String getUname()
    {
        return uname;
    }
    public void setEmail(String email)
    {
```

```
public String getEmail()
{
    return email;
}
public void setCl(String[] cl)
{
    this.cl=cl;
}
public String[] getCl()
{
    return cl;
}
}
```

ShowUsersServlet.java

```
import java.util.*;
import javax.servlet.*;
import java.io.*;
public class ShowUserServlet extends GS
{
    P V service(ServletRequest req, ServletResponse
                res) throws SE, IOE
    {
        ServletContext ctx = getServletContext();
        Object o = ctx.getAttribute("users");
```

```
pw.println("<body>");
pw.println("<Table border=2>");
Iterator it=l.iterator();
while(it.hasNext())
{
    Object o1=it.next();
    UseBean ub=(UseBean)o1;
    pw.println("<tr>");
pw.println("<td>"+ub.getUname()+"</td>");
pw.println("<td>"+ub.getEmail()+"</td>");
StringBuffer sb=new StringBuffer();
    String h[]=ub.getCi();
for(int i=0;i<h.length;i++)
    {
        sb.append(h[i]); // Append method is available
                         in StringBuffer.
        sb.append(",");
    }
pw.println("<td>"+sb.toString()+"</td>");
    pw.println("</tr>");
}
pw.println("</table></b>");
pw.close();
```

set classpath = c:\rootfolder\WEB-INF\...

2.1.0.jar; ;c:\classpath\;.

→ commons-loging 1.0.4 paste into lib folder it located tomcat lib.

Date
23/8/11

→ while compiling NewUserServlet.java, we will get errors, eventhough we have set servlet-api.jar in the classpath.

→ Because we have imported a package which is given by apache.

→ So, Apart from servlet-api.jar file, we need to set to additional jar files in the classpath.

1. Commons-beansUtils.jar
2. " -collections.jar.

→ the above two jar files can be downloaded seperately.

→ we need to copy the above two jar files and also Commons-logging.jar into lib folder of our web-application.

→ ** Adding jar-files to classpath, is for compiling our Servlets and adding jar files into lib folder is for Execution of

Weblogic :-

→ Hard Deployment :-

(i) web-logic is an application server, which provides domains for deploying our web-applications into the servers.

(ii) When we install web-logic server, we will get a by-default domain to deploy our applications WL-Servers.

(iii) Apart from the default-domain we can also create a new-domain for deploying our-applications into the server.

(iv) Hard-deployment into web-logic server is nothing but copying our web-application root directory into the autodeploy (staging) folder of the domain.

(v) To hard-deploy our web-application into weblogic, we need to copy the root directory into c:\bea\weblogic90\samples\domains\wl-server\autodeploy folder

(vi) Start the weblogic servers

Start → programs → BEa products → weblogic 9.0 →

(vii) open the Browser & Type the following Request.

http://localhost:7001/BookApp/book.html

oracle http Server Run 8080 port so we change the tomcat Server port.

(ii) Console Deployment :-

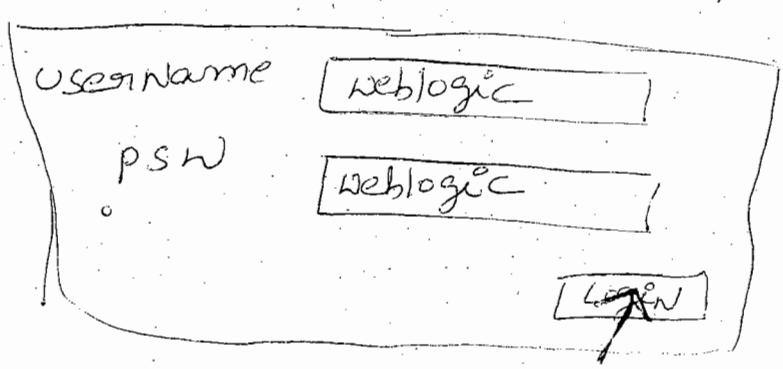
-> Create a war file for the web-application.

c:\BookApp> jar cvf booktest.war *.*

-> start the servers.

-> open the Browser & type the following url in addressbar

http://localhost:7001/console



-> At leftside Select deployments -> click on lock & Edit button -> install -> upload files (click)

-> Browse (first one) -> select war file (BookTest.war) ->

Next -> select war file -> Next -> Next

-> Next -> Finish -> Save -> Activate changes

-> deployment -> bookTest -> Start

→ open the browser and type the following request in the addressbar.

http://localhost:7001/warfile name(bookTest/book.html)

** Creating A User-Domain in web-logic

→ By creating Domains in the server, the web-logic server maintainers becomes easy.

→ if the multiple domains are created then we can deploy applications into domains, so even though we have same application name but we won't get any loss of an application, because the applications can be deployed into different domains.

→ To create a domain the following steps are Required.

Step 1:

Start → programs → BEA products → Tools → Configuration wizard → Next → Next → Enter user-name & psw → Next → Next → Next → Enter Domain Name → Create → Done.

To deploy our web-application into the domain we need to copy the root folder into

C:\bea\user_projects\Domains\Domain Name(bookTest)\autodeplay.

→ Start the Server

Start → programs → bea → user projects → web_domain → startadmin Server.

Diff b/w WebServer & Application Server (Apache tomcat) × (weblogic) × ×

WebServer	Application Server
→ A web-server provides, only a web-container, (or) Servlet-Container.	→ An Application Server provides both WebContainer & an EJB Container.
→ A web-server only supports War files deployment.	→ An application server supports both War & Ear files deployments.
→ WebServer only handles	→ An application server

less amount of middle-ware services.	provides more amount of middle-ware servic
→ webservers doesn't support clustering.	→ Application servers supports clustering.

→ Note: (i) An application server is also called as a web-application server.

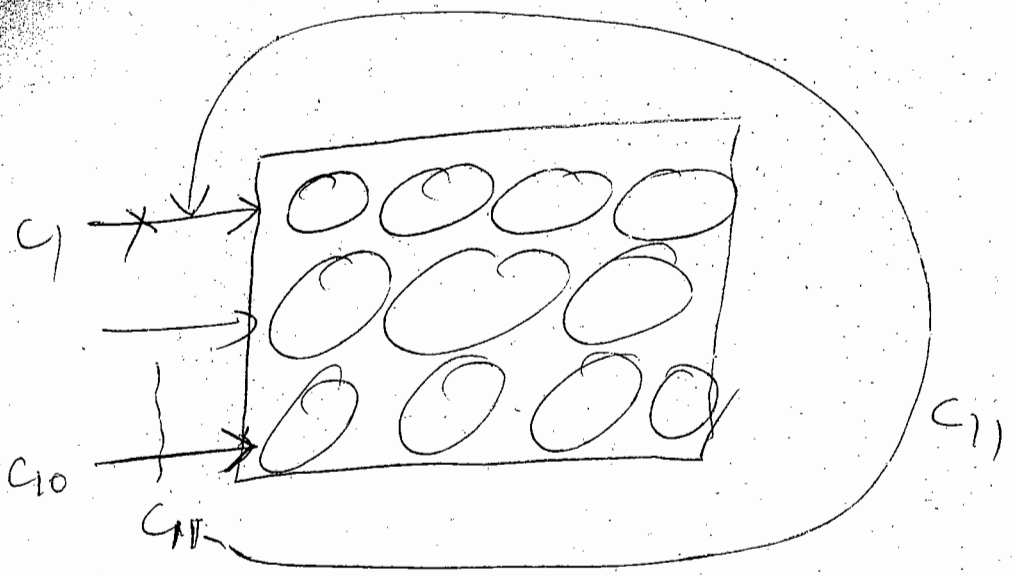
→ An application server also contains a webContainer.

(ii) we can not run ETB applications in a tomcat-server. because tomcat-server doesn't Contain an ETB Container.

(iii) Middle-ware services (or) The Real-time services required by the projects & those are like pooling, Cashing, transactions, security etc.

(iv) A webserver doesn't support clustering, it means, when server Capacity exceeds, the web-server automatically dis-

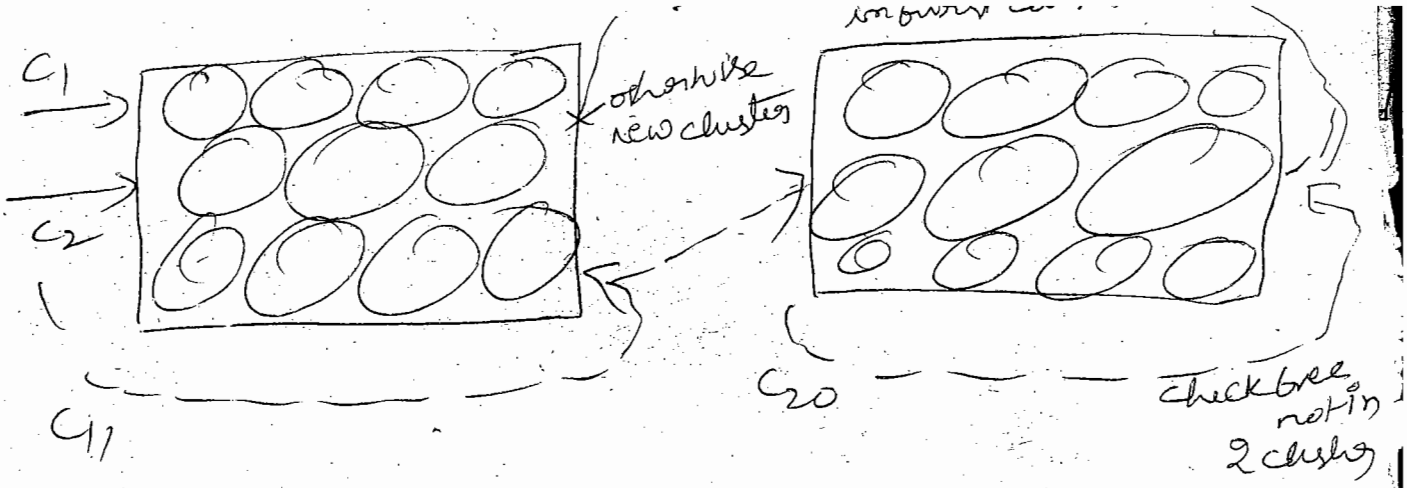
assigns that space to the new client.



⇒ In case of an application server cit follows cluster model.

→ when that server instant capacity exceeds than automatically another instance of the server will be started & client request will be redirected to new instance.

→ If the new instant capacity also exceeds by the client, than an application server first verifies whether any free-space is available in the instances (or) Server instances of the cluster or not, If not new instance of the server is created into the cluster.



* Servlet to Database Communication :-

-> we have the following 3 ways to Communicate from a servlet to a database.

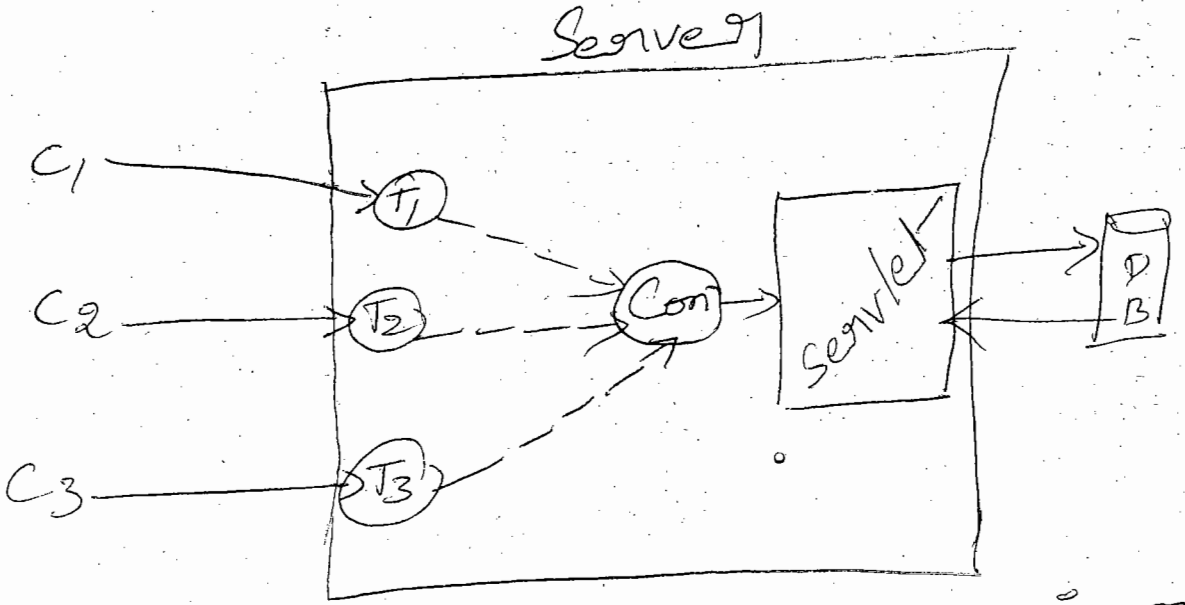
-> (i) open the database connection in the init() method, close the connection in destroy() method & Remaining processing at service() method.

-> (ii) open the database Connection, database operations & closing the Connections & all operations in service() method.

-> (iii) Getting the Connection from a Connection pool to Execute operations on database.

approach-(i)

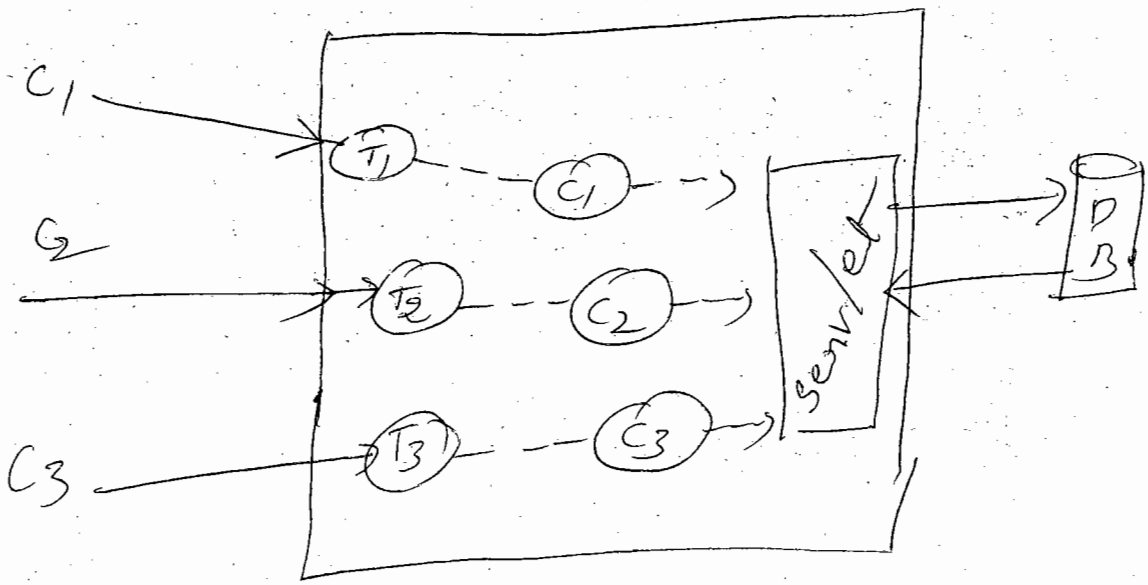
→ In this approach only one Connection with database is open & multiple threads servicing the clients requests. will share single database Connection.



→ The advantage of this approach is, only one Connection is opened with database. So burden on the database server is reduced.

→ The drawback of this approach is only one Connection is shared by multiple threads, so Connection object becomes Non-Thread safe. it means if one [illegible] modifies it other threads got

In this approach for each thread servicing the client request, a seperate connection with database is Created.



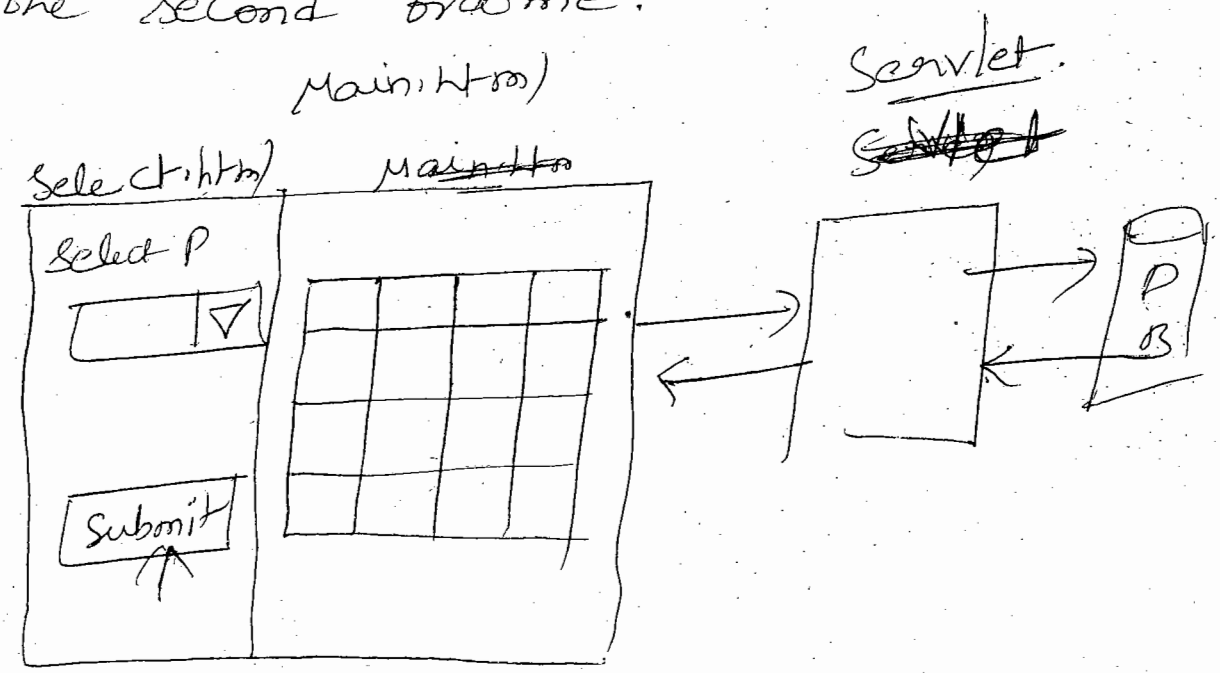
→ The advantage of this approach is each thread as its own connection, if one thread modify the Connection object it doesn't effect on remaining Threads, it means the Connection object is thread-safe.

→ The Drawback of this approach is burden on the database server will be increased.

Note: To overcome the drawback of approach I & approach II we get servlet with Technique.

Example:

→ In this example, we devide a page into two frames on Browser & request is given to the servlet from first frame & Response will be displayed in the second frame.



Logic: 1

→ Servlet Reads the parameter value & page Number

→ If client has not given pageno parameter than servlet takes pageno as ① & if client as given the page no, then servlet will take that pagenumber.

```
String sPageNo = req.getParameter("pageNo");
if(sPageNo == null)
{
    pageNumber = 1;
}
else
{
    pageNumber = Integer.parseInt(sPageNo);
}
```

= = = = =

logic 2:-

→ we need to calculate the starting index by depending on pageNumber.

→ move the cursor directly to the starting index and then display the three records of that page.

→ If we want to move the cursor directly to a particular Record then our ResultSet must be a scrollable ResultSet.

→ In JDBC, if we want to create a scrollable ResultSet then we need to use type & mode parameters.

StartIndex = pageNumber * no of Records per page - no of records per page + 1

```
rs.absolute(startIndex);
i=0;
do
{
    i++;
    ---
    ---
    ---
}while(i<=3 && rs.next());
```

logic 3 :-

→ Find total no of records available in the Table.

→ To get total-number of records, we need Count of the records from the Table.

→ Find total no of records by Executing a Select operation & store the Count into the variable

```
ResultSet rs2 = stmt.executeQuery("select count(*) from Emp");
rs2.next()
```

→ 14

calculate the no of pages required for displaying the links.

→ The number of pages are calculated by using The total no of records / No of Records per page.

→ If total number of records are greater than number of pages * No of Records per page then we need to increment number of pages by 1.

no of pages = Total No of Records / no of Records per Page

```
if (Total No of Records > (no of pages * no of Records per page))
{
    no of pages++;
}
```

Logic-5 :-

---> Display the hyperlinks using anchor Tag & for each hyperlink send a request to the servlet with pageNo = link No.

```
for(int i=1; i<= no of Pages; i++)
{
    pw.println("<a href=PS?pageNO="+i+">"+"
        +i+"</a>");
}
```

code: : Page No: 120

appl 9 of the Land bk.

date 27/8/11:- Servlet ↓ with ↓ Connection Pooling

if we use DriverManager class to obtain a database Connection in a java application like a desktop application or a servlet or an EJB etc, then we have the following problems

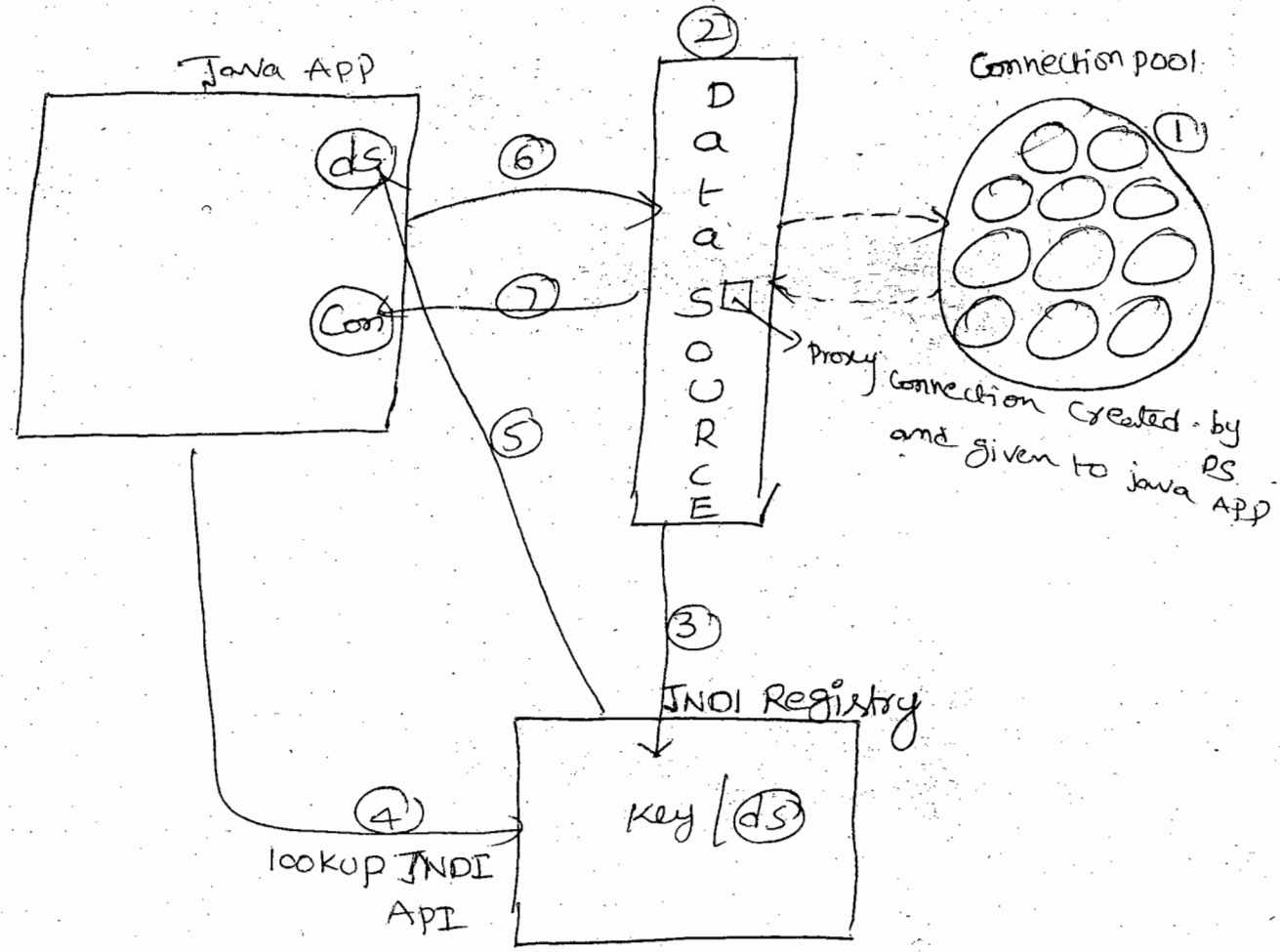
(i) java-application is talking with database and Exchanging the information like username, password, before opening the Connection. it takes some amount of time.

(ii) DriverManager provides a physical Connection to the java application & it is a non-reusable Connection. so each application has to open & close the Connections seperately. in this case some time is taken for allocating & deallocating the memories.

→ In order to overcome the above problems ~~to~~ get using DriverManager we have an alternative mechanism for java to database communication called as DataSource.

→ Datasource has a ConnectionPooling implementation. so java applications are going to

Connections with the database and these are reusable connections, so the application has no need to wait until the connection is opened & on the Database server the burden is reduced.



① Server administrator creates a Connection pool with a database.

② the administrator also create a mediator to access the ConnectionPool, we call this mediator as a Datasource object.

③ The datasource object will be binded into the JNDI Registry by the administrator, with some key, we call this key also as a JNDI

④ Java application uses JNDI API and it will search for the key in the Registry, we call this operation as a look-up operation

⑤ Java application gets datasource object from the Registry.

⑥ Java application, communicates with Data-source to obtain a Connection from the pool.

⑦ DataSource internally gets a Connection from the pool & creates a proxy Connection & finally that proxy Connection is given back to the Java application.

Note:-

DataSource will Return the original Connection back to the pool, whenever the proxy Connection is closed in the Java application.

IMP points:-

(i) Connection pool contains dynamic behaviour. it means the pool has any idle connections for a long period of time than automatically those are removed.

(ii) one Connection pool is purely dedicated for one database. it means, it is not possible to store multiple-databases connections into a single

(iii) HT (a time, ...

proxy Connection is possible. it means, proxy Connections can be created one after another but at a time possible physical Connection, but not all at a time possible.

iv) Each Connection pool has some capacity & it never exceeds its maximum capacity.

Date

29/8/11

Configuring a connection pool in web-logic 9.0

→ Start the Weblogic server

Start → programs → BEA products → weblogic server 9.0

→ open the browser & type the following URL at address bar.

http://localhost:7001/console

- Enter username & password. (weblogic & weblogic) click on Login button.

→ At left side, Expand services → Expand Jdbc → select Data sources → click on lock & Edit button → NEW → Enter the following details

Name: [ ] → AnyName (pool Name)

JNDI Name: [oracle JNDI]

database Type: [oracle]

=> Next(click) => @

Enter the following Connection properties.

DatabaseName: satya ← Global database name

HostName: localhost

Port No: 1521

UserName: scott

PSW: tiger

Confirm PSW: tiger.

@ click on next button.

=> click on Test Configuration Button => Next => select. Examples servers => Finish => click on pool name (satyapool) => click on Connection Pool => Enter capacity values => Save => Activate changes.

If a Servlet want to get connection from the Connection pool, than in Servlet following JNDI coding is Required.

① // Properties p = new Properties();

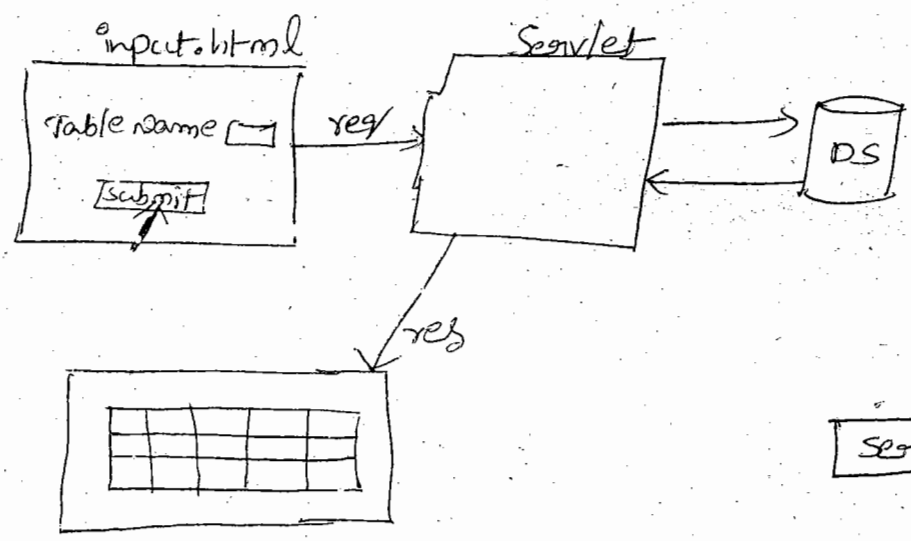
p.put(Context.INITIAL_CONTEXT_FACTORY, "weblogic.jndi.WLInitialContextFactory");

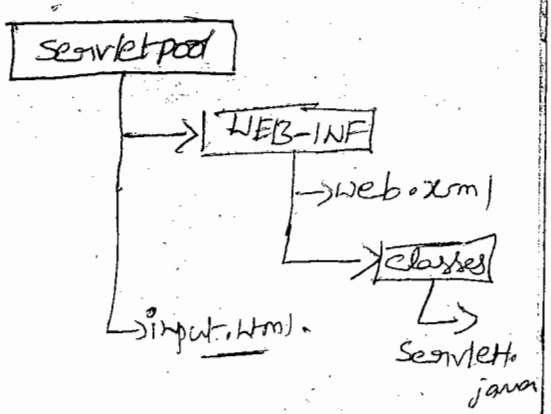
p.put(Context.PROVIDER_URL, "http://localhost:7001");

DataSource ds = (DataSource) o;

Connection con = ds.getConnection(); // ①

EX:-





<!-- input.html ---->

<form action = "Servl">

TableName: <input type=text name="t1">

<input type = submit value="click">

</form>

⇒ Servlet1.java

import java.io.*;

" " .util.*;

import javax.naming.* // JNDI purpose

import java.sql.*;

```
class Servlet1 extends HttpServlet
{
  public void doGet(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException
  {
  try
  {
    String s1 = request.getParameter("e1");
    // print e1 ---
    Statement stmt = con.CreateStatement();
    ResultSet rs = stmt.executeQuery
                   ("select * from e"+s1);
                   ↓ Gap compulsory
    PrintWriter pw = response.getWriter();
    if(rs != null)
    {
      pw.println("<table border=2>");
      while(rs.next())
      {
        pw.println("<tr>");
        pw.println("<td>"+rs.getString(1)+
                   "</td>");
        pw.println("<td>"+rs.getString(2)+"</td>");
```

```
        pw.println("</table>");
    }
    else
    {
        pw.println("no records in the Table"/"no Table in Database");
    }
    }
    }
    catch(Exception e)
    {
        e.printStackTrace();
    }
    }
}
```

= = =

```
<web-app>
  <servlet>
    <servlet-name> pool </servlet-name>
    <servlet-class> Servlet </ ---- >
    </servlet>
  <servlet-mapping>
    <servlet-name>
    <url-pattern>
  </servlet-mapping>
```

<S-n>
</url-pattern>

Jar file

c:\bea\weblogic90\server\lib\weblogic.jar

Note:

* To compile the above servlet, we need to set some server given jar file in the class-path.

* It is possible to compile a servlet by using jar-file given by one-server & we can run that servlet on another server.

* In case of weblogic-server, the jar file given is weblogic.jar & it is available at the above specified location

classpath = c:\bea\weblogic90\server\lib\weblogic.jar;%classpath%

→ weblogic.jar file is used for compiling JSP, Servelet & EJB applications also.

→ Servlet-api.jar suitable only servlet applications.

30/8/11 → Deploy the web-application root folder into c:\bea\weblogic90\samples\domains\wl-server\autodeploy folder.

→ Start the web-logic server.

http://localhost:7001/ServletPool/input.html

Applet to servlet Communication:-

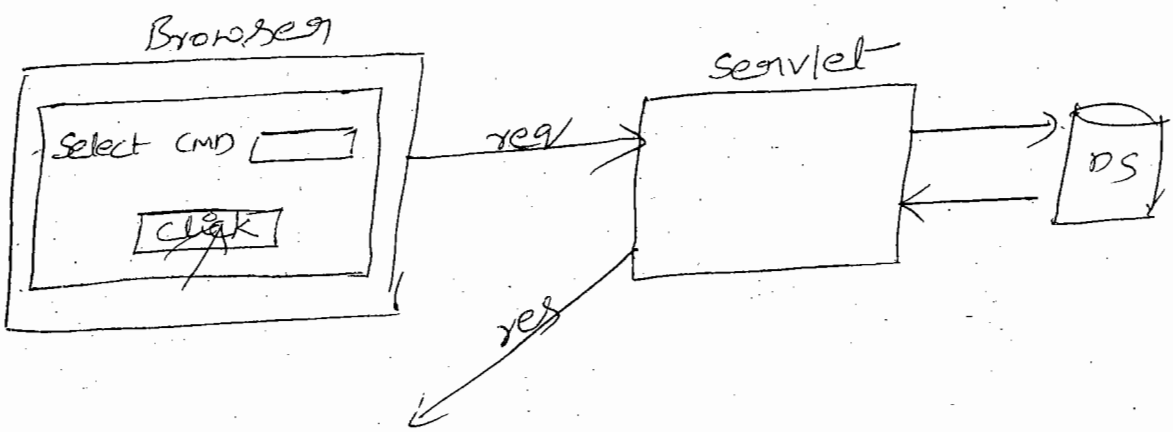
→ Generally we communicate from Html to a servlet.

→ In case of html we are replacing with an applet.

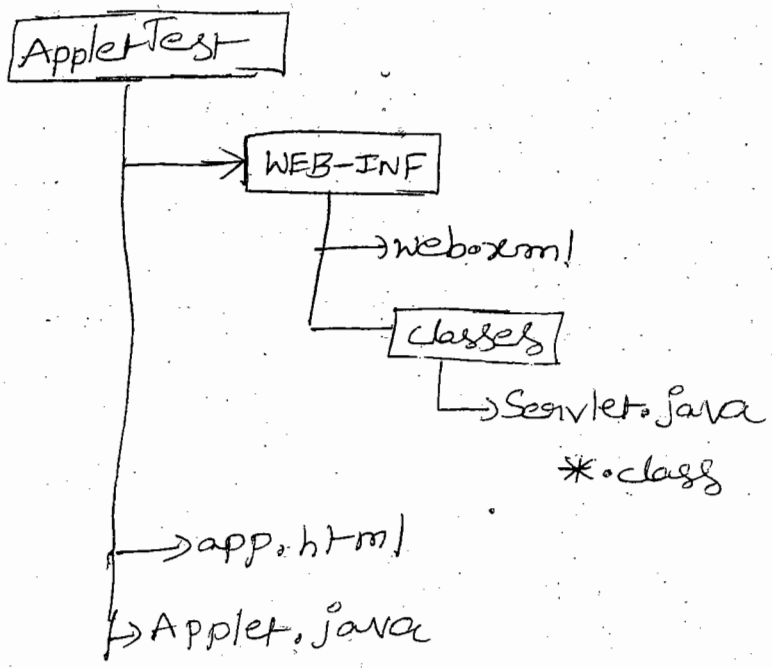
→ In applet to servlet Communication, applet provides presentation on to the browser by accepting input values from the client.

→ Inorder to integrate or Communicate from an applet to servlet we need to use a class called URL.

→ URL is a class given in java.net package.



Directory structure



1) <!-- app.html -->

<applet code = "TestApplet.class" width=500
height=200>

</applet>

2) Applet.java

import java.Applet.*;
import java.awt.*;
import java.awt.event.*; } ~~use~~ interface mechanism doesn't applicable to Pkgs
import java.net.*;

public class Applet extends Applet implements
ActionListener

{

Label l;
Button b;
TextField tf;

public void init()

{

l = new Label("Enter Select cmd");

t = new TextField(20);

b = new Button("Get Data");

add(l);

add(t);

add(b);

b.addActionListener(this);

}

public void paint(Graphics g)

{

setBackground(Color.Green);

}

public void actionPerformed(ActionEvent ae)

{

try

{

String s = tf.getText();

URL u = new URL("http://localhost:2011/AppletTest/Srv1?cmd="+s);

}

catch(Exception e)

{

e.printStackTrace();

}

}

1) Two types of events
(1) Low level → Remaining all
(2) Symantic

Applet follow by default Flow Layout

Frame follow by default Bordered Layout

X

AppletContext ctx = getAppletContext();

ctx.showDocument(u,"_self");

Servlet1.java

```
import javax.servlet.*;
import java.io.*;
import java.sql.*;

public class Servlet Extends GS
{
  public void service (ServletRequest req,
                       ServletResponse res)
                       throws ServletException,
                              IOException.
  {
  try
  {
    String s1 = req.getParameter ("cmd");
    PrintWriter pw = res.getWriter();
Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver");
Connection con = DriverManager.getConnection
                    ("jdbc:odbc:oradsn",
                     "scott", "tiger");
    Statement st = con.CreateStatement();
    ResultSet rs = st.ExecuteQuery (s1);
    pw.println ("<table border=2>");
    while(rs.next())
    {
      pw.println ("<tr>");
```

```
pw.println("<td>" + rs.getString(1) + "</td>");
pw.println("<td>" + rs.getString(2) + "</td>");
pw.println("<td>" + rs.getString(3) + "</td>");
pw.println("</tr>");
}
pw.println("</table>");
pw.close();
rs.close();
con.close();
}
catch(Exception e)
{
e.printStackTrace();
}
```

http://localhost:2011/AppletTest/appointm)

Date
31/8/11

RequestDispatcher interface (included Servlet 2.2)

→ In servlet 2.0 & 2.1 versions, we have a concept called servlet chaining. it means moving the control from one servlet to another servlet.

→ Instead of defining the entire logic in a single servlet, we can devide the logic into multiple servlets, and we can execute them one after another we call this concept as chaining of servlet

→ In servlet 2.0 & servlet 2.1, this linking b/w servlet is done by administrator at server side. but in servlet 2.2, the servlet chaining concept is moved from ~~admin~~ administrator side to programmer side by introducing RequestDispatcher

→ By using RequestDispatcher we can perform only two operations.

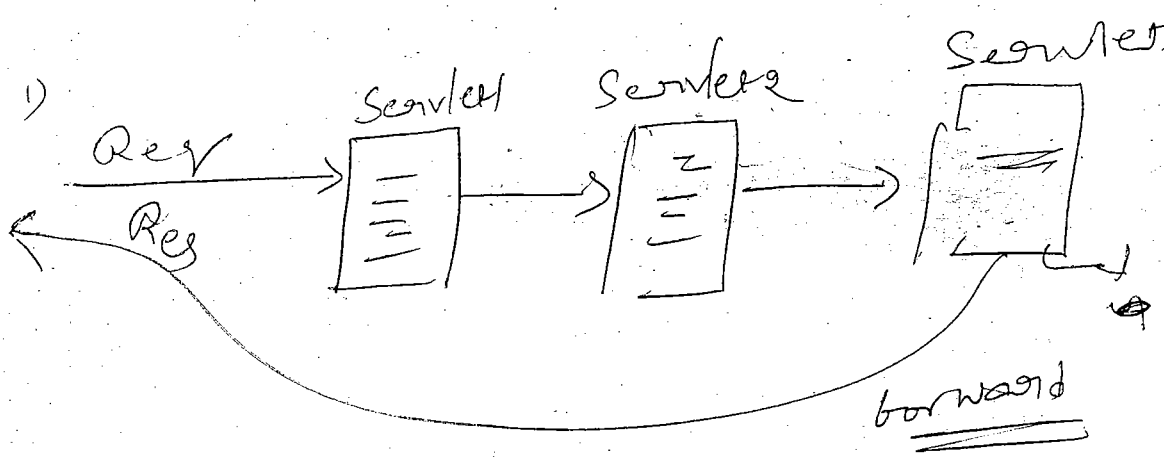
1. we can forward request from one servlet to another servlet.
2. we can include the response of

in RequestDispatcher interface 2 methods are provided-

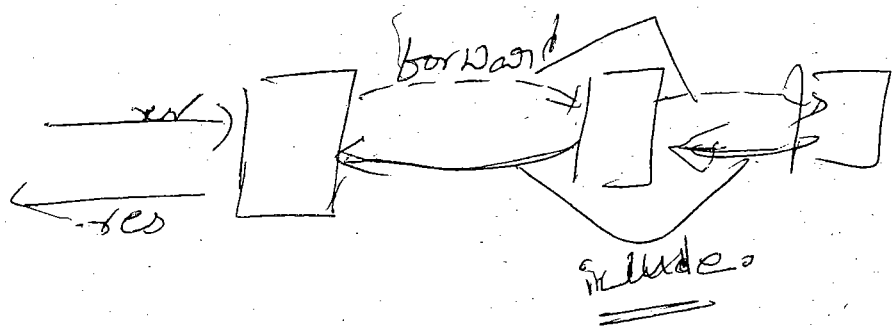
(1) forward(req,res);

(2) include(req,res);

1)



2)



forwarding

→ while forwarding a given request from one servlet to another servlet, the last servlet in the chain will produce response back to the client.

→ while forwarding a client request from one servlet to another servlet, there will be no intermation given back to the client. It means the forwarding technique work at serverside and doesnot intimate the client browser.

→ while forwarding, a client request is forwarded from one servlet to another. It means one pair of

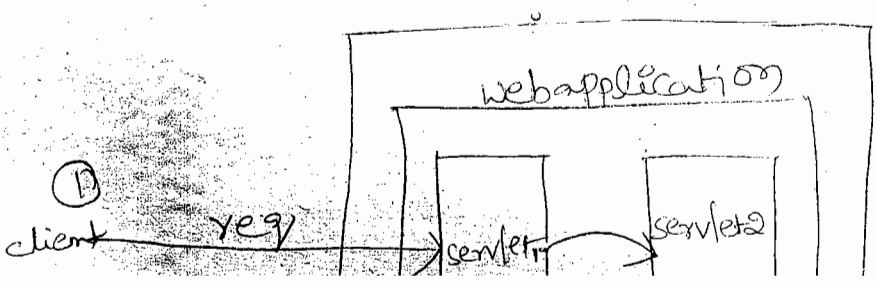
- Same request & response objects are continued in the entire forwarding chain. (2)

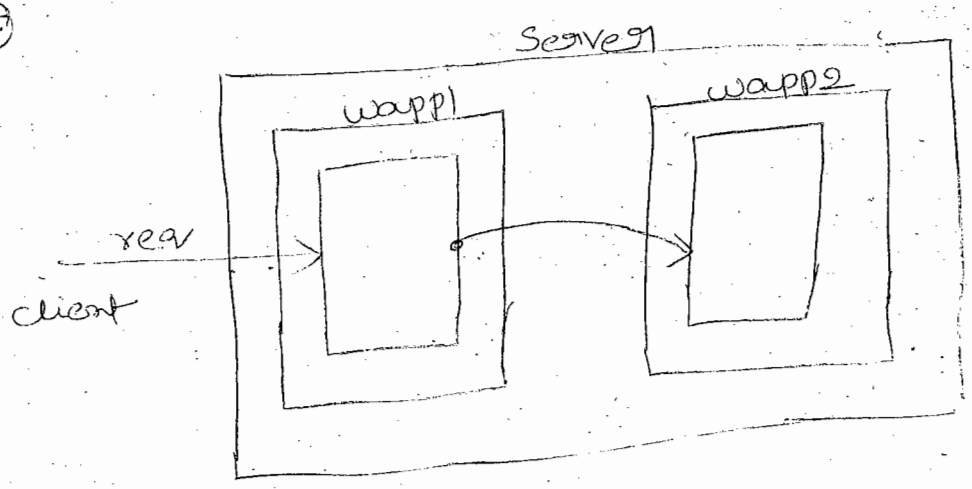
-> while forwarding a request from one servl to another servlet, the request parameters a also automatically forwarded to the destination.

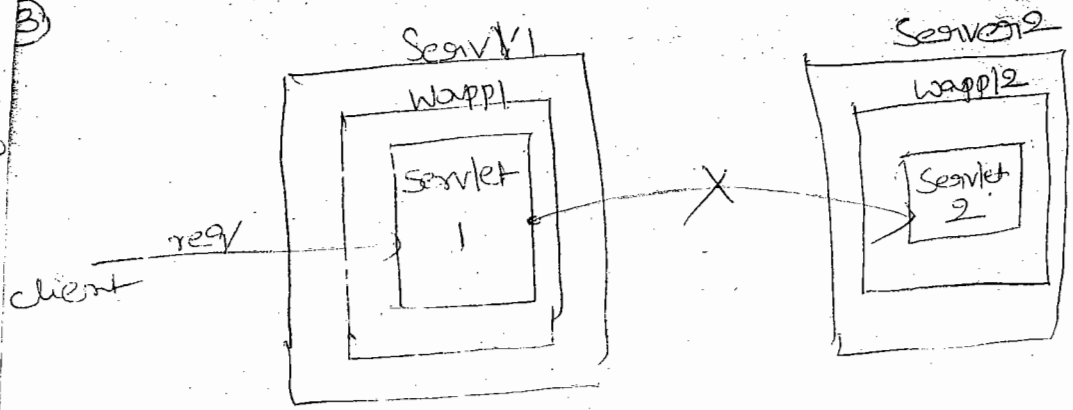
-> Forwarding a request from one servlet to another servlet permenently. It is nothing but moving the ctrl permenantly to one servlet to another, so it is not possible to forward to multiple servlet from a single source servlet. (3)

-> while forwarding, ~~from a single source se~~ the destination resource must be java enabled resource only (servlet (or) JSP, (or) html.)

-> while forwarding a request, both source & destination servlets must exist either within the same webapplication (or) different webapplicati of the same server. but it is not possible to forward a request from one server to another server.







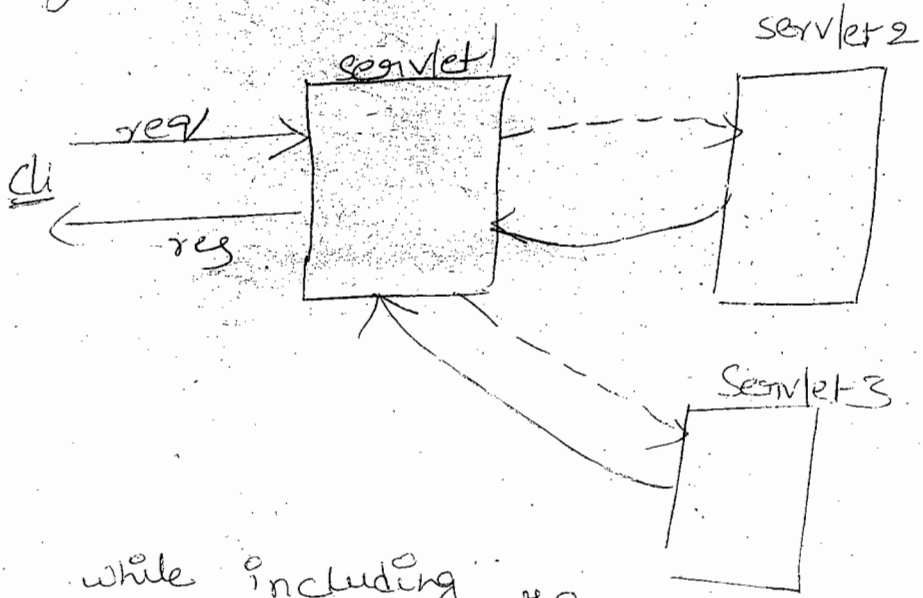
=> including

→ while including the response of one servlet into another servlet, the response will be partially generated by source or partially generated by destination & the total response will be given back to the client by the source servlet.

→ while including the response, the include operation is done at serverside. and no intimation is given back to the client so the client is unknown about the include operation.

→ while including, the webcontainer will only generated one pair of request & response objects and

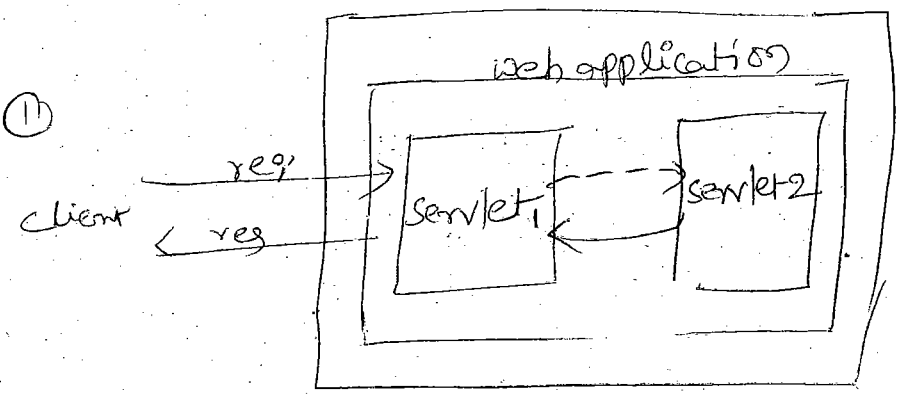
-> while including the response the ctrl is temporarily shifted from a source to a destination and again the ctrl goes back to the sourced with the Generated response. so it is possible to include multiple destination response into a single source.

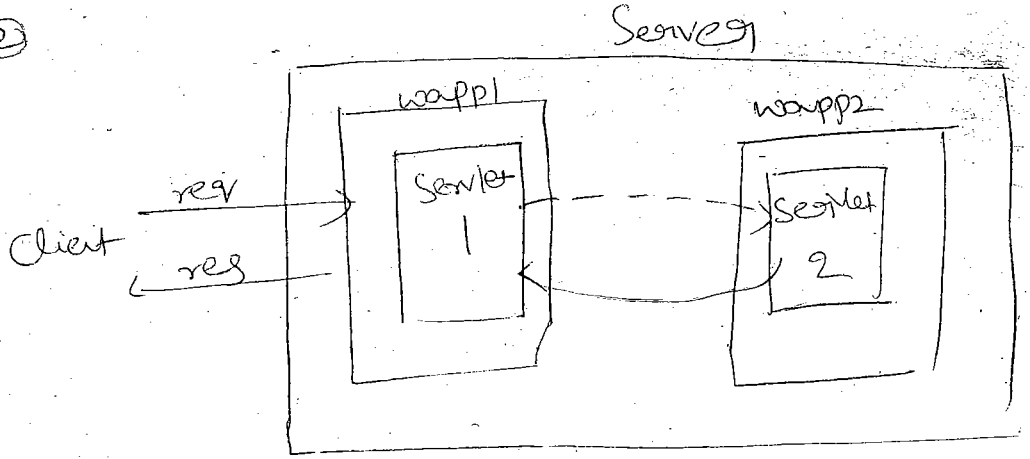


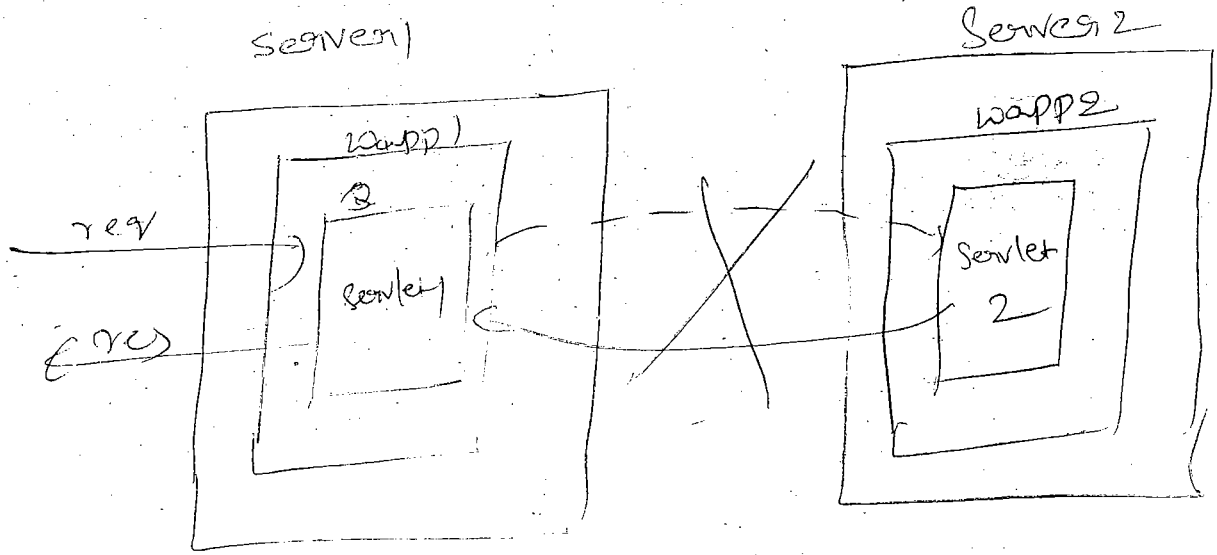
-> while including the response, the ctrl is temporarily shifted to the destination along with ctrl the request parameters are also transferred to the destination automatically.

-> while including, destination resource must be a java enabled resource only

-> while including the response, both source & destination resources must be available either in same webapplication or in different web application of the same server but it is not possible to communicate across different servers.


web application
req
client
res
Servlet1
Servlet2


Server1
wappl
wapp2
req
client
res
Servlet 1
Servlet 2


Server1
Server2
wappl
wapp2
req
res
Servlet 1
Servlet 2

obtaining RequestDispatcher object

Date 3/9/11

① RequestDispatcher rd = ctx.getNamedDispatcher(String servletName);

② RequestDispatcher rd = ctx.getRequestDispatcher(String path);

③ RequestDispatcher rd = request.getRequestDispatcher(String path);

approach-①

→ If we want to forward (or) include, we need a request dispatcher object in the source servlet & it should point to destination servlet.

→ In this approach, we can get request dispatcher object through the logicalname of the destination servlet.

→ among the all approaches, this approach is simple way of getting a RequestDispatcher object because we no need to pass any path as a parameter

ex:-

```
Servlet
service( )
{
```

Dummy servlet

metho

→

rebu

→ G

begin

ex:-

C

approa

→ In

meth

→ g

path

objec

→

Date 3/9/11

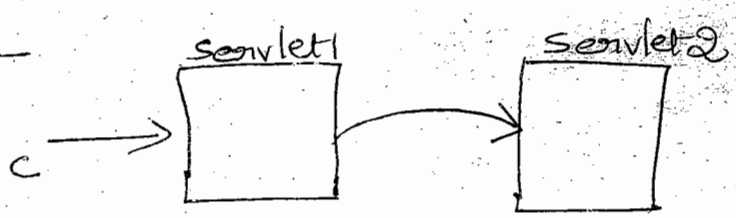
approach-2

In this approach we use getRequestDispatcher method given by Context object.

→ In this approach we need to pass Context relative path, to get RequestDispatcher object.

→ Context relative path means the path must begin with "/" and root directory.

Ex:-



web.xml

servlet1 —> /srv1.

servlet2 —> /srv2.

servlet1

service()

RD rd = ctx.getRequestDispatcher("/wapp1/srv2");

rd.forward(req, res)

approach-3

→ In these approach, we call getRequestDispatcher() method given by ServletRequest object.

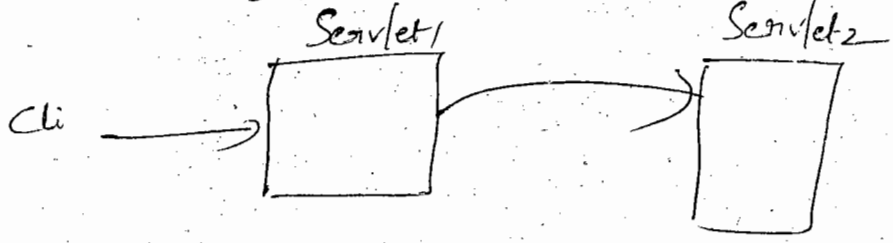
→ In this approach, we need to pass request relative path as a parameter, to get RequestDispatcher object.

→ Request relative path means we need to

→ While writing the Request relative path, if we want to come back any directory then use "..".

→ It may or may not begin with '/'.

②



③

web.xml

Servlet1 → /srv1

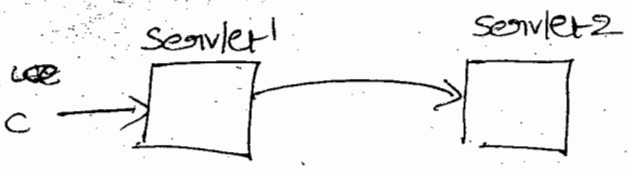
Servlet2 → /srv2

Servlet1

service()

```
RD rd = req.getRequestDispatcher("/srv2");
rd.forward(req, res);
```

⇒ The following Example is to differentiate the context relative & request relative path.



web.xml

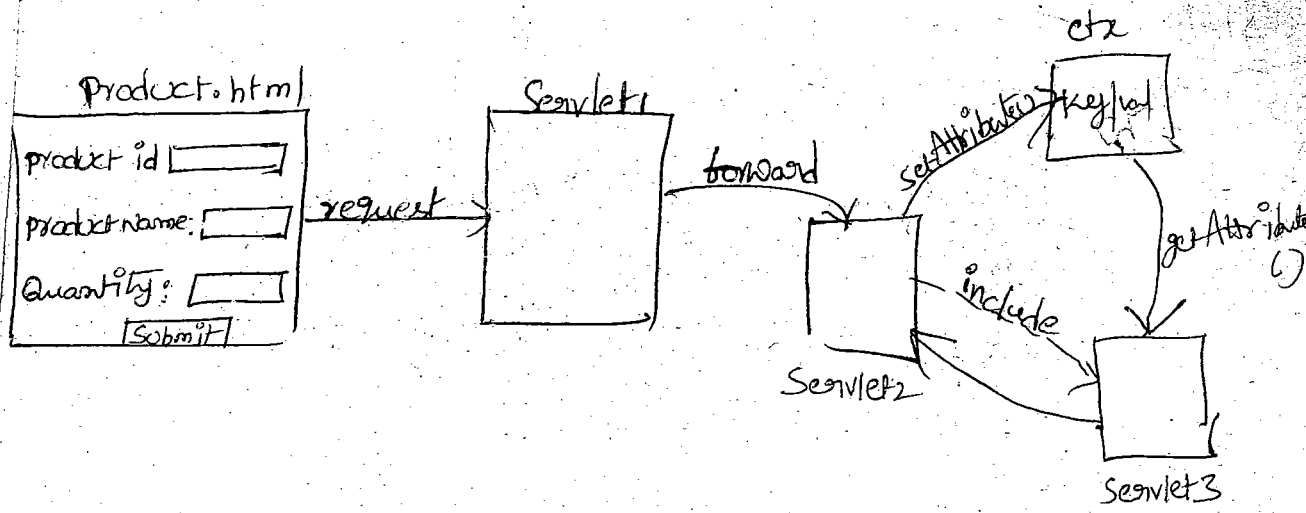
Servlet1 → /aa/bb/cc/srv1

Servlet2 → /aa/bb/mm/srv2

Servlet1

service()

```
RD rd = ctx.getRequestDispatcher("/wapp2/aa/bb/mm/srv2");
req.forward(req, res)
(or)
```



RequestDispatcherApp

- WEB-INF
  - web.xml
  - classes
    - Servlet
    - Servlet2
    - Servlet3
    - *.class
- product.html

handbook

Page NO: 127, ex:NO: 13

"Srv1" ⟶ correct

"/Srv1" ⟶ Not correct.

"./Srv1" ⟶ Correct (specify current webapplication)

Date

4/9/11

Servlet Redirection

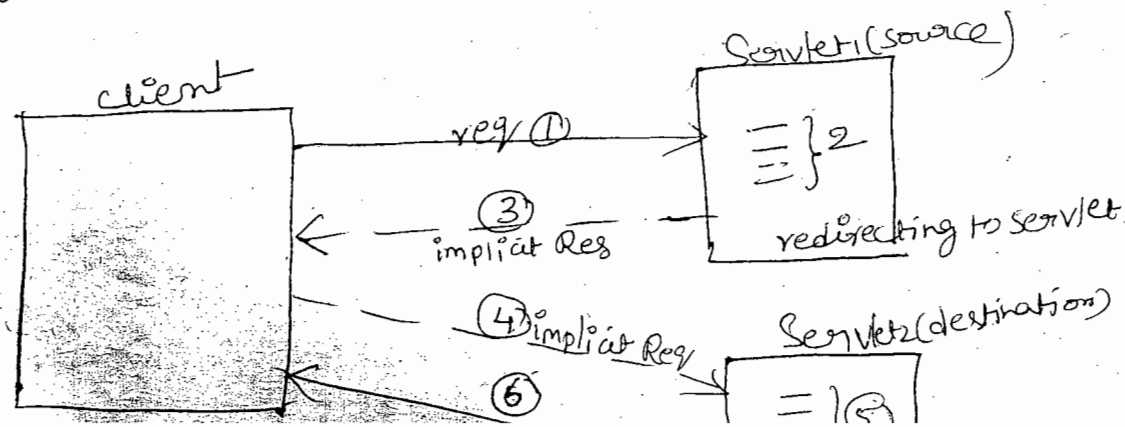
→ Servlet Redirection is also belongs to chaining of servlets.

→ If a client request is redirected from one servlet to another servlet i.e., from a source to destination then first of all the control implicitly goes to the browser, intimating the client and then the control is shifted implicitly from browser to second servlet (or) destination.

→ At the time of redirecting a request, the Redirection is not done only at servers side. First of all the client is intimated and it then after, the control goes to the destination.

→ In a redirection technique, the redirection information is provided to the client also.

→ In order to do redirection, we need to call method sendRedirect(), given by HttpServletResponse interface.



→ "URL" one.

addre reque to or www.

→ G URL Comm

→ In are (i) (ii) (iii) a mat the que

**

Eg

on
URL" in addressbar will be changed from old one to new one.

chaining → For example, if we type www.bea.com in the addressbar and when we send a request then the request is ~~www.bea.com~~ redirected to bea website. to oracle website and the addressbar is changed to www.oracle.com/bea/index.html.

→ Generally we use Redirection when an old website URL is changed to a new URL or when we are communicating from one server to another server.

→ In Redirection mechanism the control is permanently shifted from source to a destination, so it is not possible to redirect to multiple destinations at a time.

→ In case of Redirection minimum 2 requests are generated from the client side.

(i) Given by the client explicitly

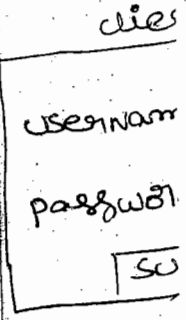
(ii) Generated implicitly while redirection.

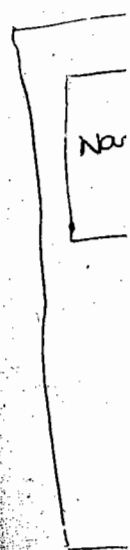
(iii) in Redirection, only ctrl will be redirected from a source to a destination, but not the data automatically. so if we want to redirect the data also then, we should add explicitly in the form of a query string.

*** The following are the differences b/w forwarding & Redirection techniques

Forwarding	Redirection
operation, the	

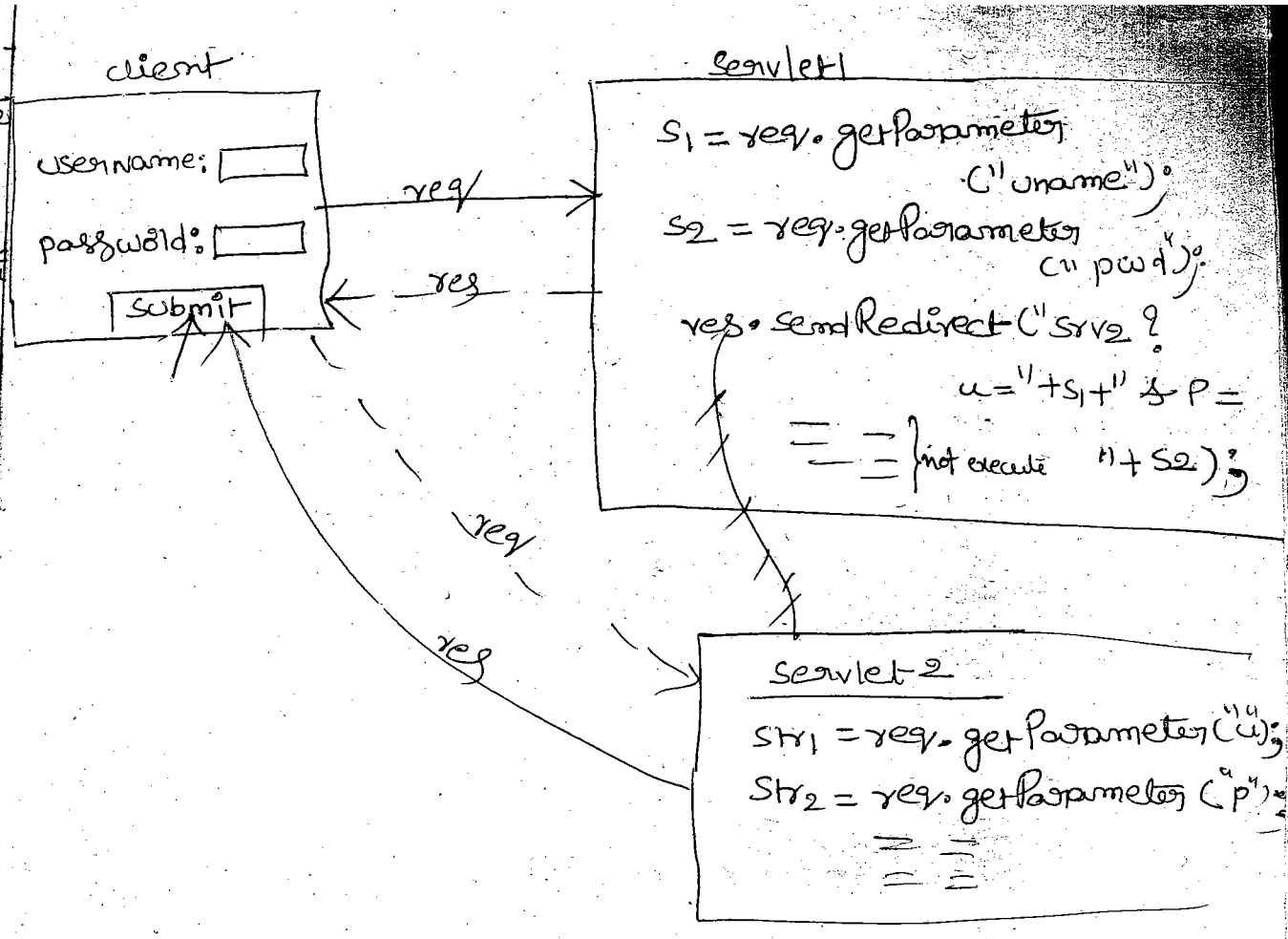
and the information is given to the client.	→ During Redirection operation the information is given to the client and then redirected to the destination.
→ In forwarding, one request only forwarded to the destination. so container creates a single pair of request & response objects.	→ In Redirection, implicitly a new Request is generated so a minimum of two pairs of request & response objects are generated. by the container
→ In forwarding both the control and the data are forwarded to the destination. (by default)	→ In redirection, only control is redirected, but not the data. (by default)
→ In forwarding, the destination resource must be java Enabled resource only.	→ In Redirection, the destination resource can be either java (or) Non-java resource also.
→ In forwarding, both source & destination resources must run within the same server. it is not possible to communicate across the servers.	→ In redirection, it is possible to communicate, either within the server (or) even across servers also.



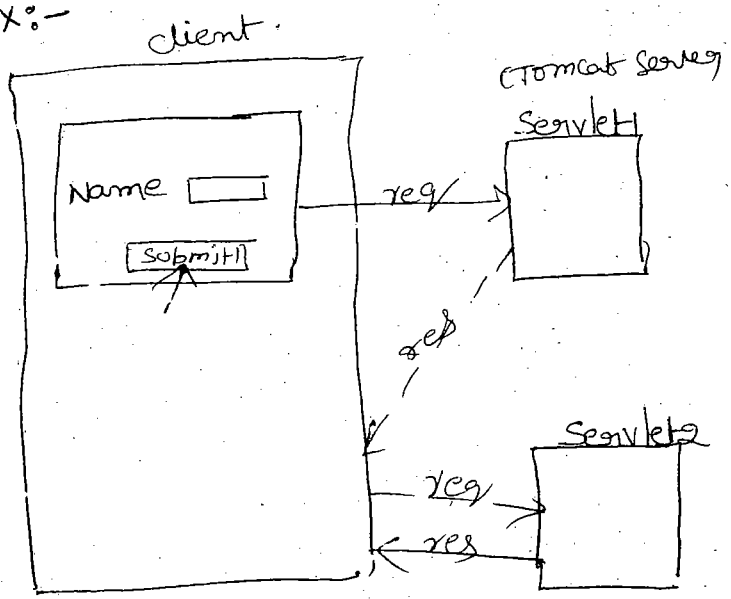


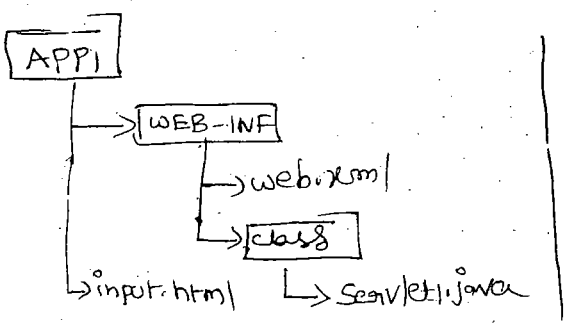
Note:-

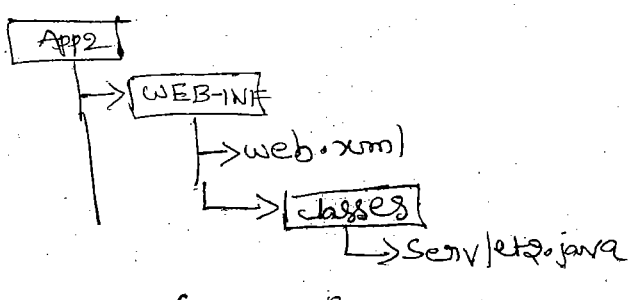
while re-directing the requests, if we want to attach the request values also, then we should explicitly add query string in the url of destination like the following


client
UserName:
Password:
Submit
req
res
Servlet1
S1 = req.getParameter("uname");
S2 = req.getParameter("upwd");
res.sendRedirect("Srv2?u="+S1+"&p="+S2);
req
res
Servlet2
Str1 = req.getParameter("u");
Str2 = req.getParameter("p");

EX:-


client
Name
Submit
req
(Tomcat Server)
Servlet1
web
Servlet2
req
res
(Weblogic server)


APP1
WEB-INF
web.xml
classes
input.html
Servlet1.java
(Tomcat Server)


App2
WEB-INF
web.xml
classes
Servlet2.java
(Weblogic server)

Tomcat Server / App1

```
<!-- input.html -->
<center>
<form action = "srv1">
Enter Name: <input type = "text"
                   name = "uname"><br>
<input type = submit  value = "click">
</center>
```

Servlet1.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Servlet1 extends HttpServlet
{
    public void doGet(HttpServletRequest req,
                      HttpServletResponse resp)
                      throws SE, IOE
    {
        String s1 = req.getParameter("uname");

        res.sendRedirect("http://localhost:7001/App2/
                          srv2?p1="+s1);
    }
}
```

Weblogic

```
import
import
import
{
    pw
    {
    }
}
```

Steps

→ C

→

→

→

goes

to

so

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
{
    public void doGet(HttpServletRequest req, HttpServlet
                      Response res) throws SE, IOE
    {
        String s1 = req.getParameter("p1");
        PrintWriter pw = res.getWriter();
        pw.println("from weblogic :" + v1);
        pw.close();
    }
}
```

Steps

→ Deploy app1 into tomcat & App2 into weblogic server.

→ start both tomcat & weblogic servers.

→ open the browser & type the following request

http://localhost:2011/App1/input.html

→ when we click the Submit button then the request goes to servlet1 and it is redirected from servlet1 to servlet2 that is presenting in weblogic server.

so the Address bar changed in the following way

http://localhost:7001/App2/Srv2?p1=Satya.

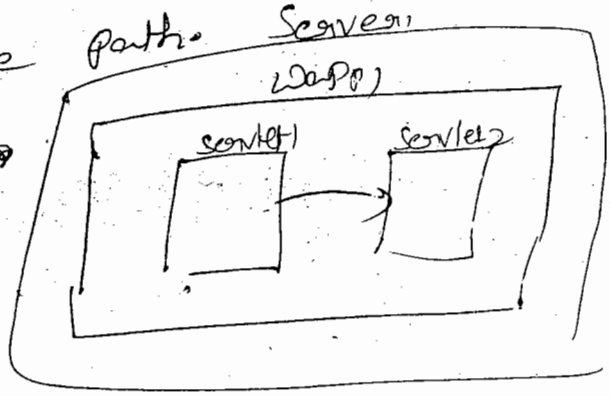
Imp points

(i) Redirection works only at HttpServlet. (c) Cause sendRedirect() is given by HttpServletRes Serve but not ServletResponse.

(ii) Forwarding works for both GenericServlet & also HttpServlet. because to forward() method we can pass ServletRequest, ServletResponse (or) HttpServletRequest, HttpServletResponse.

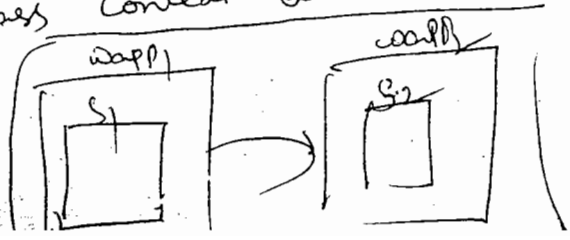
⇒ while Redirecting, we need to provide the Path like the following.

(a) if source and destination are available with same webapplication then we need to pass request relative path. Server



res. sendRedirect("Srv2")

(b) if source & destination are available within the same server but diff webapplication then we need to pass context Relative Path



Date (IMP)

→ a

→ whe stor

→ so

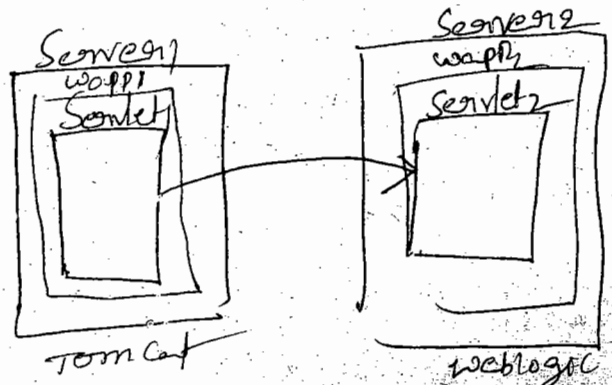
→ to sta

→ in

→ &

res.sendRedirect("/wapp2/srv2");

(c) if servlet1 & servlet2 available at different servers then we need to pass absolute path.



res.sendRedirect("http://localhost:7001/wapp2/srv2");

Date: 6/9/11

Session-Tracking

→ Session is nothing but it is a time-interval b/w a login & logout.

→ For example after Entering username & psw when we click on Login button, then one session starts & it will be continued until we say logout.

→ by default HTTP protocol is a stateless protocol so, managing the ssion is not possible.

→ In order to manage the sessions, we need to convert the HTTP behaviour from stateless into statefull.

→ Inorder to convert HTTP protocol from stateless into statefully we need session-Tracking.

→ session-Tracking is the combination of both session management & state management.

recognize that these multiple requests are given from same-client only.

→ State management means, at server side information related to a client should to be stored so that the data is shareable across multiple requests given by the client.

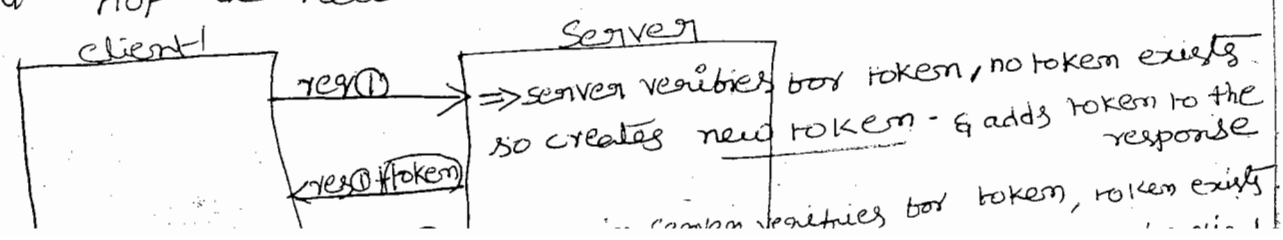
Session Management :—

In session management, server first verifies, whether the client is a new client or old client by observing a token in the request.

→ If no token exists than server will treat that the client is new and creates a token for that client.

→ while sending the response back to the client, than along with response that token is also sent back to the client.

→ when a client is next-time sending the request to a server than along with the request, the token is also sent to the server. So server recognizes that the client is existing but not a new client.



quest & response. so by using this token exchanging Technique, session-management is done by the server.

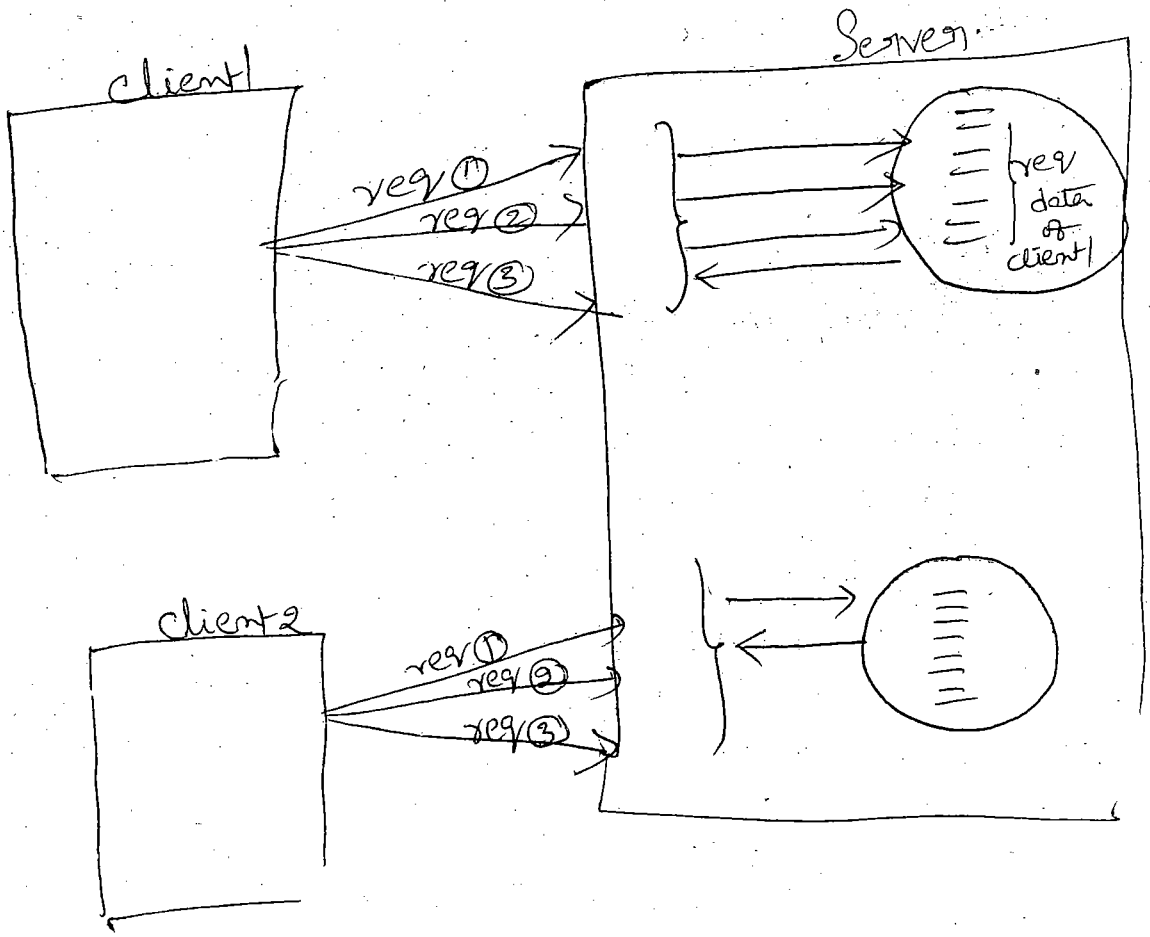
State Mgmt :-

→ when first-time a client request is given to a server than along with new token, an object is also created, for storing the information related to a client.

→ when the same client is sending next request to the server, than the information of the client will be accessed from that object.

→ for each client, one object is created at server-side for maintaining the data of the client.

→ The number of clients requested a server is equall to the number of objects created at server. these objects are for state management.



For complete session-tracking i.e., for session management & state management, at server side a token & an object both are created at serverside. Here token is for session mgmt and an object is created for state mgmt. 5)

Date 8/9/11

Session-Tracking Technically 6)

Session-Tracking means, Creating a session-ID & session-object for managing session & state of the client.

Technically, Session-ID is transfered b/w server & client in the form of a cookie.

From Server to client the cookie will be transfered by storing in ~~request~~ response header and from client to server, the cookie will be transfered by storing in request header.

→ The cookie created by server & stores session-ID into it with the key called jsessionid.

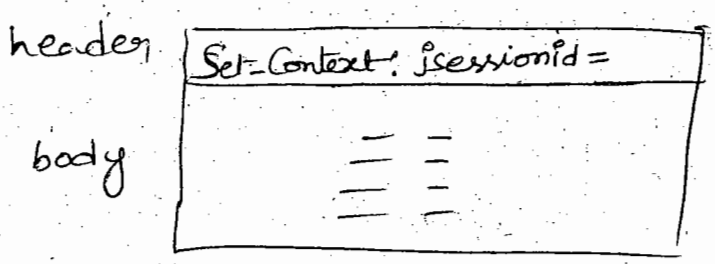
→ A cookie is created at server-side & stored on the client browsers.

Diagram:

client —req①→ server

1) session id created + session object created
2) session id stored in session object
3) request processed

5) cookie is added to response object, in header part



6) Finally, response is given back to client.

** client ~~request~~ + cookie → server

⇒ server identifies the client that client is existing client

⇒ uses session id & session object, already created by the client.

Session-Tracking with Servlet API:—

In Servlet we have following four Techniques for Session-Tracking They are

1. HttpSession (interface)
2. cookies — Partial Session-Tracking (Html-mgmt)
3. Hidden fields — Partial Session-Tracking (Html-mgmt)
4. URL-rewriting

1, 2, 3, 4 → Complete SessionTrack (Sessionmgmt + Htmlmgmt)

1) HttpSession Interface

Date 9/9/11

=> HttpSession Technique is for both session management & state management.

=> Using Servlet API, we can associate the state with the sessions. it means we can store the client information in a session object & we can read that information whenever it is required.

=> As a Servlet programmer, we can perform state management operations & session management will be taken care by the Container.

=> In order to get session object for storing state of a client, we need to ask the Container to give a session object. this can be done by calling getSession() method given by HttpServletRequest interface.

=> while asking for a session, container will create sessionID also. for managing session of the client.

=> servlet, we need to store the into HttpSession interface.

ing are the different ways

n object.

we
envle
=>
usin
ID &
-ect
=> if
we r
of

In
Exc
get
the
new
ses
ne
ses
bac
g
is
dy
is

9/9/11 → In order to work with HttpSession we need to extend our servlet class from HttpServlet.

→ As a programer, we can only store & read the data using a session object, but we can not create session ID & session object. The creation of sessionID & session object will be taken care by the container.

→ In order to work with session object, first of all we need to get the session object by using any one of the following 3 approaches.

```
HttpSession session = request.getSession()
"                 " = "       "  (true);
"                 " = "       "  (false);
```

In the above three approaches, approach 1 & 2 are exactly same. The functionality is, when we call getSession() method, container first verifies whether these request is given by an existing client or a new client.

If the client is existing then already associated session of the client is returned. If the client is new then container creates a new session ID & new session object and then that session object is given back to the servlet application.

In approach 3, container verifies whether the client is existing client (or) new client. If existing then already associated session object is given & if the client is new then the container returns null value.

In approach3, new session is not created by the container.

Accessing Session object :-

⇒ Accessing a session object is nothing but, storing the Data into a session object (or) reading the Data from a session object.

⇒ if we want to store the Data in a session object (or) if we want to read the Data from a session object then we need to use attributes.

In servlet API, we can store attributes (or) we can read attributes through the methods related to attributes API.

(i) setAttribute(String key, Object value);

(ii) getAttribute(key);

(iii) removeAttribute(key);

(iv) getAttributeNames();

⇒ In servlet api we can store attributes into 3 objects.

(i) request (ii) session (iii) Context.

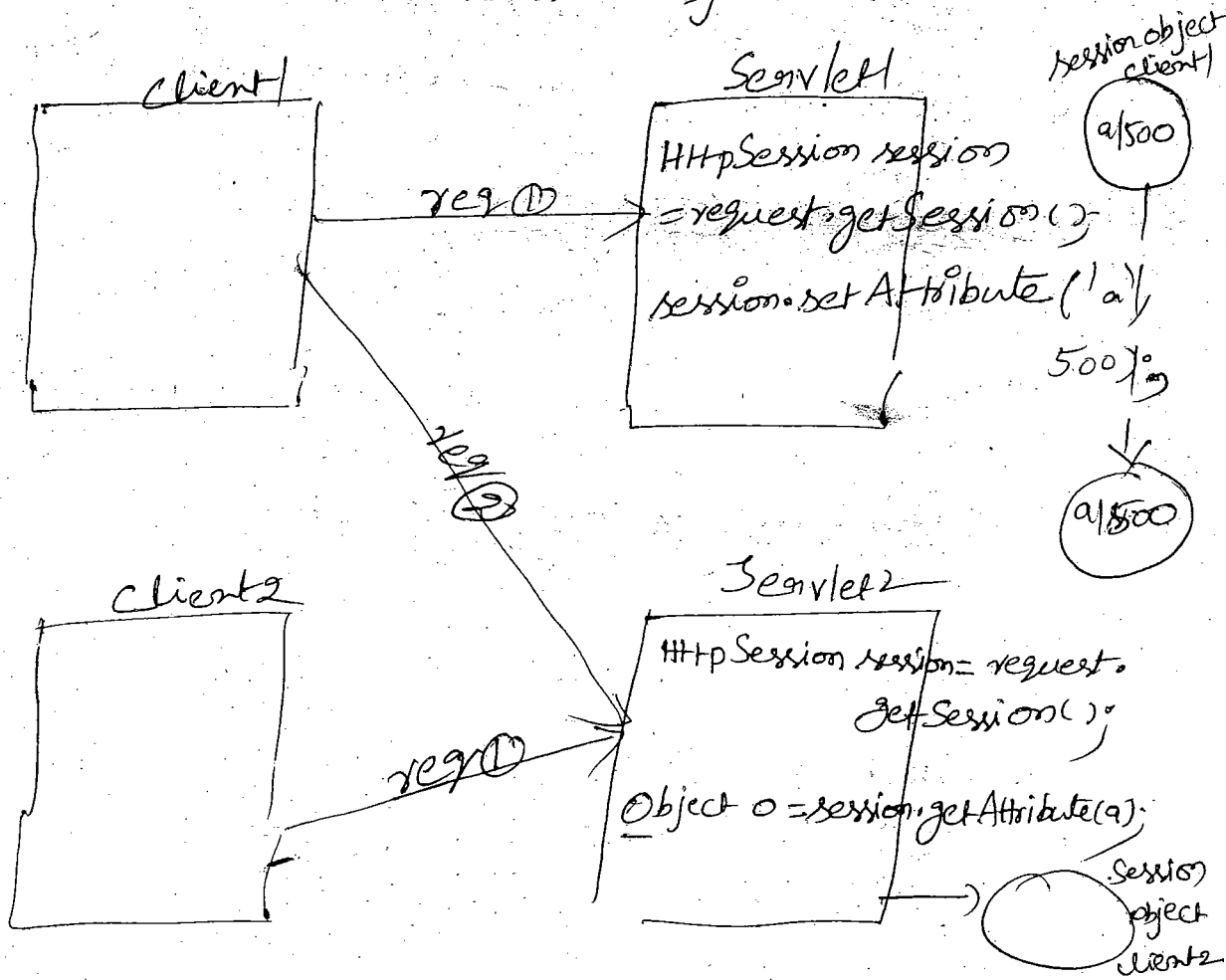
⇒ In request & context objects, we can store attributes and also parameters.

⇒ In a session object we can store attributes but not parameters.

⇒ if we store some data in the session

Containers.

⇒ If we want to use (or) store the data of the client when the client is visiting the server for multiple ~~clients~~ times. then we need to store that client data in a session object.



→ According to the above diagram, when client1 is sending the request to servlet2 then the object 'o' contains the value 500. and when client2 is sending the request to servlet2, the object o contains the value. null. because for new client a new session object is created and in that session object there is no attribute called 'a'. so the object o contains null.

→ whenever a session object is created by the container, the container will store some additional information into session object

state(new or old), Creation-time, Ideal-time etc

→ In order to read the additional information stored in the session object by the container we need to call some session-management methods given in HttpSession interface. In HttpSession Interface we have some session-mgmt methods and some state-management methods.

Session management methods:-

1) getId() 2) IsNew() 3) getCreationTime()

4) getLastAccessedTime() 5) getMaxInactiveInterval()

6) setMaxInactiveInterval() 7) invalidate()-

State Management methods-

setAttribute(K, V);
getAttribute(K);
removeAttribute(K);
getAttributeNames();

Date 10/9/11

① getId()

This method is used to get the sessionId stored in a session object. The sessionId will be a large string value created by the container.

```
String str = session.getId();
pw.println("sessionId = " + str);
```

2) isNew()

→ This

(3)

(4)

NewSession → returns true

existingsession → " false.

```
if(session.isNew())
{
} //logic-1.
else
{
}     logic-2.
```

(3) getCreationTime();

This method is going to return session creation time in the form of milliseconds (ms). These milliseconds are calculated from 1970 Jan 1st 00:00 hrll.

we need to convert the milli-seconds value into System-date & Time format by using date class given in java.util. package.

```
long ms = session.getCreationTime();
java.util.Date d = new java.util.Date(ms);
pw.println(d.toString());
```

(4) getLastAccessedTime()

This method will returns the previous request time within a session of client. This method also returns milliseconds which are calculated from 1970 Jan 1st 00:00 hrll.

The first & second request the session creation-time, last-accessed time both are one & same. From third request onwards we will get a difference b/w session creation time & last accessed time.

```
long ms = session.getLastAccessedTime();
```

Setting session-TimeOut period :-

→ While working with sessions at server-side, for each session there is some expiration-time set by the container.

→ By default most of the containers will set expiration time as ½hr (or) 1.

→ If a request gap from client / to another is exceeding the expiration time-set then the session of the client at server-side will be removed.

⇒ We can increase or decrease the expiration time-set by the container. We can do it in any one of the following 2 ways.

(i) programatic approach.

(ii) Declarative " .

→ In programatic approach, we need to call

setMaxInactiveInterval() method inorder to set the expiration time. In this case the input value should be in seconds.

session.setMaxInactiveInterval(300);

→ In declarative approach, if we want to set the session Time-out period, then we need to configure session-Config element in web.xml file.

web.xml

```
<web-app>
    ---
    <session-config>
        <session-timeout>
```

matic & declarative approaches, then programatic value is considered by the container.

The following are the two differences b/w programatic & declarative approach

(a) In programatic approach, the input value will be in seconds. so we can set least session time-out as '1' second. but in declarative approach, the least session time-out period will be 1 minute.

(b) In programatic approach u can change the expiration time from one servlet to another servlet, in declarative approach one time-period we can set it is commonly applicable for all servlets

=> getMaxInactiveInterval()

This method is used to get the session expired time in the form of seconds.

long s = session.getMaxInactiveInterval();

=) invalidate();

This method is used to expire a session immediately.

-> if we click on logout or signout links in a website then internally invalidate() method is called to expire the session of the client.

-> we can expire the session of a client in any one of the following two ways.

(1) By setting the expiration time gap b/w request.

(2) By calling invalidate() method.

The following application is to print session information on to browser?

```
import javax.servlet.*;
import javax.servlet.http.*;
public class Session extends
                    HttpServlet
{
  public void doGet(HttpServletRequest
                    req, HttpServletRespon
                    se res) throws ServletExce-
                    ption, IOException
  {
    HttpSession hs = req.getSession(true);
    PrintWriter pw = res.getWriter();
    pw.println("<h2>");
    pw.println("session id=" +hs.getId()+"<br>");
    pw.println("session state=" +hs.isNew()+"
                <br>");
    pw.println("Inactive Time =" +hs.getMax
                InactiveInterval
                ()+"<br>");
    long ms = hs.getLastAccessTime();
    java.util.Date d1 = new java.util.Date(ms1);
    pw.println("Last access time=" ~~this~~ gel
                + d1.toString());
    pw.println("</h2>");
    pw.close
    pw.println();
```

SessionApp1
→ WEB-INF
→ web.xml
→ classes
→ SessionServlet.java
*.class

Ex:=>

C
La
the
to
→
w
Cont
→
nee
the
→
~~clie~~
by
the
→
tho
clie

http://localhost:2011/sessionApp1/session

SessionID = 4E5459EBHED

Session state = TRUE

Inactive Time = 1800

Creation Time = Mon Sep 12 19:32:55 IST 2011

Last Accessed Time = Mon Sep 12 19:32:59 IST 2011

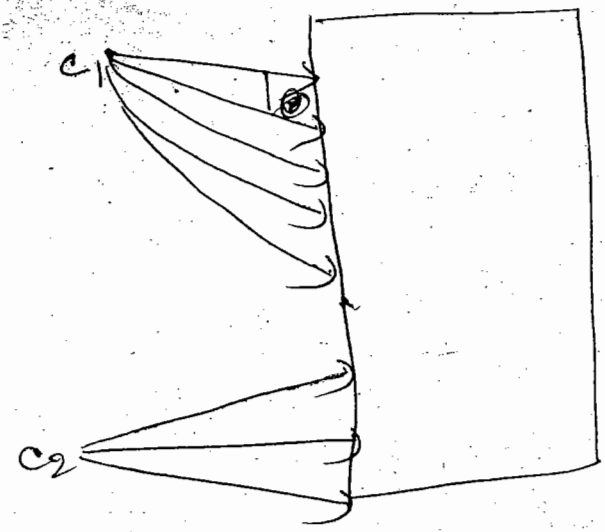
Ex :=> the following web-application is for counting the number of times request is given by a client to the servlet, by using session of the client.

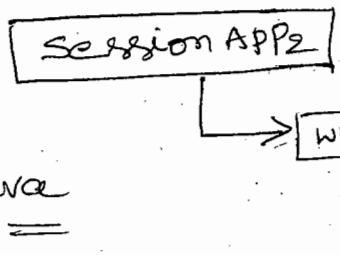
→ To find individual count of each client, we must use session object, because each client contains a seperate session.

→ If the client is new, i.e., if the session is new then start count with 1, & store it into the session object.

→ If the client is existing then, get previous ~~client~~ count value from session & increment it by 1, & again store the new count value back to the session object.

→ If we use context object in this application then we will get common count visited by all clients, but we can not get individual count.





CountServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class CountServlet extend-3 HttpServlet
{
    public void doGet (HttpServletRequest req,
                       HttpServletResponse res);
    {
        int count=0
    HttpSeqvletession hs = req.getSession();
     it (hs.isNew())
      {
          count=1;
       hs.setAttribute("counts", new Integer(count));
      }
      else
       {
```

```
        Count++;
        hs.setAttribute("count", new Integer(count));
    }
    PrintWriter pw = res.getWriter();
    pw.println("<h2> u r visit count=" + count +
                                    "</h2>");
    pw.close();
  }
}
```

web.xml

```
<web-app>
  <servlet>
    <servlet-name> session </s-n>
    <servlet-class> CountServlet </s-c>
  </servlet>
  <servlet-mapping>
    <servlet-name> session </s-n>
    <url-pattern> /session </s-p>
  </s-m>
  <session-config>
    <session-timeout> 1 </session-timeout>
  </session-config>
</web-app>
```

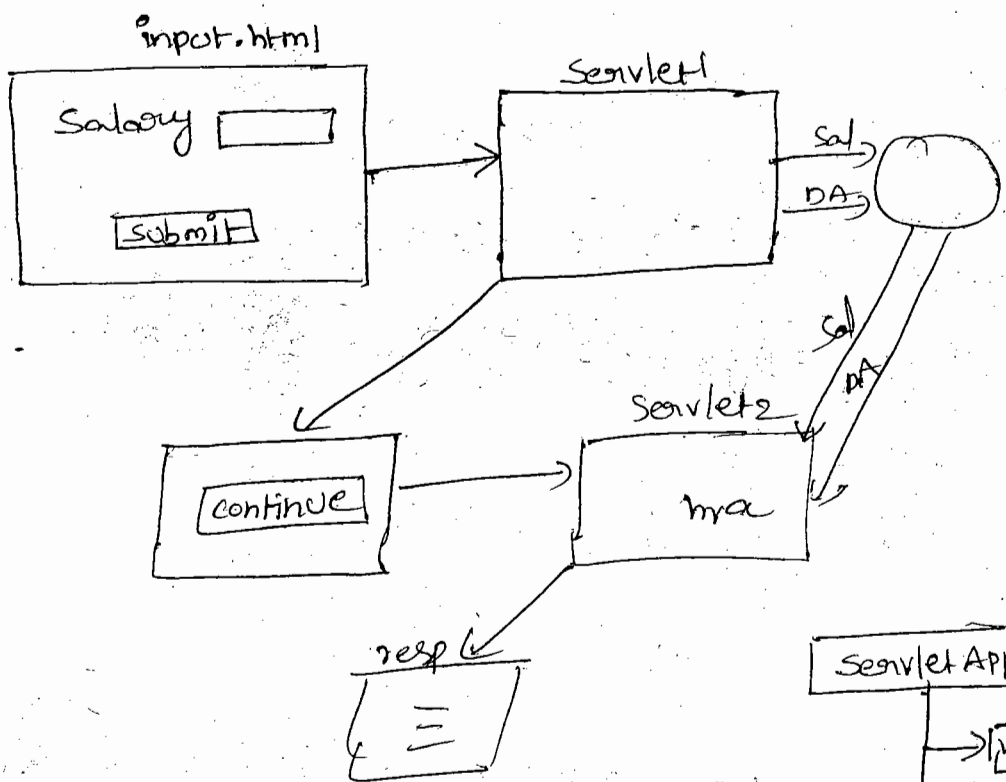
→ when we run the above Example by opening multiple browsers then we will get individual Count

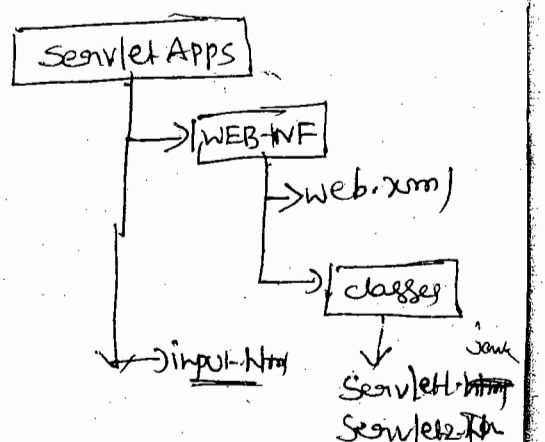
If we click on refresh button of the browser after continuous waiting-time of 1 minute then again count starts at one(1). because after 1 minute the previous session of the client will be expired & a new session will be started.

Ex:

The following web-application is for using session object & across two servlets visited by the client?

In first servlet, we store salary & DA values and 2nd servlet we read both values from session & we are calculating HRA & printing the response on browser?





input.html

<form method="GET"

<form action="srv1">

<input type=submit value="click">
</form>

ServletO.java

```
import javax.servlet.*;
"      "  .*;
"  .http.*;
public class SessionExp3 extends HttpServlet
{
    public void doGet(HttpServletRequest req, Http
                      ServletResponse res)
                      throws SE, IOE
    {
        String s1 = req.getParameter(s);
        int i = Integer.parseInt(s1);
        HttpSession hs = req.getSession();
        hs.setMaxInactiveInterval(10);
        hs.setAttribute("sai", i);
        Double dA = i*0.1;
        hs.setAttribute("DA", dA);
    }
    PrintWriter pw = res.getWriter();
    pw.println("<form action="$rv2">");
    pw.println("<input type=submit value=
                "Continue">");
    pw.println("</form>");
    pw.close();
}
```

Servlet2.java Date: 14/9/11

```
import javax.servlet.*;
import javax.servlet.http.*;
Public class servlet2 extends HttpServlet
{
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws SaLE, IOE
    {
        HttpSession hs = request.getSession()
Object
String s2 = hs.getAttribute(sal);
        int i = integer.parseInt(s2); to
                                  ->0.String()
String s3 = hs.getAttribute(DA);
object
int j = double
Double integer.parse(s3);
                          Double -> to 0.String()
Double k = i*0.20;
        PrintWriter pw = response.getWriter();
            pw.println("salary is =" +i);
            pw.println(" DA is =" +j);
            pw.println("HRA is =" + k);
        }
        pw.close();
    }
```

Cookies:

→ In http session Technique, container internally generates a cookie bcz transferring the session ID b/w server & client.

→ Apart from container generated cookie, a servlet pgmer can also generate cookies bcz storing the data of a client.

3) → If a servlet pgmer wants to create a cookie then Cookie class of javax.servlet.http pkg is used.

→ Servlet pgmer generated cookies are used for state mgmt purpose.

→ With Cookies of Servlet API partial session tracking is possible.

g) → when sessionID is not required and when less number of input values are, submited by a client than instead of using HttpSession Technique, we can use cookies Technique to reduce the burden on a server.

→ A cookie is an object containg text information in the form of key/value pair and it is created on a server, sent it back to the client and stored on a client browser.

→ In servlet pgming, bcz storing the client information, we need to create cookie object.

⇒ These objects are created to Cookie class.

→ Cookie name & value pair req-

Cookie c = new Cookie(name, value)

String type

→ In a servlet, if we create any cookie then container doesn't transfer the cookies from server to Browser or viceversa. this work should be done by servlet only.

From a servlet, if we want to transfer cookie back to the browser then we need to add that cookie object to response object.

→ To add a cookie to the response object, we use addCookie() method.

→ To read cookies from browser to a servlet, we need to call getCookies method given by request object and it returns an array type of cookie class.

response.addCookie(c1);
response.addCookie(c2);
} Cookies are sent from server to Browser

Cookie c[] = request.getCookie(); // cookies are read from browser to servlet

Types of Cookies

→ 1) in-memory cookie :—

→ By default a cookie is an in-memory cookie.
→ An in-memory cookie lives on a browser until that browser is destroyed (closed).
→ An Memory-cookie contains its expiration

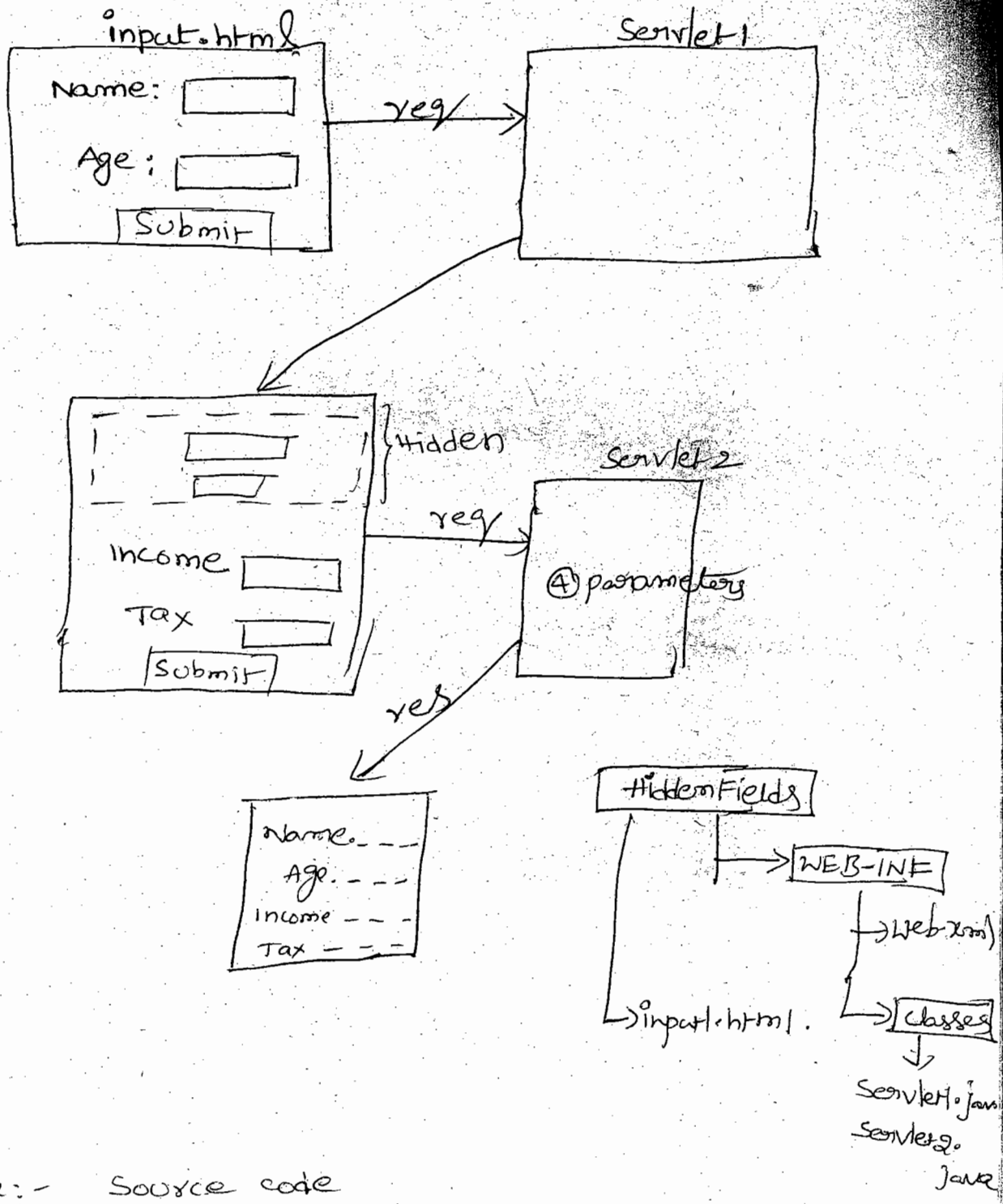
browser when a browser is closed & reopened

Persistent cookie :-

If we set some expiration time to the cookie than the cookie is converted from in-memory type to persistent type. In case of persistent cookie, it lives on a browser until its expiration time is reached. it means, eventhough we close & reopen the browser but still the cookie exists on the browser.



16/9/11



Ex:- Source code

page No: 134, AP-19

Note:

→ The Hidden fields Technique can be applied for both GenericServlet & also for HttpServlet.

→ But to use HttpSession & cookies Technique we need HttpServlet. because in HttpSession & Cookies Technique, we need HttpServletRequest &

4. URL - Rewriting

→ while working with HttpSession Technique complete session - tracking is possible. it has a major drawback.

→ in HttpSession Session-id is transbered from server to browser and next from browser to server by storing in a cookie. when cookie is transbered from server to browser, a browser can accept that cookie or even it can reject the cookie. if the cookie is rejected then when next request is given to the server, then along with request- a cookie doesn't go to the server.

→ in the above case the container will treat the client as a new client again & again creates a new session-ID & session object for that client. it means the session tracking is not working properly.

→ Finally HttpSession drawback is, if browser accepts cookie then the Technique works properly and if browser rejects cookie then the Technique becomes failed. inorder to overcome the drawback of HttpSession Techniq

- url - rewriting technique

transfered in the form of cookie and also the session-ID will be appended to the url when next request is going to the server, eventhough cookie is rejected but session-ID goes to the server from url. So that Container recognizes that this client is an existing client but not as a new client.

→ url-Rewriting is entirely HttpSessionTechnique only & the sessionID add to the address bar by using a method called

encodeUrl().

```
UrlRewriting = HttpSession + encodeUrl()
```

→ In url-rewriting mechanism sessionID is added to the addressbar with a key called JsessionID.

→ To reject cookies from web-sites on to internet explorer then the following step is required

Tools → internet options → privacy tab → move the settings ticker on to the top (then cookies are blocked) → Apply → ok

How can we say that a web-site is using Url-Rewriting Technique or not?

A:- if ~~JSESSIONID~~ sessionid is printed in the addressbar then we can say that Url-Rewriting Technique is used.

Why All Real-time websites uses Url-rewriting mechanism?

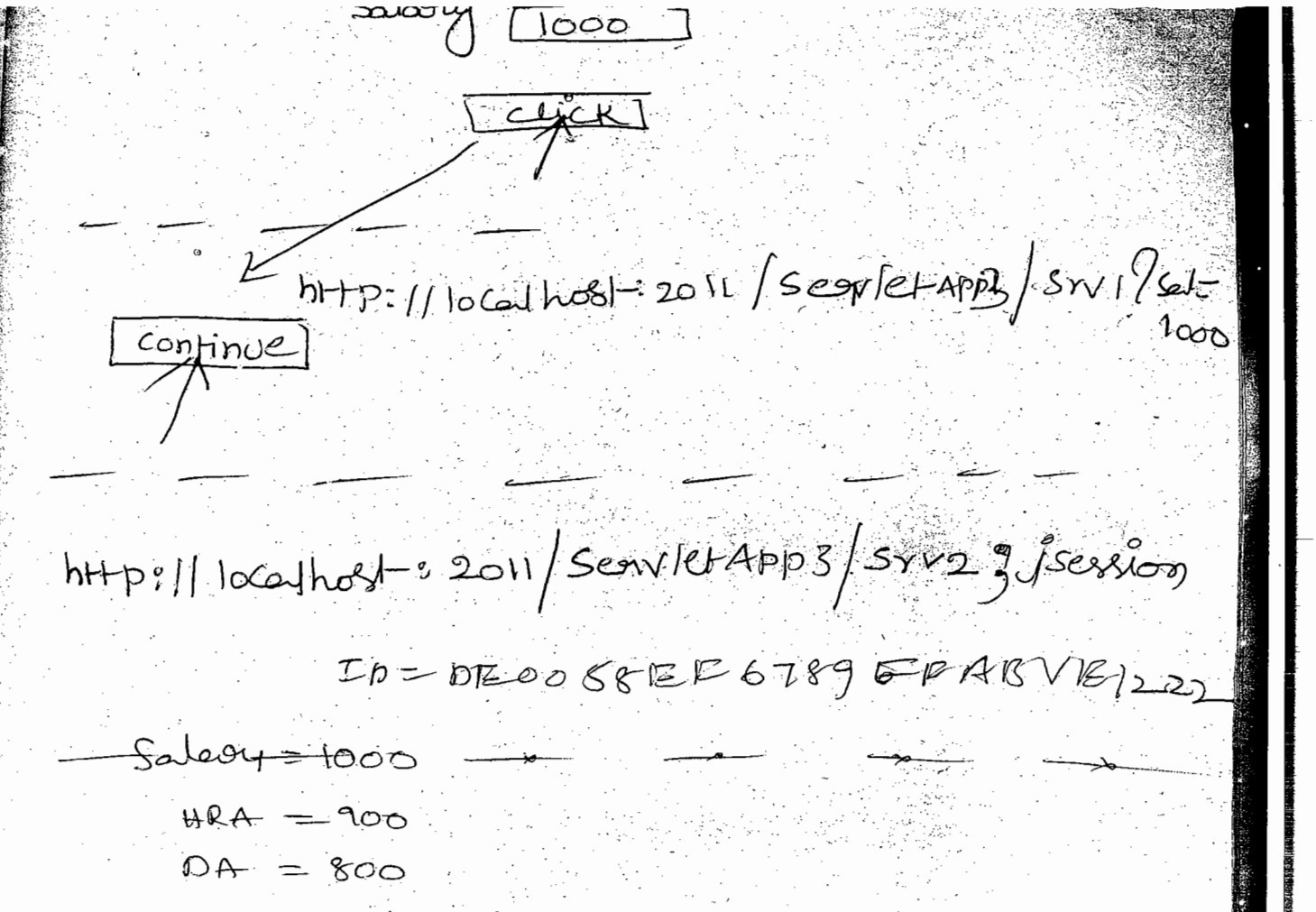
A:- The Real-time applications developed at server-side (or) unknown whether a client browser is accepting (or) rejecting the cookies but sessions should work properly eventhough cookies are accepted (or) rejected by the browser. for these reason the real-time websites uses Url-rewriting mechanism.

Ex:-

The sessionapp3 is now can apply .. to URL Rewriting we do the following changes in .. Servlet1.

```
pw.println("<form action="+res.encodeUrl("Srv2")+">");
```

Execution:


salary 1000
CLICK
http://localhost:2011/ServletApp3/srv1?sal=1000
Continue
http://localhost:2011/ServletApp3/srv2;jsessionID=DE0058EF6789EFABVIE122
Salary=1000
HRA=900
DA=800

Date
17/9/11

Image Response

→ To send an image as a response from the servelet back to the browser the following changes are required.

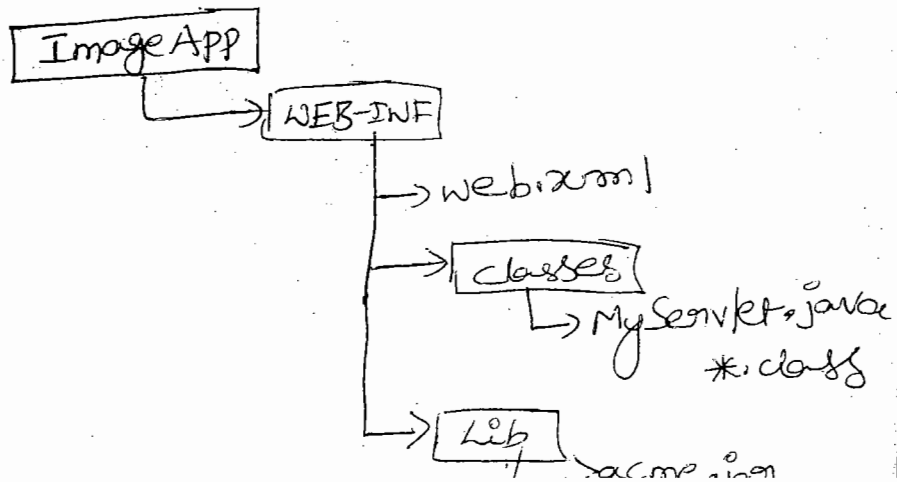
(i) Set MIME type (or) Content type as image by using ~~Application~~ image/~~zip~~.gif

(ii) To transfer image response, we need ~~servlet output~~ ServletOutputStream as an outputStream to write the response. in this case PrintWriter class is not suitable.

(iii) use an image encoder class, to store that image file into the outputStream object.

Ex:

The following Example is to send an image as a response back to the browser when a request given to the browser.



```
import javax.servlet.http.*;
import java.io.*;
import java.awt.Image;
import javax.swing.ImageIcon;
import Acme.JPM.Encoders.GifEncoder;

public class MyServlet extends HttpServlet
{
    Public void doGet(HttpServletRequest request, HttpServlet
                                        Response Response)
{
        response.setContentType("image/gif");
        ServletOutputStream out = response.get
                                        outputStream();
    //obtain image icon
    ImageIcon imageIcon = new ImageIcon
                                ("c:/rabbit.gif");
    //obtain image from image icon
    Image image = imageIcon.getImage();

//Encode the image and send to the
output stream
    GifEncoder g = new GifEncoder(image, out)
        g.encode();
        out.close();
}
```

-> in the above servlet programs we have to use a third Party class called GifEncoder, so we need a jar file to set in the class Path called acme.jar

-) to compile the above servlet we need the following two jar files

(i) servlet-api.jar
(ii) acme.jar.

-) we need to copy acme.jar file into lib folder, in Folder to Execute the web-application at server-side.

-> copy the image in the specified drives

=) Request

http://localhost:2011/imageAPP/srv1)

Exception Handling in Servlet

(i) when a client request is given to servlet, if any exception is occured at servlet code then its stacktrace information will be printed on to the browser.

(ii) The end-user is unknown about end-user can not

instead of sending exception on to the browser, we can send some error message in a client understandable format back on the browser.

(iv) to do this we need to apply exception handling in servlet.

(v) In a servlet, exception can be handled in two approaches.

(i) programatic approach.

(ii) Declarative "

→ In programatic approach, we should include try & catch blocks in the servlet code & if an exception occured we should send an error page as a response back on the client browser from the catch blocks.

ex:

```
public class TestServlet extends HttpServlet
{
  public void doGet() throws SE, IOE
  {
    try
    {
      ==
    }
    catch()
    {
      response.sendError("/error.html");
    }
  }
```

→ In Programatic approach, we should include try & catch blocks in each servlet individually & it increases burden on the servlet pgmer. instead of handling the Exceptions in each servlet seperately, we can also handles the exceptions by configuring it into web.xml file we call it as a declarative approach.

→ In case of declarative approach Exception handling become global for all servlets & jsp's available in a web-application.

→ To configure an exception we should use <error-page> tag in the web.xml

Ex:-

web.xml

```
<web-app>
  — — —
  — — —
  — — —
  <error-page>
    <exception-type> java.lang.Exception
    </exception-type>
    <location> /error.html </location>
  </error-page>
  — — —
  — —
</web-app>.
```

```
<web-app>
  = =
  = =
  <error-pages>
      <error-type> java.lang.NullPointerException </error-type>
      <location> /error2.html </location>
  </error-page>

  <error-pages>
      <error-type> java.lang.ArithmeticException </error-type>
      <location> /error3.html </location>
  </error-pages>
  — — —
  — —
</web-app>
```

Note: If exception is handled in a servlet (programmatically) & declaratively (web.xml) then the Preference will be given to Programmatic approach.

→ If we apply Exception-handling in a Servlet by Putting try & catch blocks then it is Called local exception handling & if we handle the exception through web.xml then it is Called Global - exception handling.



JSP

Java Server pages (JSP)

Q) Why Jsp?

A:- The first technology given for the development of web-applications is Common Gateway Interface (CGI) with CGI technology, there are some drawbacks, so Sun-microsystem released another technology for the development of web-applications called Servlet.

⇒ while working with servlet technology, Sun-microsystem identified some problems

a) servlet technology need in depth "Java code".

b) Servlet Technology is failed to attract the developers working at other Technologies into sun-microsystem technologies side.

⇒ In order to get the market, the microsoft released a webtechnology called "Active Server page" (ASP)

⇒ "ASP" contains a set of predefined tags to develop the web-applications, but it doesn't required in depth coding.

⇒ To overcome the problems in servlet technology developers are migrated

to ms- technologies side.

⇒ In order to overcome the drawbacks of Servlet Technology and to make competation to ASP, Sunmicrosystems released another web-technology called "Jsp".

① CGI
② Servlet
③ server pages

a) Asp -> MS
b) Jsp -> SUN
c) Psp -> oracle.

Differences b/w servlet and Jsp

Servlet	Jsp
① Servlet need lot of java code.	① Jsp need simple tags.
② In servlets, if we modify the code, then we need recompilation, reloading and	② In Jsp, if we do any modifications, then just we need to click on refresh button and recompiling reloading are not required so it is not a time-consuming process.

③ In Servlet, we need to implement business logic, presentation logic combined. It is not possible to seperate business logic & presentation logic. we call these problem as a page centric problem.

④ In Servlet we do not have implicit objects. It means, if we want to use an object then we need to get object explicitly from the server

⑤ In Servlet, by default session mgmt is not enabled. we need to enable explicitly

⑥ Servlet accepts all protocols request

⑦ In a Servlet, all packages must be import on the top of servlet.

③ In JSP, we can seperate, then business logic from the presentation logic, by using Java Bean Technology. so page centric problem is eliminated.

④ In JSP, we have implicit object support.

⑤ In JSP session mgmt is automatically enabled.

⑥ JSP will only accepts http protocol request.

⑦ In JSP packages can be import anywhere i.e., top, bottom or middle.

⑧ In servlet, service() need to be overriden

⑨ In servlet, we have only three scopes i.e.,

request
session
Application

⑩ servlet only supports dynamic including

⑪ Servlet init() we can use parameters.

⑧ In Jsp we ... override _jsp_service()

⑨ In Jsp, we have four scopes.

page
request
session
Application.

⑩ jsp supports both static and dynamic including

⑪ jsp init() does not use the parameters.

Q) what is a Jsp?

Jsp is another web Technology given by "SUN" for the development of dynamic web-pages on to the client browsers.

=> A Jsp is called as page but not a program, because a jsp Contains totally tags.

=> A servlet is a program, we insert "Complex business code".

=> Jsp page writer is called as "author" and a servlet programmer is called as "developer".

=> Every jsp is internally converted into a servlet and by the server (container)

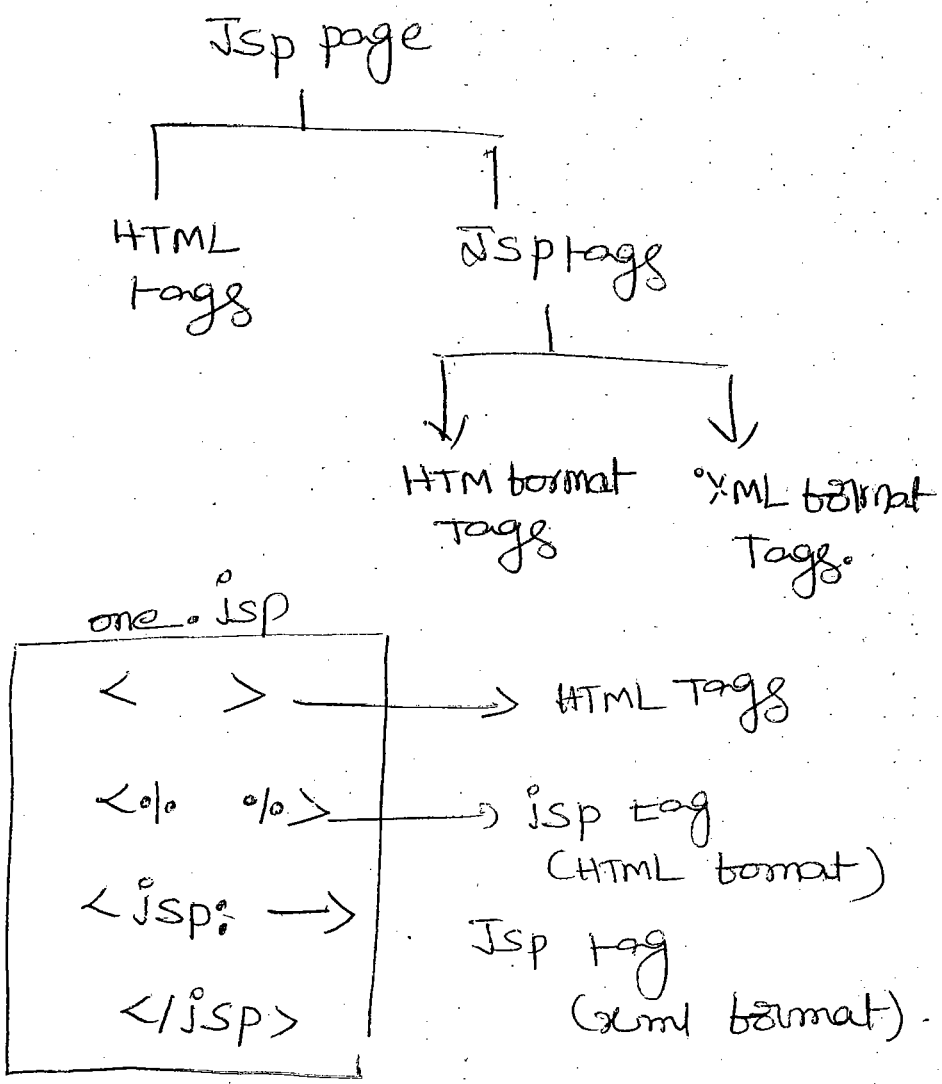
Jsp Tags

-> A Jsp page Contains both "html tags" and "jsp tags".

-> HTML tags will produce "static output" and jsp tags will produce "dynamic output".

-> Because of jsp tags we call a jsp has a

"dynamic web page".



Jsp Translation :-

=> Each jsp page is internally translated into an equivalent Servlet.

=> In a Jsp page construction, the programer writes some html tags and most of the jsp tags inside it.

=> Every jsp tag written in the page will be internally converted to an equivalent Java code

by the containers. we call this process as "Translation".

⇒ In Jsp runtime, there are two phases

1) Translation phase.
2) Request Processing phase.

⇒ Translation phase means converting a jsp into an equivalent servlet and then generating class file of the servlet.

⇒ Request processing phase means, executing service method of Jsp and generating response data on to the client browser.

⇒ when first time request given to a jsp then translation phase occurs first and then after service phase will be executed.

⇒ From next request from the jsp, only request processing phase execution but translation phase is not executed because jsp is already translated.

⇒ If a request is given to the jsp after it has modified then again translation phase is executed and after that request processing phase/service

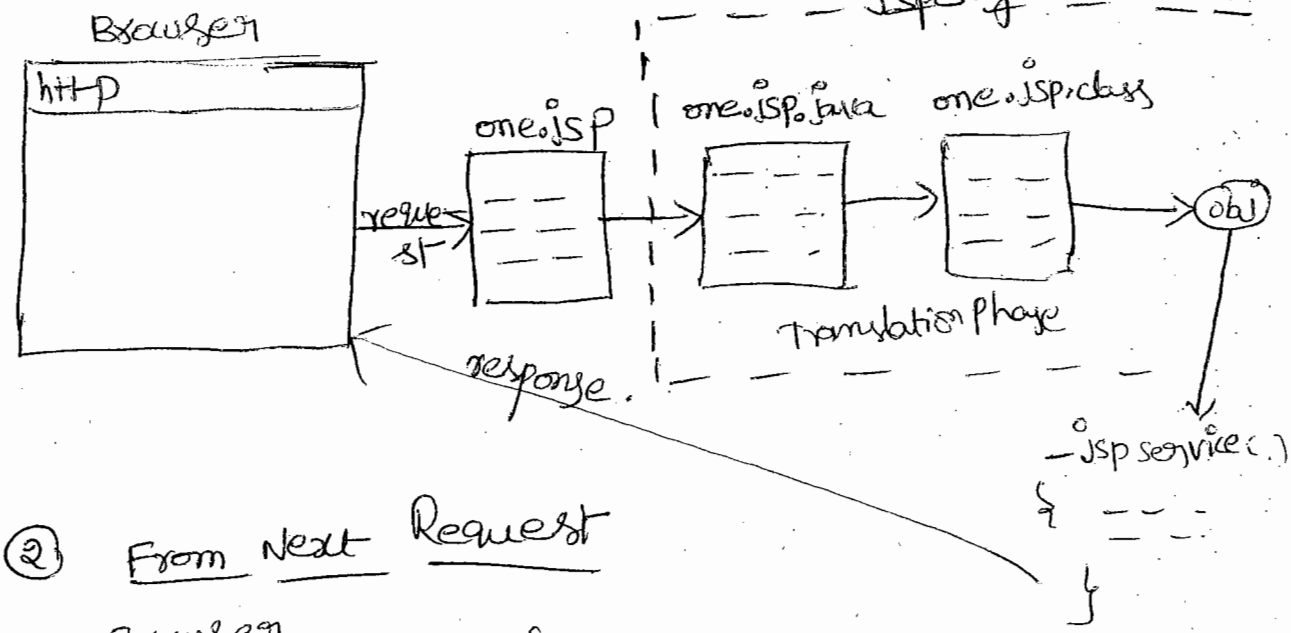
⇒ In the following

(i) when first request is given to the jsp.

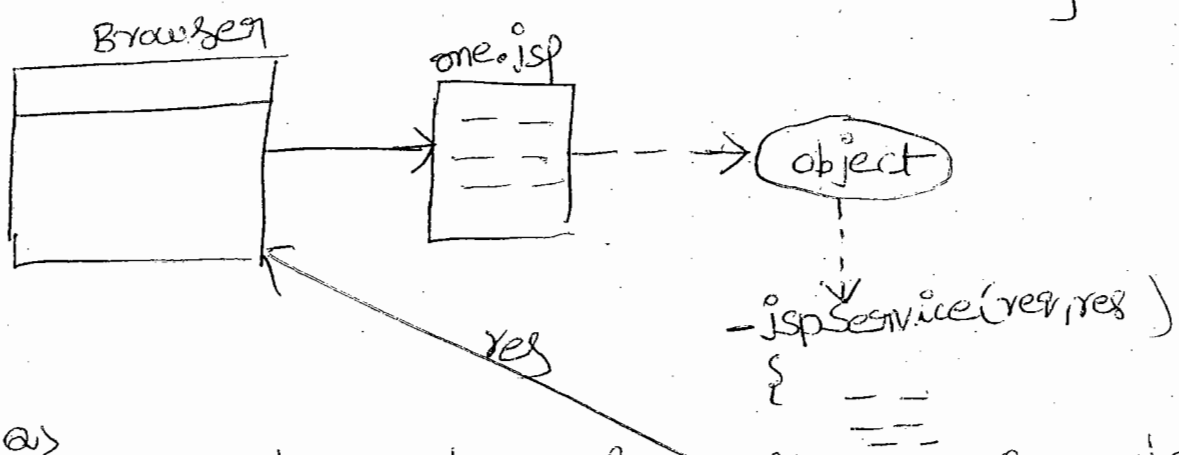
(ii) when a jsp is modified.

Jsp Translation

when first request of when Jsp is modified



② From Next Request



Q) How the web container will recognize when a jsp is modified or not?

A:- Jsp container will use file Compression tools like Araxis to verify whether a jsp is modified or not.

⇒ File Comparision tools internally uses timestamp information to check whether modified

cations are done on a file or not.

Note:

1) If we shutdown & restart the server then only requestProcessing phase will be executed. because, when we shutdown the server, only servlet object is destroyed but the servlet java and class files are not deleted from the server.

2) If we remove the class file from the server then partial translation occurs.

Lifecycle - methods of jsp

① jspInit();
② _jspService(req,res)
③ jspDestroy();

→ As a jsp programer, we can override jspInit() and jspDestroy(). but we cannot override _jspService() to inform the programer that we should not override. we inform the service() the method name is started "underscore" symbol.

⇒ when a jsp is translated into an equivalent servlet then that servlet class extends some base class provided by the server. but "indirectly"

④ <%

a+b, c+d;

%>

⑤ <%= a, b %> -> Invalid, becoz it is equll to out.println(a, b)§

We can't print the vooiable by seperating with.

⑥ <%= a+b; %> -> Invalid, because it is eqult to out.println(a+b;)§

⑦ <%= request.getParameter("uname") %> -> valid.

In this expression we have used an implicit object request, This expression print "uname" Request parameter value on to the Browser.

In jsp, the implicit objects are allowed into the tags, which are inserted into _jspService method.

An expression tag _jspService method only.

So implicit objects are allowed in the expression tag.

In declaration tag, implicit objects are not allowed because declaration tag goes to outside of _jspService method.

Ex:-

```
<%!
    String s = request.getParameter("uname") %>
```

↓

Invalid

Ex:-

one.jsp

```
<%!
①
    private int a=10, b=20;
    private String s="satya";
%>
②
<%= a+b %>
<%= s %>
<%= a+" "+b %>
```

inserted outside the service method

Declaration tag

Expression tag

inside the service method

one_jsp.java

```
public class one_jsp extends HttpJspBase
{
    (1)
    Private int a=10, b=20;
    private String s="string";

public void _jspService(req, res) throws SE, IOE
{
    =
    (2)
    out.print (a+b);
    "    "  (s);
    "    "  (a+" "+b);
    =
}
```

o/p

30

satya

10 20

_jsp

③ Scriptlet tag :-

These tag is used to insert java code in a jsp. eventhough there are multiple tags given in jsp but still we need to insert some small piece of java code also in a jsp.

scriptlet tag is used to implementig the business logic in a jsp.

scriptlet tag goes to _jspService(), during the translation. It means scriptlet tags are executed to each request.

```
<%
    Java code
%>
```

Ex:-

```
<%
    String s1 = request.getParameter("name");
    String s2 = request.getParameter("pwd");
    if(s1.equals(s2))
        out.println("ok");
    else
        out.println("sorry");
```

_jspService goes to

→ We can use implicit objects of the JSP in a scriptlet tag also, because scriptlet tag goes to -_jspService() only.

→ In a JSP implicit objects are allowed in expression tag & scriptlet tag but not in the declaration tag.

Ex:-

```
<%!
    int a=100;
%>
```
→ Declaration Tag

```
<%
    int a=49;
%>
```
→ Scriptlet tag

```
<%= a %>
<%= this.a %>
```
→ Expression tag.

=) In the above Example we can create variable a in both declaration tag & Scriptlet Tag. ✓

⇒ In this declaration,

declaration tag variable → global variable

scriptlet tag variable → local variable.

Expression tag also insert inside the _jspService so it give the preference to local variable i.e, scriptlet tag variable. if we want to print global variable we refer it by using 'this' keyword.

③ <%!

```
public void m1()
{
=
}
%>
```

} Invalid.

⇒ ⊗ Invalid. Because the scriptlet tag goes to _jspService method. In java nested method definitions are not allowed.

In a scriptlet we can declare variables but not methods. because variables will become

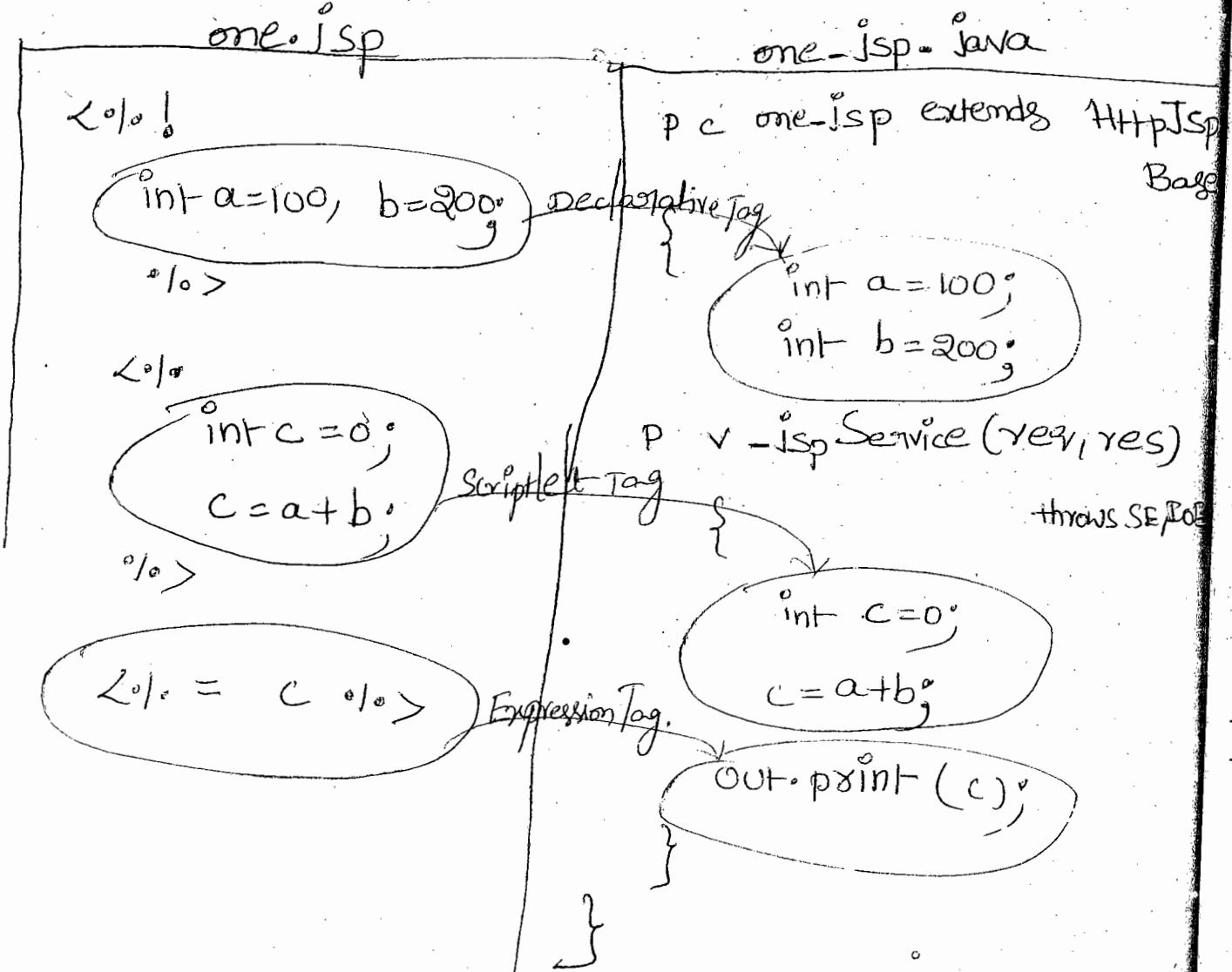
(4)

```
<%
    int a = 40;%>
    <%= a %>
%>
```

} Invalid.

Invalid. because we can not write one scripting element in another scripting element.

Ex:



=> The scripting elements of jsp can we written either in HTML syn or in XML syn.

① HTML syn	XML syn
I Declaration tag <%! int a=200, b=100; %>	I Declaration tag <jsp: Declaration> int a=200, b=100; </jsp: Declaration>
II Expression tag <%= expression %>	II Expression tag <jsp: expression> expressions </jsp:expression>
III ScripHet Tag <% Java code %>	<jsp: scriplet> java code </jsp: scriplet>

JSP comments

(i) JSP Comment | Hidden comment

(ii) HTML comment

(iii) Java Comment.

JSP comment

This type of comment is called as hidden comment because it is invisible when a JSP is translated into a servlet internally

Syn:-

<%-- comment --%>

Ex:-

<%-- one.jsp --%>

(ii) HTML comment

It is a common tag to HTML, XML and JSP.

In a jsp page, if we write HTML Comments, then these Comments are visible - jspService() method of the internal servlet.

Syn:-

```
<!-- comment -->
```

Ex:-

```
<!-- one.html -->
<!-- web.xml -->
```

(3) Java comment

We can write either a single or multiple line Comments of java in a Scriptlet tag of the jsp java Comments are allowed only inside scriptlet tag because we can insert java code in jsp only at Scriptlet tag.

Syn:

```
// Single Comment
/* ---- */ multiline.
```

Ex:

```
<%
// if Condition
```

Configuring a jsp in web.xml

Configuring a jsp into web.xml file is optional. because jsp is a public-file of the web-application.

A jsp is called a public file, & A Servlet is called a private file & the web-application because jsp file stored in root directory of the web Application, & servlet class file is stored in subdirectory of the webapplication.

→ Because jsp is a public file, we can directly send a request from a browser by using its filename.

→ If we want to configure a jsp in web.xml file then the xml elements are same as servlet-configuration. we need to replace servlet-class

element with jsp_file. element

Ex:- web.xml

```
<web-app>
    <servlet>
    <servlet_Name > test </s-N>
    <jsp_file > /one.jsp </jsp_file>
    </servlet>
    <servlet_Mapping>
        <s_n> test </s-n>
        <u-p> /sr1 </u-p>
    </s_m>
</web-app>
```

If we configure a jsp in web.xml then we can send request to the jsp either by using jsp_file name or by using its <url-pattern>.

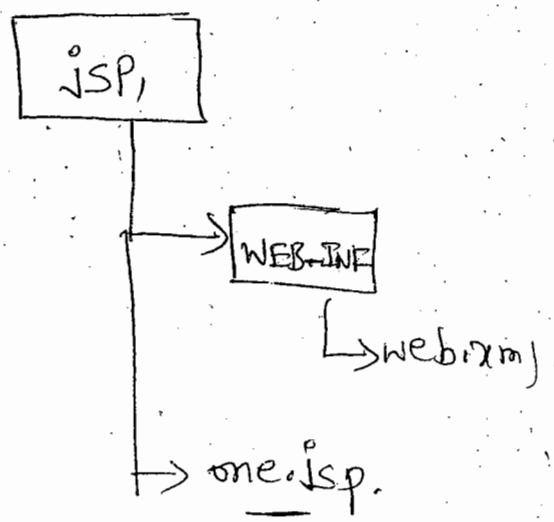
http://localhost:2011/root/one.jsp

http://localhost:2011/root/sr1

Ex:10

The following jsp application is for counting the number of Requests given to the jsp.

Ex:- one.jsp




```
<%-- one.jsp --%>
<%!
    int count=0;
%>
<%
    count++;
%>
<font color='green' size=22>
This is the request: <%= count %>
</font>
```

⇒ web.xml

```
<web-app>
</web-app>
```

O/P http://localhost:2011/jsp1/one.jsp

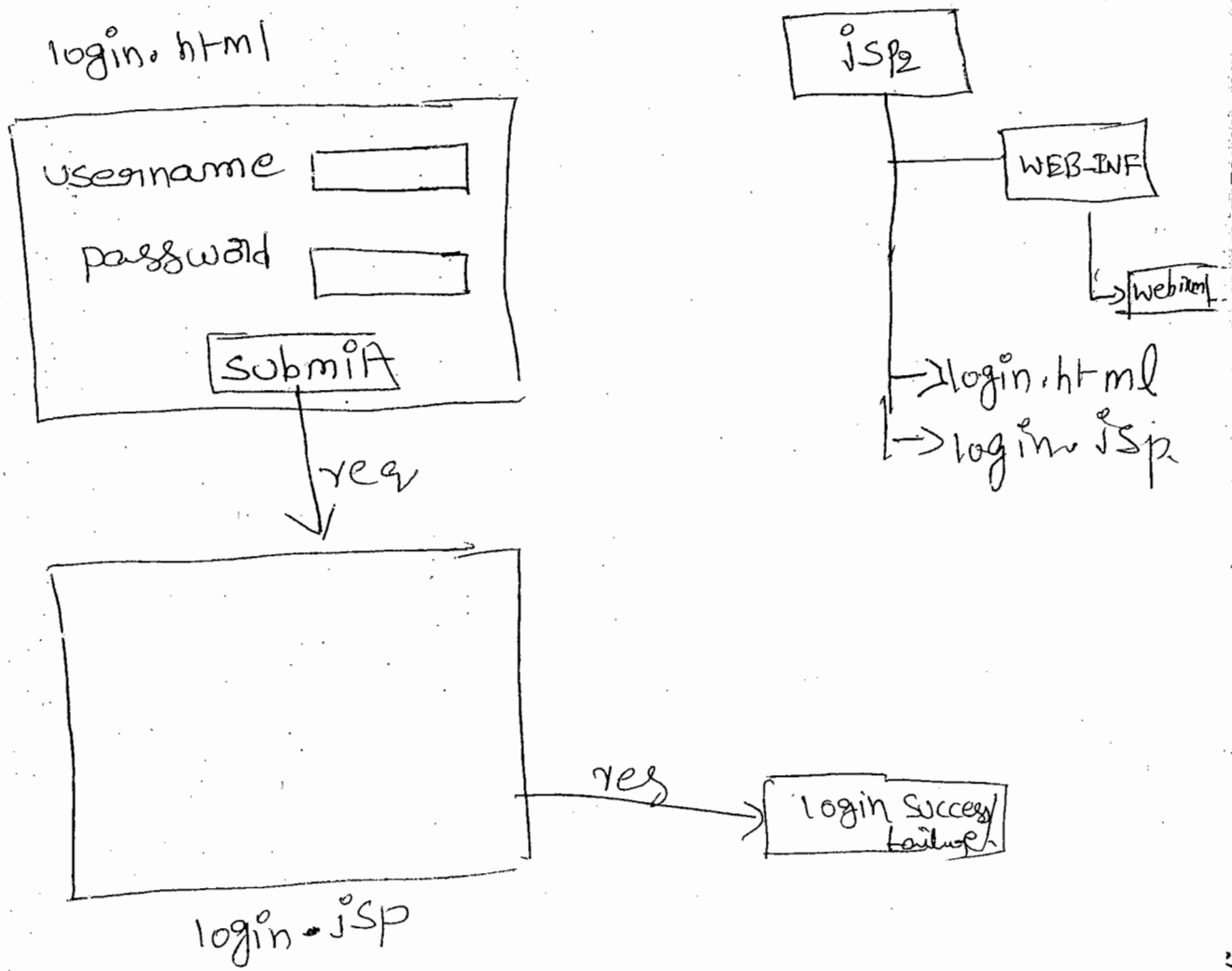
This is request:1.

The Servlet class generated by the container is available at the following location.

C:\Tomcat 6.0\work\Catalina\localhost\jsp1\org\apache\jsp

pkg folder

Ex:-2 ⇒ The following example is for communicating from HTML to jsp for a login app?



Login.html

```
<!-- Login.html -->

<form action="Login.jsp">

UserName: <input type=text name="t1"><br>

password: <input type=password name="t2"><br>

<input type="submit" value="click">

</form>.
```

Login.jsp

```
<%-- login.jsp -->

<%

String s1= request.getParameter("t1");

String s2 = "                "("t2");

int k = verify(s1, s2);

if(k==1)

out.println("login success...");

%>

<%!  int verify (String p, String q)
```

```
{
  if(p.equals("satya") && p.equals("sathya"))
    return 1;
  else
    return 0;
}
%>
```

=> If jsp is translated after giving the first request,

<load-on-startup> 1 </load-on-startup>

(1) jsp life-cycle are provided by two interfaces of the jsp-API

(i) Javax.Servlet.jsp.JspPage

(ii) Javax.Servlet.jsp.HttpJspPage.

-> JSP page interface has provided two life-cycle methods

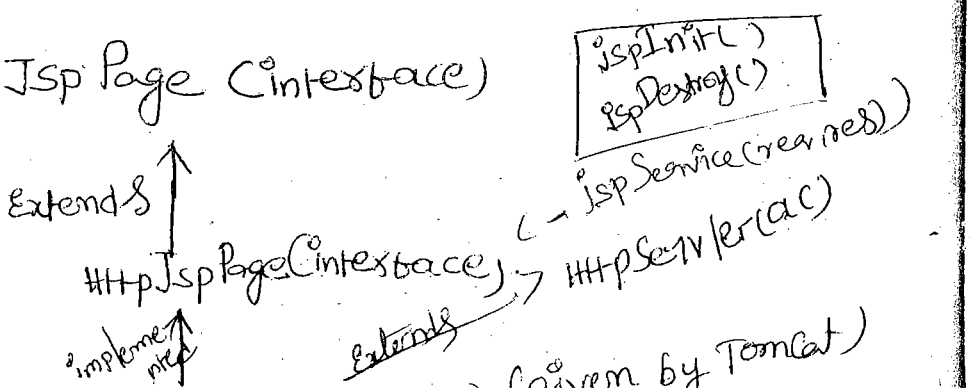
(i) jspInit()

(ii) jspDestroy()

-> HttpJsp page interface has provided a single life-cycle methods called _jspService(req,resp).

-> In case of Tomcat Server the Tomcat Container Extends the internal Servlet from HttpJspBase class.

-> HttpJspBase is an abstract class Extended from HttpServlet and implemented from HttpJsp Page Interface.



→ In HttpJspBase class given by Tomcat, there is one abstract method. so the class is given as an abstract class.

→ In HttpJspBase class, both Servlet life-cycle methods and jsp life-cycles are overridden and jsp cycle methods are called from Servlet life-cycle methods.

→ A webContainer can only called servlet life cycle but not jsp life-cycle. so container calls servlet life-cycle and Servlet-life cycle calls jsp life-cycle.

Ex:-

```
public abstract class HttpJspBase extends HttpServlet
                                  implements HttpJspPage
{
    public void init(ServletConfig config) throws ServletException, IOException
    {
        jspInit();
    }
    public void service(req, res) throws SE, IOE
    {
        _jspService(req, res);
    }
    public void destroy()
    {
        jspDestroy();
    }
    public void jspDestroy()
    {}
    public abstract void _jspService(req, res)
}
```

Note => For every Servlet class internally created by the "Container", the base class is "HttpServlet" indirectly.

Jsp Elements

① Scripting elements ② Directive Elements ③ Action Elements

④ Custom Elements.

① Scripting Elements

① Declarations ② Expression ③ Scriptlet.

Declaration:-

It is used to create variable & methods in a jsp.

-> The variable and methods will become to global to that jsp. It means in that jsp we can use those variables any where and we can call those methods any where.

-> At the time of trasbation, Container inserts the decloration tag into the class. so the variables will become instance variables and methods will become instance methods.

-> Syntax:

```
<%!
    variables;
    methods;
%>
```

Example:

one.jsp

```
<%
 int x=10;
void m1()
 {
   ≡
 }
%>
```

one_jsp.java

```
public c one_jsp extends
                    HttpJspBase
{
    int n=10;
}
    void m1()
    {
      ≡
    }
P V _jspService (req, res)
                    throws SE, IOE
{
   ≡
}
```

② Expressions

① `<%= a+b %>` } converted ---> out.print(a+b) goes to _jspService(req,res)

② `<%= a+b c+d %>` } invalid, becoz of two expressions

③

```
<%
 a+b+" "+c+d
```

} valid --> out.println(a+b+" "+c+d);

-->

Adding client-side validation to the Ex:

→ If you don't apply validation on client-side to the input then even though the input is not given, the request will be submitted to the server.

→ If a request is submited to the server by without validating the input at client side than the no of round trips b/w clients & server will be increased. So before we submit the request to server, first we need to verify whether the client input is entered (or) not. This is possible by Enabling the client-side validation through javascript.

→ Before submiting the request to server, the first javascript function will be executed and if that fn returns true then the request will be submitted to the server and otherwise the request will not be submited.

=> the Advantage of client-side validation is, we can reduce the no of round trips b/w client and a server.

=> To add a JavaScript function, to the html or jsp page then we should add the script tag under head section and function calling should be done from the body section

=> Jo call javascript function, whenever Submit is clicked then we need to add a javascript event handler to the <form> tag called on Submit

<form action = "Arithmetic.jsp" name = "f1"
onSubmit = "return fun1(this)">

```
<!--- input.html --->
<html>
  <head>
    <script language = "JavaScript">
    function fun1(form)
    {
      var x = document.f1.t1.value;
        alert(x);
      var y = document.f1.t2.value;
        alert(y);
    if (x == "" || isNaN(x))
    { alert("A value is not a number");
```

```
else if (y=="" || isNaN(y))
{
  alert("B value is not a number");
  return false;
}
else
  return true;
}
</script>
</head>
<body>
<center>
<h2>
<form name="f1" action="Arithmetic.jsp"
                onSubmit=return fun(this)">
Enter a: <input type=text name="t1"><br>
Enter b: <input type=text name="t2"><br>
<input type=Submit value="Add"
```

Jsp Directives

=> Directives doesn't produce any output on to the browser directly.

=> Directives are used to add some additional behaviour to the Servlet, during translating a jsp into a Servlet.

=> In jsp we have the following three directives:

① Include ② page ③ taglib.

=> A Directive starts with <%@ and ends with %>

<%@ directivename attribute %>

① Include

-> This include directive is used to include the code of one.jsp into another jsp during translation time.

-> If we include one jsp into another jsp then during translation time, the jsp engine creates a single servlet internally.

-> If we write include directive then code of one jsp is included to another jsp, but not the response at runtime, so we call the include directive as static including (or) Compile time including (or) inline including.

-> In a single jsp page, we can include any no of other jsp's It means we don't have any restriction.

-> while including, both Source and destination Pages must be jsp pages only.

Ex:- f1.jsp

=

<%@ include file = "f2.jsp" %>

=

<%@ include file = "f3.jsp %>

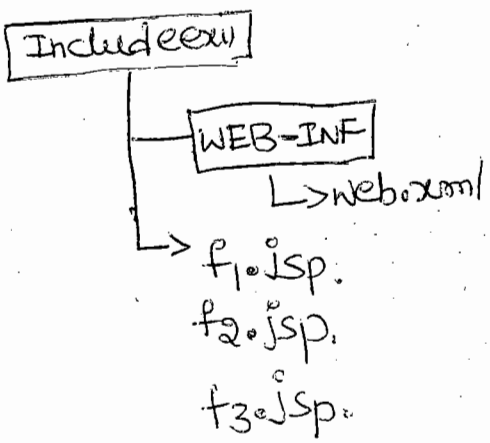
f2.jsp

f3.jsp

f1.jsp.java

Note:-

In Include directive, only one attribute and that is "file".



f1.jsp

```
<%-- --- f1.jsp --- --%>

<%
    out.println("hello");
%>

<h1>
<%@ include file="f2.jsp" %>
<b1>
<%@ include file="f3.jsp" %>
```

f2.jsp

```
<%-- --f2.jsp --- --%>

<%
    out.println("Good mooning");
%>
```

f3.jsp

```
<%-- --f3.jsp --%>
<%= new java.util.Date() %>
```

Request:- http://localhost:2041/includeex/f1.jsp

** note: include directive reduces the burden on a server, because of multiple jsp's Internally, only one servlet is created for one servlet only one object is required. It means no of servlets. objects are reduced

(2) Page Directives

=> This page directive is for setting important behaviours to a servlet during the translation.

=> The important behaviours means like exception handling, session mgmt, packages importing, mime type settings etc.,

=> Syn:

<%@ page attribute %>

Attribute:

(i) Language:-

-> The attribute is given to provide the language supported by the "jsp tags".

-> The default language and the only supported language is "Java".

<%@ page language = "java" %>

(ii) Import:-

-> These attribute is used to import with predefined & userdefined packages into a jsp.

=> while importing the packages into a jsp, we can import any where in the jsp. It means at the beginning of jsp(or) in the middle & at the end.

=> we can import multiple packages also at a time by seperating package name with either with a coma, semicolon.

⇒ Even though we can import the packages at multiple places in the jsp. but during translation all import are placed on top of the Servlet.

⇒ The only one attribute of page directive, that can be "repeated with different values" is import attribute.

Ex:-

```
<%@ page import = "java.util.*, java.sql.*" %>
        ≡
<%@ page import = "java.net.*" %>
        ≡
<%@ page import = "java.text.*" %>
```

③ session

⇒ bol Example-

i) one.jsp

```
<%@ page session = "false" %>
        ≡
<%@ page session = "true" %>
```

} wrong

Ex2

ii)

<%@ page session="false" %>

Ex:-3 one.jsp

<%@ page session="false" session="true" %> → wrong

<%@ page session="true" session="true" %> → Correct

demo.jsp

```
<%-- demo.jsp --%>
<%@ page import="java.util.*" session="true" %>
<h3>
The current Date & Time is: <%= new Date() %> <br>
Session Id : <%= session.getId() %> <br>
Inactive time : <%= session.getMaxInactiveInterval() %> <br>
Session state (new / old) : <%= session.isNew() %> <br>
</h3>
```

4) Error page

-> while executing a jsp, if any exception is occured than that exception stackTrace will be printed on the client browser.

-> If we want to transfer the control to another jsp whenever an exception is occured in the current jsp page then we need to add error page attribute of the page directive.

-> If we add error page to a jsp then the control is

transfered to the error page, only if the exception is occured. otherwise the control will not be transfered.

→ If we add errorPage attribute for the page directive in a Jsp then we call it as "local"

Exception handling

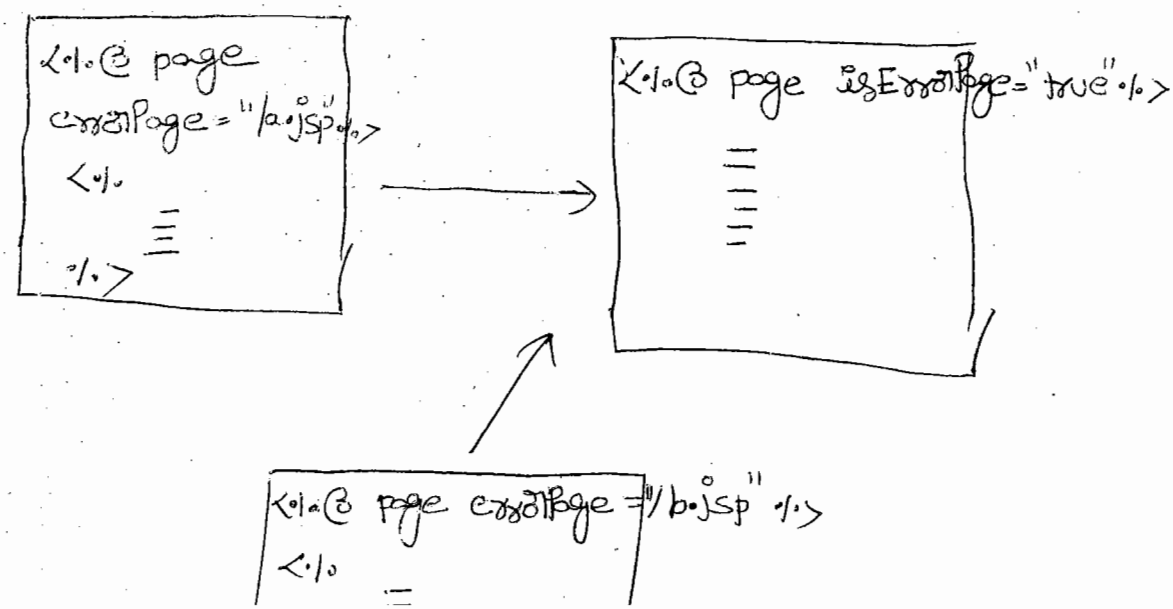
5) isErrorPage

→ This attribute is used to make a jsp as an error page

→ The default value of this attribute is false. It means by default a jsp is not acting as an error page.

→ If we write isErrorPage = "true" then that jsp is acting as an errorpage.

→ If a jsp is acting as errorpage then only an implicity object called "exception" is available in that jsp page. otherwise we cannot access are used exception object in the jsp page.



6) isThreadSafe :-

⇒ The default value of this attribute is "true"

⇒ By depending on the value of isThreadSafe, a container allows either multiple threads (or) single thread to access the jsp page.

⇒ If we write isThreadSafe = "false" then container allows single thread at time to access the jsp page.

→ If isThreadSafe = "true", then container allows multiple threads to access the jsp at a time.

→ If we write isThreadSafe = "false" then the internal servlet created by the container implements "SingleThreadModel" interface.

⑦ Buffer :-

→ the default value of this attribute is "8kb".

→ we can increase (or) decrease the buffer size by using this attribute.. buffer

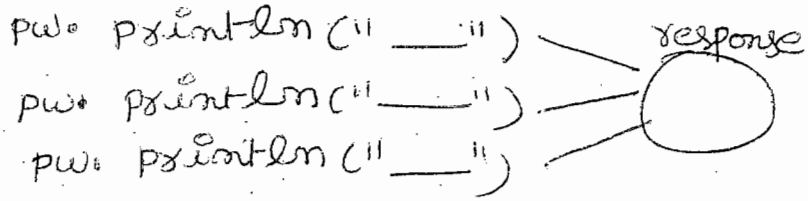
→ In jsp, we use an implicit object for generating response called "out".

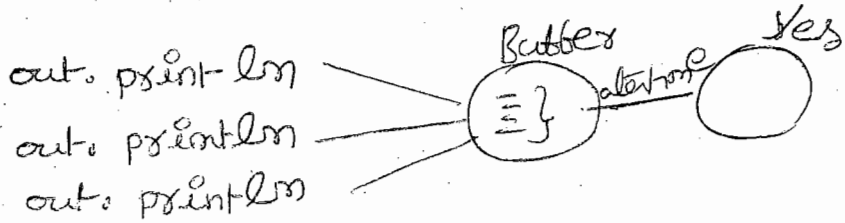
⇒ "out" is an object of "JspWriter" class and it makes use of buffers internally.

⇒ JspWriters is similar to printwriters but the difference is print-Writers doesn't use buffers but Jsp writers makes use of "Buffers"

JspWriters = print-Writers + Buffers

Ex:





First time generated instruction stored in JIT Compiler

JspWriters = print-Writers

⇒ A buffers is a like a cache memory and it first time an application Executed then it makes some slow performance. but when next time onwards it increases the performance of an application.

⇒ In a Jsp page we can do the buffers settings by using buffers attribute

⇒ if we want to make use ... we should make buffer as name.

⇒ Automatically the dat will be opened into response object from a buffer by internally calling flush method.

⇒ we can also explicit call the flush method by using autoflush statement.

*autoFlush

```
<%@ page buffer=10kb autoflush="true" %>
```

→ this attribute is used to set flush mode in jsp flushing means synchronizing the data from buffer memory to response object.

⇒ The default value of autoFlush is True.

⇒ If autoflush mode is disable to writing autoflush is equal to false then in a jsp page the programmer has to call the flush method explicitly

⇒ If buffer is remove than the programer has to set autoflush is equal to false otherwise jsp Container throws an exception

Extends:-

⇒ these attribute is used to provide the class name from which we want to Extends internal servlet class.

→ when a jsp is Converted into equivalent servlet then the internal servlet class Extends some base class which is provided by the Containers Instead of Extending from the Containers given class, we can

also specify our class name by writing Extends attribute

one.jsp

```
<%@ page Extends="satya" %>
```

```
public class one.jsp Extends Satya
{
  =
}
```

→ If we want to specify our class name then our class should Extend the server given class one our class should extend Http servlet and it should implement http jsppage interface.

→ In case of tomcat server the class satya should Extend HttpJspBase class one the class satya should Extends HttpServlet and it should implements Http.Jsppage interface.

```
p class satya Extends HttpJspBase
{
  =
}
```
(a)

```
p c satya Extends HttpServlet implements
                                HttpJspPage
{
  =
}
```

Content type:

① <%@ page contentType = "text/html" %>

(or)

<% @ response.setContentType("text/plain"); %>

-> These attribute is used to set mime type, the default mime type is text/html.

-> If we want to set mimeType in a jsp then we have two ways.

(i) We can call setContentType() or we can use contentType = "text/html"

info

<%@ page info = "this is a sample page" %>

-> This attribute used to write some description in a jsp page, we can also write description by using comment tags but it is not possible to read the description written in the comment tags

-> In case of info attribute we can need the comment one description whenever it is required by calling a method getServletInfo

<%= getServletInfo() %>

(or)

<% String s = getServletInfo();
out.println(s);
%>

isELIgnored

=> EL means Expression language

& by default EL is not Executed in jsp &

<%@ page isELIgnored="false" %> → Enable

=> In a jsp by default EL is disabled: it means EL will not be Executed -

=> The default value of isELIgnored="true", if we make this attribute as false then Expression language will be Executed.

XML:

<jsp:directive.include file="two.jsp" />

<jsp:directive.page import= session ... />

Note:

* jsp directive can be written using xml syntax also

<jsp:directive.page "import="java.util.*" />

⇒ standard actions is used for predefined tags.

⇒ custom tags are used for userdefined tags.

⇒ standard actions base for what and what logics are used in presentation and business logic.

⇒ xml tags are used for standard actions

⇒ usebean tag is used for "object" and check 4 object of the id is reference of object.

standard actions

→ standard actions are given in jsp to seperate the presentation logic and the business logic of the jsp.

→ The name is given as a standard action, because each action as a predefined meaning and as a programer it is not possible to change the meaning the tag.

→ standard actions are not completely seperating the presentation logic with the business logic. but partial seperation of the this two logics is possible.

→ each standard action follows "xml" syntax. We do not have any "equivalent" html syntax.

→ The standard actions given by jsp are

1) <jsp:useBean> 2) <jsp:setProperty> 3) <jsp:getProperty>
4) <jsp:forward> 5) <jsp:include> 6) <jsp:param>
7) <jsp:params> 8) <jsp:plugin> 9) <jsp:fallback>

1) <jsp:useBean>

→ In jsp, it is possible to seperate the presentation logic from its businesslogic

→ If we want to seperate both presentation logic and business logic in jsp. we need to create a java bean.

→ we can implement the business logic into a javabean and we can call that business logic from a jsp.

→ The advantage of seperating business logic into a javaBean class is, we will get reusability. It means that javaBeans business logic can be called from any jsp (or) multipe jsp's.

→ In a jsp, if we want to call the business method of the javabean class, then first of all we need an object of the javabeen class into a jsp.

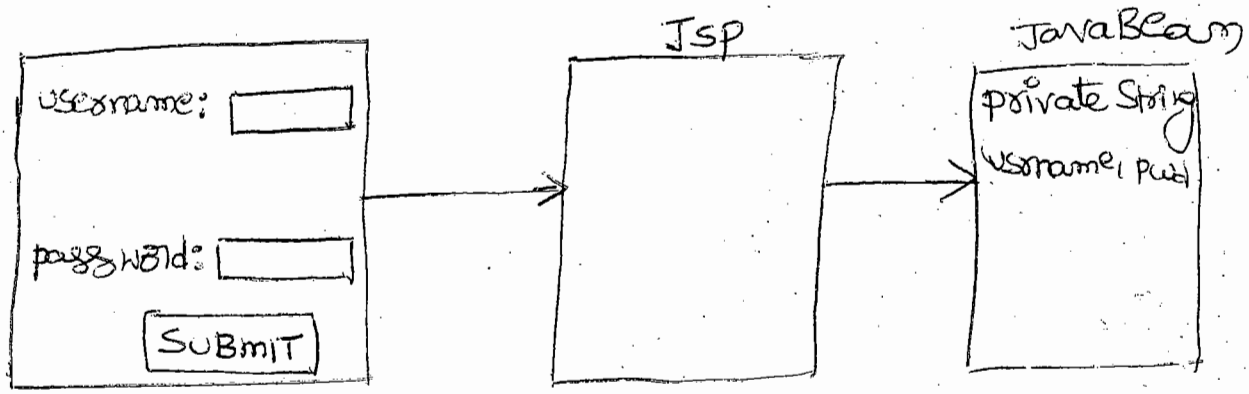
→ In order to create a javabean object in a jsp we need "<jsp:useBeans tag".

(10) another object reader from jsp

→ To say a java class as a javabean, the following rules are required: i) the class must be stored in a pkg. (ii) class must be public (iii) class should implements serializable (iv) class must have public default constructor.

Date ex:1

5/8/11.

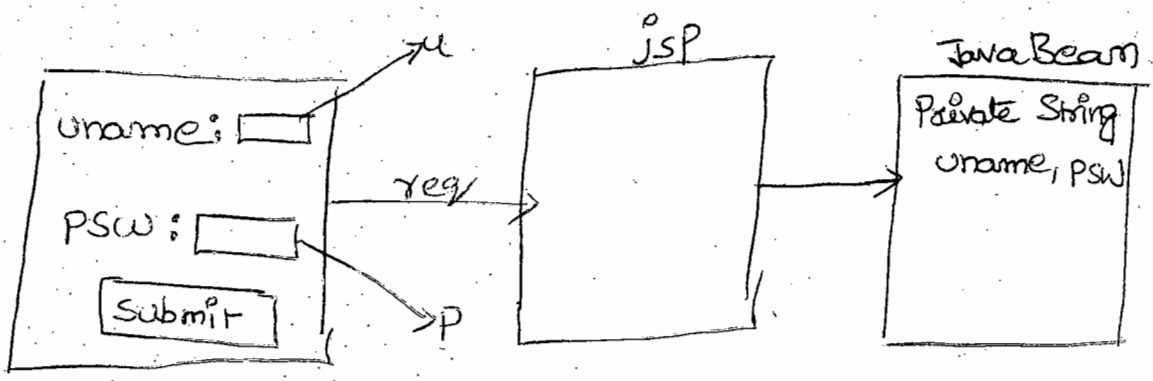


(i) <jsp:setProperty /> must be used inside <jsp:useBean>.

(ii) In the above example, request parameter names & Bean class variable names both are same. so we used property="*"

```
<jsp:useBean id="id1"
             class="pack1.LoginBean" />
<jsp:setProperty name="id1"
                 property="*" />
</jsp:useBean>
```

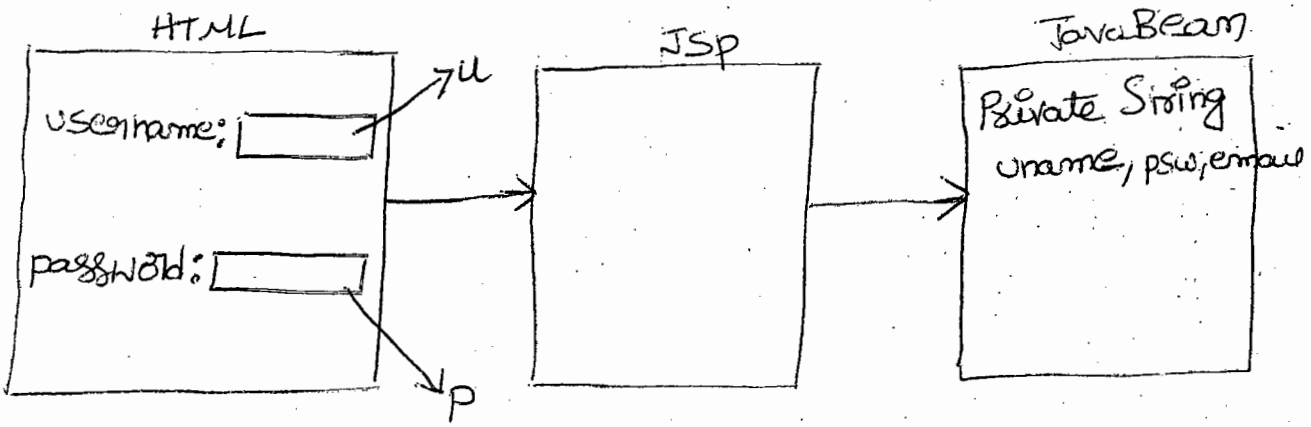
Ex:-2




```
<jsp:useBean id="id1"
             class="pack1.LoginBean" />
<jsp:setProperty name="id1"
                 property="uname"
                 param="u" />
<jsp:setProperty name="id1"
                 property="pwd"
                 param="p" />
</jsp:useBean>
```

=> In the above Example, request parameters names & Bean class property names can not matching so we can not write property="*", so we have set properties individually an additional attribute called param is required

Ex: ③




```
<jsp:useBean id="id1"
             class="pack1.LoginBean" />

<jsp:set-Property  name="id1"
                   property="uname"
                   param="u" />

<jsp:set Property  name="id1"
                   property="pwd"
                   param="p" />

<jsp:set Property  name="id1"
                   property="email"
                   value="abc@gmail.com" />

</jsp:useBean>.
```

⇒ value attribute used to Directly set the value into the JavaBean variable through JSP.

jsp: setProperty totally contain 4 attributes they are

1. name
2. property
3. param
4. value.

⇒ possible settings

1. name, property
2. name, property, param
3. name, property, value
4. name, property, param, value ✗ (not possible)

3. <jsp:getProperty>:

This standard action is used for reading value of a JavaBean property into a jsp. whenever this tag is used in the jsp then getter method of the property is call.

⇒ It is not possible to read all properties of the JavaBean at a time by calling the getter methods, it means we can not write property="*".

⇒ Always <jsp:getProperty> tag must be used at outside of <jsp:useBean>.

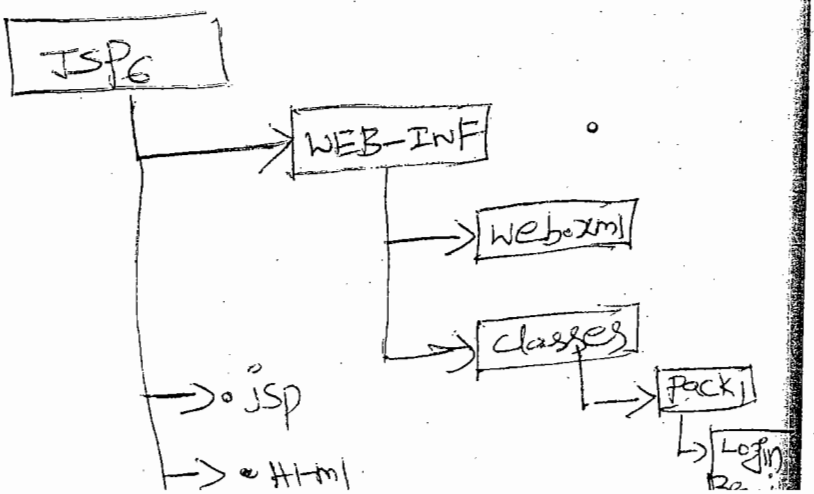
⇒ only two attributes are allowed into <jsp:getProperty> those are name & property.

<jsp:getProperty name="id1" property="uname" />

<jsp:getProperty name="id1" property="pwd" />

Ex:- The following Example is from jsp to JavaBean Communication for setting values into the Bean and getting values from the Bean:-

Directory structure



① Login.html

```
<!-- login.html -->
<form action = "login.jsp">
username : <input type = text name = "uname">
<br>
password : <input type = password name = "pwd">
<br>
<input type = submit value = "click">
</form>
```

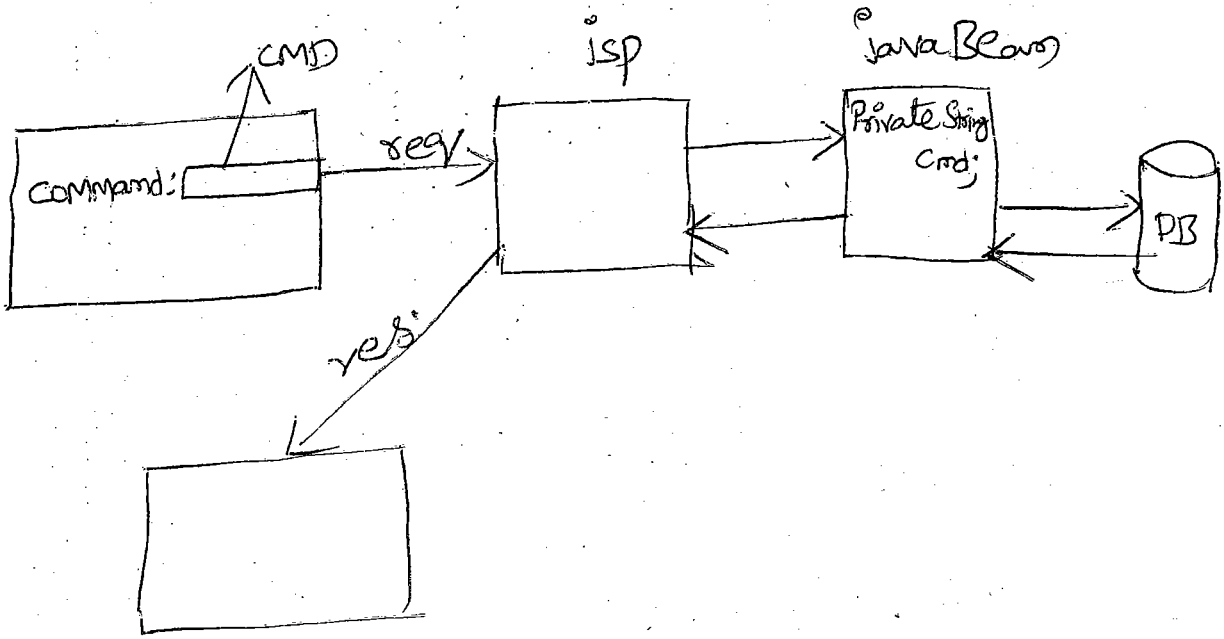
② Login.jsp

```
<%-- Login.jsp --- %>
<jsp:useBean id = "id1"
    class = "pack1.LoginBean">
    <jsp:setProperty name = "id1" property = "*"
    />
</jsp:useBean>
The username is : <jsp:getProperty
    name = "id1" property = "uname"
    />
<br>
<jsp:getProperty
```

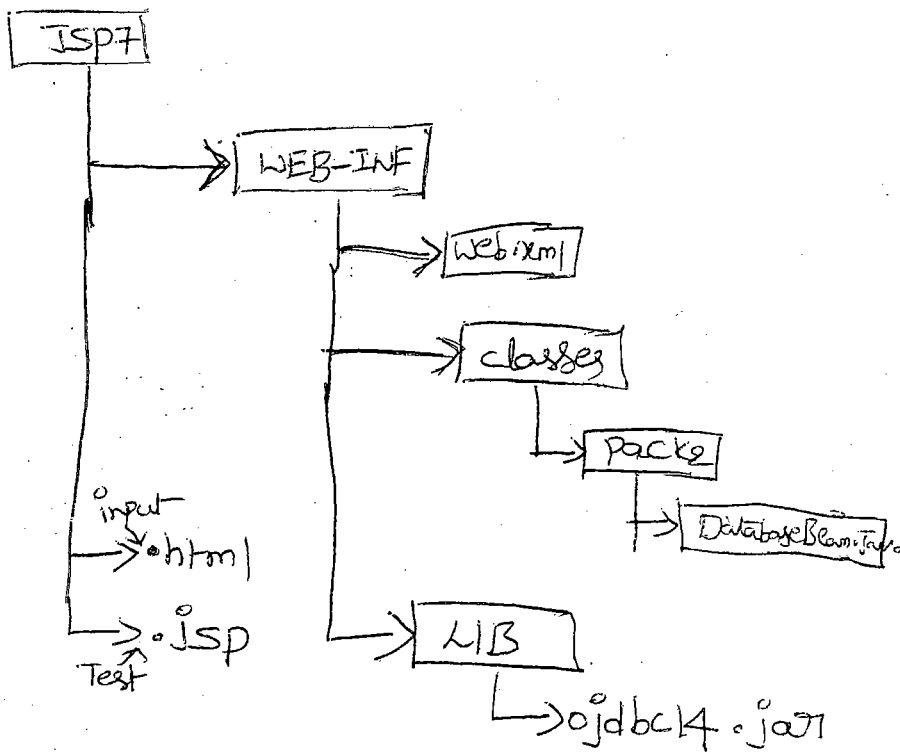
// LoginBean.java

```
package pack;
public class LoginBean
{
   private String uname, pwd;
   public void setUname(String uname)
   {
        this.uname = uname;
   }
public String getUname(S)
   {
        return uname;
   }
public void setPwd(String pwd)
   {
        this.pwd = pwd;
   }
   public String getPwd()
   {
        return pwd;
   }
}
```

Ex: (2)



Directory structure



① <!--- enter1.html ---->

<h1>

<form action = "test.jsp">

command : <input type = "text" name = "cmd"

style = "font_size:30px">

<input type = submit value = "execute">

</form>

</h1>

.

② <%--- Test.jsp ---%>

<jsp:useBean id = "id1" class = "pack2.DatabaseBean">

<jsp:setProperty name = "id1" property = "*" />

</jsp:useBean>

<%

int K = id1.bm();

if(K == -1)

{

```
out.println(" DDL command executed ");
}
if(k>=0)
{
  out.println(" DML Command executed ");
}
if(k==-10)
{
  ArrayList al = id1.getData();
  Iterator it = al.iterator();
  while(it.hasNext())
  {
    Object o = it.next();
    out.println(o.toString() + "<br>");
  }
}
<%@ page import="java.util.*" %>
```

→ In the above JSP we are calling business method of JavaBean.

following 3 value

-1 → DDL CMD is executed

+ve → DML CMD is executed

-10 → select CMD is executed.

If select CMD Executed then we are calling method calling getData() of the Beanclass to get an arraylist from the bean class.

Because of ArrayList & Iterator we have imported java.util package using page directive.

// DatabaseBean.java

```
Package pack2;
import java.sql.*;
import java.util.*;

public class DataBaseBean
{
    private String cmd;
```

```
ArrayList al = new ArrayList();
public void setCmd(String cmd)
{
    this.cmd = cmd;
}
public String getCmd()
{
    return cmd;
}
public int bm()
{
    int m = 0;
    try
    {
class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
DriverManager.getConnection("jdbc:odbc:oradsn",
                            "scott", "tiger");
Statement st = con.CreateStatement();
    if(st.execute(cmd))
    {
      ResultSet rs = st.getResultSet();
```

```
{
    al.add (rs.getString(1)+ " " + rs.getString(2));
  }
  return -10;
}
else
{
  m= st.getUpdateCount();
  }
}

catch(Exception e)
{}
return m;
}

public ArrayList getData()
{
    return al;
  }

}
```

⇒ In the above bean class, we have used type 4 driver, so we donot require any jar file into lib folder.

⇒ In the above bean class, we have called Execute() method, because Enduser can type Either select or Non-select cmd.

⇒ Execute() verifies whether the given cmd is select or Non-select and then returns true for select operation, return false for Non-select operation.

⇒ Execute() need 2 supporting methods

(i) getResultSet();

(ii) getUpdateCount();

getResultSet() → called for select operation,

getUpdateCount() → " " Non-select "

⇒ In the above bean class we have used an arraylist for storing the rows of the ResultSet object. we are Returning Arraylist() to the jsp

** We can not Return Directly ResultSet object from Javabeans to Jsp because ResultSet object is not a Serialize object.

⇒ We have taken an arraylist, & arraylist is a serialized object we can transfer it back to Jsp.

** Note:

(i) In Java, all collection objects are Serialize objects.

(ii) In Java, legacy (old) collections are Synchronized, but collection framework classes are not synchronized.

Direct communication from Jsp to a Database

(i) If we want to Directly Communicate from jsp to a database then we should open the database Connection, we should Execute the Cmd, finally we should close the Connection within jsp page itself.

(ii) In a jsp if we want to open & close the database Connection, we can override life-cycle init(), & destroy() methods of jsp.

(iii) if we want to override init() & destroy() of a jsp then we need to use Declaration tag of jsp.

(iv) we need to use scriptlet tag for inserting logic required for the Cmd Execution

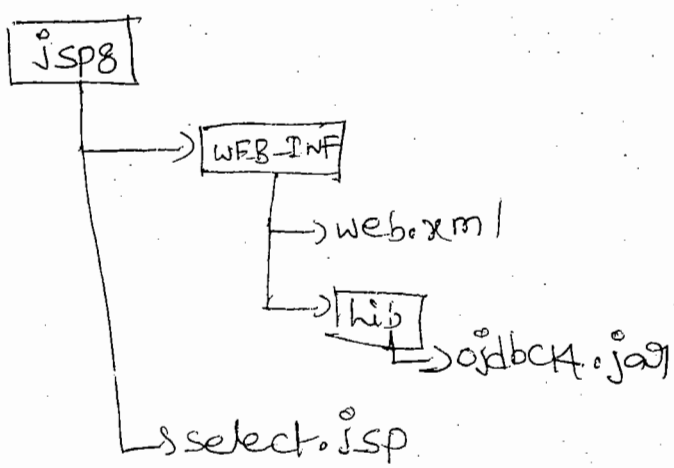
(v) If we want to open the Connection at init() & if we want to close it in the destroy() then we need to declare connection, stmt object as a Global variable.

```
<%!
    Connection con;
    Statement stmt;

    P v jspInit()
    {
        // open connection
    }
    P v . jspDestroy()
    {
        // close connection
    }
%>

<%
    == logic.
%>
```

Directory Structure



```
<!-- select.jsp -->

<%@ page import="java.sql.*" %>

<%!
    Connection Con;
    Statement stmt;

public void jspInit()
{
try
{
 class.forName("orace.jdbc.orcaleDriver");

Con= DriverManager.getConection("jdbc:
                                 orcle:
     stmt = Con.CreateStartament();
}
 catch(Exception e)
  {
       e.printStackTrace();
  }
}
jsp %>
```

```
<%
    try
    {
    ResultSet rs = stmt.executeQuery
                        ("select * from dept");
%>
<table border=2>
<%
    while(rs.next())
    {
out.println("<tr>");
out.println("<td>"+rs.getString(1)+"</td>"+
            "<td>"+rs.getString(2)+"</td>"+
            "<td>"+rs.getString(3)+"</td>");
    out.println("</tr>");
    }
%>
```

```
</table>
<%
rs.close();
}
catch(Exception e)
{
    e.printStackTrace();
}
%>
<%!
    public void jspDestroy()
    { try
      { con.close();
        stmt.close();
      }
     catch(Exception e)
      {
        e.printStackTrace();
      }
    }
%>
```

Date

Bean Scopes

Page scope (Local / Default scope)

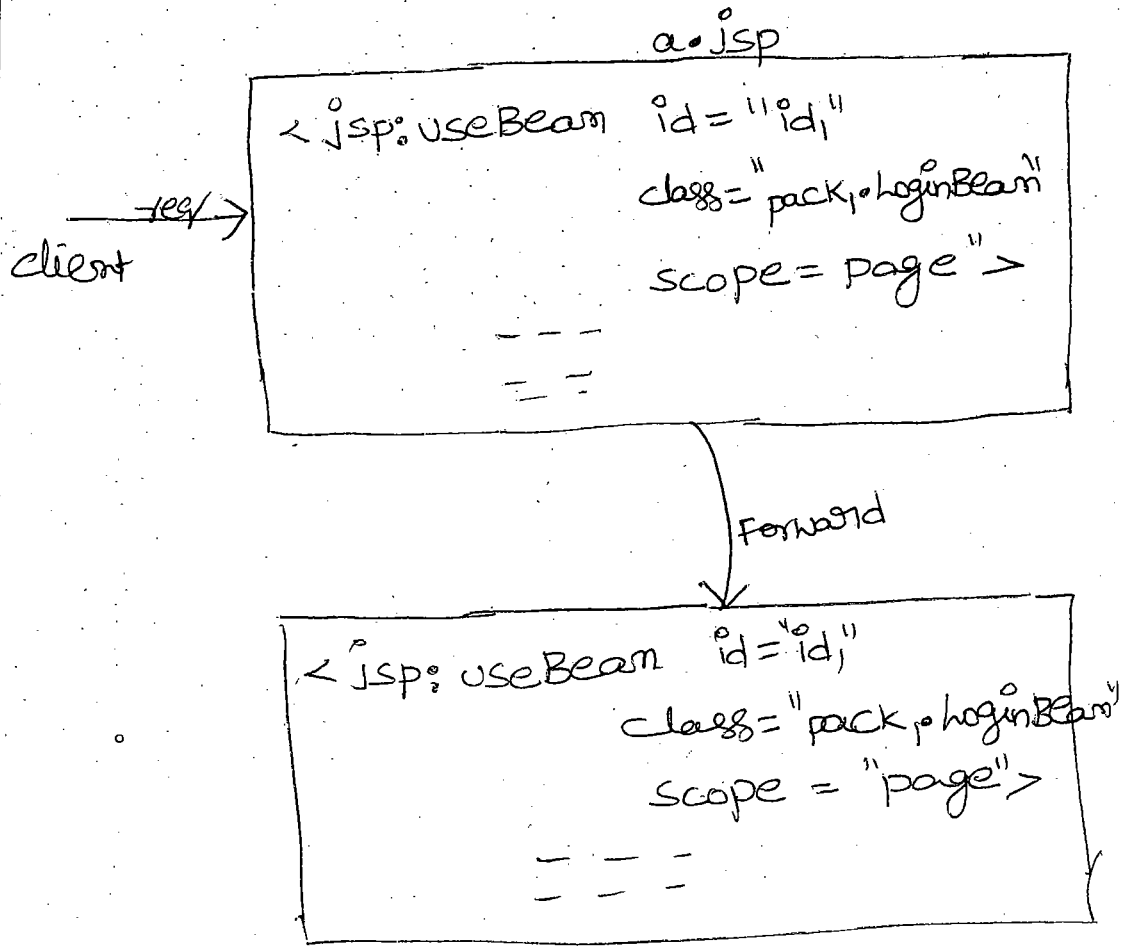
(i) Page is the least scope of a bean in jsp.

(ii) If a bean object is created in page scope that object is accesible only within that jsp.

(iii) page scope object becomes local to the jsp.

(iv) when a request is given to the jsp, the Bean object is created and then control going out of the that jsp page either to the next page back to the browser that object will be removed from the server.

(v) If we don't specify scope attribute then by default pagescope is assigned to the bean object.



-> In the above example client request is given to a.jsp and it is forwarded to b.jsp.

-> In the above case in a.jsp, the bean object "id1" is created with page scope and when the control is forwarded to b.jsp the "id1" object of a.jsp is removed and in b.jsp another object by the bean created with the name id1.

-> In the above example "id1" object is crea-

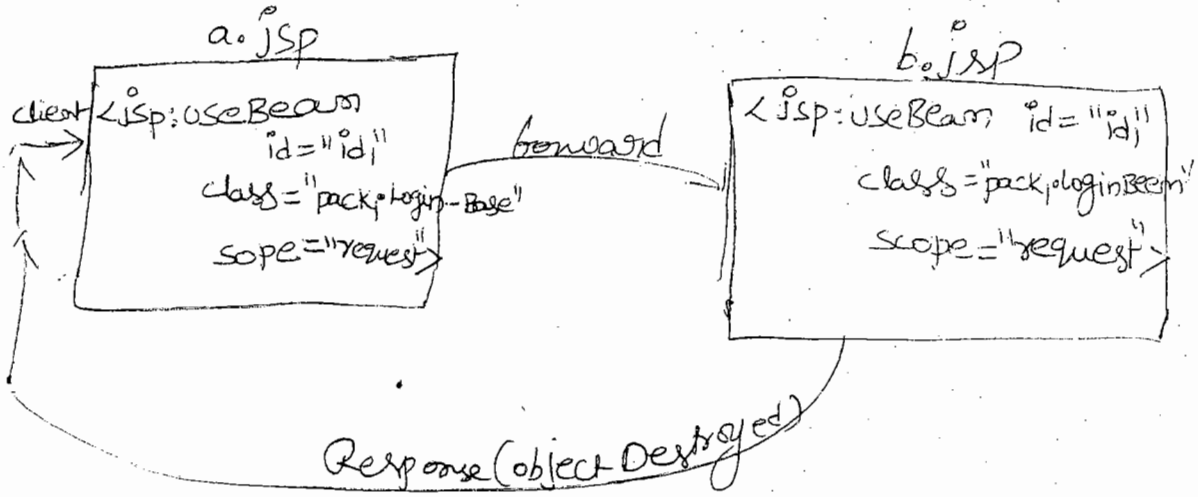
The object created in a.jsp not acceptable in b.jsp because of page scope.

(2) Request Scope :-

→ If a bean object is created with request scope then that object is available in the same jsp & if the request is forwarded to another page then in that page also the object is available.

→ Request scope is just more than page scope

→ Request scope object is removed whenever response is given back to the client



→ In the above diagram both a.jsp & b.jsp the Bean scope added as request..

→ when a client request is given to a.jsp, id1 object created with request scope. request forwarded to b.jsp, same object id1 is transferred to b.jsp also.

→ Request scope means the object is available in the current page and also continued to forwarded page.

→ In the above diagram response is given back to the client the object "id1" is removed from the server.

→ In both a.jsp & b.jsp same object "id1" is used. Seperate objects are not created.

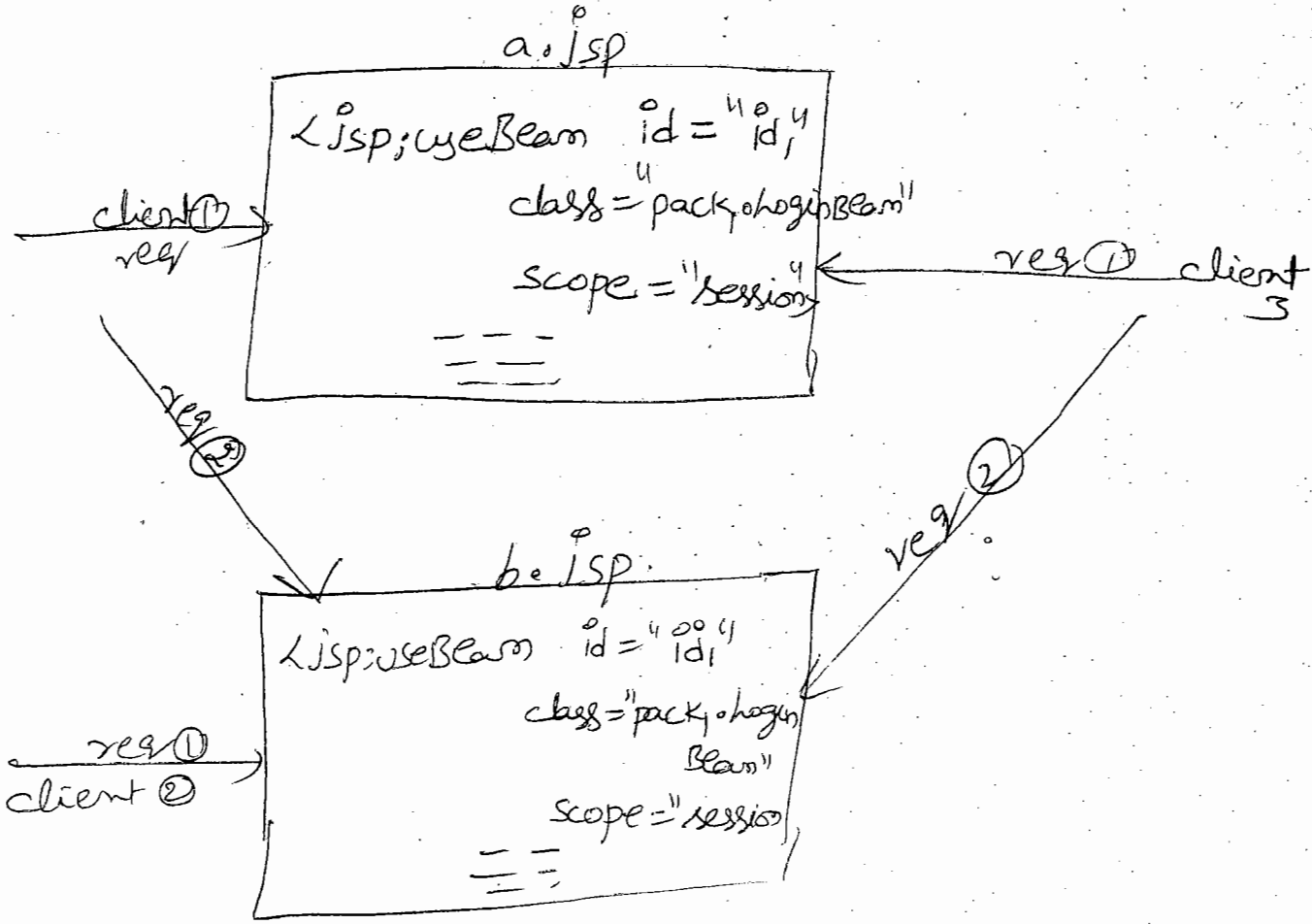
(3) Session Scope

→ when a Bean object is created in Session Scope then object will be created for each session seperately.

→ For each new client, a new session starts & For each new session a new object of the beean class is created.

→ The Bean object is removed whenever

⇒ Session scope is just more than request scope.



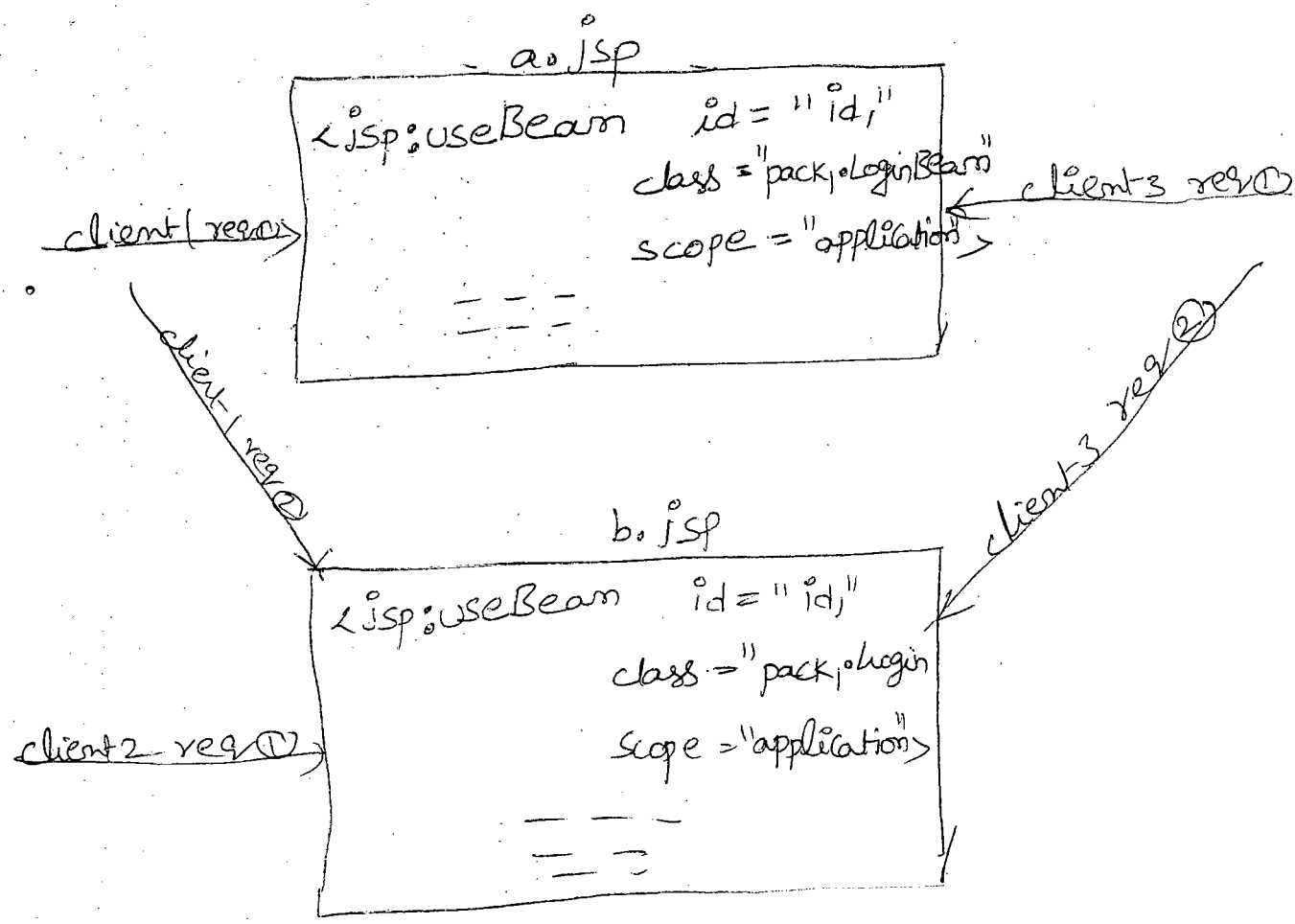
→ In the above diagram client ① & client 3 are sending first request to a.jsp & client 2 is sending first request to b.jsp for each new client a new session is started & the id1 object is created for each client (session) seperately. in the above case id1 object is

Created for 3 times because of 3 clients (sessions)

→ For client1 & client3, in b.jsp the object "id1" is not created, because both are already existing clients to the "b.jsp", and the clients are already having this sessions with "id1" objects.

→ The life-time "id1" object is until that client session is expired, but not until response is given.

(iv) Application Scope :-



→ If a Bean object is created in this scope the object become Global to the application

→ In application scope for any number of clients & for any number of jsp's only one object is created. and it is sharable across all the jsp's in the web-application

→ The highest scope of a bean object is application scope.

→ In the above diagram for all the clients & all the requests & all the Pages only one object "id," is created it is available to all jsp's because of application scope.

** what is the diff b/w page & Application Scope?

→ page scope is local & seperate object is created for each page. application scope is global and it is one time created & sharable across all pages.

** what is the diff b/w page & request scope?

A:- In page scope if the request is forwarded to the next page, the object will be removed and it is not accepted in the next page.

→ In the request scope the object is accessible to the next page if the request is forwarded

** what is the diff b/w Session & application?

→ In session scope object is created seperately for each session.

→ In application scope common object is created for all sessions.

** what is the diff b/w request scope & session scope?

* Request scope object life-time is until that request completed.

In session scope object life-time is until that client session expired.

<jsp:forward>

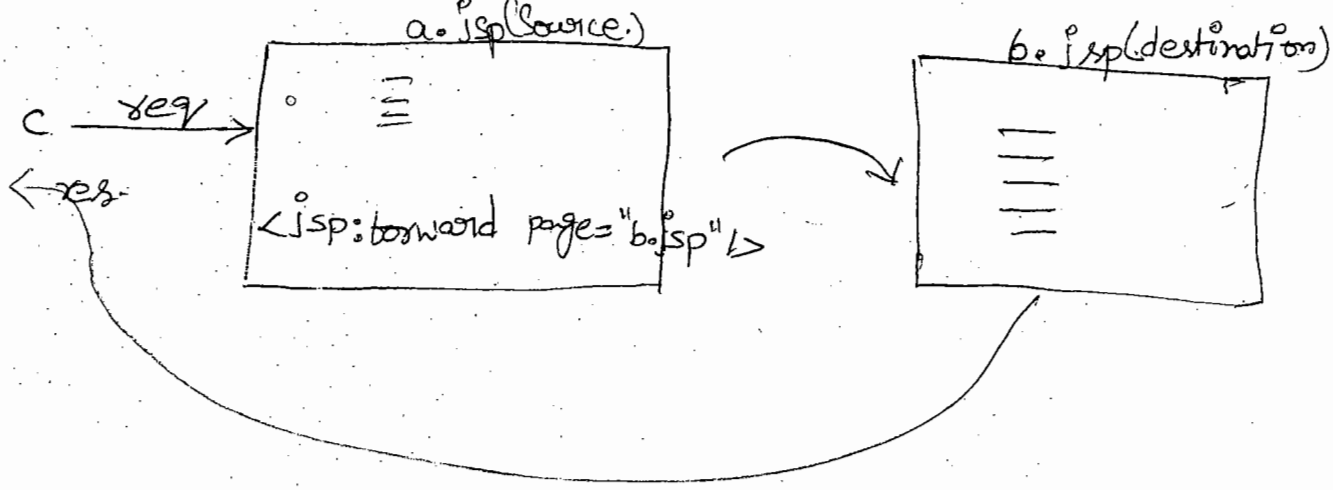
(i) This standard action is used to forward a client request from one page to another page.

(ii) This standard action internally uses request-Dispatchers forwarding mechanism.

(iii) While forwarding a client request, the Destination Resource can be a jsp page or A Servlet or an Html page.

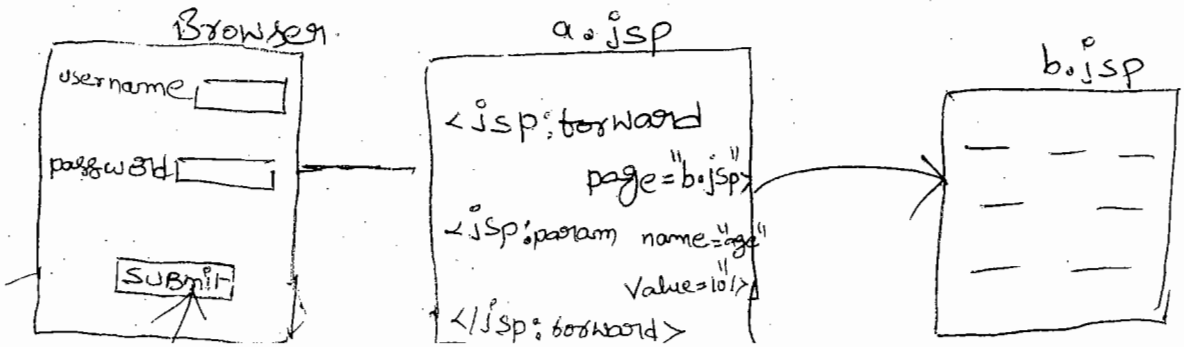
(iv) While forwarding a client request, The source & destination resources must be within the same server.

(v) While forwarding a client request, if destination is a jsp page then we should pass name of the jsp, if it is a Servlet we should pass the url-pattern of the Servlet



-> While forwarding a client request, automatically the Request will be forwarded as along with the Request parameter also forwarded.

-> Along with the request parameters, we can also add additional parameters will forwarding the request this can be done by using <jsp:param>



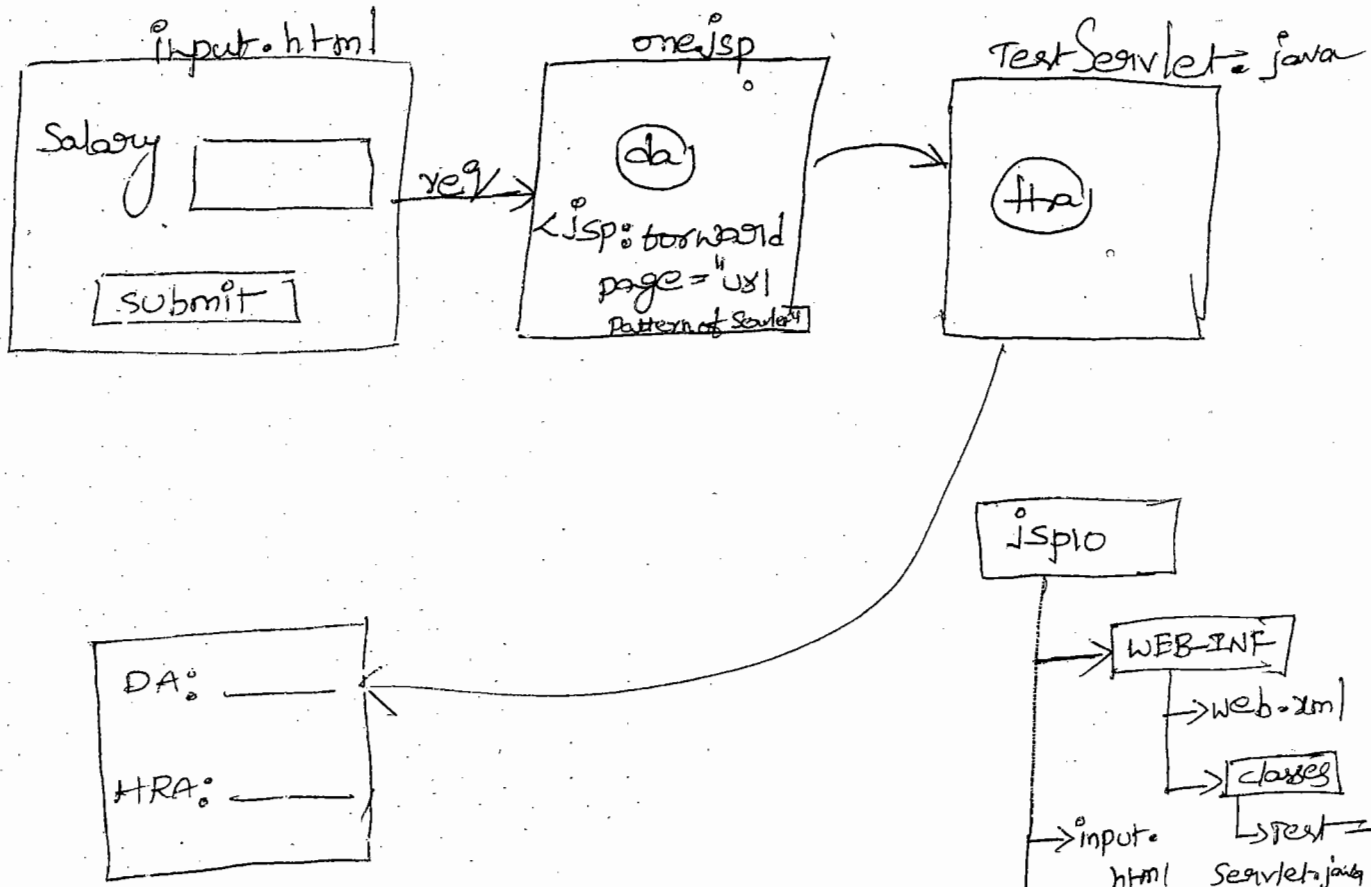
→ In the above diagram while forwarding the request from a.jsp to b.jsp automatically username & password are forwarded and along with that additional parameter age is also forwarded to b.jsp.

→ If we want to pass more than one additional parameter we should put more than one param tags inside the params.

```
<jsp:forward page="b.jsp">
    <jsp:params>
        <jsp:param name="age" value="10"/>
        <jsp:param name="email" value="—"/>
    <jsp:params>
</jsp:forward>
```

We can not use <jsp:param> & <jsp:params> individually. that means these tags must be used inside other class.

→ the following example is for forwarding a client request jsp to a servlet for calculating DA, HRA for the given salary.



input.html

```
<!-- input.html -->
<CENTER>
    <form action = "one.jsp">
    Salary: <input type= text name ="s"><br>
    <input type = submit value="click">
    </form>
</center>
```

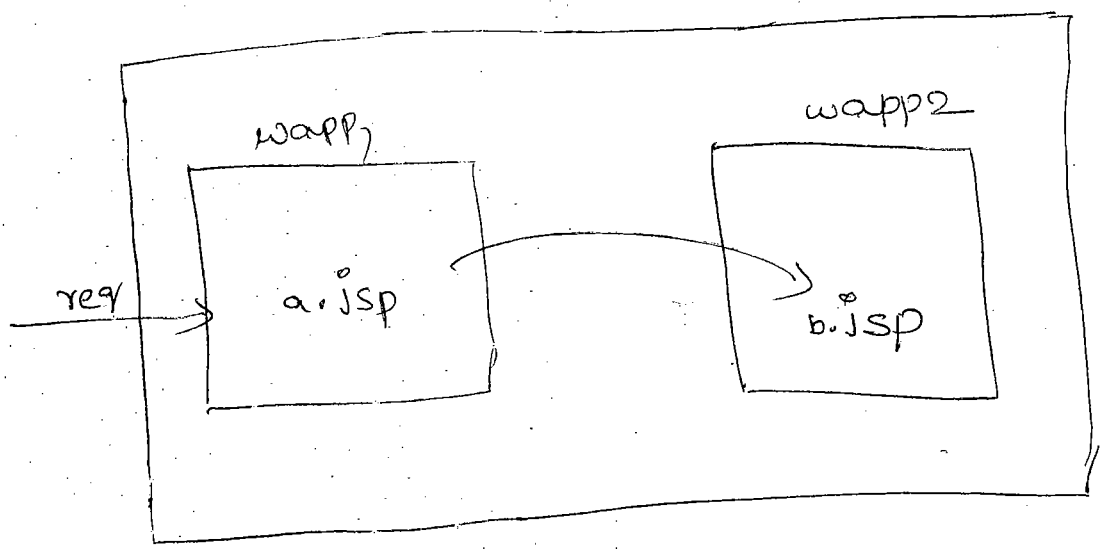
(2)
```
<!-- one.jsp -->
<%!
   double da;
%>
<%
   String s1 = request.getParameter("s");
   double sal = Double.parseDouble(s1);
      da = sal * 0.10;
   request.setAttribute("da", new double(da));
%>
<jsp:forward page="TestServlet" />
```

(3) // TestServlet.java

```
import javax.servlet.http.*;
import java.io.*;
import javax.servlet.*;
public class TestServlet extends HttpServlet
public void doGet(HttpServletRequest req, HttpServletRes res)
X   throws SE, IOE
{
   String s2 = request.getParameter("s")
      // here s2 containg salary
   double sal = Double.parseDouble(s2);
```

```
double hra = sal*0.15;
PrintWriter pw = response.getWriter();
Double x = (Double)request.getAttribute("da");
pw.println("DA" : x.doubleValue());
pw.println("hra" : hra);
}
```

while forwarding a request to destination resource is avaible in another web application; then we should mention web-application name also



<jsp:forward page="\wapp2\b.jsp" />

Note: Marker interfaces are two types

(i) Predefined

(ii) user-defined | customized.

* Hay we no that serialization can be done in two ways.

(i) Complete or total Serialization

(ii) in-complete or partial "

* whenever one class all variables are participated it is said to be complete persistency.

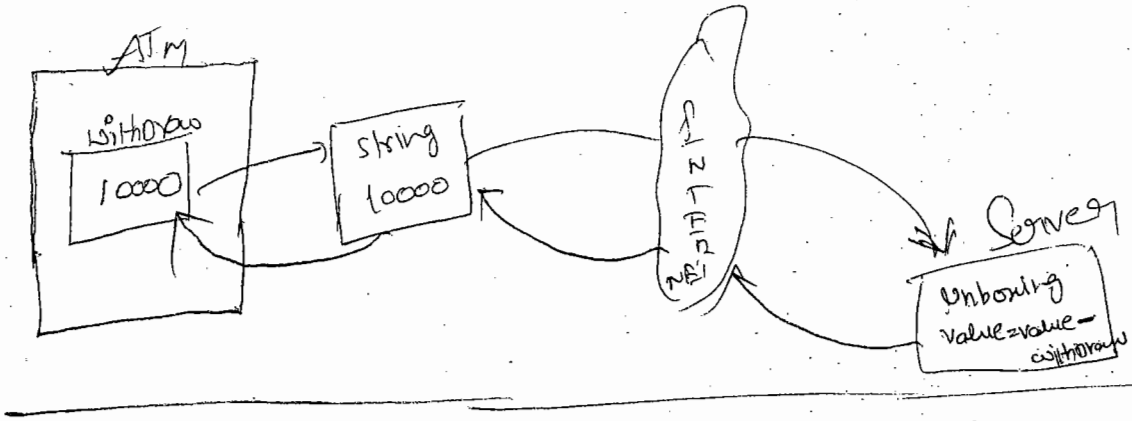
* whenever some of the members are not participated it is partial persistency.

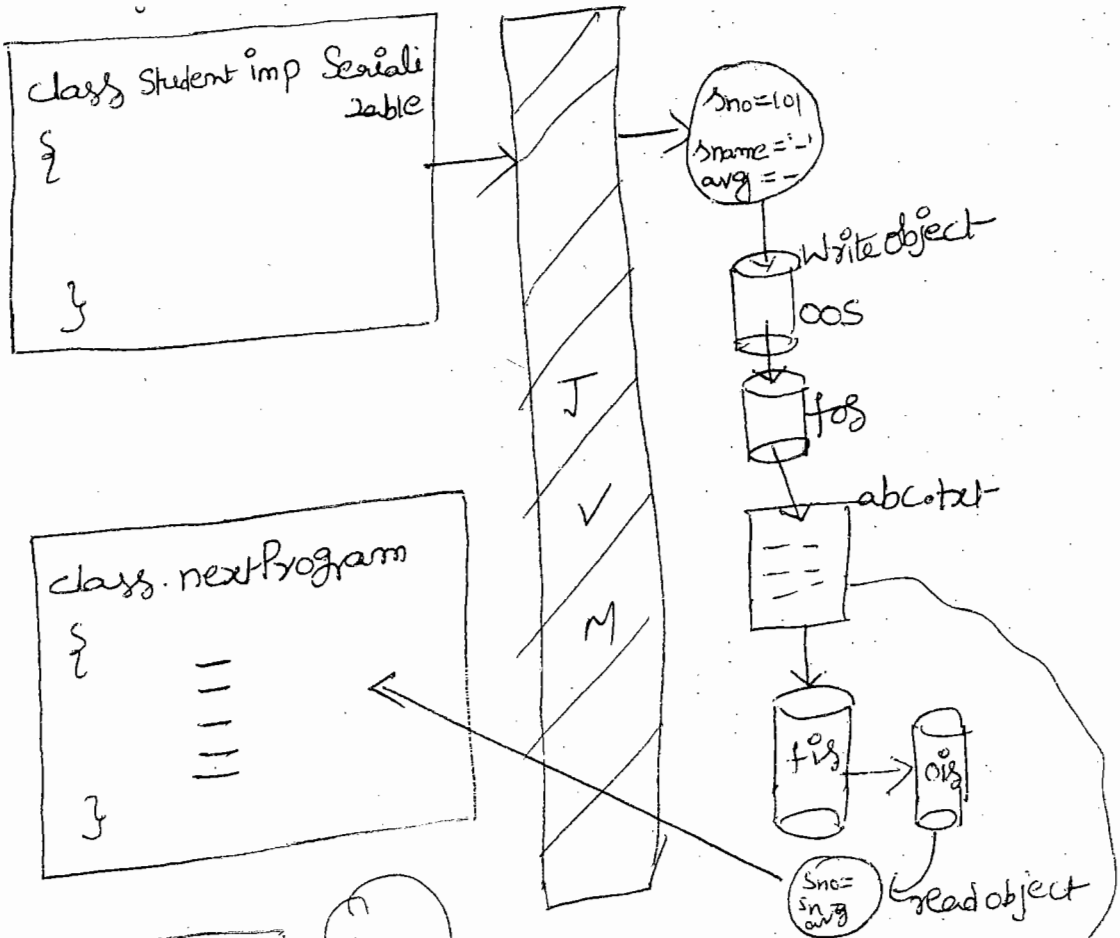
→ For this particial persistency we are using transistent modifier then those members are not involved in Serialization

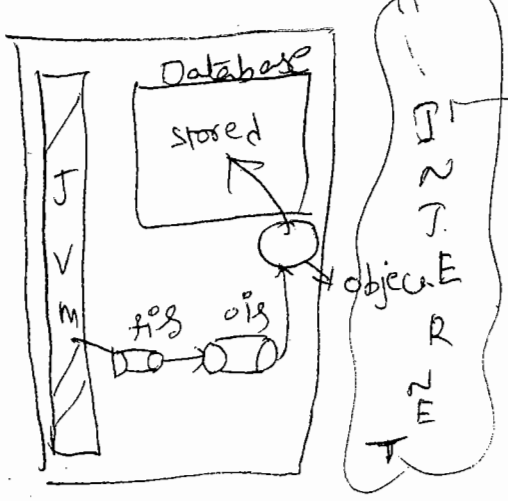
```
Trasient double avg;
```

→ Transient variables are non-static variable.

Static members are not involved in the persistency.


ATM
withdraw
10000
String
10000
CLIENT
Server
Unboxing
value=value-
withdraw


class Student imp Serializable
{
}
JVM
Sno=101
Sname=''
avg=--
Write object
oos
fos
abc.txt
class. nextProgram
{
}
fis
ois
Sno=
Sname
avg
Read object


Database
stored
JVM
fis
ois
object
INTERNET

Server

the